



UNIVERSITÀ DI TRENTO

Department of Information Engineering and Computer Science

Master's Degree in
Artificial Intelligence Systems

FINAL DISSERTATION

DENSE ASSOCIATIVE MEMORIES AND DEEP FEEDFORWARD NEURAL NETWORKS IMPLEMENTED VIA QUANTUM ANNEALER

Supervisor
Davide Pastorello

Student
Dimitri Vinci

Co-supervisor
Enrico Blanzieri
Enrico Zardini

Academic year 2023/2024

Contents

Abstract

- 1 Core concepts dissertation**
 - 1.1 Machine Learning
 - 1.2 Quantum Computation
 - 1.3 Quantum Machine Learning
- 2 Quantum Dense Associative Memories**
 - 2.1 Strategies and difficulties in developing QUBO formulation for Dense Associative Memory
 - 2.2 A cost effective dense associative memory activation functions derivation
- 3 Quantum deep neural networks via quantum dense associative memories**
 - 3.1 Training shallow and deep neural networks in energy minimization framework
 - 3.2 An analogue of backpropagation for energy minimization networks
- 4 Hybrid quantum-classical deep neural networks**
 - 4.1 Kernel formulation of deep feed-forward neural network for hybrid classical-quantum training
 - 4.2 Application of kernel formulation for hybrid quantum-classical deep neural network . .
- 5 Conclusion and further research**

Bibliography

- A Notation**
 - A.1 Generic notation employed
 - A.2 Specific symbol employed
- B Chapters integrations**
 - B.1 Machine learning
 - B.2 Quantum computation integration
 - B.3 Quantum machine learning
 - B.4 Present work integration

Abstract

In this work a theoretical analysis about how treating dense associative memories and deep feed-forward neural networks in the quantum annealing formulation is exposed¹.

Quantum annealing (QA) is an optimization process for finding the global minimum of a given objective energy function over a given set of candidate solutions (candidate states), using quantum fluctuations (more details in chapter 1.2). Furthermore, dense associative memories instead are a particular type of recurrent neural networks, capable to store and retrieve predefined memories in an energy minimization fashion (more details in following chapters). Hopfield networks /Associative Memories have a naturally perceptron interpretation [203] and at the same time Dense Associative Memories (see chapter 1.1) have a feed-forward interpretation [80] [147]. Obtaining a suitable QUBO/Ising formulation of Dense associative memories implemented in a quantum annealing setting allows for modifications (as reported in this work) to tackle the problem of implementing (training and sampling) shallow and deep neural networks via quantum annealing.

Having a formulation for quantum annealers (QA) of dense associative memories has other motivations: being a known models in the research community, Associative Memories allow for easy of interpretability of associative models by users, making easier comprehension and usage of quantum annealing by researchers who have no specific education in computer science or quantum computing [13]. Further there is a direct analogy between Associative Memories energy and quantum annealing minimization: associative networks retrieve information via energy minimization, allowing to cast it directly in annealer energy minimization. These aspects are further developed in second chapter of present document.

However Dense Associative Memories are not easy transferable to actual quantum annealers due to quantum annealers limitations: quantum annealers minimize only quadratic functions while dense associative memories comprise more-than-binary/quadratic interactions. At the same time actual hardware implementations of quantum annealers have a low number of qubits, meaning every possible QUBO formulation of a dense associative memories implemented via quantum annealer should have a limited usage of architectural resources. Both these problems are strictly related, as it will be apparent in following chapters, and it will be shown possibility to tackle both of them in an efficient way.

The proposed work achieved and exposes first proposal of a QUBO formulation suitable for a quantum annealer of dense associative memories (chapter 2), first proposal for quantum deep feed-forward neural network on the annealer (chapter 3), first proposal to approach training of deep feed-forward neural network on limited qubits annealing architectures (chapter 3), first cost effective method to treat sampling and ensembling of deep neural network (chapter 4), first proposal on how treating hybrid classical/quantum training of neural network (chapter 4).

Note finally that, as it will be exposed (chapter 2) other proposals of feed-forward neural network implemented on QA arised during the development of this work but with significant limitations, in particular: most of them have higher complexity that the present work, all of them do not consider deep (more than three layer) implementation, have limitations in dealing with activation functions, do not allow for feed-forward in the annealer (meaning they either simulate the model to sample over that or try optimize training procedure deferring forward pass in classical setting), do not naturally allow for an hybrid quantum classic approach to the problem and finally do not allow for an ensembling

¹To make it easy for the reader fruition of present content, note that full list of symbols used in all work is reported in Appendix A

approach to the problem to exploit multiple annealer solutions

1 Core concepts dissertation

This section expose necessary theoretical aspects to understand produced contents.

In particular chapter 1.1 deals with a brief dissertation of the core problems and elements of machine learning. This chapter focuses more about support vector machines (machine learning most promising algorithm before deep learning revolution) and neural networks. Neural networks subsection regards in particular associative memories, a model widely studied by neuroscientific community in the eighties, and deep feed forward neural network, the core model used in the present era.

Chapter 1.2 introduces main concepts of quantum computation. It briefly mentions circuit based quantum computation and it develops a brief discussion about adiabatic quantum computation and quantum annealing, the core computational model used in this work.

Chapter 1.3 elaborates on the concept of quantum machine learning meaning the union of the concepts of machine learning and quantum computation. Focus will be placed more on proposal of quantum implementation of support vector machines, quantum associative memories and quantum neural networks. These notions are used in following chapters to develop thesis original content.

1.1 Machine Learning

Machine learning is a form of applied statistics with emphasis on the use of computers to statistically estimate complicated functions[107]. ML is generally used when human expertise does not exist, it is not easy to explain expertise, models must be customized or models are based on huge amounts of data. Machine Learning core concept is to allow machines learn from experience/data as stated in Tom Mitchell historical definition: "A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at task in T , as measured by P , improves with experience E ." [170] This definition casts the so called "learning problem" in three well defined components that are analyzed thereafter: Experience, Task and Performance.

Experience defines learning algorithm used, and it is bounded to task and performance measure. Typically experience type is differentiated on basis of availability of provided labels given specific data. In this work supervised learning is treated, in which data-set is composed of training examples, each of them associated with target labels [107].

The task is defined as the way in which user wants to process an example. An example is a collection of features, which defines the so called representation of a sample. This defines a vector $\mathbf{x}$ in R^n where each entry x_i of the vector is a feature [107]. Different machine learning tasks, based on the type of experience, are related to supervised learning. In this work classification tasks are mainly considered: the computer program is asked to specify which of k categories some input belongs to [107]. The learning algorithm produces a function $f : R^n \rightarrow \{1, \dots, k\}$ meaning the model assigns an input described by vector $\mathbf{x}$ to a category identified by numeric code y [107].

Performance measure, meaning quantitative measure of algorithm performance, is strictly related to the task. The metric of reference for classification tasks is accuracy, typically defined as proportion examples for which the model produces the correct output. Accuracy in binary classification could be a misleading metric given it cannot account for (generally) greater importance of true positives [48] [76]. So in many experiments (e.g. [245]) is customary to report AUROC/AUC, area under receiving operating characteristic (ROC) curve. ROC curve is obtained plotting true positive rate (the probability of a positive test result, conditioned on the individuals truly being positive) versus false positive rate (the probability of a positive test result, conditioned on the individuals falsely being positive) varying some parameters of learning algorithm that determine predicted class [245].

Learning process could be realized measuring and optimizing a performance metric over a test set, allowing to select best model in accordance. The "best model" is defined in accordance of different representations. Supervised models representations comprehend instance-based non-parametric func-

tions (e.g. k-nearest neighbors, kernel regression, local regression) and numerical parametric functions (e.g. support vector machines, deep learning, linear regression). For these models, learning problem is generally in NP complexity class. The general learning problem for a neural network, which consists in finding the unknown elements of a given architecture to compute a specific given function, is proved being NP-complete [203]. In this formulation of the problem, network components (also architectural) of the network could be unknown. Even if some of them are kept fixed, simply try finding parameters of a predefined function able to produce the correct output for all provided training examples is NP-complete [133] (even with two layered three-nodes neural network [28]). Some assumptions to limit complexity of the problem are necessary. Smoothness prior, meaning objective function at hand should not change much within small regions $f(\mathbf{x}) = f(\mathbf{x} + \varepsilon)$ is most widely applied assumption. Non-parametric machine learning algorithms (with smoothness assumption) have difficulties in very high dimensions where even a very smooth function can change in a different way along dimensions [107]. This is one of the motivation parametric functions are the state of the art methods in many machine learning algorithms (and the subject of interest of this work).

Parametric methods subsume parameters estimation concept. Point estimation attempts to provide the single “best” prediction of some quantity of interest (vector of parameters in parametric model). Let $\{\mathbf{x}_1, \dots, \mathbf{x}_m\}$ be a set of m independent and identically distributed (i.i.d.) data points. A point estimator or statistic is any function of the data:

$$\hat{\mathbf{w}}_m = g(\mathbf{x}_1, \dots, \mathbf{x}_m) \quad (1.1)$$

Function estimation tries to predict a variable $\mathbf{y}$ given an input vector $\mathbf{x}$. It assumes that there is a function $f(\mathbf{x})$ that describes the approximate relationship between y and $\mathbf{x}$. f is then approximated with a model or estimate $\hat{f}$. The function estimator $\hat{f}$ is simply a point estimator of function parameters in function space.

Statistical learning theory give assurance in how performance measures could be used to find those parameters, also when are obtained from limited number of data. The following brief exposure is based on [234]. Suppose m observations are given. Each observation consists of a pair: a vector $\mathbf{x}_i \in R^n$, $i = 1, \dots, m$ and the associated “truth” y_i . It is assumed that there exists some unknown probability distribution $P(\mathbf{x}, y)$ from which these data are drawn, i.e., the data are assumed “iid” (independently drawn and identically distributed). Now suppose to have a machine whose task it is to learn the mapping $\mathbf{x}_i \mapsto y_i$. The machine is actually defined by a set of possible mappings $\mathbf{x} \mapsto f(\mathbf{x}, \mathbf{w})$, where the functions $f(\mathbf{x}, \mathbf{w})$ themselves are labeled by the adjustable parameters $\mathbf{w}$. The machine is assumed to be deterministic: for a given input $\mathbf{x}$, and choice of $\mathbf{w}$, it will always give the same output $f(\mathbf{x}, \mathbf{w})$. The $\mathbf{w}$ parameters should be estimated by learning algorithms. Not specifying $\mathbf{w}$ conversely defines a family of learning machines. The expectation of the test error for a trained machine is therefore:

$$R(\mathbf{w}) = \int \frac{1}{2} |y - f(\mathbf{x}, \mathbf{w})| dP(\mathbf{x}, y) \quad (1.2)$$

$$R(\mathbf{w}) = E_{(\mathbf{x}_0, y_0) \sim p(\mathbf{x}, y)} [L(f(\mathbf{x}_0; \mathbf{w}), y_0)] \quad (1.3)$$

“empirical risk” $R_{emp}(\mathbf{w})$ is defined to be just the measured mean error rate on the training set (for a fixed, finite number of observations) :

$$R_{emp}(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^m |y_i - f(\mathbf{x}_i, \mathbf{w})|. \quad (1.4)$$

Considering a general error measure for a task defined over N data-points:

$$L(\mathbf{w}, \mathbf{X}, \mathbf{y}) = \frac{1}{m} \sum_{i=1}^m L(f(\mathbf{x}_i; \mathbf{w}), y_i) \quad (1.5)$$

If the training set is a representative of the underlying (unknown) probability distribution $p(x, y)$, the empirical loss is a proxy for the risk. In fact, choose some η such that $0 \leq \eta \leq 1$ then for losses taking

these values, with probability $1 - \eta$, the following bound holds :

$$R(\mathbf{w}) \leq R_{emp}(\mathbf{w}) + \sqrt{\left(\frac{h(\log(2m/h) + 1) - \log(\eta/4)}{m} \right)} \quad (1.6)$$

where h is a non-negative integer called the Vapnik Chervonenkis (VC) dimension, and is a measure of the notion of capacity, meaning capability to fit wide variety of function. In the following the right hand side of previous inequality is called the "risk bound." The second term on the right hand side is called the "VC confidence." Note finally that it is usually not possible to compute the left hand and if one knows h , one can easily compute the right hand side.

Algorithms are further divided in discriminative (e.g. support vector machines, feed-forward neural networks) and generative (e.g. naive bayes classifier, variational autoencoders). Discriminative methods focus on the decision boundary, directly mapping input data to label function. Having lots of examples at disposal, they are generally more powerful than generative models but are not designed for incorporating unsupervised data. In this work discriminative methods are considered, being the standard method to tackle classification with supervised data.

A natural question is whether classifier could be combined to enhance performances. Ensemble techniques allow for reducing generalization errors combining several models (model averaging). Ensemble technique proved themselves to be useful for every algorithms (typically machine learning contests are won by ensemble methods like [143]), also for deep learning (for example [21] [226]). Typical ensemble methods are bagging or bootstrap aggregating [35], or boosting [98]. However it's worth to be underlined that all these quoted methods require different training procedure for each one of the models used to construct ensemble. Given high computational cost of training some models (like neural networks), different ensemble methodologies exposed in next sections, like dropout, are typically employed.

To sum up three machine learning models are considered in this work

- Support Vector Machines: parametric, numeric, discriminative a widespread model in the category of Kernel methods
- Associative memories: parametric, numeric, discriminative
- Deep Neural Network: parametric, numeric, discriminative

Kernel Methods and Support Vector Machines

Classification is often computed as feature similarities, meaning dot product between feature representation. If feature mappings of a classification algorithms appear only in dot products the kernel trick, discovered by [3], consists in replacing these dot products with an equivalent kernel function:

$$k(\mathbf{x}, \mathbf{x}') = \Phi(\mathbf{x})^T \Phi(\mathbf{x}') \quad (1.7)$$

A valid kernel is a (similarity) function defined in Cartesian product of input space $k: \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ corresponding to a dot product in an (arbitrary) feature space. The category of algorithms that employ the kernel trick is known as kernel machines, or kernel methods [217] [244].

The most commonly used kernel is the Gaussian kernel,

$$k(\mathbf{x}, \mathbf{x}') = \mathcal{N}(\mathbf{x} - \mathbf{x}'; 0, \sigma^2 \mathbf{I}), \quad (1.8)$$

where $\mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma})$ is the standard normal density. This kernel is also known as the radial basis function (RBF) kernel, because its value decreases along lines in $\mathbf{x}$ space radiating outward from $\mathbf{x}'$. The Gaussian kernel corresponds to a dot product in an infinite-dimensional space and compared to other kernels, it does not suffer of numerical difficulties and empirically compare favorably to sigmoid or hyperbolic tangent kernel .

The kernel trick is widely used, given it enables to learn models that are nonlinear as a function of $\mathbf{x}$ using convex optimization techniques that are guaranteed to converge efficiently limiting optimization

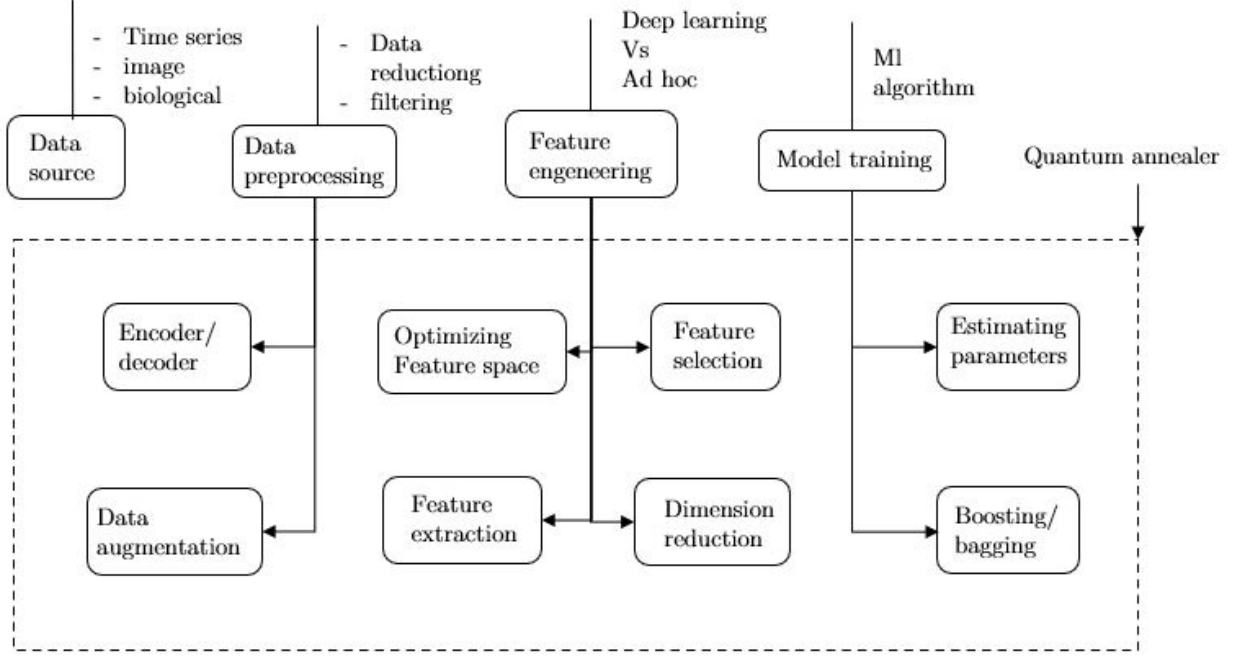


Figure 1.1: Machine learning pipeline

to kernel itself. However it also expose some drawbacks: the cost of evaluating the decision function is linear in the number of training examples, because the i -th example contributes a term $\alpha_i k(\mathbf{x}, \mathbf{x}_i)$ to the decision function leading to high computational cost of training when the dataset is large. Further, Kernel machines with generic kernels struggle to generalize well. It is interesting to note that while the k -nearest neighbors algorithm copies the output from nearby training examples also most kernel machines (local kernels) interpolate between training set outputs associated with nearby training examples. As it's exposed in next section modern motivation for deep learning is derived from studying the limitations of this so called local template matching [19]

Support vector machines are a family of kernel methods based on the following core idea: for a given learning task with a given finite amount of training data the best generalization performance will be achieved if the right balance is obtained between the accuracy attained on that particular training set and the capacity of the machine, meaning the ability of the machine to learn any training set without error. Given the previously exposed bound on empirical risk (equation 1.6) the VC confidence term (on the right side of the summation on the right of equation) depends on the chosen class of functions whereas empirical and actual risk depends on training procedure so it is ideal to find subset of the chosen set of functions such that the actual risk bound for that subset is minimized [233][39]. The VC dimension for the SVM is at most the dimension of the data in kernel space and from [234]:

- Given N data points in $R^n : X = \{\mathbf{x}_1, \dots, \mathbf{x}_m\}$ with $\|\mathbf{x}_n\| \leq r$
- Defining $\mathcal{H}_\rho$: set of classifiers in R^n having margin ρ on $\mathbf{X}$

The VC dimension of $\mathcal{H}_\rho$ is bounded by:

$$vC(\mathcal{H}_\rho) \leq \min \left\{ n, \left\lceil \frac{4r^2}{\rho^2} \right\rceil \right\} \quad (1.9)$$

To achieve this strong results, support vector machines are defined as linear classifiers selecting hyper-plane maximizing separation margin between classes. Given a training set $\mathcal{D}$, a classifier confidence margin is:

$$\rho = \min_{(\mathbf{x}, y) \in \mathcal{D}} yf(\mathbf{x}) \quad (1.10)$$

A classifier geometric margin is equivalently defined as

$$\frac{\rho}{\|\mathbf{w}\|} = \min_{(\mathbf{x}, y) \in \mathcal{D}} \frac{yf(\mathbf{x})}{\|\mathbf{w}\|} \quad (1.11)$$

Note that there is an infinite number of equivalent formulation for same hyperplane

$$\mathbf{w}^T \mathbf{x} + w_0 = 0 \quad (1.12)$$

$$\alpha (\mathbf{w}^T \mathbf{x} + w_0) = 0 \quad \forall \alpha \neq 0 \quad (1.13)$$

so it is customary to define the canonical hyperplane or the hyperplane having confidence margin equal to 1:

$$\rho = \min_{(\mathbf{x}, y) \in \mathcal{D}} yf(\mathbf{x}) = 1 \quad (1.14)$$

The SVM predicts that the positive class is present when $\mathbf{w}^T \mathbf{x} + b$ is positive. Likewise, it predicts that the negative class is present when $\mathbf{w}^T \mathbf{x} + b$ is negative. Points staying on the minimal confidence hyperplanes are called support vectors and are the only one contributing to the classification (consequently obtaining sparse classifier). It can be shown that the linear function used by the support vector machine can be re-written as

$$\mathbf{w}^T \mathbf{x} + b = b + \sum_{i=1}^m \alpha_i \mathbf{x}^T \mathbf{x}_i \quad (1.15)$$

where $\mathbf{x}^{(i)}$ is a training example, and α is a vector of coefficients. After replacing dot products with kernel evaluations (so using kernel trick), predictions are made using the function

$$f(\mathbf{x}) = b + \sum_i \alpha_i k(\mathbf{x}, \mathbf{x}_i) \quad (1.16)$$

Standard SVM are called hard margin SVM and follows this problem definition

$$\min \mathbf{w}, w_0 \quad \frac{1}{2} \|\mathbf{w}\|^2 \quad (1.17)$$

subject to:

$$y_i (\mathbf{w}^T \mathbf{x}_i + w_0) \geq 1 \\ \forall (\mathbf{x}_i, y_i) \in \mathcal{D}$$

the constraints guarantee that all points are correctly classified and minimization corresponds to maximizing the (squared) margin, obtaining a quadratic optimization problem. The Karush-Kuhn-Tucker (KKT) approach provides a very general solution to constrained optimization problem like $\min_z f(z)$ subject to $g_i(z) \geq 0 \forall i$. It introduces a non-negative variable $\alpha_i \geq 0$ called Lagrange multiplier for each constraint and rewrite the optimization problem as generalized Lagrangian, or generalized Lagrange function:

$$\min_z \max_{\alpha \geq 0} f(z) - \sum \alpha_i g_i(z) \quad (1.18)$$

The optimal solutions for this problem are the same as the optimal solutions for the original constrained problem. This particular formulation of dual problem is called Wolfe dual [39][95] and applied to the present case

$$L(\mathbf{w}, w_0, \boldsymbol{\alpha}) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^m \alpha_i (y_i (\mathbf{w}^T \mathbf{x}_i + w_0) - 1) \quad (1.19)$$

The Lagrangian is minimized wrt $\mathbf{w}, w_0$ and maximized wrt α_i , therefore solution is a saddle point.

To solve it, one calculates the derivatives of primal variables

$$\frac{\partial}{\partial w_0} L(\mathbf{w}, w_0, \boldsymbol{\alpha}) = 0 \Rightarrow \sum_{i=1}^m \alpha_i y_i = 0 \quad (1.20)$$

$$\frac{\partial}{\partial \mathbf{w}} L(\mathbf{w}, w_0, \boldsymbol{\alpha}) = 0 \Rightarrow \mathbf{w} = \sum_{i=1}^m \alpha_i y_i \mathbf{x}_i \quad (1.21)$$

Substituting them in the Lagrangian

$$\begin{aligned} \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^m \alpha_i (y_i (\mathbf{w}^T \mathbf{x}_i + w_0) - 1) &= \frac{1}{2} \sum_{i,j=1}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j - \sum_{i,j=1}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \\ &\quad - \underbrace{\sum_{i=1}^m \alpha_i y_i w_0}_{=0} + \sum_{i=1}^m \alpha_i \end{aligned} \quad (1.22)$$

$$= \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \quad (1.23)$$

$$= L(\boldsymbol{\alpha}) \quad (1.24)$$

which have to be maximized wrt the dual variables

$$\max_{\boldsymbol{\alpha} \in R^m} \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \quad (1.25)$$

subject to

$$\alpha_i \geq 0 \quad i = 1, \dots, m \quad \sum_{i=1}^m \alpha_i y_i = 0$$

that is still a convex problem. The bias can then be computed from the KKT conditions [197]

$$y_i (\mathbf{w}^T \mathbf{x}_i + w_0) = 1 \quad (1.26)$$

$$y_i \mathbf{w}^T \mathbf{x}_i + y_i w_0 = 1 \quad (1.27)$$

$$w_0 = \frac{1 - y_i \mathbf{w}^T \mathbf{x}_i}{y_i} \quad (1.28)$$

Typically regularization term to account for smooth and simpler solutions is introduced, in this case allowing margin violations [48]

$$\min_{\mathbf{w} \in \mathcal{X}, w_0 \in R, \boldsymbol{\zeta} \in R^m} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \zeta_i \quad (1.29)$$

subject to

$$y_i (\mathbf{w}^T \mathbf{x}_i + w_0) \geq 1 - \zeta_i \quad i = 1, \dots, m$$

$$\zeta_i \geq 0 \quad i = 1, \dots, m$$

A slack variable ζ_i represents the penalty for the example x_i not satisfying the margin constraint. The sum of the slacks is then minimized together with the inverse margin. The regularization parameter

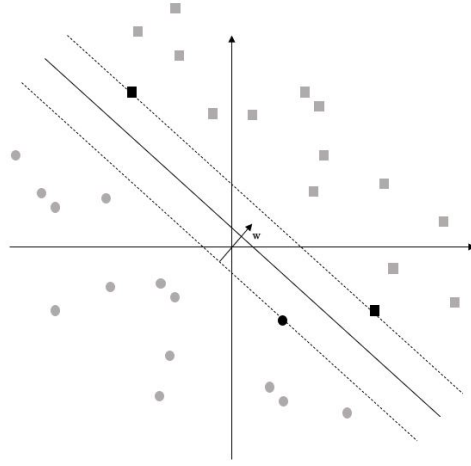


Figure 1.2: Hard margin SVM representation

C trades-off data fitting and size of the margin. Substituting derivatives in the Lagrangian as before

$$\frac{1}{2}\|\mathbf{w}\|^2 + C \sum_{i=1}^m \zeta_i = C \sum_{i=1}^m \xi_i + \frac{1}{2}\|\mathbf{w}\|^2 - \sum_{i=1}^m \alpha_i (y_i (\mathbf{w}^T \mathbf{x}_i + w_0) - 1 + \zeta_i) - \sum_{i=1}^m \beta_i \zeta_i \quad (1.30)$$

$$= \sum_{i=1}^m \xi_i \underbrace{(C - \alpha_i - \beta_i)}_{=0} + \frac{1}{2} \sum_{i,j=1}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j - \sum_{i,j=1}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j - \underbrace{\sum_{i=1}^m \alpha_i y_i w_0}_{=0} + \sum_{i=1}^m \alpha_i \quad (1.31)$$

$$= \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j = L(\boldsymbol{\alpha}) \quad (1.32)$$

obtaining finally

$$\max_{\boldsymbol{\alpha} \in R^m} \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \quad (1.33)$$

subjected to

$$\begin{aligned} 0 &\leq \alpha_i \leq C \quad i = 1, \dots, m \\ \sum_{i=1}^m \alpha_i y_i &= 0 \end{aligned}$$

With kernel trick used the dual optimization is rewritten as [33]

$$\max_{\boldsymbol{\alpha} \in R^m} \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m \alpha_i \alpha_j y_i y_j \underbrace{\Phi(\mathbf{x}_i)^T \Phi(\mathbf{x}_j)}_{k(\mathbf{x}_i, \mathbf{x}_j)} \quad (1.34)$$

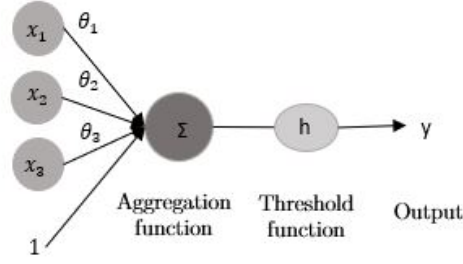


Figure 1.3: Perceptron of Rosenblatt representation; picture inspired by [107]

subject to

$$0 \leq \alpha_i \leq C \quad i = 1, \dots, m$$

$$\sum_{i=1}^m \alpha_i y_i = 0$$

The advantages of support vector machines include the fact that solution depends only on a small subset of training examples, called support vectors. Further, SVM could be easily extended to nonlinear separation via kernel trick, and finally SVM could be used with small sets of data [42]. However it should be noted there is not a clear-cut way to select best kernel for a task and design of optimal multi-class support vector machines is not an obvious task.

An important relationship between dense associative memories and kernel machines [127] should be underlined. Considering a dual feed-forward formulation of a dense associative memory network with an arbitrary number n_{out} output neurons. The network has a state at time point t is denoted $\mathbf{s}^{(t)} \in \{-1, 1\}^n$, and whose dynamic evolved according to the update rule

$$s_i^{(t+1)} = \text{sgn} \left[\mathbf{w}_i^\top \Phi \left(\mathbf{s}^{(t)} \right) - \theta_i \right] \quad (1.35)$$

where $\mathbf{w}_i \in R^n$ is the weight vector to neuron $i = 1, \dots, n$ (note that n in this case depends on dimensionality of the mapped output of Φ). In order to make the patterns $\{\xi^\mu\}_{\mu=1}^m$ attractors of the network dynamic, each neuron i is, again, independently trained on every pattern μ to produce the response ξ_i^μ when the rest of the network is initialized in ξ^μ . The problem of training the entire network can be transformed into the problem of training N separate classifiers according to $\min_{\mathbf{w}_i} \|\mathbf{w}_i\|_2$ such that $\xi_i^\mu (\mathbf{w}_i^\top \Phi(\xi^\mu) - \theta_i) \geq 1, \forall \mu, i$. Adding constraints prohibiting a neuron from having self-connections in previous formulation yields the following result (that could be called SVM formulation of dense associative memory)

$$s_i^{(t+1)} = \text{sgn} \left[\sum_{\mu} \alpha_i^\mu \xi_i^\mu k \left(\sum_{j \neq i} \xi_j^\mu s_j^{(t)} \right) - \theta_i \right] \quad (1.36)$$

(with the inner-product kernel $K(\mathbf{x}_i, \mathbf{x}_j) = k(\mathbf{x}_i^\top \mathbf{x}_j)$) and the state update of the classical dense associative formulation can be consequently identified as a special case of a sub-optimal recurrent SVM network in the kernel form, with $k = F'$, $\mathbf{A} = \mathbf{1}$, and $\boldsymbol{\theta} = \mathbf{0}$ (since $f = 0.5$). With a linear $F'(x) = x$, the MHN reduces to the classical Hopfield network making use of the linear kernel $k(\mathbf{x}_i^\top \mathbf{x}_j) = \mathbf{x}_i^\top \mathbf{x}_j$

Neural Network

Deep learning is based on concept of perceptron, a method to implement threshold logic [203]. An artificial neuron, in its perceptron implementation, is a non-linear parameterized function with restricted output range. The number of incoming edges satisfied unlimited fan in property, meaning there is not an upper bound on number inputs [203]. As it could be seen perceptron are formed by two

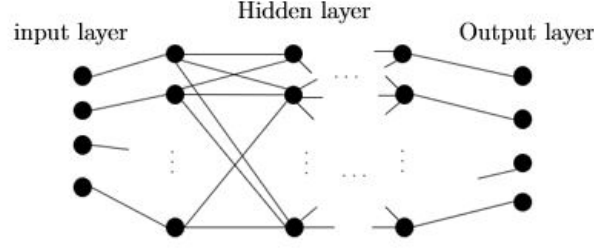


Figure 1.4: Representation of multi-layer neural network; picture inspired by [203]

core components: integration and activation functions. An integration function $\Psi : R^n \rightarrow R$, typically summation, reduces inputs to a single values. Weights are necessary to produce non-parallel separating hyper-planes and using non binary weights allows for focusing on learning this weights instead of only modifying topology to implement learning. Weights can be positive (eccitatory) or negative (relative inhibitory, note that inhibitory synapses are necessary to compute every logical functions). On the other hand an activation function $\Phi : R \rightarrow R$, produce an output taking a single value as argument

$$\hat{y} = h \left(\sum_i w_i x_i \right) = h(\mathbf{w}^T \mathbf{x}) \quad (1.37)$$

This definition is based on Roseblatt [206] which is conversely based on McCulloch Pitts Model [161]. Roseblatt approach (in respect to McCulloch approach) allows for learning bias and parameters and uses a sign function as activation function. Perceptron of Roseblatt was trained with perceptron learning rule (which is out of the scope of this work) that allows to learn weights binary classifying samples given input/output pairs. However it should be underlined that this rule supposes direct supervision over neuron not allowing to learn multiple layered architecture. This was shown to be a limitation when [159] shows that the Exclusive OR (XOR), cannot be solved by perceptron. More generally this limitation applies to every nonlinear separable function. Two sets of points A and B in an n -dimensional space are called linearly separable if $n + 1$ real numbers $w_1, \dots, w_{n+1}$ exist, such that every point $(x_1, x_2, \dots, x_n) \in A$ satisfies $\sum_{i=1}^n w_i x_i \geq w_{n+1}$ and every point $(x_1, x_2, \dots, x_n) \in B$ satisfies $\sum_{i=1}^n w_i x_i < w_{n+1}$.

More perceptrons can build a so called network architecture. A network architecture is a tuple (I, N, O, E) consisting in a set I of input sites, a set N of computing units, a set O of output sites and a set E of weighted directed edges. A directed edge is a tuple (u, v, t) whereby $u \in I \cup N, v \in N \cup O$ and $t \in R$. The input sites or input units are entry points for information into the network. Results are transmitted to the output sites. The computing units from which results are read off are called the output units. [203]. Minsky showed that a multi-layer perceptron, a particular network architecture (see feed-forward section), could in principle solve this problem. However no learning algorithm until back-propagation [159] allowed to train this multilayered architectures. This leads to deep neural network development, explored in following sections.

Even if nowadays continuous network (meaning network whose neurons produce continuous output) are more used, binary outputs units, as originally proposed and treated by McCulloch, Rosenblatt and Minsky, are still of interests because a reduced number of channel states limits effort in decoding outputs (e.g. to implement lightweight and memory limited neural network and seems also a consideration of interest for biological networks to efficient transport signals [203]).

As alternative to binary encoding, many models of neural networks use bipolar vectors. In a bipolar coding the value 0 is substituted by 1. This change does not affect the essential properties of the perceptrons, but changes the symmetry of the solution regions. Since the relative sizes of the solution regions in the weight space, represent how difficult it is to learn them, and since our learning algorithm will be asked to learn one of these functions randomly, the best strategy is to try to get regions of about the same relative size. Bipolar coding is better in n -dimensional space and for a large

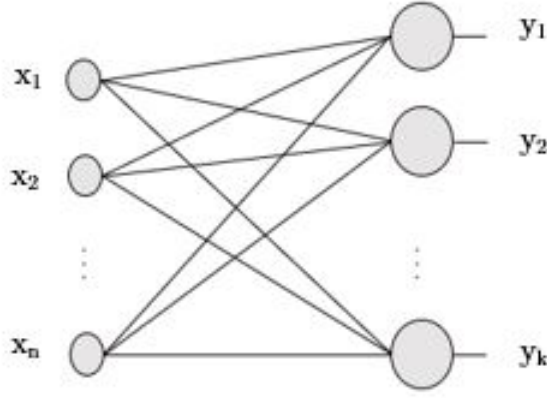


Figure 1.5: An example of hetero-associative networks without feed-backs; picture inspired by [203].

n , since two randomly selected vectors with coordinates 1 or -1 are orthogonal with a probability near 1 because each component is 1 or -1 with probability $1/2$. The expected value of the scalar product is small compared to the length of the two vectors, which is $\sqrt{n}$. Since the separating planes in weight space are defined by these vectors, they also tend to be mutually orthogonal when bipolar coding is used. On the other hand, the expected value of the scalar product of binary vectors is $n/4$, which is not negligible when compared to $\sqrt{n}$, even for large n .

In the following paragraph a binary/bipolar single layer recurrent neural network, the Hopfield model, is analyzed as long as its extension, dense associative memories.

Associative memory is defined as the ability to learn and remember the relationships between unrelated items [241]. For an artificial neural network, associative memory is the ability to retrieve the network state out of P stored network states which is closest to the input pattern in terms of Hamming distance [202]¹.

Associative networks are neural networks implementing associative memory. The goal of associative networks is to associate known input vectors with given output vectors. Contrary to networks typically called continuous mapping networks, in associative memories, neighborhood of a input vector $\mathbf{x}$, representing a memories, should be mapped to $\mathbf{y}$, the target memory, and not to a neighborhood of $\mathbf{y}$ itself.

Here some important aspects must be underlined. As said associative memories are a kind of neural networks and they share with other known neural architectures, like feed forward neural networks, a common history in their quantum implementation development as it is underlined thereafter. Further, main interest in associative networks in eighties was related to the neural plausibility associated with their locality, meaning the output is influenced only by the input, and with Hebbian learning rule, the main learning rule used with associative network [203], was employed mainly for understand brain function (e.g. [119]). For present purpose even if this property is noteworthy, other advantages are of relevance for present work.

Taxonomy of associative memory define three types of associative memories [203]. Hetero-associative networks, map m input vectors $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m$ in n -dimensional space to m output vectors $\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_m$ in k -dimensional space so that $\mathbf{x}_i \mapsto \mathbf{y}_i$. Auto-associative networks conversely associate each vector with itself i.e. $\mathbf{x}_i = \mathbf{y}_i$. Finally in Pattern recognition networks, vector $\mathbf{x}_1$ is associated with the scalar $\mathbf{i}$, so giving a label to an input pattern.

Figure 1.5 shows a typical hetero-associative networks without feed-backs which is useful to define some basic concepts. Let w_{ij} be the weight between input site i and unit j , w_{ij} be the weight between input site i and unit j and $\mathbf{W}$ denote the $n \times k$ weight matrix $[w_{ij}]$. The row vector $\mathbf{x} = (x_1, x_2, \dots, x_n)$ produces the excitation vector $\mathbf{e}$ through the computation $\mathbf{e} = \mathbf{x}\mathbf{W}$. The activation function is computed next for each unit. If it is the identity, the units are just linear associators and the output $\mathbf{y}$ is just $\mathbf{x}\mathbf{W}$. In general, m different n -dimensional row vectors $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m$ have to be associated with m k -dimensional row vectors $\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_m$

¹This definition is not generally shared but is useful for topic comprehension

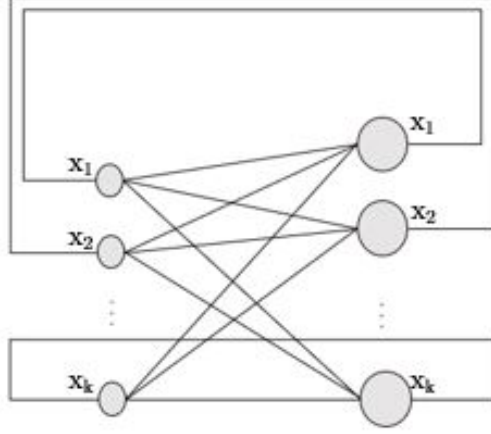


Figure 1.6: Auto-associative network with feedback; picture inspired by [203]

This work focuses on content addressable memory (CAM) category, a form of auto-associative memories that recall information by value [122]. In the case of auto associative network with feedback a $n \times n$ matrix $\mathbf{W}$ has at most n linear independent eigenvectors and n eigenvalues. The eigenvectors $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n$ satisfy the set of equations $\mathbf{x}_i \mathbf{W} = \lambda_i \mathbf{x}_i$ for $i = 1 \dots n$ where $\lambda_1, \dots, \lambda_n$ are the matrix eigenvalues. Each weight matrix with a full set of eigenvectors defines a kind of "eigenvector automaton". Assuming $|\lambda_1| > |\lambda_i|$ for $i \neq 1$. Let $\lambda_1 > 0$ and pick randomly a non-zero n -dimensional vector $\mathbf{a}_0$. This vector can be expressed as a linear combination of the n eigenvectors of the matrix $\mathbf{W}$:

$$\mathbf{a}_0 = \alpha_1 \mathbf{x}_1 + \alpha_2 \mathbf{x}_2 + \dots + \alpha_n \mathbf{x}_n \quad (1.38)$$

Assume that all constants α are non-zero. After the first iteration with the weight matrix $\mathbf{W}$, the following result is obtained

$$\mathbf{a}_1 = \mathbf{a}_0 \mathbf{W} = (\alpha_1 \mathbf{x}_1 + \alpha_2 \mathbf{x}_2 + \dots + \alpha_n \mathbf{x}_n) \mathbf{W} = \alpha_1 \lambda_1 \mathbf{x}_1 + \alpha_2 \lambda_2 \mathbf{x}_2 + \dots + \alpha_n \lambda_n \mathbf{x}_n \quad (1.39)$$

After t iterations the result is

$$\mathbf{a}_t = \alpha_1 \lambda_1^t \mathbf{x}_1 + \alpha_2 \lambda_2^t \mathbf{x}_2 + \dots + \alpha_n \lambda_n^t \mathbf{x}_n \quad (1.40)$$

It is obvious that the eigenvalue λ_1 with the largest magnitude, dominates this expression for a big enough t . The vector $\mathbf{a}_t$ can be brought arbitrarily close to the eigenvector $\mathbf{x}_1$ (with respect to the direction and without considering the relative lengths). In each iteration of the associative network, the vector $\mathbf{x}_1$ attracts any other vector $\mathbf{a}_0$ whose component α_1 is non-zero and it is called an attractor. Associative networks are considered as dynamical systems, whose attractors are those vectors that should be stored. In the case of the linear eigenvector automaton, one vector absorbs almost the whole of input space. Avoiding this requires introduction of non-linearities in the activation of the network units so that the dynamical system becomes nonlinear. Typically auto-associative memories use a bipolar coding and so use step function as non-linearity. The choice of bipolar vectors is due to a greater probability of being orthogonal than binary vectors (as said before) and that reduces cross-talk term of Hebbian learning rule, the preferred learning rule used in this type of network.

In Hebbian learning, two neurons which are simultaneously active develop a degree of interaction higher than those neurons whose activities are uncorrelated. In the latter case the interaction between the elements is very low or zero [112]. Hebbian learning is applied, updating the weight w_{ij} by Δw_{ij} . This increment measures the correlation between the input x_i at site i and the output y_j of unit j . During learning, both input and output are clamped to the network and the weight update is given

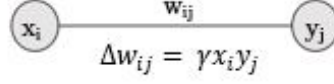


Figure 1.7: Hebbian learning rule example; picture inspired by [203]

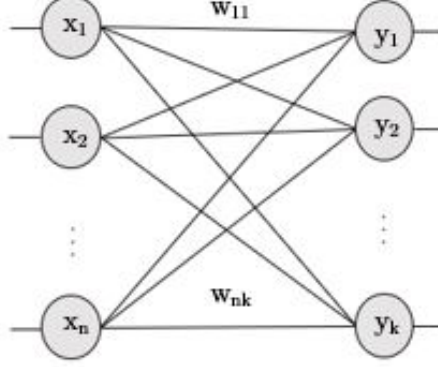


Figure 1.8: Example of a resonance network; picture inspired by [203]

by

$$\Delta w_{ij} = \gamma x_i x_j. \quad (1.41)$$

The factor γ is a learning constant in this equation. The weight matrix $\mathbf{W}$ is set to zero before Hebbian learning is started. In general, if we want to associate m n -dimensional non-zero vectors $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m$ with m k -dimensional vectors $\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_m$, Hebbian learning is applied to each input-output pair and the resulting weight matrix is $\mathbf{W}^\ell = \left[x_i^\ell y_j^\ell \right]_{n \times k}$. If the input to the network is the vector $\mathbf{x}^p$, the vector of unit excitation is

$$\mathbf{x}^p \mathbf{W} = \mathbf{x}^p (\mathbf{W}^1 + \mathbf{W}^2 + \dots + \mathbf{W}^m) = \mathbf{x}^p \mathbf{W}^p + \sum_{\ell \neq p}^m \mathbf{x}^p \mathbf{W}^\ell = \mathbf{y}^p (\mathbf{x}^p \cdot \mathbf{x}^p) + \sum_{\ell \neq p}^m \mathbf{y}^\ell (\mathbf{x}^\ell \cdot \mathbf{x}^p) \quad (1.42)$$

Having $\mathbf{x}^p \cdot \mathbf{x}^p > 0$ then $\text{sgn}(\mathbf{y}^p (\mathbf{x}^p \cdot \mathbf{x}^p)) = \mathbf{y}^p$ and the contribution on the right is called cross-talk. Cross-talk become 0 only when the patterns are mutually orthogonal.

However, Hebbian learning is not always able to reduce efficiently cross-talk term. if cross-talk becomes too large spurious states, meaning not stored pattern, tends to appear and some memorized states could also be lost. One should then resort to other learning rules. It's necessary to explicit an upper bound on the probability this could appear and that lead to limits to the number of patterns m that can be stored safely. Given an auto associative network with an $n \times n$ weight matrix the maximum capacity is $m \approx 0.18n$, which is the upper bound probability for one bit failure set to 0.01 on a Gaussian that represent the m and n selection as random [114].

Bidirectional associative memories(BAM), also called resonance networks, are a type of associative memories in which two layers are connected by bidirectional edges [144]. Bidirectional associative memories lack a form of global synchronization, being therefore even more biologically/neurally plausible, and as we will see in next sections more convenient for annealing computation. The output is achieved by requiring that all computing elements evaluate their inputs and compute their output simultaneously [203].

Output of the network can be evaluated via a synchronous or asynchronous dynamic. The first approach mentioned supposes every neuron fires simultaneously. In this case the excitation of the output units is computed using vector-matrix multiplication (input vector, weight matrix) and evaluating

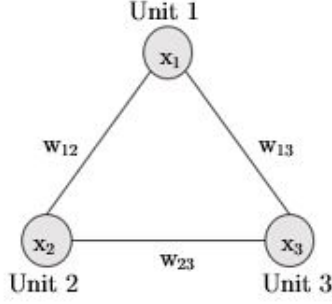


Figure 1.9: Hopfield network with three units; picture inspired by [203]

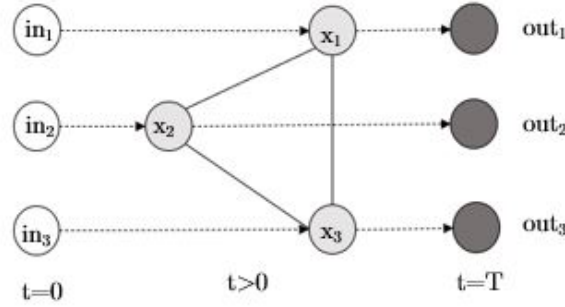


Figure 1.10: Schematic evolution of an Hopfield model of 5 neurons. The neurons are initialized (white nodes) then the networks (light gray nodes) evolves to the final configuration (dark gray nodes); picture inspired by [165]

the sign function at each node. The output of this operation is then the input to another iteration until reaching a stable state [144]. In asynchronous dynamic instead, each unit computes its excitation at random time and changes its state to 1 or -1 independently of other neurons and according to the sign of its total excitation. The probability of two units firing simultaneously is zero [203]. These two approaches have different properties. In the following sections only the second one will be considered. This is because asynchronous dynamic is more widely adopted in literature and as a consequence most learning rules and improvements are defined only in terms of asynchronous dynamic. Furthermore, it could be described via Markov Dynamic like quantum annealing making easier parallelism between them and finally it allows to deduce some interesting properties, which are worth to be quoted : an energy function connected with bi-directional memory could be extracted (see later) and it is proved that an Hopfield network with asynchronous dynamic eventually reaches a stable state at a local minimum of the energy function (see the following section).²

An Hopfield neural network (HNN) is a specific type of bidirectional associative memory of N interconnected neurons $s_1, s_2, \dots, s_n$ each of them is (typically) a bipolar threshold unit:

$$s_i = \text{sign} \left(\mathbf{w}_i^\top \mathbf{s} - \theta_i \right) \quad (1.43)$$

where the vector $\mathbf{s} \in \{-1, +1\}^n$ denotes the global state of the whole network. If the $n \times n$ weight matrix $\mathbf{W}$ of a Hopfield network is symmetric and hollow, i.e. has a diagonal of all 0s, (trivially not having a 0 diagonal cause network to oscillate between states [203]), and if the network neurons update asynchronously, then the energy at time t is defined as

$$E(t) = -\frac{1}{2} \mathbf{s}^{(t)\top} \mathbf{W} \mathbf{s}^{(t)} + \boldsymbol{\theta}^\top \mathbf{s}^{(t)} \quad (1.44)$$

²for brevity in Appendix B.1 is reported fully exposure of typical asynchronous dynamic

and can never increase [122], being a monotonically decreasing and a Lyapunov function. Given finitely many distinct states, trivially from all possible combinations of polar values for each neuron, the network eventually reaches a local energy minimum, called attractor, after finitely many asynchronous updates [122] [13].

Note also that Hopfield network could be defined as fully connected McCulloch-Pitts network in which the synaptic weights are symmetric quantities chosen according Hebb rule [50].

Hopfield models was also studied for its isomorphism with Ising model in a zero temperature setting, while in case of variable temperatures Boltzman machines, not treated here, are the reference models [203]. An Ising Hamiltonian is the one also minimized in quantum annealer, and it's exposed in detail in the next section.

Both Hopfield and Ising models (at zero temperature) dynamics could be represented by a first order Markov model. More formally both models have an associated probability measure, the Gibbs measure, that has the Markov property. A Gibbs measure extends canonical ensemble to infinite systems, in which canonical ensemble measures the probability of a system to be in a specific state[153]. In probability theory and statistics, the term Markov property refers to the memory-less property of a stochastic process. A stochastic process has the Markov property if the conditional probability distribution of future states of the process, conditional on both past and present values, depends only upon the present state. In other terms, given the present, the future does not depend on the past. A process with this property is said to be Markovian or a Markov process [83] ³

As underlined before, performances of content associative memories varies due to different learning rules used to compute related weight matrices.

Hebbian learning rule mentioned before define the following weight matrix $\mathbf{W}$:

$$\mathbf{W} = \sum_i^m \mathbf{x}_i \mathbf{x}_i^T \quad (1.45)$$

The capacity of retrieval of patterns free of errors is $C \approx \frac{n}{2 \log n}$ where n is the dimension of the input. The storage capacity for retrieval of patterns with a small percentage of errors is $C \approx 0.14n$. Equivalently, given that capacity is defined as number of patten stored over number of neuron, if it is lower than 0.138 there is perfect recall while if capacity is above 0.138 it appears a phase transition to spin glass phase that impairs recall capability [167][162]. Hebb rule is incremental as it is a sum over individual patterns and local since the synaptic weights depend only on the value of the adjacent neurons. ⁴

As a final note (the importance will be made apparent in next sections) a parallelism between Hopfield model and other common used machine learning methods is underlined: It is possible to offer a proof that Hopfield networks are equivalent to the nearest-neighbor algorithm (a predecessor of kernel machines) [156][84] and it is possible to transform a learning problem in a Hopfield network with n units into a learning problem for a perceptron of dimension $n(n+1)/2$. Consequently, we could consider Hopfield network a predecessor of deep neural network [84]. It is known that a perceptron is not capable to face all possible problems (e.g. XOR problems) and that leads directly to the next section. Both considerations together show hint of a bound between neural networks and kernel machines that will be exploited in chapter 4 and analyzed in next sections.

Krotov and Hopfield introduced the energy function:

$$E = - \sum_{\mu=1}^m F(\mathbf{s}^T \boldsymbol{\xi}^\mu) \quad (1.46)$$

where F is a nonlinear interaction function (named activation) and m is again the number of stored patterns. In present formula F(x) could be any smooth function. One can derive an iterative synchronous update rule that leads to decrease of the energy. Here asynchronous updates is considered

³for brevity include example of markovian dynamic in appendix B.1

⁴more informations regarding learning rule is in Appendix B.1

X	Y	Z
-1	-1	-1
-1	1	1
1	-1	1
1	1	-1

Figure 1.11: Table containing input values (X, Y) and corresponding output (Z) from [147]

via flipping one unit at a time. The update rule is then

$$s_i^{(t+1)} = \text{Sign} \left[\sum_{\mu=1}^m \left(F \left(\xi_i^\mu + \sum_{j \neq i} \xi_j^\mu s_j^{(t)} \right) - F \left(-\xi_i^\mu + \sum_{j \neq i} \xi_j^\mu s_j^{(t)} \right) \right) \right] \quad (1.47)$$

The rule updates a unit in such a way that the energy of entire configuration of the network never decrease [147] (more detail of dynamic can be found in [34]).

This new type of CAM has the following intriguing properties: In the generalized Hopfield model with interaction function $F(x) = x^p$ one can store up to $m = \alpha_p n^{(p-1)}$ patterns, if small errors in the retrieval are tolerated [102]. Note that here x^p is intended as a rectified version of the corresponding polynomial interaction function, meaning $F(x) = 0$ if x is less or equal 0. In the same model, one can store $m = \frac{n^{(p-1)}}{c_p \log n}$ patterns for $c_p > 2(2p-3)!!$, if one wants a fixed pattern to be a fixed point of the dynamics T introduced in the precedent point with a probability converging to 1 [80].

It seems that in contrast to classical Hopfield Networks, Dense Associative Memories do not have a weight matrix. Instead, the energy function is the sum of a function of the dot product of every stored pattern with the state pattern [199]. This is also underlined in the example provided in [147]. The problem is the following: given two inputs x and y produce an output z such that the truth table in figure is satisfied. Treating this task as an associative memory problem, the four examples could be embedded as input-output triplets x, y, z in the memory. Resulting network has $n = 3$ identical units, two used for the inputs and one for the output, and $m = 4$ memories which are the four lines of the truth table. Thus, the energy for a dense associative memory is equal to

$$E_p(x, y, z) = -(-x - y - z)^p - (-x + y + z)^p - (x - y + z)^p - (x + y - z)^p \quad (1.48)$$

and if evaluated on the corners of the cube $x, y, z = 1$, it reduces to

$$E_p(x, y, z) = \begin{cases} 0, & p = 1 \\ C_p, & p = 2, 4, 6, \dots \\ C_p xyz, & p = 3, 5, 7, \dots \end{cases} \quad (1.49)$$

where coefficients C_p denote numerical constants. The network clearly cannot solve this problem for $p = 1$ and $p = 2$, since the energy does not depend on the spin configuration. For an odd $p \geq 3$ the network provide a solution. Given two inputs, x and y , one can choose the output z that minimizes the energy. This leads to the update rule

$$z = \text{Sign} [E_p(x, y, -1) - E_p(x, y, +1)] = \text{Sign}[-xy] \quad (1.50)$$

The capability to deal with the XOR task suggests the capability of this network to approximate a three-layered (shallow) neural network, so able to deal with non-linearly separable problems. In fact in [147] authors show duality property, stating the possibility to interpret dense associative memory/new Hopfield as three-layered feed forward neural networks, where derivative of dense associative memory/new Hopfield activation become feed-forward activation. Defining the network, represented

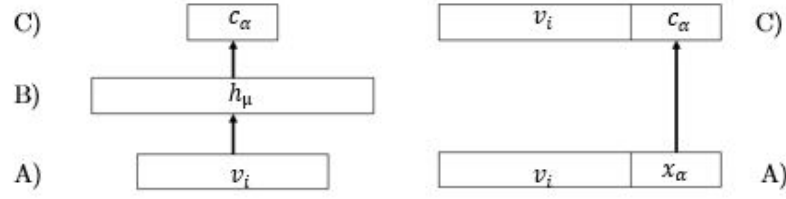


Figure 1.12: Duality displayed. On the right a dense associative memory / new Hopfield network, on the left the dual feed-forward neural network that approximates; picture inspired by [147]

in situation A on the right of the image, having x_a neurons used for classification, set to -1 initially, and that can be flipped $+1$, v_i neurons clamped treated as a constant to input. Note that after applying g as below x_α become c_α that is non-binary.

This network evolves, toward B, with following updates over neurons asynchronously ⁵

Elements to be learned are ξ_α^μ , meaning the connection between hidden and output layer, so number of memories defines number of hidden layers neurons. If the sign function is changed toward a differentiable function, gradient descent could be used to obtain best weights, meaning memories, to classification neuron. Authors in [147] provide experiments of classical three layered neural network trained via gradient descent in this way. In particular using as error measure

$$C = \sum_{\alpha=1}^m \sum_{k=1}^k (c_\alpha - t_\alpha)^{2p}, \quad (1.51)$$

Where k is number of classes, m is number data, t_α is set to -1 for wrong classes, $+1$ true class, so classification is done in a one vs all fashion, in which unit with greater c_α is considered the proposed class.

Given duality, the activation function from the visible layer to the hidden layer is equal to the derivative of the energy function $f(x) = F'(x)$. Authors provided this dense associative memory/new Hopfield activations

- $F(x) = \log(\cosh(x))$, approximating $\tanh$ [147]
- $F(x) = \log(1 + \exp(x))$, approximating logistic [147]
- Rectified polynomial

$$F(x) = \begin{cases} x^p, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

having second order approximating ReLU function [147]

other non-linearities considered in this work were proposed in the following years with other notable works as it's exposed thereafter.

One of them, by [80], extended the energy function of original dense associative memories/new hopfield by using an exponential interaction function :

$$E = - \sum_{i=1}^m \exp(\mathbf{x}_i^T \boldsymbol{\xi}) \quad (1.52)$$

where m is again the number of stored patterns. This energy function leads to an exponential storage capacity $C \cong 2^{\frac{n}{2}}$, approximating by duality property cross entropy function.

In a following work, author of [147], faced the problem of neural plausibility of this network, that as said before was one of the more important reason that cause diffusion of original Hopfield model. Dense associative memories do not have binary interaction as in classical Hopfield/Ising model.

⁵The author do not explicitly show a demonstration of this duality so a demonstration is provided in Appendix B.1

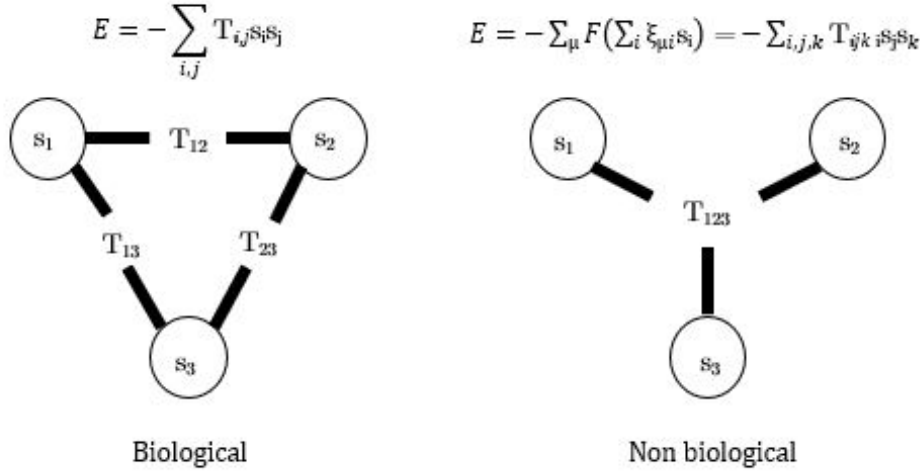


Figure 1.13: A depiction of multiple interactions; picture inspired by [148]

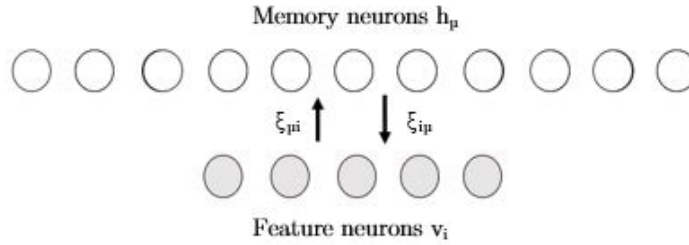


Figure 1.14: A depiction of two body synapses dense associative network of [148]; picture inspired by [148]

Consider two binary networks as in the image 1.13 which depicts a three bipolar neurons $\sigma_1, \sigma_2, \sigma_3$ case. The classical Hopfield network (In image 1.13 on the left) have a weight matrix $T_{ij} = \sum_{\mu} \sigma_{\mu i} \xi_{\mu j}$ being the outer product of memory vectors. In this case the matrix T_{ij} is interpreted as a matrix of synaptic connections between cells i and j . Considering also a Dense Associative Memory network (image 1.13 on the right) with cubic interaction term $F(x) = x^3$. In this case the corresponding tensor $T_{ijk} = \sum_{\mu} \xi_{\mu i} \xi_{\mu j} \xi_{\mu k}$ has three indices, thus presenting more than binary interactions. The same authors of [148] proposed this possibility to recover two body synapses, or as they call neural plausibility, via the network depicted below

They coupled an additional set of n_h "complex neurons", or "memory neurons", or hidden neurons to the n_f feature neurons. The dynamical system in $n_f + n_h$ variables, as defined below, has an energy function describing its dynamics whose minima, stable points of this dynamics are at the same locations in the $n_f - dimensional$ feature subspace as the minima in the corresponding Dense Associative Memory system. Importantly, the resulting spikes of action potentials in a feature neuron produce input "currents" into a memory neuron returning to baseline with a time constant . Note that time constants for the two groups of neurons are denoted by τ_f and τ_h . The "currents" of the feature neurons are denoted by v_i , and the "currents" of the complex memory neurons are denoted by h_{μ} . A matrix $\xi_{\mu i}$ denotes the strength of synapses from a feature neuron i to the memory neuron . The synapses are assumed to be symmetric, meaning $\xi_{i\mu} = \xi_{\mu i}$. The outputs of the memory neurons and the feature neurons are denoted by f_{μ} and g_i , which are non-linear functions of the corresponding currents. The dynamic of the two layers is then

$$\begin{cases} \tau_f \frac{dv_i}{dt} = \sum_{\mu=1}^{n_h} \xi_{i\mu} f_{\mu} - v_i + i \\ \tau_h \frac{dh_{\mu}}{dt} = \sum_{i=1}^{n_f} \xi_{\mu i} g_i - h_{\mu} \end{cases} \quad (1.53)$$

where I_i denotes the input "current" into the feature neurons. The energy function for the network can be written as

$$E(t) = \left[\sum_{i=1}^{n_f} (v_i - I_i) g_i - L_v \right] + \left[\sum_{\mu=1}^{n_h} h_\mu f_\mu - L_h \right] - \sum_{\mu,i} f_\mu \xi_{\mu i} g_i \quad (1.54)$$

Here authors introduced two Lagrangian functions $L_v(\{v_i\})$ and $L_h(\{h_\mu\})$ for the feature and the hidden neurons. They are defined through the following equations, so that derivatives of the Lagrangian functions correspond to the outputs of neurons

$$f_\mu = \frac{\partial L_h}{\partial h_\mu} \quad (1.55)$$

$$g_i = \frac{\partial L_v}{\partial v_i} \quad (1.56)$$

Note that in this formulation energy monotonically decreases on the dynamical trajectory and by using bounded activation function the energy is bounded from below. Note also that the first term depends only on the feature neurons, the second term depends only on the hidden neurons, and the third term is the "interaction" term between the two groups of neurons. This last aspect seems to solve the problem of two body synapses for QUBO formulation in annealer but thereafter it will be shown how it is not the case. Another problem to be noted is that Hopfield and Dense Associative Memories retrieve a state initializing neurons to the desired state to be retrieved. This is not possible in the classical quantum annealer setting, as it will be apparent in following section. However this problem could be circumvented simply subtracting to the energy function a term given by the product of each neuron of the memory we want to retrieve and an appropriate bias as done in [218] [177]. Finally to recover the energy for the case of an exponential non-linearity they stated

$$L_h = \sum_{\mu} F(h_\mu) \quad (1.57)$$

$$L_v = \sum_i G(v_i) \quad (1.58)$$

With G being sign function and F being exponential or polynomial function. Here a determinant assumption shows up: the "steady state condition" meaning that visible and hidden neurons do their calculation with a negligible time difference. This allow to obtain from dynamic equations previously state, supposing at least hidden neurons can assume continuous values, obtaining:

$$h_\mu = \sum_{i=1}^{n_f} \xi_{\mu i} v_i \quad (1.59)$$

authors of [148] also propose a new non-linearity $F(x) = G\left(\sum_i \xi_{\mu i} \frac{v_i}{\sqrt{\sum_j v_j^2}}\right)$ where G is a nonlinear function of choice. The importance of Dense Associative Memories comprehends also actual neural networks trending topic like self-attention. This success is due to the importance of Transformer [235] model which represents the state of the art in natural language understanding and computer vision community for a lot of different tasks. The transformer model has as its core the attention mechanism. [199] shows the possibility to represent attention via Dense Associative Memories model. As said before, [80] extended the energy function of original dense associative memories/new Hopfield by using an exponential interaction function:

$$E = - \sum_{\mu=1}^m \exp(\mathbf{x}^T \boldsymbol{\xi}) \quad (1.60)$$

where m is again the number of stored patterns. Eliminating additivity, the energy can also be written

as:

$$E = -\exp(\text{lse}(1, \mathbf{X}^T \xi)) \quad (1.61)$$

where $\mathbf{X} = (x_1, \dots, x_m)$ is the data matrix (matrix of stored patterns). The log-sum-exp function (LSE) is defined as:

$$\text{lse}(\beta, z) = \beta^{-1} \log \left(\sum_{l=1}^m \exp(\beta z_l) \right) \quad (1.62)$$

this energy function works on continuous patterns [80] The final energy function for a dense associative memory attention is the following

$$E = -\text{lse}(\beta, \mathbf{X}^T \xi) + \frac{1}{2} \xi^T \xi + \beta^{-1} \log m + \frac{1}{2} M^2 \quad (1.63)$$

where M is the largest norm of all stored pattern, N is the number of stored memories and $\frac{1}{2} \xi^T \xi$ is used to bound energy below given negative sign of LSE function

The performance of machine learning algorithms, and lot of other IT algorithms, typically depends heavily on the representation of the data. In classical machine learning a lot of time should expended to find best features for a task, given it is typically not trivial what features should extracted. A natural way to deal with this task is to use machine learning also to learn best data representation, which is typically called representation learning. Representation learning goal is the capability to disentangle factors of variation that explain observed data. Factors of variation are separate sources of influence, typically not directly observed and present in large numbers for every piece of data. Neural networks solve representation learning introducing representation that are expressed in term of other simpler representation via a specific layered architecture [107]

Layered architectures are those in which the set of computing units N is subdivided into ℓ subsets $N_1, N_2, \dots, N_\ell$ in such a way that only connections from units in N_1 go to units in N_2 , from units in N_2 to units in N_3 , etc. The input sites are only connected to the units in the subset N_1 , and the units in the subset N_ℓ are the only ones connected to the output sites. In the usual terminology, the units in N_ℓ are the output units of the network. The subsets N_i are called the layers of the network. The set of input sites is called the input layer, the set of output units is called the output layer [203]. Deep learning layer are so divided in [107] visible layers, meaning input and output layers and hidden layers, those with no direct connections from or to the outside. Usually the units in a layer are not connected to each other. It is also customary to define width, number of units in a layer and depth, number of layers in a network [107]. More than two hidden layer depth networks are called deep neural networks.

[52] shows 2-layer net (shallow networks) with linear output can approximate any continuous function over compact domain to arbitrary accuracy (given enough hidden units). So it could be questioned why is useful to employ deep neural network. In fact, more hidden units increase capacity of the network meaning number of possible solutions implemented by the network. Every units separates the input space in two half-space. Two layer define convex cluster of points in inputs space (if number of hidden units is of dimension n with an input space of dimension $n+1$) while three layer define any union of convex polytope [203].

After initial successes (Recurrent Long-Short Term Memory Networks to treat sequence data [120], [155] solve OCR problem) neural networks went temporarily out of fashion given lack of data and other engineering problems (vanishing gradients, over-fitting) at least until 2006 when greedy layer-wise pre-training of Hinton allowed to pre-train neural networks (deep belief net, [117]). Finally neural networks reach apex of popularity with Alexnet [146] given capability to offer a natural way to handle structured inputs and have obtained remarkable results in multiple tasks [107] (e.g. object recognition [145] [128], image captioning, semantic segmentation, speech recognition [118], machine translation [225], [10], geometric deep learning, style transfer, image generation, poetry generation, autonomous driving).

Most modern neural networks are trained (because of reasons of statistical efficiency and consis-

⁶some architectural archetypes have arisen, during neural networks history, which are considered in this work. If reader is not aware of them, they are reported in appendix B.1

Batch gradient descent at iteration k
Require: Learning rate ε_k Require: Initial Parameter 1. While stopping criteria not met do 2. Compute gradient estimate over N examples 3. $\mathbf{g} = +\frac{1}{m} \nabla_{\mathbf{w}} \sum L(f(\mathbf{x}_i, \mathbf{y}_i))$ 4. Apply Update $\boldsymbol{\theta} = \boldsymbol{\theta} - \varepsilon \mathbf{g}$ 5. End while

Figure 1.15: Pseudo-code of batch stochastic gradient descent; picture inspired by [107]

tency, see [107]) using maximum likelihood principle: Consider a set of m examples $X = \{\mathbf{x}_1, \dots, \mathbf{x}_m\}$ drawn independently from the true but unknown data-generating distribution $p_{\text{data}}(\mathbf{x})$. Let $p_{\text{model}}(\mathbf{x}; \mathbf{w})$ be a parametric family of probability distributions over the same space indexed by $\mathbf{w}$. $p_{\text{model}}(\mathbf{x}; \mathbf{w})$ maps any configuration $\mathbf{x}$ to a real number estimating the true probability $p_{\text{data}}(\mathbf{x})$. The maximum likelihood estimator for $\mathbf{w}$ is then defined as

$$\mathbf{w}_{\text{ML}} = \arg \max_{\mathbf{w}} p_{\text{model}}(X; \mathbf{w}) = \arg \max_{\mathbf{w}} \prod_{i=1}^m p_{\text{model}}(\mathbf{x}_i; \mathbf{w}) \quad (1.64)$$

This product over many probabilities can be inconvenient for various reasons (e.g. numerical underflow). To obtain a more convenient but equivalent optimization problem, logarithm of the likelihood is used:

$$\mathbf{w}_{\text{ML}} = \arg \max_{\mathbf{w}} \sum_{i=1}^m \log p_{\text{model}}(\mathbf{x}_i; \mathbf{w}) \quad (1.65)$$

Dividing by m a version of the criterion that is expressed as an expectation with respect to the empirical distribution $\hat{p}$ defined by the training data is obtained:

$$\mathbf{w}_{\text{ML}} = \arg \max_{\mathbf{w}} E_{\mathbf{x} \sim \hat{p}_{\text{data}}} \log p_{\text{model}}(\mathbf{x}; \mathbf{w}) \quad (1.66)$$

maximum likelihood estimator can be generalized to estimate a conditional probability $P(\mathbf{y} | \mathbf{x}; \mathbf{w})$ in order to predict $\mathbf{y}$ given $\mathbf{x}$ as in supervised learning. If the examples are assumed to be i.i.d., then this can be decomposed into

$$\mathbf{w}_{\text{ML}} = \arg \max_{\mathbf{w}} \sum_{i=1}^m \log P(\mathbf{y}_i | \mathbf{x}_i; \mathbf{w}) \quad (1.67)$$

The cost function employed with maximum likelihood trained network is simply the negative log-likelihood, equivalently described as the cross-entropy (using cross-entropy was discovered in 1980 to reduce saturation and slow learning) between the training data and the model distribution. This cost function is given by

$$L(\mathbf{w}) = -E_{\mathbf{x}, \mathbf{y} \sim \hat{p}_{\text{data}}} \log p_{\text{model}}(\mathbf{y} | \mathbf{x}) \quad (1.68)$$

The specific form of the cost function changes from model to model, depending on the specific form of $\log p_{\text{model}}$.

Optimize weights of a deep Neural networks in maximum likelihood setting, meaning via loss minimization (which is done via backpropagation algorithm ⁷), shows to be a non-convex optimization problem, meaning a problem where the objective or any of the constraints are non-convex. One of the reason for this rely in non-linearities used to compute neural network outputs, which are typically not convex in parameter: trivially there always possible permutation of parameters inside a neural networks

⁷a brief digression and explanation of backpropagation in appendix B.1

which produce same result, this is called identifiability problem [107] (see later). The algorithm of choice in non-convex optimization of neural network is gradient descent and its variant, in particular mini-batch gradient descent, a type of first order optimization algorithms, meaning algorithms using only first derivative information of the objective function to achieve minimization (see image 1.15 for an explanation). There are several problems that could arise applying gradient descent to loss minimization. In particular deep neural networks shows local minima problem. Some minima arise from model identifiability problem. A model is said to be identifiable if a sufficiently large training set can rule out all but one setting of the model parameters. Models with latent variables like neural network are not identifiable and show weight space symmetry, equivalent network obtained swapping weight vectors incoming to neuron in same layer, and other form of symmetries. The local minima are equivalent in cost function values and are not a problem. Other local minima instead arise from non-convexity and could be problematic if they have higher value of cost function in respect of global minima. Saddle point, another criticality in optimizations, are points where Hessian matrix has positive and negative eigenvalues. Saddle points tend to be common in high dimensional spaces and in particular in neural networks [75], however saddle points seem empirically to be more problematic for second order models that search for critical points [108]. Finally in optimization there could be wide flat regions of constant value, degenerate locations, in which gradient and hessian is zero, while even if rare, pose a major problem for optimization algorithms. Cliff structure of gradients are instead solved via gradient clipping and typical only of recurrent structures. Local minima seem to be the most difficult elements in gradient descent but in recent history they were redimensioned because of pretty good local minima obtainable with mini-batch gradient descent. However they still seem to be a major concern. [108] shows that much of run-time of training is due to length of trajectory to arrive solution, spending most of its time tracing out a wide arc around mountain shaped function. It would be desirable having algorithm capable of direct "jump" this mountain. Other criticalities include initialization, which seems to play a major role [169], and gradient descent slowness in flat region of errors. To escape last problem larger iteration steps are needed but note that this is precluded by length of gradient which is too small in these problematic regions [203]. Finally unexpected results can happen because of many functions composed together are changed simultaneously using updates computed on other function remain constant, also second order optimization algorithm ignore higher than two interactions. Batch-norm solve partially this problem normalizing first and second order statistics of each layer. This reduce effect of lower layers but do not account for nonlinear statistics and relationship between layers. There are also corner cases in which also lower layers without nonlinear effects or non-linearity can have an effect [129]. It would be useful to have an optimization algorithm capable of considering these type of interactions.

Most neural network units output smoothly increase their output values and this make optimization via local search feasible [107]. This has suggested some researchers to employ metropolis family of algorithm to optimize neural networks. Methods like simulated annealing are also strictly related to continuation methods. Continuation methods are a family of strategies that can make optimization easier by choosing initial points to ensure that local optimization spends most of its time in well-behaved regions of space. Methods belonging to continuation methods like curriculum learning are shown to be very effective [18]. One possibility is then optimize all cost function with annealing. This method has serious concerns: it has not guarantees to find local minima and have to deal with continuous space defining then discrete step [88] [138] [17] [219] [94]. An alternative is then to interplay gradient descent with annealing. This approach showed effective to escape local minima, ridge etc. [40] [229] [185] [142]. It is also possible, to optimize a solution with simulated annealing after having found it: this is a simplification of previous approaches that calls annealer only after find a solution to refine it [201] [150] [176]

In the previous section importance of ensemble models were stated. The typical way to increase performances of neural networks makes use of a technique derived from ensemble techniques but with lower computational cost, called dropout. Each time an example is loaded into a mini-batch, a binary mask is randomly sampled. The mask is sampled independently for each hidden units and used via multiplication to suppress neuron contributions whose binary mask entry value is zero. The probability of sampling a mask value of one (causing a unit to being included) is a hyper-parameter. In the case

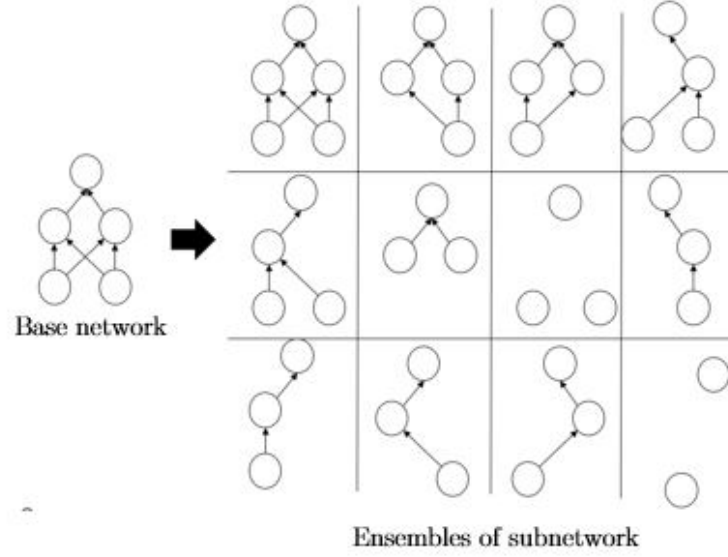


Figure 1.16: How dropout allow representing network ensemble; picture inspired by [107]

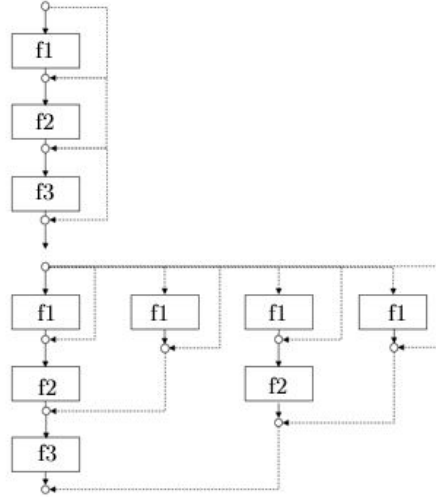


Figure 1.17: How skip connection allow represents ensemble of network; picture inspired by [107]

of dropout, each sub-model defined by mask vector μ defines a probability distribution $p(y \mid \mathbf{x}, \mu)$. The arithmetic mean over all masks is given by

$$\sum_{\mu} p(\mu) p(y \mid \mathbf{x}, \mu) \quad (1.69)$$

where $p(\mu)$ is the probability distribution that was used to sample μ at training time. Because this sum includes an exponential number of terms, it is intractable to be evaluated, except when the structure of the model permits some form of simplification. However sample only a subset of binary masks was found to be sufficient, with typically 20 masks commonly used.

[85] shows an interesting approach for treating deep neural network trained with gradient descent approach as a form kernel machine. This method is explored in chapter 2.3.1.

1.2 Quantum Computation

With end of Moore's law an increasing concern about limits of CPU, GPU and GPUs based super-computer has raised [74][141]. Of all possible computational paradigms, quantum computation is

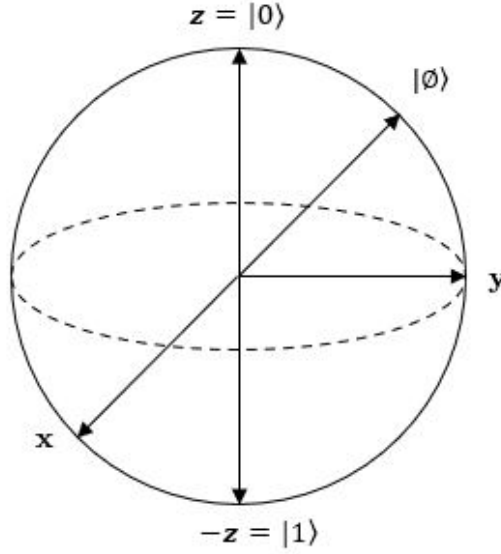


Figure 1.18: Bloch sphere representation. A state vector can be visualized as a point on the surface of the three-dimensional unit Bloch sphere. By convention the north and south poles correspond to the standard basis states $|0\rangle$ and $|1\rangle$, respectively. We will also be interested in an alternative pair of basis states $|x+\rangle$ and $|x-\rangle$ that lie on the positive and negative x axes

one of the main approach that is thought be able to overcome classical CPU and GPU computation. There are great expectation about quantum computation [194] and quantum supremacy, meaning capability of quantum computation to solve classical untreatable problems, has been experimentally demonstrated for highly theoretical model [7]. In this section a brief treatise about quantum computation is exposed focusing on computational models of quantum computations and computational advantages of quantum computation paradigms. After that, this section focus over adiabatic quantum computation approach and quantum annealing optimization will be analyzed.

Basics of Quantum Computation

The state of a classical algorithm can be represented by a register R of bits, each having value 0 or 1. The state of a quantum algorithm is instead represented by a register Q containing qubits. A state vector with a single qubit can be visualized as a point on the surface of the three-dimensional unit Bloch sphere (see figure 1.18). By convention the north and south poles correspond to the standard basis, meaning states $|0\rangle$ and $|1\rangle$, respectively (note however that quantum computation consider also an alternative pair of basis states $|x_+\rangle$ and $|x_-\rangle$ that lie on the positive and negative x axes).

Qubits have peculiar properties called superposition and entanglement that are exploited in quantum computation.

Superposition means potential for quantum states to exhibit correlations that cannot be accounted for classically. In particular, a qubit can be in states 0 and 1 simultaneously. A superposition state $|\phi\rangle$ corresponds to a qubit state vector containing two complex numbers α and β . The state vector denotes a linear combination of two basis states chosen conventionally to be $|0\rangle$ and $|1\rangle$: that is, $|\phi\rangle = \alpha|0\rangle + \beta|1\rangle$. Superposition states cannot be directly observed. Measurement device defines a pair of orthogonal bases and when a qubit is measured with respect to a given basis, qubits instantly "collapse" or are projected to one of the basis states: to $|0\rangle$ (observed as classical state 0) with probability $|\alpha|^2$, and to $|1\rangle$ (observed as 1) with probability $|\beta|^2$. Being α and β probabilities, it is stated:

$$|\alpha|^2 + |\beta|^2 = 1 \quad (1.70)$$

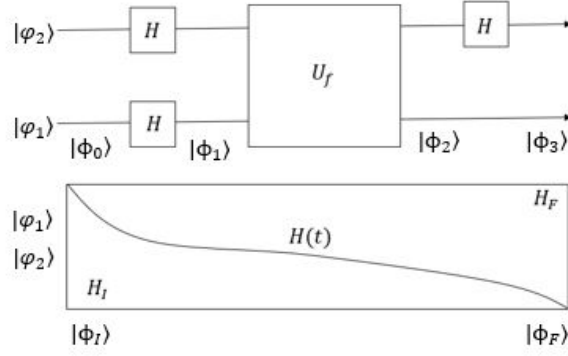


Figure 1.19: Circuit vs quantum annealing computation; picture inspired by [215]

where $|\alpha|$ is the magnitude of the complex number: $|\alpha| = |x + iy| = \sqrt{x^2 + y^2}$. Basis states are also called the observable states of the qubit [164]. Superposition is thought to guarantee exponential speedup via the so called quantum parallelism [103]. A set of n qubits can be considered to be all 2^n possible binary patterns at once, thus any operation performed on the qubits is performed on all 2^n patterns in parallel. Coherence of the qubit set is a necessary condition to show superposition. A quantum system is said to be coherent if it is in a linear superposition of its basis states. If a system that is in a linear superposition of states interacts in any way with its environment, the superposition is destroyed. This loss of coherence is called decoherence and is governed by the wave function [237].

Entanglement describe capacity of quantum particles such as qubits to interact in such a way that after the interaction, they are no longer independent: that is, their α 's and β 's are linked. Mathematically, states that cannot be factorized as a tensor product are said to be entangled, while those that can be factorized are not. Entanglement between quantum particles can persist even when the particles are physically separated. Entanglement also persists in time, through gate transformations and measurements. The amount of information contained in an entangled state of N qubits grows exponentially instead of linearly, which is the case for classical bits. There is no analog to entanglement in classical computation. Entanglement is necessary for a quantum circuit to act on all superposed states in constant time.

Given previous considerations, two qubit states $|\phi_1\rangle = (\alpha_1, \beta_1)^T$ and $|\phi_2\rangle = (\alpha_2, \beta_2)^T$ can be combined to form $|\Phi\rangle = |\phi_1\phi_2\rangle$ by applying the tensor product $\otimes$ to their state vectors:

$$(\alpha_1, \beta_1)^T \otimes (\alpha_2, \beta_2)^T = (\alpha_1\alpha_2, \alpha_1\beta_2, \beta_1\alpha_2, \beta_1\beta_2)^T \quad (1.71)$$

Setting $\gamma_1 = \alpha_1\alpha_2, \gamma_2 = \alpha_1\beta_2, \gamma_3 = \beta_1\alpha_2$ and $\gamma_4 = \beta_1\beta_2$, we can represent $|\Phi\rangle$ in terms of its bases as follows

$$|\Phi\rangle = |\phi_1\phi_2\rangle = (\gamma_1, \gamma_2, \gamma_3, \gamma_4)^T = \gamma_1|00\rangle + \gamma_2|01\rangle + \gamma_3|10\rangle + \gamma_4|11\rangle, \quad (1.72)$$

where

$$1 = |\gamma_1|^2 + |\gamma_2|^2 + |\gamma_3|^2 + |\gamma_4|^2. \quad (1.73)$$

These values represent the probabilities that the qubit pair will be observed in each possible state. A greater number of entangled qubits is simply represented via tensor product between each of their states.

There are two main distinct quantum computing paradigms : quantum gate computing ⁸, meaning sequences of quantum mechanical operators used for manipulating quantum registers and thus to perform computations, and adiabatic quantum computing.

In fact, in NISQ era, the implementation of quantum hardware is hardened by noise and quantum errors [168]. Quantum physics is a linear theory, meaning that maps of one state onto another are executed by linear operators, without loose of any information. In fact, the current challenge in the implementation of a quantum computer (especially for quantum gate computational strategy) lies in the difficulty to maintain coherence in a multiple qubits system in order to simulate unitary dynamics

⁸for brevity this approach is briefly exposed in appendix B.2

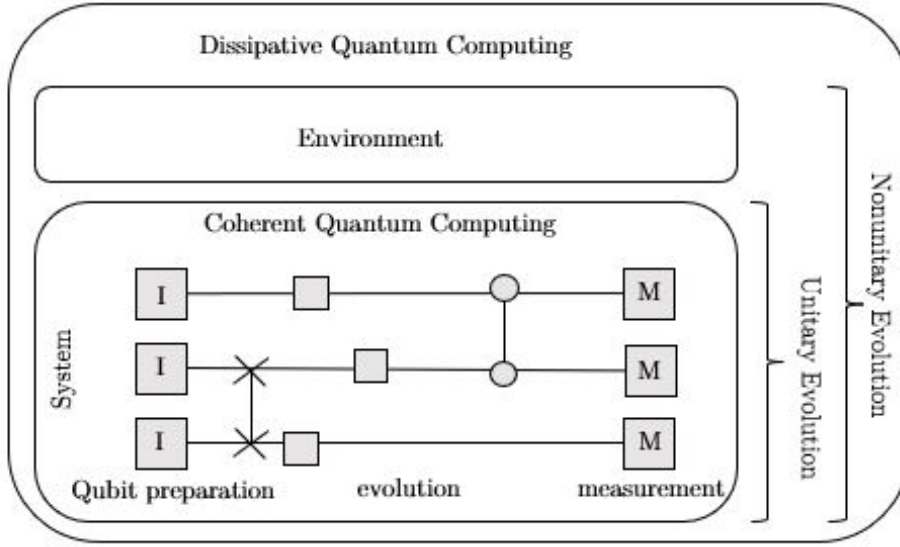


Figure 1.20: The problem of noise; picture inspired by [215]

which preserve probabilities. Real systems interact with an environment, thus introducing effects of dissipation, loss in the population of quantum states, and decoherence, destruction of quantum state correlations [216].

The other main quantum computation paradigm, adiabatic quantum computation (AQC), is exposed. AQC was original proposed by [91] and it was shown [92][2] that quantum gate model and AQC are polynomially equivalent.

Supposing have initialized a register Q to some known state as before, where n qubits are particles in a quantum dynamical system that evolves over time according to certain forces acting on it. Define the (superposition) state of Q at time t as $|\Phi_t\rangle$. The forces involved are from external sources and from interactions among qubits (including entanglement). At any time t , these forces are completely characterized by a time-varying Hamiltonian $\mathcal{H}_t$. Mathematically $\mathcal{H}_t$ is an $n' \times n'$ Hermitian matrix, meaning it has specific property of symmetry and decomposability. Every observable state $s = (0 | 1)^n$ has an associated energy, which is a real scalar that depends on $\mathcal{H}_t$. The energy spectrum is the set of all possible energies in the system, of size at most n' . The ground state s_g is an observable state having minimum energy over all basis states. A state that is not the ground state is said to be excited. The Hamiltonian serves a dual role in this framework. Firstly, It can be considered an operator that changes the state of the system: $\mathcal{H}_t |\Phi_t\rangle \rightarrow |\Phi_{t'}\rangle$. Secondly, the eigenstates of $\mathcal{H}_t$ correspond to the observable states of the system, and their eigenvalues to the energies of those eigenstates. Thus, if $|\Phi_t\rangle$ is an eigenstate one can measure its energy λ since $\mathcal{H}_t |\Phi_t\rangle \rightarrow \lambda |\Phi_t\rangle$

According to the Adiabatic Theorem if a quantum system starts out in the ground state of a Hamiltonian operator which then gradually changes over a period, the system will end up (with high probability) in the ground state of the resulting Hamiltonian [30][164]. Adiabatic Theorem [30] allows describe certain property of quantum particle processes which evolve according to the Schrodinger equation:

$$i\hbar \frac{d}{dt} |\Phi_t\rangle = \mathcal{H}(t) |\Phi_t\rangle \quad (1.74)$$

Here i is the imaginary unit and $\hbar = h/2\pi$ where h is Planck's constant ($\approx 6.63 \times 10^{-34}$ Joule-second). Suppose $\mathcal{H}(t)$ describes an AQC algorithm acting on Q , which evolves over time $t : 0 \rightarrow t_f$. Let $s(t)$ be strictly decreasing from 1 to 0 as $t : 0 \rightarrow t_f$. Let $\tilde{\mathcal{H}}(s)$ be the equivalent Hamiltonian in the time scale $s : 0 \rightarrow 1$, which evolves at a rate determined by t_f . Let $\tau(s)$ determines the rate at which the

Hamiltonian changes as a function of s . Now

$$\frac{d}{ds} |\Phi_s\rangle = -i\tau(s)\tilde{\mathcal{H}}(s) |\Phi_s\rangle \quad (1.75)$$

Assume that $\tilde{\mathcal{H}}_s$ has a non-degenerate ground state throughout time s . Let δ_s denote the spectral gap, the positive difference between the eigenvalues of the ground state and of the first excited state, at any time s . Let δ_m be the minimum spectral gap $\delta_m = \min_s (\delta_s)$. Now suppose $\tau(s)$ is bounded by

$$\tau(s) \gg \frac{\left\| \frac{d}{ds} \tilde{\mathcal{H}}(s) \right\|}{(\delta_m)^2} \quad (1.76)$$

where $\|\cdot\|$ is the matrix norm induced by the L_2 metric. Let $|\Phi_g\rangle$ denote the ground state of the final Hamiltonian. The adiabatic theorem states that if Q is in ground state at $s = 0$, δ_m is strictly greater than 0 throughout time s and the process evolves slowly enough then it will finish in ground state ($|\Phi_{t_f} = \Phi_g\rangle$) with sufficiently high probability. As a general rule it is easy to show that the numerator in previous formula is bounded above by a low-degree polynomial in n . Therefore, computation time t_f is polynomial in n exactly when the minimum spectral gap δ_m is inverse polynomial in n . A level crossing corresponds to a situation where $\delta_m = 0$ and therefore the adiabatic theorem does not apply. One reason for choosing $\mathcal{H}_I$ to operate in a basis that is orthogonal to that of $\mathcal{H}_F$ is that level crossings are highly unlikely to occur. This theorem provides lower bound on transition time in function of spectral gap. This last values is often out of reach of computation for many problems so the transition time must be selected as hyper-parameter. Wrong guesses increase the probability that the solution returned is not optimal: this problem is faced in detail for example by [204] to prove quantum advantage of Grover search, or [6] which tries to estimate spectral gap.

The AQC algorithm is finally described by a time-varying Hamiltonian $\mathcal{H}(t)$ specified by three components[164], meaning an initial Hamiltonian $\mathcal{H}_I$ chosen so that the ground state of the system is easy to find, a final, also called problem, Hamiltonian $\mathcal{H}_F$ that encodes the objective function so that the ground state of Q is an eigenstate of $\mathcal{H}_F$ having minimum eigenvalue and an adiabatic evolution path, a function $s(t)$ that decreases from 1 to 0 as $t : 0 \rightarrow t_f$, for some elapsed time t_f . Supposing a simple linear path $s(t) = 1 - t/t_f$, the Hamiltonian $\mathcal{H}(t)$ creates a gradual transition from $\mathcal{H}_I$ to $\mathcal{H}_F$ according to

$$\mathcal{H}(t) = s(t)\mathcal{H}_I + (1 - s(t))\mathcal{H}_F. \quad (1.77)$$

AQC has some advantages in respect of quantum gate models. Instead of using complicate scheme to counteract decoherence (p.e. error correction systems) AQC is robust against decoherence because the computation takes place in ground state. Given qubits decohere in low energy states or ground states, chance of failure are lower. With this paradigm it is possible to accept design simplification relaxing computation goals to achieve specific results, for example quantum annealing chips (that are explored later) are built to solve only one NP-hard optimization problem, the Ising model. It is also generally accepted in quantum computation that computation times must be much smaller than decoherence times to ensure adequate success probabilities. However theoretical [45], and experimental [29], evidences show that the adiabatic model is inherently robust in this regard: the success of the computation is not related to the phase of the ground state, and therefore success probabilities are not affected by dephasing (a type of decoherence).

Quantum annealing

Quantum annealing (QA) is an heuristic search approach for solving problems in combinatorial optimization, originally developed for implementation in classical hardware. Quantum annealing is a variation of simulated/thermal annealing (SA) in the family of the so called Metropolis/Hastings algorithms [208]. All methods for annealing are based on Monte Carlo algorithm, which in order to generate sample states, repeats a large amount of random sampling on a hypercube of dimension N , the space of solutions for the problem [208]. Consequently QA and SA are heuristic local search method meaning meta-heuristic, a problem-generic solution strategy for solving NP hard problems on

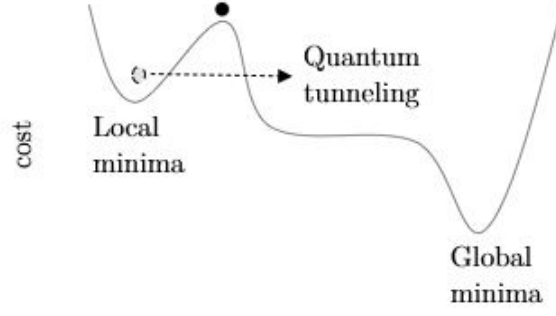


Figure 1.21: Landscape of energy minimization problem; picture inspired by [215]

par with an algorithm paradigm [164]. The idea appears to have been independently suggested by several researches [164]. Many authors use terms AQC and QA interchangeably but as [205] pointed out AQC algorithm refers to an algorithm in the universal AQC model while QA algorithm imply some relaxation of properties of AQC models (e.g. restriction on the final Hamiltonian to represent a classical objective function, not all computation take place in ground state)

QA algorithms could run in different physical implementations. An ideal AQC platform, meaning adiabatically closed system, have no interference from environment. In this case algorithm is probabilistic, meaning it uses a random source to perform computation, and adiabatic theorem may be used to probabilistically bound trade-off between computation time and probability that system finishes in ground state. Another possibility is using physically realized quantum platform: note that in this case, as other open systems, it cannot be perfectly isolated from its environment: theoretical AQC guarantees do not apply, so in this case QA is heuristic, an algorithm that has weak or nonexistent theoretical guarantees on run time and/or solution quality. In present document the focus is on the second case.

The most known QA platform is the D-Wave Quantum Annealer platform, whose actual Advantage Quantum Process Unit (QPU) can actually accommodate around 5000 qubits, each one directly connected to twelve other qubits via Pegasus topology⁹. There are other initiatives building computers in the AQC framework that are quoted here for completeness. [221] implements AQC via nuclear magnetic resonance, implementing 3 qubit, for solving maximum cut problem. [248] describes adiabatic quantum computer to factor integer 143 on four qubits from hydrogen nuclei in molecules of 1-bromo-chlorobenzene and each spin manipulated using bursts of radio waves.

As said before annealing approaches are in the family of local search approaches to solve problems. Given a minimization problem defined on n binary variables $x = x_1 \dots x_n$, an objective function $f(x)$ that assigns a cost to each solution $x \in \{0, 1\}^n$ is defined. The solution space D^n is represented by a graph with nodes as solutions and edges defined by a neighborhood rule. Together, the graph and the objective function create a landscape, where hilltops correspond to solutions with high cost and valleys to solutions with low cost. A local search algorithm starts at some initial node, or meta-state, in the space and then iterates, stepping from node to neighbor node/state and generally heading toward low-cost regions of the space, or energy, until terminated by some stopping rule. To empower local search avoiding local minima, heuristic search methods and in particular annealing methods are proposed. [164][208]

The simulated annealing strategy[140], at each iteration, selects a random neighbor of the current solution. If the new solution has lower cost, it becomes the current solution. If the new solution has higher cost, the uphill move is accepted according to a probability that depends on the temperature T and on the difference between costs. Let $\Delta = c_{\text{new}} - c$ where c and $c_{\text{new}} \in \{0, 1\}^n$, this probability is often defined as

$$p[\text{Accept}(T, \Delta)] = \min \left(1, e^{-\Delta/T} \right) \quad (1.78)$$

⁹Given results here presented do not apply only to D-Wave platform, author defer a more detailed description of D-Wave architecture in appendix B.2

The temperature parameter T is initialized at some high value and then decremented at each iteration according to a cooling schedule .

QA instead of using a temperature parameter T , uses a transverse field coefficient γ , also called the tunneling width/quantum field strength parameter[208], to control the traversability, or selection of meta-states, of the solution landscape. As T , γ starts at a high value allowing the algorithm in principle to tunnel into all search space or at least a wide part of search space (see later), and it is gradually decremented over time toward a single quantum coherent state that encode the solution, in a quantum collapse, according to a given schedule. Quantum annealing introduces a transverse field Hamiltonian, also called the disordering Hamiltonian, $\mathcal{H}_D$ that does not commute with $\mathcal{H}_F$. Scaling it with γ , creates a new time-dependent Hamiltonian,

$$\mathcal{H}(t) = \mathcal{H}_F + \Gamma(t)\mathcal{H}_D \quad (1.79)$$

The disordering Hamiltonian $\mathcal{H}_D$ introduces kinetic energy to the annealing process in the form of quantum fluctuations of the solution space, changing in the amount of energy at a point in space for extremely short time lapses as a result of the uncertainty principle [208]. The tunnel effect/quantum tunneling controlled by γ allows to take advantage of wave matter duality in order to traverse through the solid interstate potential barriers, in this case of the problem landscape, by quantum jumps [208]. Decreasing $\Gamma(t)$ moves the system closer to $\mathcal{H}_F$ (ground state) while dampening the quantum fluctuations. Too fast annealing time impedes wave function collapsing correctly and phase transition toward ground state does not occur because functional shape of meta states does not adapt the associated probability distribution in time. The capability to adapt to this phase transition is called adiabaticity. Residual entropy remains in zero point energy/ground states due to this violation of adiabaticity, and causes too wide interstate energy barriers to be traversed with tunneling. When the algorithm is implemented on a quantum platform, it is not in a single state a given time t , but rather in a superposition of states, with probabilities determined by $\mathcal{H}(t)$ [164]. Following Heisenberg principle, a particle exploring landscape does not occupy a specific point but is associated to a probability density related to the place it occupies. In other word superposition should allow exploring more neighbours at the same time.[208]

QA has some intriguing properties that were underlined under classical implementations. It have shown a convergence bound for QA analogous for SA. Assuming that $\Gamma(t)$ decreases according to

$$\Gamma(i) \geq \frac{b}{(i+1)^{c/n}} \quad (1.80)$$

for constants b, c , the algorithm guarantees to reach ground state with a probability approaching 1 as $i \rightarrow \infty$. This convergence rate is better in n than that of SA. In some restricted cases QA can be proven to converge in polynomial time when the corresponding SA approach has an exponential lower bound [171]. [187] also point out that in nearly all cases where QA and SA have been compared empirically by numerical simulation or sampling, QA has shown more rapid convergence to ground state than SA. However, this results were not always confirmed experimentally [79]. Finally QA performance do not degrades according to the height of barriers in the solution landscape [79]

As said before quantum annealing minimizes an Ising Hamiltonian. Created by Ernst Ising in 1920 in his doctoral dissertation, Ising model original purpose was to explain how short range interactions give rise to long range correlative behavior allow for predicting potential for phase transitions, meaning a small change in a parameter such as temperature or pressure that causes large-scale qualitative changes in the state of a system. Phenomena of ferromagnetism, in particular spontaneous magnetisation phase transitions were originally considered as Ising model subject. A lattice is a finite set of regularly spaced points in a space of dimension $d=1,2,3$. Each line segment between lattice is called a bond and d lattice sites are called nearest neighbors if there is a bond connecting them.

A binary variable, traditionally σ , for each lattice site is assigned. An assignment of values to every site is called configuration [47] [203].

Ferromagnetic interpretation (there exists also lattice gas interpretations) supposes every lattice site is composed of magnetic material. Every atom has a magnetic moment, or angular momentum

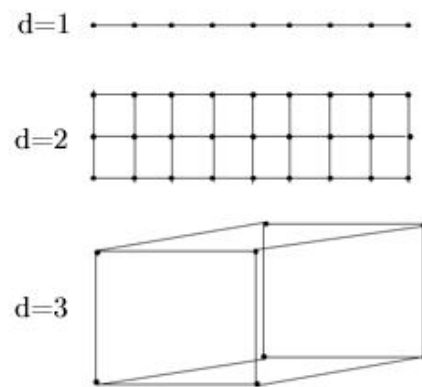


Figure 1.22: Lattice in multiple dimensions; picture inspired by [47]

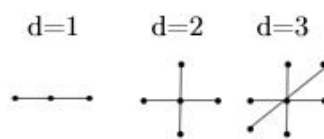


Figure 1.23: Lattice nearest neighbors interactions in multiple dimensions; picture inspired by [47]

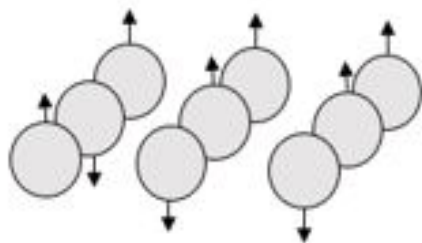


Figure 1.24: Particles with two possible spins; picture inspired by [203]

[208], with two possible values: up and down. The model assumes only short range nearest neighbor interactions between particles and single interactions of lattice sites with an external field. The total magnetic field h_i of the i atoms given the external field h^* is

$$h_i = \sum_{j=1}^n w_{ij}s_j + h^* \quad (1.81)$$

where w_{ij} represents the magnitude of the magnetic coupling between the atoms labeled i and j . Coupling is categorized in antiferromagnetic coupling if i is different of j , ferromagnetic coupling instead. The potential energy or the total energy/hamiltonian E of a certain state of an Ising material system has the form

$$E = -\frac{1}{2} \sum_{i,j}^n w_{ij}s_i s_j + \sum_i^n -h^* s_i \quad (1.82)$$

And a magnetized configuration has less energy of a non-magnetized configuration. If a magnetic material is inserted in a magnetic field with a certain temperature, and magnetic field intensity, all magnets interacting with each other cause each other to flip their spins until equilibrium and this determines a magnetization. If the external field is turned off, for high temperature, the lattice return demagnetized while for low temperature it remains a residual tendency to stay magnetized; this phenomenon is called spontaneous magnetization. The fact that there is a critical temperature in which this spontaneous magnetization appear only for more than 1D Ising models lead this model to oblivion until forties when the 2D lattice model was analyzed: [101] showed Ising problem is NP-hard with a 2D grid and nonzero h , and the so called Ising problem was formulated (for brevity see [47] for an Ising problem formulation description).

The Ising Model, defined on spins $s_i \in \pm 1$, is closely related to two problems on binary variables $x_i \in \{0,1\}$. The first is Quadratic Unconstrained Binary Optimization (QUBO). The other one is Weighted Maximum 2 Satisfiability which is not considered here. QUBO problem is defined as follows. Given an $n \times n$ matrix of weights Q_{ij} the problem is to assign binary values to n variables s_i for $i \in 1 \dots n$, to minimize the function

$$f_{qubo}(x) = \sum_{i,j} Q_{ij}x_i x_j. \quad (1.83)$$

note here notation make use of x instead of s variables to underline that variable involved are binary not bipolar

1.3 Quantum Machine Learning

In the current era two interesting phenomena are leading research. First, given the need for more computational power, artificial intelligence which recurs to GPU is now transitioning to GPU-based supercomputers [74][126]. This is due to new challenges machine learning [175][198] like scale data generated (20% rate of data increasing every year [116]), hardware limitations (the current trend tech advancement in term computational power will be saturated), computation complexity (many machine learning problems non-convex in nature [132]) and cost (due to data increasing). Secondly there is a concern limits of GPU and GPUs based supercomputer will be reached very soon [74][141]. In fact the growth of artificial intelligence in real world applications coincides with the end of Moore's law [74]. There are also methodological concerns regarding classical approach machine learning[125]: machine-learning and in particular deep learning could ignore classical sampling theorem meaning generalization, robustness and interpretability could not be guaranteed: adversarial example show to be a way to fool network capability [8]. Furtherly ML shows cognitive impairment, meaning less robustness and versatility than human. This is reflected in huge amount of data needed. Interest in heterogeneous compute platforms tailored for different aspect has begun [74][196]. As stated by [74][26][77] one of the potential areas for which quantum computers will make a difference in computing is machine learning. Quantum machine learning is suggested [125][157][26] to be able to find solutions not available classically as a consequence of quantum supremacy and as alternative to use full extent

data [175][5]. Application of quantum computation to machine learning is called Quantum machine learning, a term coined by [26].

Using D-Wave for training classifiers is a novel perspective. The first idea is of [11], where its authors, using a D-Wave One chip based on a C_4 chimera graph with 52 working qubits, constructed two types of binary classifiers: a deterministic one, providing separation function to classify part of speech tag, and a probabilistic one, outputting a distribution of classifications. To implement them, they optimized the weights that improve classification performances, as long as other soft and hard constraints, sending iteratively QUBO instances to annealer. Authors of [11] compared their approach to an alternative method involving support vector machines (SVM) their learning algorithm seems to improve results [183][181][182]. Nowadays, using annealer in classification tasks has demonstrated better generalization on small datasets via ensembles, superior performance with lower learning iteration and was already used for ¹⁰ machine learning related problems (like least-squares solutions to polynomial systems of equations [44], better loss to exploit quantum advantage [9], latent variable sampling for generative models [246][82], sampling data [82]), reinforcement learning[213], unsupervised learning (2-means and binary clustering [149], soft-clustering [137] [211], bi-clustering [93], automated feature selection [174], unsupervised probabilistic machine learning (like deep belief network or restricted Boltzmann machines)[74], sparse representation extraction [184]), supervised learning (binary classification [81], qBoost with binary classification [86], support vector classification [245], linear binary classifier based on PMW for transcription factor binding, ensemble of weak classifier to solve Higgs optimization problem [172][179][180]).

Quantum annealing based SVM

Due to historical importance and being the only example of a quantum implementation of a complex and actually used machine learning algorithm, it is relevant to expose results of [245]. In [245], based on SVM formulation of [217], author proposed first implementation of quantum SVM trained on annealer. They optimized soft constraint Wolfe dual formulation of support vector machine, with an hybrid linear/Gaussian kernel, meaning Gaussian kernel including linear kernel as corner case[136]

$$k(\mathbf{x}, \mathbf{x}') := \begin{cases} \text{rbf}(\mathbf{x}, \mathbf{x}') & (\gamma > 0), \\ \mathbf{x} \cdot \mathbf{x}' & (\gamma = -1) \end{cases} \quad (1.84)$$

to calculate support vectors they encode weights of KKT solutions in a discrete encoding

$$\alpha_n = \sum_{l=0}^{d-1} b^l a_{dn+l}, \quad (1.85)$$

where $a_{dn+l} \in \{0, 1\}$ are binary variables, d is the number of binary variables to encode α_n , and b is the base used for the encoding. In practice, author obtained good results for $b = 2$ or $b = 10$ and a small number of d (see also the list of arguments given below). The advantages of this formulation comprehend encoding explicit zero solutions given real encoding of variable make a zero result less probable, SVM solutions sparsity, box constraint satisfaction, simply extension of the encoding to fractional number by replacing the base b^l with b^{l-l_0} for some $l_0 \in N$. They use minor-miner embedding algorithm as default of Ocean package testing their implementation over a protein basis sequence classification task evaluated with AUROC/AUPRC.

A following work of [25] test performances measured in respect of accuracy and f1 score over Iris and Standard Banknote Authentication datasets comparing it to qSVM optimized with D-Wave hybrid, qSVM trained with simulated annealing and classic SVM. Results show that SA and hybrid produce same decision boundary, classic SVM seem to have smoother boundaries achieving greater generalization: in fact all QPU qSVM, Hybrid qsvm and SA qsm seems very sensible to dataset. this is confirmed in other works like [251] and QPU qSVM seems to have great limitation in data dimensions. However in further works [193] QPU SVM seems to have more confidence rather than

¹⁰a full review of quantum machine learning based on quantum annealer is in [175]

qSVM implementation with SA. [43] shows that compared to circuit based quantum support vector machines, QPU qSVM implemented over advantage annealer architecture, has enough capacity to treat remote sensing tasks. However the algorithm scaling with dimension remains an open question. Remote sensing classification with qSVM ensemble approach on QPU [42] shows robust classification results. There were also extensions to multi-class of qSVM approach. As an example qSVM is applied to flow separation problem [251], developing one vs all, a winner takes all approach in which the largest confidence classifier among all class specific classifier is selected. In [78] authors developed both one vs all and one vs one approach but using lowest energy samples as break tie condition to select greatest margin classifiers. Authors applied it to a 2d, three classes, synthetically generated dataset and iris dataset

Quantum annealing based neural network

Quantum neural networks are models, systems or devices that combine features of quantum theory with the properties of neural networks [215]. Only one year after Schor algorithm [220] were published demonstrating potential power of quantum computation, [134] authors first proposed the idea to bind neural network and quantum computation. This proposal, even if detached from the discussion about quantumness of the brain [134][97][110], has attracted considerable attention in scientific community.

In the following section Quantum implementation with a focus on annealer implementation is exposed. For a fully-fledged discussion comprehending also AQO and circuit models see appendix B.3. Note that in this work, focus is posed on continuous, also called in early literature, simulation, architectures like AQO and QA. These approaches in respect of discrete ones appeared to be really effective in treating NP problems, like neural network training [252], and in fact first example of quantum neural network [4] is a simulation/continuous approach.

The states of [215] must be quoted and is still valid nowadays: “QNN research remains an exotic conglomeration of different ideas under the umbrella of quantum information”. In literature could not be founded uniform evaluation metrics, a single trending idea, same properties stated as fundamental. So in future it could be useful to take into account comparisons between architectural choices and also properties of the developed architecture itself. This should also take into account recent developments in classical algorithms, like exponential storage capacity for Dense Associative Memories, as it’s tried to be done in this work. Good guidelines for analysis of quantum neural networks are proposed by [215]:

- The initial state of the quantum system encodes any binary string of length n . The QNN produces a stable output configuration encoding the one state of 2^n possible output binary strings of length n which is closest to the input by some distance measure.
- The QNN reflects one or more basic neural computing mechanisms
- The evolution is based on quantum effects, such as superposition, entanglement, and interference, and it is fully consistent with quantum theory.

And a criticality from same authors: a fundamental challenge appears in the integration of the non-linear, dissipative dynamics of attractor based NNs and linear, unitary quantum theory. In neural computing, the incoming signal $\sum_{j=1}^n w_{ji}x_j$ of the n neurons $x_j, j \neq i$ connected to neuron x_i by weights w_{ij} gets mapped to the output of neuron x_i by a non-linearity. This non-linearity leads to the attractor-like dynamics fundamental for associative memory: as saw before without nonlinearity an eigenvalue dominate all attraction field. Practically, this is called dissipative behavior because non-linearity losses some information and this is important to filter out patterns. If one introduces quantum mechanics, he has necessarily to look at firing probabilities instead of deterministic values. However, mapping firing probabilities onto one another by a nonlinear function contradicts the basic principle of the linear evolution in quantum theory, where linear superpositions of solutions of the Schrödinger equation play an important role. One apparent exception are measurements, which can be understood as a probabilistic step-function processes

As pointed out by [115] there are three aspects that can be enhanced and if modelled together define a complete implementation of feed-forward neural network architecture in the annealer: the

neural network implementation by itself, loss function whose minimization defines task, model training algorithm. The first point has to be expanded. It should be pointed out that feed-forward neural network as said before is a broad term so it's useful to define possible proposed neural network on the annealer as

- deep feed-forward neural network vs shallow feed-forward neural network a deep neural network implementation require more than three layer
- binary feed-forward neural network versus a continuous (or general) feed-forward neural network a binary neural network models only binary weights and neuron values while a continuous (or general) feedforward neural network has necessarily to model more complex nonlinear activation like ReLU.

There are actually three works about possible implementation of feedforward neural networks [115] [210] in annealer architecture. Given criteria before exposed, it should be underlined how no authors have already proposed a suitable method in how treating deep feed-forward neural architectures in QA setting and no authors have already defined continuous feed-forward neural architectures with nontrivial non-linearity. Given their results are not exploited in present document their description is left in appendix as long with comments regarding their criticalities. ¹¹

Other quantum annealing trending approach

For completeness, here other tendency that seem have appeared along implementation of algorithms above exposed are briefly treated. First, hybrid classical-annealer approaches. As said before interest in heterogeneous compute platforms tailored for different aspect has begun [74][196]. Authors, like in [125], exposed hybrid approaches capable to employ different architectures for different task in which they are proven to be effective. Purely quantum approaches have in fact some disadvantages [109], like limited knowledgeable users, difficult in translate a problem in quantum formalism, lack of benchmarks. Due to these aspects, developing hybrid classical-quantum system for machine learning classification tasks is expected to perform better than using a classical machine learning pipeline under limited data, high dimensional data, high cost and time and improving generalization [175]. QA machine learning algorithm implementations shows comparable, and sometimes increasing, performances in deal with sampling phases of some probabilistic unsupervised machine learning models, so it could be advantageous leaving matrix multiplication involving algorithm to specialized hardware, meaning GPUs while doing inferences in QA framework [74].

As an example of a trending technique, [1] authors used quantum annealer to sample as a part of Deep Belief network training showing comparable results with classical contrastive divergence training and was applied in hybrid algorithms to lung cancer detection [240]. Another trend is using ensembling techniques using multiple solutions provided by the annealer. qSVM [245] originally underline effectiveness of using multiple solutions provided by the annealer training process of qsvm. This idea naturally arises from the fact that optima over training set is not necessary the optima for test data or real data setting. Comparing quantum SVM and classic svm [245] over a fake 2d dataset they note as different energy sample produce different separation surface. They divide protein sequence classification train dataset for each binary classification task in 50 slices of 30 datapoints and train a SVM for each slice with same hyper-parameters founded in a previous task. They combine 20 lowest energy solution for each slice averaging over 50 slice per binary task. This ensembling approach obtain AUROC superior to classic SVM. Another work [184] show performance improvement using multiple solutions of sparse representations calculated by annealer (this approach is called quantum augmentation). Note however that ensembling approaches to improve solution quality is at the beginning and

¹¹during writing of the present thesis started in 2020 the first of this works was not published however author find relevant to analyze them to underline differences in approach followed and what are advantages of present work formulation

only a few works have been done over this possibility.

2 Quantum Dense Associative Memories

In this chapter it will be shown how implements quantum dense associative memories (chapter 2.2). Lack of global synchronization of Hopfield network and Dense Associative Memories as a type of BAM, meaning not need to have a way to synchronize each units [203], is an intriguing aspect. This is coherent with QA annealing and its hardware implementations which which does not offer a way to control directly which units do its calculation: there are methods to constraint which sets activate like reverse annealing but their capabilities are not completely adherent to a mechanism of global synchronization. Associative memories under asynchronous dynamic have also a natural energy interpretation that is compliant with quantum annealing minimization. Both these aspects make appealing to implement dense associative memories via annealer formulation.

However a necessary question to be answered is why implementing quantum Dense Associative neural network. Apparently works of [147][80] show exponential memory storage of classical dense associative memories, eliminating one of the core reason for searching for quantum formulation of Hopfield memories, meaning exponential storage due to quantum computation properties, however there are other reasons it is worth to implement it.

First, historical and biological importance of associative memories, like Hopfield networks and Dense Associative Memories, being one of the early neural networks methods and source of first proposal of fast quantum algorithm implementation [200]. Implementing quantum dense associative memories inherits same historical, biological and architectural importance.

Secondly, both Hopfield networks and dense associative memories become intractable to be computed classically due to their fully connected nature [168][199]. This tendency was already underlined in [215].

Another motivation regards local minima escaping. There is actually no way to escape local minima and errors of classical Hopfield neural networks [203][218] and classically Dense Associative Memories implementing Markov dynamic have also local minima. Randomized strategies for problems, like quantum computing, are a way to break complexity and polynomial nondeterministic algorithms are often more powerful than deterministic counterparts [252], leading to possibility to improve capacity of both models.

Furtherly associative memories, like Hopfield networks and Dense Associative Memories, are interpretable and widely known by neurocomputing community and dense associative memories being their generalization could inherit also this property. Recently, authors of [13], relaunched the usage of Hopfield network to solve problems via it's isomorphism with ising model: they considered a k-rooks problem solving via Hopfield and AQO. One of the challenges in adiabatic quantum computing is devise suitable problem Hamiltonians. If the problem at hand is a minimization problem that can be tackled by a Hopfield network, this challenge is minor because it is easy to construct Hamiltonians for the energy functions of Hopfield networks.

Associative memory models could be adapted to solve a wide variety of hard combinatorial optimization problems which could belong also to NP difficulty class. As an example TSP is a NP problem already analyzed via Hopfield model. However Bruck and Goodman [37] showed that a polynomial network bounded by the size of the problem, is unable to find the global minimum of the energy function of NP-complete problems encoded as Hopfield networks with a 100% guarantee. If one try to transform all local minima of the Hopfield network into an optimal solution of the combinatorial problem, the size of the network explodes exponentially. Other considerations are shown to limit capability of associative memories in solving NP tasks. In particular, let L be an NP-complete decision problem and H a Hopfield network with several weights bounded by a polynomial on the size of the problem.

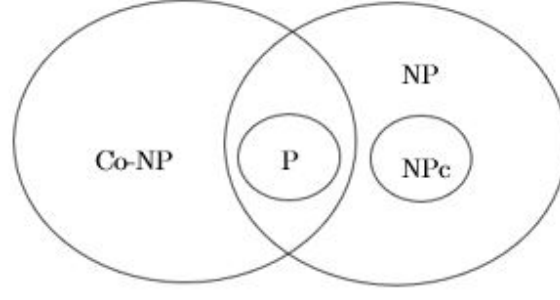


Figure 2.1: A depiction of NP and co-NP classes; picture inspired by [203]

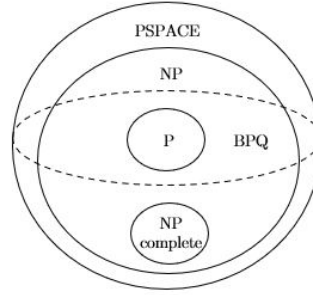


Figure 2.2: A depiction of quantum complexity theory classes; picture inspired by [203]

If H can solve L , meaning 100% success rate, then $NP = co-NP$ (demonstration in [203]). Note that if NP is different of $co-NP$, which is the complexity class that contains the complement of all NP problems, than P is different from NP and given this consequent is considered generally true it results that capability of associative memories to help in NP solving seems to be modest. However [203] suggests that associative memories could be competitive for problem resolution with usage of special hardware, like AQO. And in fact, the before mentioned proposition do not hold for BQP complexity class, since is not the same of NP problems, and quantum dense associative memories implementation could help finding solutions for NP problems.

Quantum dense associative memories as proposed thereafter could also be only way to implement exponential storage capacity in QA setting. Quantum parallelism of a classical hopfield model implemented in transverse field could be thought to be sufficient to obtain exponential memory storage so dense associative memories could be perceived as not necessary. However, this could not be the case. Authors of [215], [171] analyzed a x-y quantum Ising model simulating a neural network. Quantum fluctuations play the same role as thermal fluctuations and are consequently not able to influence the macroscopic behavior of the network other than a classical model with nonzero temperature would do. This suggest quantum Hopfield networks in AQO setting are not capable of exponential memory storage.

As Hopfield networks can be used as core component for more complex architecture via perceptron analogy of Hopfield network[200], Dense Associative Memories expand this capability allowing for a feed-forward implementation of neural network. For many authors (i.e. [252]) analog computation like QA could be a solution to high complexity problems and consequently implementing quantum deep feed-forward neural networks via QA implementation of Dense Associative Memories could improve performances of deep neural networks in terms of, as reported in previous chapter, performances/quality of solutions via annealer new training procedures and possibility to exploit ensembling approach of the annealer and speed of solutions via possibility of sampling them via annealer.

2.1 Strategies and difficulties in developing QUBO formulation for Dense Associative Memory

Ising model and classical Hopfield model suppose only binary interactions [47] while dense associative memory allows for non-binary interactions configuring itself as PUBO problem [32]. Removing this interactions is one of the core problem that have to be faced.

Authors of [148] presented a possible solution to handle this non-binary interaction, where a general energy function can be written as

$$E(t) = \left[\sum_{i=1}^{n_f} (v_i - i_i) g_i - L_v \right] + \left[\sum_{\mu=1}^{n_h} h_\mu f_\mu - L_h \right] - \sum_{\mu,i} f_\mu \xi_{\mu i} g_i \quad (2.1)$$

that allows to recover Dense Associative Memories energy function with a determinant assumption: the “steady state condition” meaning that visible and hidden neurons do their calculation with a negligible time difference. Supposing at least hidden neurons can assume continuous values, this allows to obtain from dynamic equations previously state:

$$h_\mu = \sum_{i=1}^{n_f} \xi_{\mu i} v_i \quad (2.2)$$

For present work purpose this is not a viable choice. If hidden neurons are bipolar like in Ising model, or binary in QUBO formulation, then the bipolar hidden neurons h_μ could not be equal to the sum of other bipolar values of visible layer v_i . Consequently this steady state condition can not hold and this formulation is of no use in translating this problem to annealer.

In the next subsections it will be shown how to fix this problem but now it's interesting to show what other possibilities could be used to eliminate more than two body synapses with binary/bipolar neurons. Literature regarding treating high order interactions in annealing typically face it with native annealer platform provided mechanisms [64] or polynomial reductions [72]. Native mechanisms in the platform here considered (D-wave) regard in particular make_quadratic method [69], relying on approximation of the more than quadratic terms in HUBO (higher order unconstrained optimization) provided by user. This is done via reduction by substitution methods described below. Given aim of this work is constructing directly a native energy formulation, meaning without more than quadratic terms, to optimize it directly, this possibility will be ignored. Polynomial reductions [72] implies users have to simplify energy function at hand to reach a native formulation. This could be done with two main techniques which operate on QUBO formulation, meaning on binary variables. Note that Ising based quantum annealer like D-Wave employ a bipolar native formulation however a simple algebraic substitution is sufficient to translate a formulation one into another. This can be simply achieved substituting in the original bipolar energy formula every spin $s_i \in \{-1, 1\}$ with

$$s_i = 2x_i - 1 \quad (2.3)$$

having

$$x_i \in \{0, 1\}$$

In Reduction by substitution, user reformulates a non-quadratic polynomial to Ising/QUBO using this substitution to reduce a triplet interaction to pairwise,

$$x_1 x_2 x_3 = \min_z \{x_4 x_3 + mP(x_1, x_2; x_4)\} \quad (2.4)$$

where $m > 1$ is a penalty weight and P is some quadratic penalty function that has lowest value when $x_4 = x_1 x_2$ [72]. To obtain equal energy, the design of M and P is necessary and nontrivial.

Another possibility is Minimum selection. This technique of reduction by minimum selection exploits the identity,

$$x_1 x_2 x_3 = \max_{x_4} \{x_4 (x_1 + x_2 + x_3 - 2)\} \quad (2.5)$$

where x_1, x_2, x_3 are binary variables in some pseudo-binary function being minimized and x_4 is the ancillary binary variable introduced to substitute quadratic terms for cubic terms. Substituting terms in the form of $ax_1x_2x_3$, where a is a real number, with

$$ax_1x_2x_3 = \begin{cases} ax_4(x_1 + x_2 + x_3 - 2) & a < 0 \\ a\{x_4(x_1 + x_2 + x_3 - 1) + (x_1x_2 + x_2x_3 + x_3x_1) - (x_1 + x_2 + x_3) + 1\} & a > 0 \end{cases} \quad (2.6)$$

Or equally for $a > 0$

$$ax_1x_2x_3 = -a \min_{x_4} \{-x_4(x_1 + x_2 + x_3 - 2)\} \quad (2.7)$$

This method has the advantage of being more straightforward, could be easily showed constructing a table with energy/variable combination and has remarkable property of maintaining same energy minima, which is important for the present work approach as it will be underline thereafter. There is also an extensive version of this equality [130] which can be applied to polynomials of order higher than two which is not very well known in the community of QA (e.g. [106][13]) and by author knowledge was not previously quoted in QA related work. Defining

$$y_1 = \sum_{i=1}^n x_i \quad (2.8)$$

$$y_2 = \sum_{i=1}^{n-1} \sum_{j=i+1}^n x_i x_j = \frac{y_1(y_1 - 1)}{2} \quad (2.9)$$

Where x_i are free variables and n is the degree of a polynomial in binary variable that one wants to make first order. Then, if $a < 0$

$$ax_1 \cdots x_n = \min_{x_{n+1}} ax_{n+1} \{y_1 - (n - 1)\} \quad (2.10)$$

where x_{n+1} is a new auxiliary variable. For $a > 0$ instead:

$$ax_1 \cdots x_n = a \min_{x_{n+1} \dots x_{n+n'}} \sum_{i=1}^{n'} x_{n+i} (a'_{i,n'} (-y_1 + 2i)) + ay_2 \quad (2.11)$$

Where

$$n' = \left\lceil \frac{n-1}{2} \right\rceil \quad (2.12)$$

$$a'_{i,n'} = \begin{cases} 1, & \text{if } d \text{ is odd and } i = n' \\ 2, & \text{otherwise} \end{cases} \quad (2.13)$$

And $x_{n+1} \dots x_{n+n'}$ are n' auxiliary variable.

Given it is not feasible to rely on steady state condition anymore the only remaining possibility is trying minimize original dense associative memory energy formulation of [147] :

$$E = - \sum_{\mu=1}^m F \left(\sum_i^n \xi_i^\mu s_i \right) \quad (2.14)$$

F could be every possible continuous function. In the following section an exponential energy function will be considered given the claimed exponential storage capability which is dense associative memories greatest appeals. Note analogous considerations could be done also for other energy functions.

As an example to develop suitable formulation, consider two memories of length three

$$\xi_1 = [\xi_1^1, \xi_2^1, \xi_3^1]$$

$$\xi_2 = [\xi_1^2, \xi_2^2, \xi_3^2]$$

where ξ_i^μ is the bipolar value of the memory μ in position i . Denote as $\mathbf{s} = [s_1, s_2, s_3]$ the neurons vector of an associative memory with same length of the beforementioned memories. Dense Associative Memory energy for this two memories would be

$$E = -e^{(\xi_1 \cdot \mathbf{s})} - e^{(\xi_2 \cdot \mathbf{s})} \quad (2.15)$$

Taking in consideration the first term ¹

$$-e^{(\xi_1 \cdot \mathbf{s})} = -e^{\xi_1^1 s_1 + \xi_2^1 s_2 + \xi_3^1 s_3} = -e^{\xi_1^1 s_1} e^{\xi_2^1 s_2} e^{\xi_3^1 s_3} \quad (2.16)$$

Quantum annealers can not handle exponentiation of variables, but given only bipolar values of neurons and memories are available, every factor in the above product can have only these possible values

$\xi_i^{\mu_k}$	s_i	$e^{\xi_i^{\mu_k} s_i}$
+1	-1	e^{-1}
-1	+1	e^{-1}
+1	+1	e^1
-1	-1	e^1

and every single factor in the energy can be expressed as

$$e^{\xi_i^\mu s_i} = e^{-1} \frac{(1 - (\xi_i^\mu s_i))}{2} - e \frac{(-1 - (\xi_i^\mu s_i))}{2} = \frac{(e^{-1} + e)}{2} + \frac{(+e - e^{-1}) \xi_i^\mu s_i}{2} \quad (2.17)$$

Substituting this equality in first term of energy function

$$-e^{\xi_1^1 s_1} e^{\xi_2^1 s_2} e^{\xi_3^1 s_3}$$

one obtains

$$\begin{aligned} -e^{(\xi_1 \cdot \mathbf{s})} = & - \left(\frac{(e^{-1} + e)}{2} + \frac{(+e - e^{-1}) \xi_1^1 s_1}{2} \right) \left(\frac{(e^{-1} + e)}{2} \right. \\ & \left. + \frac{(+e - e^{-1}) \xi_2^1 s_2}{2} \right) \left(\frac{(e^{-1} + e)}{2} + \frac{(+e - e^{-1}) \xi_3^1 s_3}{2} \right) \end{aligned} \quad (2.18)$$

one can already note that this substitution implies more than binary interactions of order equal to the length of the considered memory of the net; however, as a proof of concept, let's develop a little bit the expression

$$\begin{aligned} = & - \left\{ \left[\left(\frac{(e^{-1} + e)}{2} \right)^2 + \frac{(e^{-1} + e)}{2} \left(\frac{(+e - e^{-1}) \xi_1^1 s_1}{2} + \frac{(+e - e^{-1}) \xi_2^1 s_2}{2} \right) \right. \right. \\ & \left. \left. + \frac{(+e - e^{-1})^2 \xi_1^1 s_1 \xi_2^1 s_2}{4} \right] \left[\frac{(e^{-1} + e)}{2} + \frac{(+e - e^{-1}) \xi_3^1 s_3}{2} \right] \right\} \end{aligned} \quad (2.19)$$

$$\begin{aligned} = & - \left\{ \left(\frac{(e^{-1} + e)}{2} \right)^3 + \left(\frac{(e^{-1} + e)}{2} \right)^2 \left(\frac{(+e - e^{-1}) (\xi_1^1 s_1 + \xi_2^1 s_2 + \xi_3^1 s_3)}{2} \right) \right. \\ & \left. + \frac{(+e - e^{-1})^3 \xi_1^1 s_1 \xi_2^1 s_2 \xi_3^1 s_3}{4} + \frac{(e^{-1} + e)}{2} \left(\frac{(+e - e^{-1}) \xi_1^1 s_1 \xi_3^1 s_3}{2} + \frac{(+e - e^{-1}) \xi_2^1 s_2 \xi_3^1 s_3}{2} \right) \right\} \end{aligned} \quad (2.20)$$

¹for the second one considerations are the same

The third term in the equality above shows already ternary interactions

$$+ \frac{(+e - e^{-1})^3 \xi_1^1 s_1 \xi_2^1 s_2 \xi_3^1 s_3}{4}$$

In general one can decompose the following energy function in this way

$$E = - \sum_{\mu=1}^m F \left(\sum_{i=1}^n \xi_i^\mu s_i \right) = - \sum_{\mu=1}^m e^{\sum_{i=1}^n \xi_i^\mu s_i} = - \sum_{\mu=1}^m \prod_{i=1}^n e^{\xi_i^\mu s_i} \quad (2.21)$$

$$= - \sum_{\mu=1}^m \prod_{i=1}^n \left(\frac{(e^{-1} + e)}{2} + \frac{(+e - e^{-1}) \xi_i^\mu s_i}{2} \right) \quad (2.22)$$

Which needs $m * (2^n - \binom{n}{2} - \binom{n}{1} - \binom{n}{0}) = m * (2^n - \binom{n}{2} - \binom{n}{1} - 1)$ auxiliary variables where m is the total number of memories and n is the length of the memories. This could be easily proved by induction but requires having proved two other statements. First statement is

$$\sum_{k=0}^n \binom{n}{k} = 2^n \quad (2.23)$$

Which can be proved also by induction. Given for $n \in N$ if $n > 0$

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k} \quad (2.24)$$

The base case $n = 0$ is

$$\sum_{k=0}^0 \binom{n}{k} = \binom{0}{0} = 1 = 2^0 \quad (2.25)$$

Let then $n > 0$ and suppose that the statement is true for $n - 1$. Consequently:

$$\sum_{k \in N} \binom{n}{k} = \sum_{k \in N} \left[\binom{n-1}{k-1} + \binom{n-1}{k} \right] = \sum_{k \in N} \binom{n-1}{k-1} + \sum_{k \in N} \binom{n-1}{k} = 2^{n-1} + 2^{n-1} = 2^n \quad (2.26)$$

Which proves the statement. Regarding second statement, one defines the set of all possible combination without repetition of $C(n, k)$ having $|C(n, k)| = \binom{n}{k}$. If one fixes n , then can define a tuple in which each element of the set is assigned to a random position. One can call this tuple as $C'(n, k)$. A function P over this tuple can then be defined:

$$P : C' \rightarrow N \quad (2.27)$$

Given $c \in C'(n, k)$ then $P(c) = \prod_{i=1}^k s_i$. Further, one can define a function $T : N \rightarrow C'$ which given the index i retrieves the corresponding element of the tuple. Finally, the composite function $S = T(P(i))$ is defined, over

$$S : N \rightarrow N \quad (2.28)$$

For easy manipulation author refers to S_i as the S function applied to the tuple $C'(k, i)$ Given a product of the form

$$\prod_{i=1}^n (a + b s_i) \quad (2.29)$$

where $a, b \in R$ and s_i a binary variable. The second statement which has to be proven is

$$\prod_{i=1}^n (a + b s_i) = \sum_{i=1}^n a^{n-i} b^i \sum_{j=1}^{|s_i|} S_i(j) + a^n \quad (2.30)$$

The demonstration is the following. For $n = 3$

$$\prod_{i=1}^n (a + bs_i) = (a + bs_1) (a + bs_2) (a + bs_3) \quad (2.31)$$

$$= a^3 + a^2b(s_1 + s_2 + s_3) + ab^2(s_1s_2 + s_1s_3 + s_2s_3) + b^3s_1s_2s_3 \quad (2.32)$$

$$= \sum_{i=1}^3 a^{n-i} b^i \sum_{j=1}^{|s_i|} s_j(i) + a^3 \quad (2.33)$$

Supposing property holds for n , then for the case $n + 1$

$$\left(\sum_{i=1}^n a^{n-i} b^i \sum_{j=1}^{|S_i|} S_j(i) + a^n \right) * (a + bs_{n+1}) \quad (2.34)$$

$$= a^{n+1} + \sum_{i=1}^n a^{n+1-i} b^i \sum_{j=1}^{|S_i|} S_j(i) + a^n bs_{n+1} + \sum_{i=1}^n a^{n-i} b^{i+1} s_{n+1} \sum_{j=1}^{|S_i|} S_j(i) \quad (2.35)$$

$$= \left(\sum_{i=1}^{n+1} a^{n+1-i} b^i \sum_{j=1}^{|S_i|} S_j(i) + a^{n+1} \right) \quad (2.36)$$

Having proved both statements, one can prove that the complexity of previous quantum formulation is $m * (2^n - \binom{n}{2} - \binom{n}{1} - 1)$. As it was previously showed for memories of length $n = 3$ an auxiliary variable is needed for each memory given each ternary interaction has to be reduced via [130] and in fact:

$$m * \left(2^3 - \binom{3}{2} - \binom{3}{1} - 1 \right) = m * 1 = m \quad (2.37)$$

Then one can suppose the complexity calculation is true for $n \in N$ and $n > 3$. For the case $n + 1$ one can define following set of equalities

$$a = \frac{(e^{-1} + e)}{2} \quad (2.38)$$

$$b = \frac{(+e - e^{-1})}{2} \quad (2.39)$$

$$- \sum_{\mu=1}^m \left(\sum_{i=1}^{n+1} a^{n+1-i} b^i \sum_{j=1}^{|S_i|} S_j(i) + a^{n+1} \right) = \sum_{\mu=1}^m \left(- \sum_{i=1}^{n+1} a^{n+1-i} b^i \sum_{j=1}^{|S_i|} S_j(i) - a^{n+1} \right) \quad (2.40)$$

Note that every term in $S_j(i)$ index is a correspondingly i -order monomial. Using [130] substitution one can reduce these polynomials to order two introducing a corresponding extra auxiliary variable when their order is greater than 2. Finally, simply note that the total number of monomial terms present in the quantum formulation is

$$\sum_{\mu=1}^m \left(\sum_{i=1}^{n+1} a^{n+1-i} b^i \binom{n}{k} + a^{n+1} \right) \quad (2.41)$$

$$= \sum_{\mu=1}^m \left(\sum_{i=1}^{n+1} a^{n+1-i} b^i \binom{n}{k} + a^{n+1} \binom{n}{0} \right) \quad (2.42)$$

$$= \sum_{\mu=1}^m \left(\sum_{i=0}^{n+1} a^{n+1-i} b^i \binom{n}{k} \right) = \sum_{\mu=1}^m 2^n = m 2^n \quad (2.43)$$

Given one not need to reduce single, double and costant term, the total number of auxiliary variables

needed is then

$$m * \left(2^n - \binom{n}{2} - \binom{n}{1} - \binom{n}{0} \right) \quad (2.44)$$

However, total number of qubits employed, meaning the space complexity ², is then $\Theta(2^n)$. This is a too massive increase of variables considering limited qubits capability of actual quantum annealer hardware. In next section a possibility to retrieve a cost-effective Dense Associative Memory with only binary interactions is discussed. ³

2.2 A cost effective dense associative memory activation functions derivation

A derivation of a cost-effective formulation for quantum dense associative memories will be shown thereafter. For simplicity the derivation will be shown starting from the beforementioned [148] work on binarization of interactions and considering rectified polynomial activation. Recall the energy exposed in the paper

$$E(t) = \left[\sum_{i=1}^{n_f} (v_i - i_i) g_i - L_v \right] + \left[\sum_{\mu=1}^{n_h} h_\mu f_\mu - L_h \right] - \sum_{\mu=1, i=1} f_\mu \xi_{\mu i} g_i \quad (2.45)$$

Two assumptions have to be introduced. Lagrangian additive from which follows

$$L_h = \sum_{\mu=1}^{n_h} F(h_\mu) \quad (2.46)$$

$$L_v = \sum_{i=1}^{n_f} G(v_i) \quad (2.47)$$

Where for our case $G(x) = |x|$. Second assumption regards the fact that dynamic of memory neuron is fast. From which follows

$$h_\mu = \sum_{i=1}^{n_f} \xi_{\mu i} g_i \quad (2.48)$$

For a Dense Associative Memory

$$s_i = g_i = g(v_i) = \frac{\partial L_v}{\partial v_i} = \text{Sign}[v_i] \quad (2.49)$$

And with a bipolar variable v_i

$$g_i = g(v_i) = \frac{\partial L_v}{\partial v_i} = \text{Sign}[v_i] = v_i \quad (2.50)$$

From here on it's safe to suppose v_i be bipolar variable, instead h_μ could not be treated as binary given previously mentioned steady state condition:

$$h_\mu = \sum_{i=1}^{n_f} \xi_{\mu i} g_i \quad (2.51)$$

²note thereafter space complexity is expressed only in term of number of qubits needed to represent each memory, considering number of memories m as a constant

³For brevity, author of present work decided to not discuss in main text a further possibility that one could be tempted to follow, which is presented in Appendix B.4

Taking the energy again

$$E(t) = \left[\sum_{i=1}^{n_f} (v_i - i_i) g_i - L_v \right] + \left[\sum_{\mu=1}^{n_h} h_\mu f_\mu - L_h \right] - \sum_{\mu=1, i=1} f_\mu \xi_{\mu i} g_i \quad (2.52)$$

Substituting $g_i = v_i$ and $h_\mu = \sum_{i=1}^{n_f} \xi_{\mu i} g_i$

$$E(t) = \left[\sum_{i=1}^{n_f} (v_i - i_i) v_i - L_v \right] + \left[\sum_{\mu=1}^{n_h} f_\mu \sum_{i=1}^{n_f} \xi_{\mu i} v_i - L_h \right] - \sum_{\mu=1, i=1} f_\mu \xi_{\mu i} v_i \quad (2.53)$$

$$= \left[\sum_{i=1}^{n_f} v_i^2 - i_i v_i - L_v \right] + \left[\sum_{\mu=1}^{n_h} f_\mu \sum_{i=1}^{n_f} \xi_{\mu i} v_i - L_h \right] - \sum_{\mu=1, i=1} f_\mu \xi_{\mu i} v_i \quad (2.54)$$

Substituting $L_v = \sum_{i=1}^{n_f} G(v_i)$ and $G(x) = |x|$

$$E(t) = \left[\sum_{i=1}^{n_f} v_i v_i - i_i v_i - \sum_{i=1}^{n_f} |v_i| \right] + \left[\sum_{\mu=1}^{n_h} f_\mu \sum_{i=1}^{n_f} \xi_{\mu i} v_i - L_h \right] - \sum_{\mu=1, i=1} f_\mu \xi_{\mu i} v_i \quad (2.55)$$

$$= [n_f - i_i v_i - n_f] + \left[\sum_{\mu=1}^{n_h} f_\mu \sum_{i=1}^{n_f} \xi_{\mu i} v_i - L_h \right] - \sum_{\mu=1, i=1} f_\mu \xi_{\mu i} v_i \quad (2.56)$$

$$= - \sum_{i=1}^{n_f} i_i v_i + \left[\sum_{\mu=1}^{n_h} f_\mu \sum_{i=1}^{n_f} \xi_{\mu i} v_i - L_h \right] - \sum_{\mu=1, i=1} f_\mu \xi_{\mu i} v_i \quad (2.57)$$

In the original paper demonstration two terms $\sum_{\mu=1}^{n_h} f_\mu \sum_{i=1}^{n_f} \xi_{\mu i} v_i$ and $-\sum_{\mu=1, i=1} f_\mu \xi_{\mu i} v_i$ cancel out leading L_h return to nonlinear function of sum of term. However, in present case, it is necessary to deal with lack of steady state condition; given

$$h_\mu = \sum_{i=1}^{n_f} \xi_{\mu i} v_i \quad (2.58)$$

and being the product $\xi_{\mu i} v_i$ equal to -1 or $+1$, the value h_μ is comprised between $[-n_f, +n_f]$. The representation of this interval could be stated in two complement encoding

$$[-2^{r-1}, 2^{r-1} - 1] = [-n_f, +n_f] \quad (2.59)$$

Using $r = \log(n_f + 1) + 1$ auxilliary variables. Note that given h_μ being expressed via two complement encoding, this represents with minimal number of binary variable the numerical interval. So h_μ can be substituted in the energy function by $z_\mu = \sum_{l=0}^{r-1} 2^l \varepsilon_{\mu, l} - 2^r \varepsilon_{\mu, r}$. Adding to the total energy $(\sum_{i=1}^{n_f} \xi_{\mu i} v_i - z_\mu)^2$ one enforces z_μ equal to $\sum_{i=1}^{n_f} \xi_{\mu i} v_i = h_\mu$.

Follow now original paper derivation, having recovered steady state condition

$$E(t) = - \sum_{i=1}^{n_f} i_i v_i + \left[\sum_{\mu=1}^{n_h} f_\mu \sum_{i=1}^{n_f} \xi_{\mu i} v_i - L_h \right] - \sum_{\mu=1, i=1} f_\mu \xi_{\mu i} v_i \quad (2.60)$$

with simplification stated above

$$= - \sum_{i=1}^{n_f} i_i v_i - L_h \quad (2.61)$$

With

$$L_h = \sum_{\mu=1}^{n_h} F(h_\mu) \quad (2.62)$$

Adding the following term to the energy to enforce $h_\mu = z_\mu$

$$+ \left(\sum_{i=1}^{n_f} \xi_{\mu i} v_i - \sum_{l=0}^{r-1} 2^l \varepsilon_{\mu, l} - 2^r \varepsilon_{\mu, r} \right)^2 \quad (2.63)$$

one obtains

$$E(t) = - \sum_{i=1}^{n_f} i_i v_i - \sum_{\mu=1}^{n_h} F \left(\sum_{l=0}^{r-1} 2^l \varepsilon_{\mu, l} - 2^r \varepsilon_{\mu, r} \right) + \sum_{\mu=1}^{n_h} \left(\sum_{i=1}^{n_f} \xi_{\mu i} v_i - \sum_{l=0}^{r-1} 2^l \varepsilon_{\mu, l} - 2^r \varepsilon_{\mu, r} \right)^2 \quad (2.64)$$

Introducing

$$n_h (\log(n_f + 1) + 1) \quad (2.65)$$

auxiliary variables. Now the question is how to model efficiently F function which appears in original and [148] energy formulation. This will lead to abandon the [148] formulation toward two next formulations. Note that in this section, y_μ is used to denote dot product between memory μ and the input and m is used to denote total number of memories: $y_\mu = \sum_{i=1}^n \xi_i^\mu s_i$. Further, the presented energy function are defined over QUBO, or binary variables. However memories and inputs are provided in bipolar fashion so the equality 2.3 is applied as a substitution to every qubit s_i appearing in the energy, leading to $y_\mu = \sum_{i=1}^n \xi_i^\mu * (2x_i - 1)$, having $x_i \in \{0, 1\}$. Start considering rectified polynomial; its total energy function can be implemented in two step. First, inserting in energy function a term that measures the value to which each qubit/memory product corresponds

$$\sum_{\mu=1}^m \gamma_\mu (y_\mu - z_\mu)^2 \quad (2.66)$$

where

$$z_\mu = \sum_{l=0}^{r-1} 2^l \varepsilon_{\mu, l} - 2^r \varepsilon_{\mu, r} \quad (2.67)$$

γ_μ is simply a real-valued constant that can be set to enforce the equality constraint. Note that this constant, and the consequent penalty for violation, should be greater than the possible negative energy contribution that the annealer can achieve violating it, which is general the negative contribution to energy that a full memory-neuron match do to the total energy (in the ReLU case, it is the square of the maximum possible value that a dot product memory/neuron can achieve). Two complement encoding, which is a minimal encoding, allows to evaluate only $\varepsilon_{\mu, r}$ to understand if the product memory/qubit is positive. That introduce r auxiliary variables for each memory, where $r = \log(n + 1) + 1$. Secondly, a term in the energy function is introduced which if the product qubit/memory is positive adds a quadratic terms equal to the square of this product

$$\sum_{\mu=1}^m (1 - \varepsilon_{\mu, r}) (y_\mu)^2 \quad (2.68)$$

This approach introduces $i < j$ auxiliary variables for each memory:

$$\sum_{\mu=1}^m (1 - \varepsilon_{\mu,r}) (y_{\mu})^2 = \sum_{\mu=1}^m (1 - \varepsilon_{\mu,r}) \left(\sum_{i=1}^n \xi_i^{\mu} * (2x_i - 1) \right)^2 \quad (2.69)$$

$$= \sum_{\mu=1}^m (1 - \varepsilon_{\mu,r}) \left(\sum_{i=1}^n (\xi_i^{\mu} * (2x_i - 1))^2 + 2 \sum_{i < j} (\xi_i^{\mu} * (2x_i - 1) * \xi_j^{\mu} * (2x_j - 1)) \right) \quad (2.70)$$

$$= \sum_{\mu=1}^m (1 - \varepsilon_{\mu,r}) \left(\sum_{i=1}^n (4(\xi_i^{\mu} x_i)^2 + (\xi_i^{\mu})^2 - 4(\xi_i^{\mu})^2 x_i) \right. \\ \left. + 2 \sum_{i < j} (4\xi_i^{\mu} \xi_j^{\mu} x_i x_j - 2\xi_i^{\mu} \xi_j^{\mu} x_i - 2\xi_i^{\mu} \xi_j^{\mu} x_j + \xi_i^{\mu} \xi_j^{\mu}) \right) \quad (2.71)$$

Note that if one develops the last equality, the only term with more than binary interaction is $-\sum_{i < j} \xi_i^{\mu} \xi_j^{\mu} \varepsilon_{\mu,r} x_i x_j$ which requires in total $m(n^2 - n)/2$ auxiliary variables and has a space complexity of $\Theta(m(n^2 - n)/2 + mn) = \Theta(n^2)$. As an alternative with less space complexity, one can instead introduce two terms. The following one uses first bit of the encoding as indicator to check if product memory/input y is positive, meaning $\varepsilon_{\mu,r}$ is zero if y is negative, and sets a new variable to the encoding value or zero otherwise depending of this indicator. There are $r-1$ new variables $t_{\mu,l}$ for each memory.

$$\sum_{\mu=1}^m \sum_{l=0}^{r-1} \gamma_{\mu,l} \cdot (t_{\mu,l} - (1 - \varepsilon_{\mu,r}) \varepsilon_{\mu,l})^2 \quad (2.72)$$

$$= \sum_{\mu=1}^m \sum_{l=0}^{r-1} \gamma_{\mu,l} \cdot (t_{\mu,l} - \varepsilon_{\mu,l} + \varepsilon_{\mu,l} \varepsilon_{\mu,r})^2 \quad (2.73)$$

$$= \sum_{\mu=1}^m \sum_{l=0}^{r-1} \gamma_{\mu,l} \left((t_{\mu,l})^2 + (-\varepsilon_{\mu,l})^2 + (\varepsilon_{\mu,l} \varepsilon_{\mu,r})^2 - 2t_{\mu,l} \varepsilon_{\mu,l} + 2t_{\mu,l} \varepsilon_{\mu,l} \varepsilon_{\mu,r} - 2\varepsilon_{\mu,r} (\varepsilon_{\mu,l})^2 \right) \quad (2.74)$$

$$= \sum_{\mu=1}^m \sum_{l=0}^{r-1} \gamma_{\mu,l} (t_{\mu,l} + \varepsilon_{\mu,l} + \varepsilon_{\mu,l} \varepsilon_{\mu,r} - 2t_{\mu,l} \varepsilon_{\mu,l} - 2\varepsilon_{\mu,r} \varepsilon_{\mu,l} + 2t_{\mu,l} \varepsilon_{\mu,l} \varepsilon_{\mu,r}) \quad (2.75)$$

The term $+2t_{\mu,l} \varepsilon_{\mu,l} \varepsilon_{\mu,r}$ in the last equation can be decomposed in binary interaction via [130] presented before. This approach introduces other $r - 1$ auxiliary variables $t'_{\mu,l}$. Also for enforcing this constraint constants $\gamma_{\mu,l}$ are introduced (same observations of γ_{μ} constants hold). After that subtract to total energy the square of the value y which is equal to product of the new introduced variables and encoding variables

$$\sum_{\mu=1}^m \sum_{l=0}^{r-1} 2^l t_{\mu,l} \left(\sum_{i=0}^{r-1} 2^i \varepsilon_{\mu,i} \right) \quad (2.76)$$

So the total energy function is

$$E = \sum_{\mu=1}^m \left(\gamma_{\mu} (y_{\mu} - z_{\mu})^2 + \sum_{l=0}^{r-1} \gamma_{\mu,l} (t_{\mu,l} + \varepsilon_{\mu,l} \varepsilon_{\mu,r} - 2t'_{\mu,l} t_{\mu,l} - 2t'_{\mu,l} \varepsilon_{\mu,l} - 2t'_{\mu,l} \varepsilon_{\mu,r} + 4t'_{\mu,l}) \right. \\ \left. - \sum_{l=0}^{r-1} 2^l t_{\mu,l} \left(\sum_{i=0}^{r-1} 2^i \varepsilon_{\mu,i} \right) \right) \quad (2.77)$$

This formulation introduces in total $m \cdot (3r - 2) = m(3\log(n + 1) + 1)$ auxiliary variables with a space complexity of $\Theta(m(3\log(n + 1) + 1) + m * n) = \Theta(m(3\log(n + 1) + 1 + n)) = \Theta(n)$.

Another method with different scalability arises if one considers the fact the dot product between

a bipolar memory array and the qubits array, both of length n , can result in only $n + 1$ different values. The demonstration is trivial. The dot product of two vectors $\mathbf{a} = [a_1, a_2, \dots, a_n]$ and $\mathbf{b} = [b_1, b_2, \dots, b_n]$ is defined as:

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^n a_i b_i = a_1 b_1 + a_2 b_2 + \dots + a_n b_n \quad (2.78)$$

Which can be seen as the sum of the elementwise product of the two arrays. It is clear that the elementwise product of two bipolar arrays is still a bipolar array. The number of possible bipolar arrays of length n is 2^n but lot of them admit the same sum, as said before $n + 1$. If this last statement is proved, consequently the dot product of two bipolar array admit only $n + 1$ values. To prove it via induction:

Define the function $L : B^n \rightarrow N$ as $L(n)$ that count the number of possible different valued sum of a bipolar array of length n . For an array of length $n = 1$ only two possible values are possible, $+1$ and -1 , and $L(n) = n + 1 = 1 + 1 = 2$ so base case is proved. Supposed this is true for $n = k$

$$L(k) = k + 1 \quad (2.79)$$

For the case $k + 1$

$$L(k + 1) \stackrel{?}{=} (k + 1) + 1 = k + 2 \quad (2.80)$$

Given an array of length n , one can count the number of elements with value $+1$, which is denoted as l thereafter. The number of elements with value -1 is $n - l$. It is clear that two arrays with same length n and same l have the same sum, from ordinary properties of summation. The number l can span from 0 to n . Consequently

$$L(n) = \sum_{i=0}^n 1 = n + 1 \quad (2.81)$$

In the case at hand

$$L(k + 1) = \sum_{i=0}^k 1 + 1 = \sum_{i=0}^{k+1} 1 = k + 2 \quad (2.82)$$

which proves the statement. Considering for simplicity to put all possible results of the dot product neuron/memory in an array $\mathbf{k}$ of length $n+1$. To construct this new formulation, first, a term is inserted in the energy function which measures the value which corresponds to the value of qubit/memory dot product

$$\sum_{\mu=1}^m \gamma_{\mu} (y_{\mu} - z_{\mu})^2 \quad (2.83)$$

where

$$z_{\mu} = \sum_{l=1}^{n+1} k_l \varepsilon_{\mu,l} \quad (2.84)$$

$\varepsilon_{\mu,l}$ is a binary variable and k_l is the l -th element of $\mathbf{k}$. This step introduces $n+1$ variables for each memory. Then, one adds a term that adds a quadratic contribution to the energy function if the corresponding $\varepsilon_{\mu,l}$ is one

$$\sum_{\mu=1}^m \sum_{l=l^+}^n (\varepsilon_{\mu,l}) (k_l)^2 \quad (2.85)$$

Where l^+ is lower index of $\mathbf{k}$ array which has a positive value, having rectified quadratic polynomial add a quadratic contribution to energy only if dot product memory/input is positive. So the total energy function is

$$E = \sum_{\mu=1}^m \left(\gamma_{\mu} (y_{\mu} - z_{\mu})^2 + \gamma'_{\mu} \left(\sum_{l=1}^{n+1} \varepsilon_{\mu,l} - 1 \right)^2 - \sum_{l=l^+}^{n+1} (\varepsilon_{\mu,l}) (k_l)^2 \right) \quad (2.86)$$

note that second term enforce the fact that only a qubit which represents dot product total value is active at a time, γ'_μ is a real constant to enforce this constraint (same observations of γ_μ constants hold). Complexively this method adds $m(n+1)$ auxilliary variables, with a total space complexity of $\Theta(m(n+1) + mn) = \Theta(m(2n+1)) = \Theta(mn) = \Theta(n)$.

From this last approach one could conveniently obtain all other activations of dense associative memories: exponential, tanh, logistic, attention and spherical. Exponential energy is analogous rectified polynomial having a switch energy value when $y < 0$. An example of possible exponential energy following this second method is

$$E = \sum_{\mu=1}^m \left(\gamma_\mu (y_\mu - z_\mu)^2 + \gamma'_\mu \left(\sum_{l=1}^{n+1} \varepsilon_{\mu,l} - 1 \right)^2 - \sum_{l=1}^{n+1} (\varepsilon_{\mu,l}) e^{k_l} \right) \quad (2.87)$$

Modelling tanh activations, which value is computed with $\ln(1 + e^x)$, requires a similiar approach of exponential

$$E = \sum_{\mu=1}^m \left(\gamma_\mu (y_\mu - z_\mu)^2 + \gamma'_\mu \left(\sum_{l=1}^{n+1} \varepsilon_{\mu,l} - 1 \right)^2 - \sum_{l=1}^{n+1} (\varepsilon_{\mu,l}) \ln(1 + e^{k_l}) \right) \quad (2.88)$$

A logistic activation can be obtained simply as

$$\cosh(x) = \frac{e^x + e^{-x}}{2} \quad (2.89)$$

$$\ln\left(\frac{e^x + e^{-x}}{2}\right) = \ln(1/2) + \ln(e^x + e^{-x}) = \ln(e^x (1 + e^{-2x})) = x + \ln(1 + e^{-2x}) \quad (2.90)$$

From this result one proceeds like in tanh. Spherical energy function can be formulated instead as

$$E(t) = - \sum_{\mu} F \left(\sum_i \xi_{\mu i} \frac{v_i}{\sqrt{\sum_j v_j^2}} \right) \quad (2.91)$$

$\sqrt{\sum_j v_j^2}$ is a constant term in a bipolar implementation (the square of a bipolar variable is always equal to 1, so its summation is a constant), F can be every possible nonlinear function mentioned above. As a side note, here a possible attention formulation in a quantum dense associative memory fashion is provided. Recall attention formulation is

$$E = \frac{1}{2} \sum_{i=1}^{n_f} v_i^2 - \log \left(\sum_{\mu} \exp \left(\sum_i \xi_{\mu i} v_i \right) \right) \quad (2.92)$$

where $\frac{1}{2} \sum_{i=1}^{n_f} v_i^2$ is a constant so the only term that has to be treated is

$$\log \left(\sum_{\mu} \sum_i \exp(\xi_{\mu i} v_i) \right) \quad (2.93)$$

being logarithm a monotonic function depends on

$$\sum_{\mu} \exp \left(\sum_i \xi_{\mu i} v_i \right) \quad (2.94)$$

A simple way to deal with it could be calculating exponential term inside logarithm and apply loga-

rithm after energy minimization process.

3 Quantum deep neural networks via quantum dense associative memories

In this section it will be presented how train (deep) quantum feedforward neural networks via quantum dense associative memories formulations developed in previous section.

Recall [147] having defined dense associative memory with energy

$$E = - \sum_{\mu=1}^m F(\xi_i^\mu s_i) \quad (3.1)$$

With $F(x)$ is some smooth function, summation over index i is assumed, it is possible to define the network, represented in figure 3.1 as A, having n_c neurons x_α used for classification and v_i neurons clamped, meaning treated as a constants to input. Note that α indicates which class is related with this neuron value. This network evolves with following asynchronous update of neurons (as represented in figure 3.1 in which network evolves toward B)

$$c_\alpha = g \left[\beta \sum_{\mu=1}^m \left(F \left(-\xi_\alpha^\mu x_\alpha + \sum_{\gamma \neq \alpha} \xi_\gamma^\mu x_\gamma + \sum_{i=1}^n \xi_i^\mu v_i \right) - F \left(\xi_\alpha^\mu x_\alpha + \sum_{\gamma \neq \alpha} \xi_\gamma^\mu x_\gamma + \sum_{i=1}^n \xi_i^\mu v_i \right) \right) \right] \quad (3.2)$$

in this case authors of [147] used x_α to indicate only a single classification neuron while x_γ are optional classification neurons that are constant during evolution for a specific classification neuron

$$c_\alpha \approx g \left[\beta \sum_{\mu=1}^m F' \left(\sum_{i=1}^n \xi_i^\mu v_i \right) (-2\xi_\alpha^\mu x_\alpha) \right] = g \left[\sum_{\mu=1}^m \xi_\alpha^\mu F' \left(\sum_{i=1}^n \xi_i^\mu v_i \right) \right] = g \left[\sum_{\mu=1}^m \xi_\alpha^\mu f \left(\sum_{i=1}^n \xi_i^\mu v_i \right) \right] \quad (3.3)$$

Provided the activation function from the visible layer to the hidden layer is equal to the derivative of the energy function $f(x) = F'(x)$, an example of this associative memory with two feature neurons v_1, v_2 , binary classification and two memories is given in figure 3.1.

Authors of [147] started with initial state of -1 for classification neurons that lead to the dynamic shown in figure 3.1. however this is not a concern for the duality property to hold. In fact, McLaurin polynomial expansion requires only $n - 1$ differentiability for an n order polynomial expansion, ϵ is not necessary greater than zero and F and F' are defined for positive and negative values. So in principle also an evolution from neuron +1 to -1 is admitted. In fact when one resorts to a quantum annealer, the state in which the annealer is initialized is not known apriori: it can be in $x_\alpha = -1$ or $x_\alpha = 1$ initial state. The final energy state of annealer follows either one of figure 3.2 arrows starting from one of the states reported in figure.

Note how this formulation embeds the entire model in annealer allowing encoding in c_α qubits the output of forward pass of the network; this allow its usage for sampling and to be used for neural network optimization, see next section.

3.1 Training shallow and deep neural networks in energy minimization framework

The key question is how training this network in energy minimization framework. A 0-1 loss like term C in the form of squared error for a single datapoint can be simply provided with

$$C = (c_\alpha - y^*)^2 \quad (3.4)$$

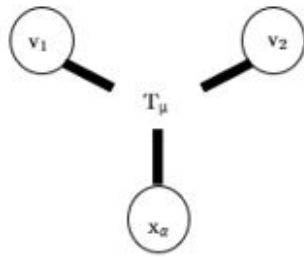
Notation

$$d = \sum_{\mu=1}^2 \left(\xi_{\alpha}^{\mu} x_{\alpha} F' \left(\sum_{i=1}^2 \xi_i^{\mu} v_i \right) \right)$$

$$A = F' \left(\sum_{i=1}^2 \xi_i^1 v_i \right) \quad B = F' \left(\sum_{i=1}^2 \xi_i^2 v_i \right)$$

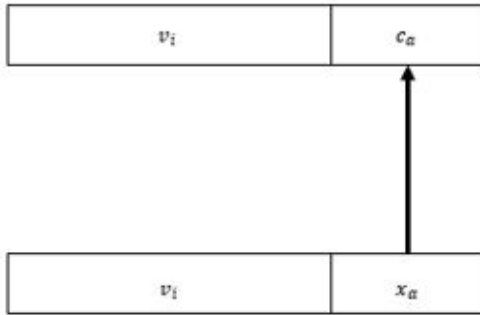
$$a = \sum_{i=1}^2 \xi_i^1 v_i \quad b = \sum_{i=1}^2 \xi_i^2 v_i$$

Associative Network architecture



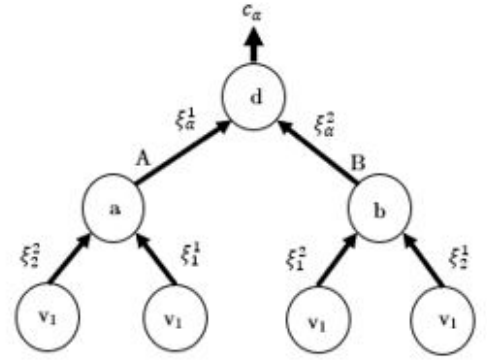
$$E = - \sum_{\mu} T_{\mu} v_1 v_2 x_{\alpha} = - \sum_{\mu} F \left(\sum_i \xi_{\mu i} \sigma_i \right)$$

Associative evolution



$$c_{\alpha} = \text{Sign} \left[\beta \sum_{\mu=1}^2 \left(F \left(-\xi_{\alpha}^{\mu} x_{\alpha} + \sum_{i=1}^2 \xi_i^{\mu} v_i \right) - F \left(\xi_{\alpha}^{\mu} x_{\alpha} + \sum_{i=1}^2 \xi_i^{\mu} v_i \right) \right) \right]$$

Feedforward Network architecture

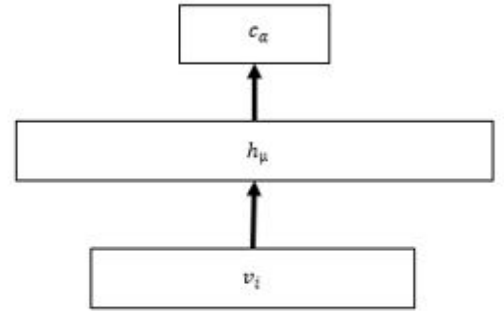


Feedforward evolution

C)

B)

A)



$$c_{\alpha} = \text{Sign} \left[\sum_{\mu=1}^2 \left(\xi_{\alpha}^{\mu} x_{\alpha} F' \left(\sum_{i=1}^2 \xi_i^{\mu} v_i \right) \right) \right]$$

Figure 3.1: Visualization of dense associative memory as associative network and feed-forward with its relative equations. Note however that ξ_i^{μ}

Markov dynamic of classical dense associative network

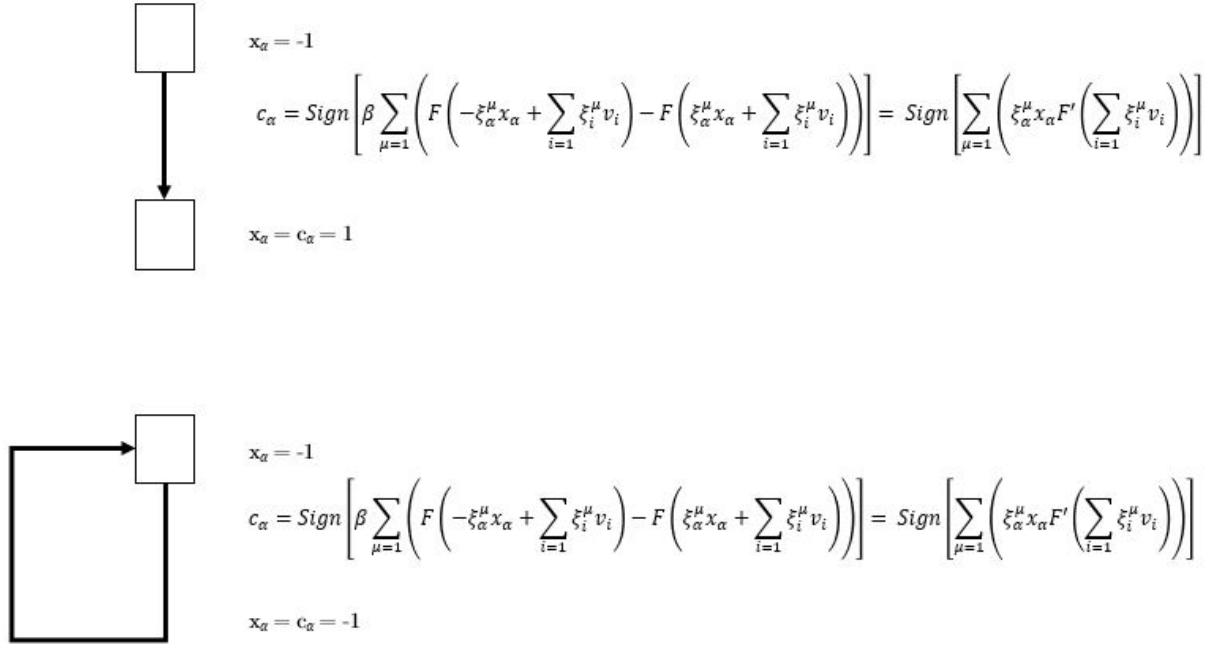


Figure 3.2: Markov dynamic of dense associative memory as feed-forward neural network

Markov dynamic of quantum associative network

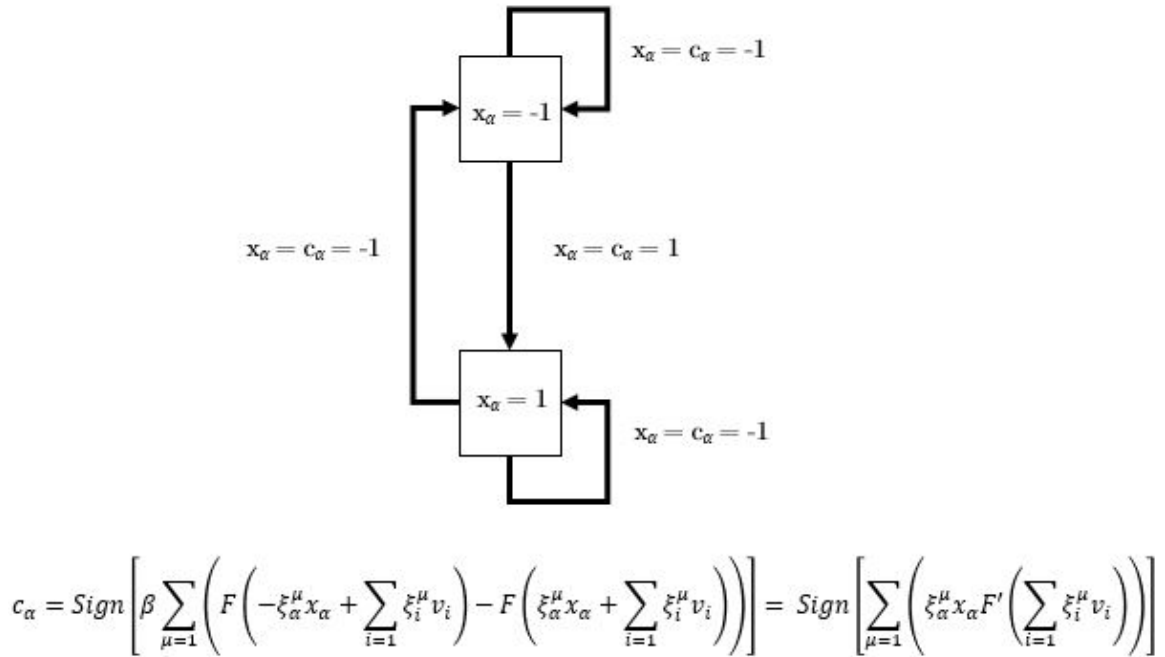


Figure 3.3: Markov dynamic of dense associative memory as feed-forward neural network implemented in annealer

or in multi-class case as done in [147]

$$C = \sum_{\alpha=1}^{n_c} (c_{\alpha} - y_{\alpha}^*)^2 \quad (3.5)$$

Where y_{α}^* is set to -1 for wrong classes, +1 true class. It is straightforward to extend this possibility to other typical machine learning tasks. A simple multi-class classification setting is achieved with more than one neuron c_{α} . Given c_{α} is used to provide binary classification value and being classes in multi-class classification mutual exclusive, it is possible simply enforcing mutual exclusion of positive c_{α} via a penalty term added to global network energy; thereafter n_c is used to denote number of classes:

$$k_{\alpha} \left(1 - \sum_{\alpha=1}^{n_c} \frac{c_{\alpha} + 1}{2} \right)^2 \quad (3.6)$$

Where k_{α} is a suitable positive coefficient. Note that this penalty is minimized only when only one neuron is active and does not add auxiliary variables. Similarly a multi-label classification setting is obtainable simply removing the penalty, while a regression setting is obtainable removing the penalty and considering c_{α} as part of a real representation of a target function. Obviously if the number of datapoint is greater than one than the loss is obtained summing this square error term for every datapoint in the dataset, e.g. for the case of binary classification if more than one datapoint is present

$$C = \sum_{\delta}^{|D|} (c_{\alpha,\delta} - y_{\delta}^*)^2 \quad (3.7)$$

and in multiclass setting

$$C = \sum_{\delta}^{|D|} \sum_{\alpha=1}^{n_c} (c_{\alpha,\delta} - y_{\alpha\delta}^*)^2 \quad (3.8)$$

This total loss could be simply added to the total energy

$$E_{\text{TOT}} = E(E_1, \mathbf{w}) + C \quad (3.9)$$

Where C is one of the possible ways to measure label discrepancy mentioned before. Note that training a neural network via minimization of beforementioned energy for a multiclass/multilabel setting can be done in a single annealing step, without employing complex schema like one vs. all or one vs. one as one necessarily has to do for qSVM formulation of [245]. Note that exploiting Markov dynamic neural network is not inserted in 0-1 Loss, instead only classification qubits are exploited to compute loss results, not introducing more than binary interactions, E_1 is the energy of dense associative memory in a suitable representation for a quantum annealer, E is a transformation of the energy provided as first function argument, substituting to original bipolar memories representations ξ_i^{μ} a weight representation w_n provided in $\mathbf{w}$. Note in fact that in this case weights can be made binary achieving binary neural networks or instead could be extended via a binary encoding as used in [245]. In this last case the n weights of a neural network could be defined as

$$w_n = \sum_{l=0}^{d-1} b^l s_{dn+l} \quad (3.10)$$

where $s_{dn+l} \in \{-1, +1\}$ are bipolar variables, d is the number of binary variables to encode w_n , and b is the base used for the encoding.

As an example here is illustrated procedure to do training minimization in dense associative memory

framework

$$E_{TOT} = E(E_1, W) + C \quad (3.11)$$

$$= E(E_1, W) + (c_\alpha - y_\alpha^*)^2 \quad (3.12)$$

note that for simplicity it is temporarily considered single data-point case. Considering as example a ReLu activation function as presented in previous section

$$E_1 = \sum_{\mu=1}^m \left(\gamma_\mu (y_\mu - z_\mu)^2 + \gamma'_\mu \left(\sum_{l=1}^{n+1} \varepsilon_{\mu,l} - 1 \right)^2 - \sum_{l=l^+}^{n+1} \varepsilon_{\mu,l} (k_l)^2 \right) \quad (3.13)$$

where y_μ in original associative setting is the dot product spin s_i , $y_\mu = \sum_i^n \xi_i^\mu s_i$. Similarly to [147] setting of image classification inputs are considered a constant v_i and weights, that have to be learned, can be treated as spin values: the result of energy minimization of the loss E_{TOT} via QA provides than optimal weights to be employed for the task at hand. Note that in the present case

$$y_\mu = \sum_{l=0}^{d-1} b^l s_{0,l} s_\alpha + \sum_{i=1}^n \sum_{l=0}^{d-1} b^l s_{i,l} v_i \quad (3.14)$$

It is clear that if input data $\mathbf{v}$ can assume any possible real value in its entry it is necessary to introduce an approximation so that one can enforce the constraint

$$\gamma_\mu (y_\mu - z_\mu)^2 \quad (3.15)$$

having

$$z_\mu = \sum_{l=1}^{|k|} \varepsilon_{\mu,l} k_l \quad (3.16)$$

In particular one have to decide a number $|k|$ of predefined k_l values (hyperparameter) and the constraint is then minimized when the difference $y_\mu - \sum_{l=1}^{|k|} \varepsilon_{\mu,l} k_l$ is as low as possible so that $\varepsilon_{\mu,l}$ signals which k_l value is closer to the input. To do classification a single neuron has to be reserved for output s_α and an hyperparameter β has to be used as in [147]:

$$E(E_1, W) = \sum_{\mu=1}^m \gamma_\mu \left(\sum_{l=0}^{d-1} b^l s_{0,l} s_\alpha + \sum_{i=1}^n \sum_{l=0}^{d-1} b^l s_{i,l} v_i - z_\mu \right)^2 + \sum_{\mu=1}^m \gamma'_\mu \left(\sum_{l=1}^{|k|} \varepsilon_{\mu,l} - 1 \right)^2 - \beta \sum_{\mu=1}^m \sum_{l=l^+}^{|k|} \varepsilon_{\mu,l} (k_l)^2 \quad (3.17)$$

note here neuron index in y_μ term starts with one supposing 0 index reserved for classification qubit weights. Note that this term presents more than binary interactions that have to be decomposed

$$\begin{aligned} \left(\sum_{l=0}^{d-1} b^l s_{0,l} s_\alpha + \sum_{i=1}^n \sum_{l=0}^{d-1} b^l s_{i,l} v_i - z_\mu \right)^2 &= \left(\sum_{l=0}^{d-1} b^l s_{0,l} s_\alpha \right)^2 + \left(\sum_{i=1}^n \sum_{l=0}^{d-1} b^l s_{i,l} v_i \right)^2 \\ &\quad + z_\mu^2 + 2 \left(\sum_{l=0}^{d-1} b^l s_{0,l} s_\alpha \right) \left(\sum_{i=1}^n \sum_{l=0}^{d-1} b^l s_{i,l} v_i \right) \\ &\quad - 2 \left(\sum_{l=0}^{d-1} b^l s_{0,l} s_\alpha \right) (z_\mu) - 2 \left(\sum_{i=1}^n \sum_{l=0}^{d-1} b^l s_{i,l} v_i \right) z_\mu \end{aligned} \quad (3.18)$$

Only first, fourth and fifth term have more than binary interaction, the other terms arise also in Dense Associative Memory framework and are square of single spin terms showing consequently only binary

interaction

$$\begin{aligned}
&= \left[\left(\sum_{l=0}^{d-1} b^{2l} (s_{0,l} s_\alpha)^2 + 2 \sum_{i < j}^{d-1} b^{i+j} s_{0,i} s_{0,j} \right) + \left(\sum_{i=1}^n \sum_{l=0}^{d-1} b^l s_{i,l} v_i \right)^2 + z_\mu^2 \right. \\
&\quad + \left(2 \sum_{i=1}^n \sum_{l=0}^{d-1} \sum_{l'=0}^{d-1} v_i b^{l'+l} s_{0,l'} s_\alpha s_{i,l} \right) - \left(2 \sum_{l=0}^{d-1} \sum_{l'=1}^{|\mathbf{k}|} b^l s_{0,l} s_\alpha \varepsilon_{\mu,l'} k_{l'} \right) \\
&\quad \left. - 2 \left(\sum_{i=1}^n \sum_{l=0}^{d-1} b^l s_{i,l} v_i \right) z_\mu \right] \quad (3.19)
\end{aligned}$$

The two terms with more than binary interaction are $+ \left(2 \sum_{i=1}^n \sum_{l=0}^{d-1} \sum_{l'=0}^{d-1} v_i b^{l'+l} s_{0,l'} s_\alpha s_{i,l} \right)$ require $n * d^2$ auxiliary variables and $- \left(2 \sum_{l=0}^{d-1} \sum_{l'=1}^{|\mathbf{k}|} b^l s_{0,l} s_\alpha \varepsilon_{\mu,l'} k_{l'} \right)$ requires $d * |\mathbf{k}|$ auxiliary variables. Note however that d and $|\mathbf{k}|$ are hyperparameters provided by user so could be treated as constants in total space complexity calculation. However it has to be noted that this complexity can explode especially if one wants to minimize loss in respect of multiple data having a total space complexity of $\theta(|\mathcal{D}| * (d * |\mathbf{k}| + n * d^2))$. This multiple interactions arise because of presence of two qubits interaction inside square components of the energy. Given classification qubits have to be preserved to account for the loss term, a possibility to reduce complexity can be oversee weights of classification neurons, which is exploited in next section, or use preset weights. This second possibility is illustrated thereafter in case weights are used to encode multiple values

$$E(E_1, W) = \sum_{\mu=1}^m \gamma_\mu \left(\sum_{l=0}^{d-1} b^l s_\alpha + \sum_{i=1}^n \sum_{l=0}^{d-1} b^l s_{i,l} v_i - z_\mu \right)^2 + \sum_{\mu=1}^m \gamma'_\mu \left(\sum_{l=1}^{|\mathbf{k}|} \varepsilon_{\mu,l} - 1 \right)^2 - \beta \sum_{\mu=1}^m \sum_{l=l^+}^{|\mathbf{k}|} \varepsilon_{\mu,l} (k_l)^2 \quad (3.20)$$

and in this case number of auxiliary variable needed is $|\mathcal{D}| * m * |\mathbf{k}|$. However there is still a more convenient approach to treat training of deep neural network using Dense Associative Memories in QA framework. As it will be apparent even if this method can not be easily exploited for representing deep neural network in QA framework, as will be apparent in next section, it has the the remarkable advantage of being not dependant on the number of data (which is not the case for any other neural network and qSVM [245] approach on QA). From duality it is known that

$$c_\alpha = \text{Sign} \left[\beta \sum_{\mu=1} \left(F \left(-\xi_\alpha^\mu x_\alpha + \sum_{i=1} \xi_i^\mu v_i \right) - F \left(\xi_\alpha^\mu x_\alpha + \sum_{i=1} \xi_i^\mu v_i \right) \right) \right] \quad (3.21)$$

$$= \text{Sign} \left[\sum_{\mu=1} \left(\xi_\alpha^\mu x_\alpha F' \left(\sum_{i=1} \xi_i^\mu v_i \right) \right) \right] \quad (3.22)$$

Noting that

$$\beta \sum_{\mu=1} \left(F \left(-\xi_\alpha^\mu x_\alpha + \sum_{i=1} \xi_i^\mu v_i \right) - F \left(\xi_\alpha^\mu x_\alpha + \sum_{i=1} \xi_i^\mu v_i \right) \right) = \sum_{\mu=1} \left(\xi_\alpha^\mu x_\alpha F' \left(\sum_{i=1} \xi_i^\mu v_i \right) \right) \quad (3.23)$$

and for c_α being a bipolar variable

$$\beta \sum_{\mu=1} \left(F \left(-\xi_\alpha^\mu + \sum_{i=1} \xi_i^\mu v_i \right) - F \left(\xi_\alpha^\mu + \sum_{i=1} \xi_i^\mu v_i \right) \right) = \sum_{\mu=1} \left(\xi_\alpha^\mu F' \left(\sum_{i=1} \xi_i^\mu v_i \right) \right) \quad (3.24)$$

Returning to the problem of obtaining shallow neural network training on QA one can define a cost function

$$C = \sum_{\delta} \left(\beta \sum_{\mu=1}^D \left(F \left(-\xi_{\alpha}^{\mu} + \sum_{i=1}^n \xi_i^{\mu} v_i \right) - F \left(\xi_{\alpha}^{\mu} + \sum_{i=1}^n \xi_i^{\mu} v_i \right) \right) - y_{\delta}^* \right)^2 \quad (3.25)$$

Recalling the energy for a Dense Associative Memory ReLU formulation then

$$\begin{aligned} F \left(\xi_{\alpha}^{\mu} + \sum_{i=1}^n \xi_i^{\mu} v_i \right) &= \sum_{\mu=1}^m \gamma_{\mu} \left(\sum_{l=0}^{d-1} b^l s_{0,l} + \sum_{i=1}^n \sum_{l=0}^{d-1} b^l s_{i,l} v_i - z_{\mu} \right)^2 + \sum_{\mu=1}^m \gamma'_{\mu} \left(\sum_{l=1}^{|k|} \varepsilon_{\mu,l} - 1 \right)^2 \\ &\quad - \beta \sum_{\mu=1}^m \sum_{l=l^+}^{|k|} \varepsilon_{\mu,l} (k_l)^2 \\ F \left(-\xi_{\alpha}^{\mu} + \sum_{i=1}^n \xi_i^{\mu} v_i \right) &= \sum_{\mu=1}^m \gamma_{\mu} \left(-\sum_{l=0}^{d-1} b^l s_{0,l} + \sum_{i=1}^n \sum_{l=0}^{d-1} b^l s_{i,l} v_i - z'_{\mu} \right)^2 + \sum_{\mu=1}^m \gamma'_{\mu} \left(\sum_{l=1}^{|k|} \varepsilon'_{\mu,l} - 1 \right)^2 \\ &\quad - \beta \sum_{\mu=1}^m \sum_{l=l^+}^{|k|} \varepsilon'_{\mu,l} (k_l)^2 \end{aligned}$$

Where

$$z_{\mu} = \sum_{l=1}^{|k|} k_l \varepsilon_{\mu,l} \quad (3.26)$$

$$z'_{\mu} = \sum_{l=1}^{|k|} k_l \varepsilon'_{\mu,l} \quad (3.27)$$

Considering the energy previously formulated, the total energy to be minimized is then

$$\begin{aligned} C &= \sum_{\delta} \left[\left(\beta \sum_{\mu=1}^D \left(\sum_{l=l^+}^{|k|} \left(\varepsilon'_{\mu,l} (k_l)^2 - \varepsilon_{\mu,l} (k_l)^2 \right) \right) - y_{\delta}^* \right)^2 + \sum_{\mu=1}^m \gamma'_{\mu} \left(\sum_{l=0}^{|k|} \varepsilon_{\mu,l} - 1 \right)^2 \right. \\ &\quad + \sum_{\mu=1}^m \gamma'_{\mu} \left(\sum_{l=0}^{|k|} \varepsilon'_{\mu,l} - 1 \right)^2 + \sum_{\mu=1}^m \gamma_{\mu} \left(-\sum_{l=0}^{d-1} b^l s_{0,l} + \sum_{i=1}^n \sum_{l=0}^{d-1} b^l s_{i,l} v_i^{\delta} - z'_{\mu} \right)^2 \\ &\quad \left. + \sum_{\mu=1}^m \gamma_{\mu} \left(\sum_{l=0}^{d-1} b^l s_{0,l} + \sum_{i=1}^n \sum_{l=0}^{d-1} b^l s_{i,l} v_i^{\delta} - z_{\mu} \right)^2 \right] \end{aligned} \quad (3.28)$$

Notice that, apart from $2^*m^*|k|$ auxiliary variables used to represent dense associative memory, no other variable is needed and more than binary interactions do not appear.

Quantum Deep feedforward neural networks training implemented via dense associative memories

In this section an approach to implement deep feed-forward network via dense associative memory framework is provided. If implemented in the annealer this is a cost effective methodology to implement classification (single, multi-class, multi-label) and regression and as it will be shown in next section also to implement sampling with low number of variables employed. A deep feed-forward neural network implemented via Dense Associative Memories could be built using the following equality that naturally arise from duality demonstration

$$\beta \sum_{\mu=1}^m \left(F \left(-\xi_{\alpha}^{\mu} x_{\alpha} + \sum_{\gamma \neq \alpha} \xi_{\gamma}^{\mu} x_{\gamma} + \sum_{i=1}^n \xi_i^{\mu} v_i \right) - F \left(\xi_{\alpha}^{\mu} x_{\alpha} + \sum_{\gamma \neq \alpha} \xi_{\gamma}^{\mu} x_{\gamma} + \sum_{i=1}^n \xi_i^{\mu} v_i \right) \right) \quad (3.29)$$

$$= \sum_{\mu=1}^m \xi_{\alpha}^{\mu} f \left(\sum_{i=1}^n \xi_i^{\mu} v_i \right) \quad (3.30)$$

In the annealer setting, input layer is represented as

$$\beta \sum_{\mu=1}^m \left(F \left(-\xi_{\alpha}^{\mu} x_{\alpha} + \sum_{i=1}^n \xi_i^{\mu} v_i \right) - F \left(\xi_{\alpha}^{\mu} x_{\alpha} + \sum_{i=1}^n \xi_i^{\mu} v_i \right) \right) \quad (3.31)$$

Considering $-\xi_{\alpha}^{\mu} x_{\alpha}$ and $\xi_{\alpha}^{\mu} x_{\alpha}$ as a constant, leads to approximated

$$\sum_{\mu=1}^m \xi_{\alpha}^{\mu} f \left(\sum_{i=1}^n \xi_i^{\mu} v_i \right) \quad (3.32)$$

A first naive approach is applying non-linearities to inputs of a feed-forward neural network implemented via dense associative memory like in figure 3.4 This requires an implementation of nonlinearity in an energy minimization fashion, so that if the energy is minimized then nonlinearities are applied to the input. A possible way to implement this approach, similarly to the last chapter 2.2 Dense Associative Memory formulation, can be the following one: in case of ReLU

$$\sum_{\mu=1}^m \left(\gamma_{\mu} (y_{\mu} - z_{\mu})^2 + \gamma'_{\mu} \left(\sum_{l=1}^{|k|} \varepsilon_{\mu,l} - 1 \right)^2 + \sum_{l=1}^{|k|} (\varepsilon_{\mu,l}) k_l \right) \quad (3.33)$$

where as in chapter 2

$$z_{\mu} = \sum_{l=1}^{|k|} k_l \varepsilon_{\mu,l} \quad (3.34)$$

y_{μ} in this case represents dot products between weights (memories in Dense Associative Memories formalism) and inputs ¹. It is clear that if $(y_{\mu} - z_{\mu})^2$ and $\left(\sum_{l=1}^{|k|} \varepsilon_{\mu,l} - 1 \right)^2$ quadratic constraints are satisfied (with suitable gamma real constant) then $\sum_{l=1}^{|k|} (\varepsilon_{\mu,l}) k_l$ outputs the results of nonlinearity applied to inputs/weights product. Tanh is then implemented as

$$\sum_{\mu=1}^m \left(\gamma_{\mu} (y_{\mu} - z_{\mu})^2 + \gamma'_{\mu} \left(\sum_{l=1}^{|k|} \varepsilon_{\mu,l} - 1 \right)^2 + \sum_{l=1}^{|k|} (\varepsilon_{\mu,l}) \frac{1 - e^{-2k_l}}{1 + e^{-2k_l}} \right) \quad (3.35)$$

Sigmoid nonlinearity is then implemented as

$$\sum_{\mu=1}^m \left(\gamma_{\mu} (y_{\mu} - z_{\mu})^2 + \gamma'_{\mu} \left(\sum_{l=1}^{|k|} \varepsilon_{\mu,l} - 1 \right)^2 + \sum_{l=1}^{|k|} (\varepsilon_{\mu,l}) \frac{1}{1 + e^{-k_l}} \right) \quad (3.36)$$

¹As a sidenote one can notice also that sign of last term is changed to positive given its output should reflect the value of the nonlinearity

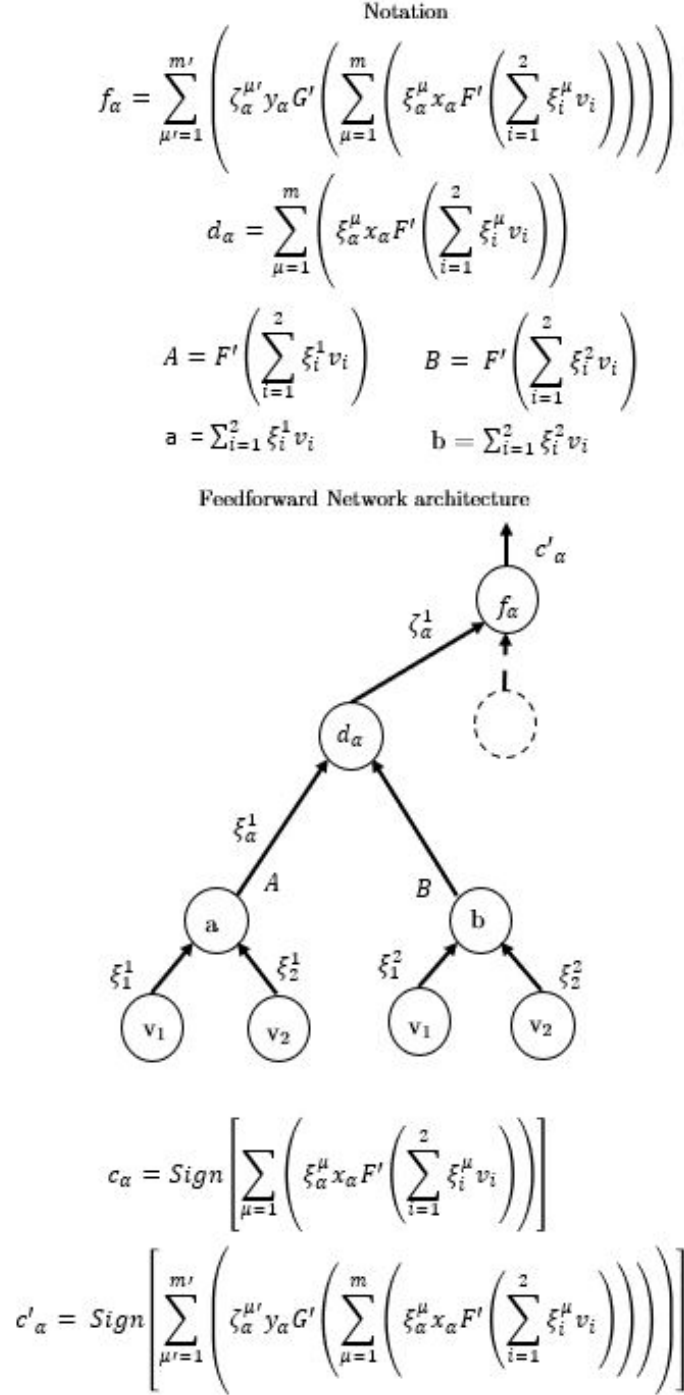


Figure 3.4: Naive implementation of deep neural network via feedforward dense associative memory

The energy QUBO representation of a two hidden layers neural network ReLU, in the same fashion of the first shallow neural network energy representation, will be then

$$E_{TOT} = E(E_1, W) + C \quad (3.37)$$

$$= E(E_1, W) + (c_\alpha - y_\alpha^*)^2 \quad (3.38)$$

where

$$\begin{aligned} E(E_1, W) = & -\beta \sum_{\mu'=1}^{m'} \sum_{l'=l_2^+}^{|\mathbf{k}'|} \epsilon'_{\mu',l'} k'_{l'} + \sum_{\mu'=1}^{m'} \gamma'_{\mu'} \left(\sum_{l'=1}^{|\mathbf{k}'|} \epsilon'_{\mu',l'} - 1 \right)^2 \\ & + \sum_{\mu=1}^m \gamma_\mu \left(\sum_{l=0}^{d-1} b^l s_{i,l} v_i - z_\mu \right)^2 + \sum_{\mu=1}^m \gamma''_\mu \left(\sum_{l=1}^{|\mathbf{k}|} \epsilon_{\mu,l} - 1 \right)^2 \\ & + \sum_{\mu'=1}^{m'} \gamma'''_{\mu'} \left(\sum_{l'=1}^{d'-1} b^{l'} s'_{0,l'} c_\alpha + \sum_{i=1}^{n'} \sum_{l'=0}^{d'-1} b^{l'} s'_{i,l'} \sum_{\mu=1}^m \sum_{l=l_2^+}^{|\mathbf{k}|} \epsilon_{\mu,l} k_l - z'_{\mu'} \right)^2 \end{aligned} \quad (3.39)$$

Note how first term returns the last layer output, third term accounts for z_μ for storing qubits values of first layer, fourth term accounts for $z'_{\mu'}$ storing second layer qubits values, n is number of input units, n' is number of hidden units. However this approach has difficulties in scalability, considering that if developed it explodes due to summation involved to treat all layer representing function that are enveloped one in the other in third term of the previous equations. Also it would be necessary to know all possible values arising after applying first and second nonlinearities in the $\mathbf{k}'$ vector.

Note that this does not depend on the usage of the first approach presented in previous section in how treating neural network on QA. In fact simply minimize over loss as done in second approach presented before simply worsen situation.

$$\begin{aligned} C = & \sum_{\delta}^D \left[\left(\beta' \sum_{\mu'=1}^{|\mathbf{k}'|} \sum_{l'=l_2^+}^{|\mathbf{k}'|} \left(\epsilon'''_{\mu',l'} (k'_{l'})^2 - \epsilon''_{\mu',l'} (k'_{l'})^2 \right) - y_\delta^* \right)^2 \right. \\ & + \sum_{\mu'=1}^{m'} \gamma''_{\mu'} \left(\sum_{l'=1}^{|\mathbf{k}|} \epsilon'''_{\mu,l'} - 1 \right)^2 + \sum_{\mu'=1}^{m'} \gamma''_{\mu'} \left(\sum_{l'=1}^{|\mathbf{k}'|} \epsilon''_{\mu,l'} - 1 \right)^2 \\ & + \sum_{\mu=1}^m \gamma'_\mu \left(\sum_{l=1}^{|\mathbf{k}|} \epsilon_{\mu,l} - 1 \right)^2 + \sum_{\mu=1}^m \gamma'_\mu \left(\sum_{l=1}^{|\mathbf{k}|} \epsilon'_{\mu,l} - 1 \right)^2 \\ & + \sum_{\mu'=1}^{m'} \gamma'''_{\mu'} \left(- \sum_{l'=0}^{d'-1} b^{l'} s'_{0,l'} + \sum_{i'=1}^{n'} \sum_{l'=0}^{d'-1} b^{l'} s'_{i',l'} \beta \sum_{\mu=1}^m \sum_{l=l_2^+}^{|\mathbf{k}|} \left(\epsilon'_{\mu,l} (k_l)^2 - \epsilon_{\mu,l} (k_l)^2 \right) - z'''_{\mu'} \right)^2 \\ & + \sum_{\mu'=1}^{m'} \gamma'''_{\mu'} \left(- \sum_{l'=0}^{d'-1} b^{l'} s'_{0,l'} + \sum_{i'=1}^{n'} \sum_{l'=0}^{d'-1} b^{l'} s'_{i',l'} \beta \sum_{\mu=1}^m \sum_{l=l_2^+}^{|\mathbf{k}|} \left(\epsilon'_{\mu,l} (k_l)^2 - \epsilon_{\mu,l} (k_l)^2 \right) - z''_{\mu'} \right)^2 \\ & \left. + \sum_{\mu=1}^m \gamma_\mu \left(- \sum_{l=0}^{d-1} b^l s_{0,l} + \sum_{i=1}^n \sum_{l=0}^{d-1} b^l s_{i,l} v_i^\delta - z'_\mu \right)^2 + \sum_{\mu=1}^m \gamma_\mu \left(\sum_{l=0}^{d-1} b^l s_{0,l} + \sum_{i=1}^n \sum_{l=0}^{d-1} b^l s_{i,l} v_i^\delta - z_\mu \right)^2 \right] \end{aligned} \quad (3.40)$$

This approach worsen the situation having to add two square terms (sixth and seventh terms in equation before) to enforce that the inputs to second layers is equal to the output of previous layers. These terms introduce more than binary interactions due to presence of $\epsilon'_{\mu,l}$ and $\epsilon_{\mu,l}$ variables. Worse

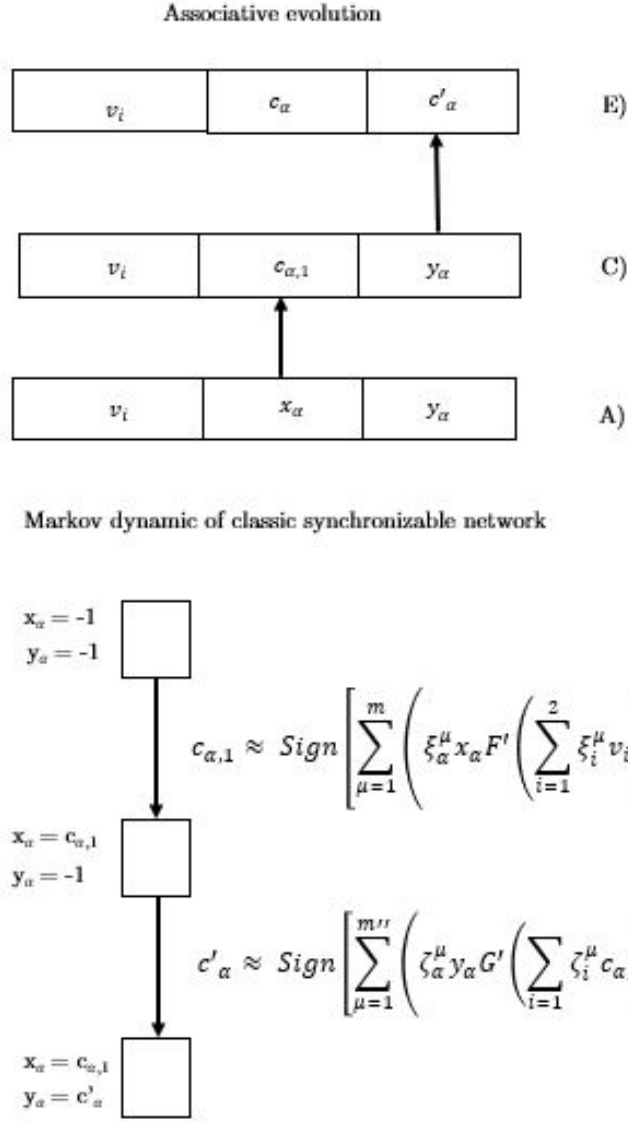


Figure 3.5: Visualization of proposed deep neural network architecture via feed-forward Dense Associative Memory before evolution

than that if one wants to add further hidden layers these terms have to envelope all other previous layers leading to increasing number of more than binary interactions to be decomposed. Instead, a convenient formulations for deep neural networks suitable to QA annealer energy minimization could be constructed exploiting Markov dynamics of Dense Associative Memories that allow to store intermediate results due to hidden layers in qubits. In this case c_α neurons could be used as intermediate output. This can be considered a binarization of the output and can be used to construct binary neural networks. An example of a 2 ReLU layered architecture, binarization not accounted, with last hidden layer containing two neurons

$$E_{TOT} = E(E_{1,1}, W) + E(E_{1,2}, W) + E(E_{2,1}, W) + E(E_2, W) + C$$

Note that L is a loss identical to the one seen previously, $E(E_2, W)$ is the energy component of second hidden layer and output layer, while $E(E_{1,1}, W)$ and $E(E_{1,2}, W)$ are the energy components modelling input, first hidden layer and binarization producing $c_{\alpha,1}$ and $c_{\alpha,2}$. Substituting loss

$$E_{TOT} = E(E_{1,1}, W) + E(E_{1,2}, W) + E(E_{2,1}, W) + (c'_\alpha - y^*)^2 \quad (3.41)$$

Associative Network architecture

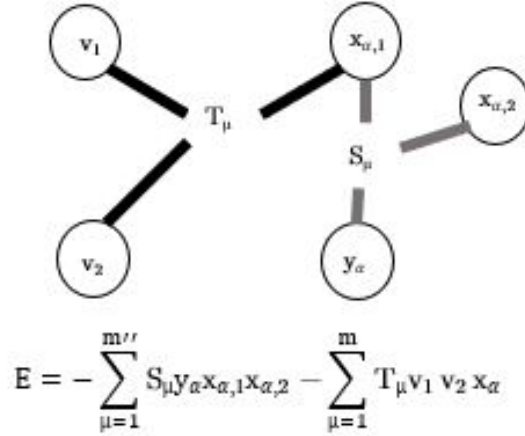


Figure 3.6: Visualization of proposed deep neural network via feedforward Dense Associative Memory during evolution

Substituting $E(E_{1,1}, W)$ and $E(E_{1,2}, W)$

$$\begin{aligned} E_{TOT} = & \sum_{\mu=1}^m \gamma_{\mu} \left(\sum_{l=0}^{d-1} b^l s_{0,l} c_{\alpha,1} + \sum_{i=1}^n \sum_{l=0}^{d-1} b^l s_{i,l} v_i - z_{\mu} \right)^2 - \beta \sum_{\mu=1}^m \sum_{l=1}^{|\mathbf{k}|} \varepsilon_{\mu,l} (k_l)^2 \\ & + \sum_{\mu'=1}^{m'} \gamma'_{\mu'} \left(\sum_{l'=0}^{d'-1} b^{l'} s'_{0,l'} c_{\alpha,2} + \sum_{i=1}^n \sum_{l'=0}^{d'-1} b^{l'} s'_{i,l'} v_i - z_{\mu'} \right)^2 - \beta' \sum_{\mu'=1}^{m'} \sum_{l'=1}^{|\mathbf{k}'|} \varepsilon'_{\mu',l'} (k'_{l'})^2 \\ & + E(E_{2,1}, W) + (c'_{\alpha} - y^*)^2 \end{aligned} \quad (3.42)$$

Note that if one wants to treat sign as a simple binarization step, one could also set weights of outputs units to one

$$\begin{aligned} E_{TOT} = & \sum_{\mu=1}^m \gamma_{\mu} \left(\sum_{l=0}^{d-1} b^l s_{0,l} c_{\alpha,1} + \sum_{i=1}^n \sum_{l=0}^{d-1} b^l s_{i,l} v_i - z_{\mu} \right)^2 - \beta \sum_{\mu=1}^m \sum_{l=1}^{|\mathbf{k}|} \varepsilon_{\mu,l} (k_l)^2 \\ & + \sum_{\mu'=1}^{m'} \gamma'_{\mu'} \left(\sum_{l'=0}^{d'-1} b^{l'} s'_{0,l'} c_{\alpha,2} + \sum_{i=1}^n \sum_{l'=0}^{d'-1} b^{l'} s'_{i,l'} v_i - z_{\mu'} \right)^2 - \beta' \sum_{\mu'=1}^{m'} \sum_{l'=1}^{|\mathbf{k}'|} \varepsilon'_{\mu',l'} (k'_{l'})^2 \\ & + E(E_{2,1}, W) + (c'_{\alpha} - y^*)^2 \end{aligned} \quad (3.43)$$

In this case there are no ternary terms in developing squares in energy function. Finally substituting last component

$$E(E_2, W) = \sum_{\mu=1}^{m''} \gamma''_{\mu} \left(\sum_{l=0}^{d''-1} b^l s''_{0,l} c'_{\alpha} + \sum_{i=1}^2 \sum_{l=0}^{d''-1} b^l s''_{i,l} c_{\alpha,i} - z''_{\mu} \right)^2 - \beta'' \sum_{\mu=1}^{m''} \sum_{l=1}^{|\mathbf{k}''|} \varepsilon''_{\mu,l} (k''_l)^2 \quad (3.44)$$

Require treat ternary interactions arising in $\left(\sum_{l=0}^{d''-1} b^l s''_{0,l} c'_{\alpha} + \sum_{i=1}^2 \sum_{l=0}^{d''-1} b^l s''_{i,l} c_{\alpha,i} - z''_{\mu} \right)^2$ which are the same of equation of 3.17 meaning introducing $n_{\alpha} * d''^2$ and $d'' * |\mathbf{k}''|$ where n_{α} is the number of intermediate c_{α} neurons. To limit complexity it is possible to avoid modelling weights of classification neurons. In this case total number of variables involved for classification of $|\mathcal{D}|$ datapoints are $|\mathcal{D}| * (m * |\mathbf{k}| + m' * |\mathbf{k}'| + m'' * |\mathbf{k}''| + 2)$. It should be noted differently from before there is only a linear increase in complexity for each new layer depending on number of weights, representation of weights,

Notation

$$f_\alpha = \sum_{\mu=1}^{m''} \left(\zeta_\alpha^\mu y_\alpha G' \left(\sum_{i=1}^2 \xi_i^\mu c_{\alpha,i} \right) \right)$$

$$d_{\alpha,i} = \sum_{\mu=1}^m \left(\xi_\alpha^\mu x_{\alpha,i} F' \left(\sum_{i=1}^2 \xi_i^\mu v_i \right) \right)$$

$$A = F' \left(\sum_{i=1}^2 \xi_i^1 v_i \right) \quad B = F' \left(\sum_{i=1}^2 \xi_i^2 v_i \right)$$

$$\mathbf{a} = \sum_{i=1}^2 \xi_i^1 v_i \quad \mathbf{b} = \sum_{i=1}^2 \xi_i^2 v_i$$

Feedforward Network architecture

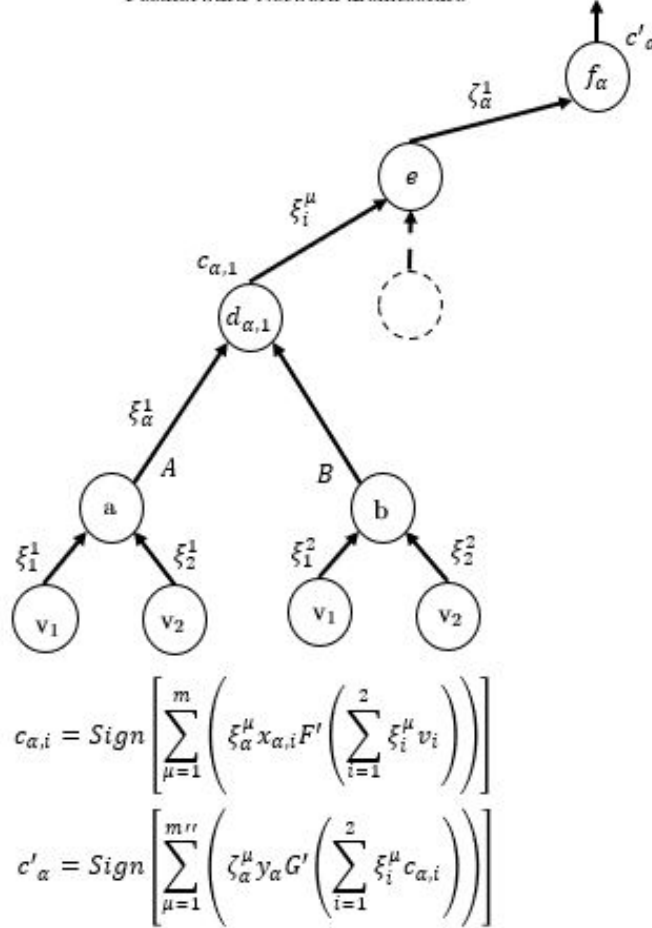


Figure 3.7: Visualization of proposed deep neural network architecture via feedforward Dense Associative Memory, alternative visualization

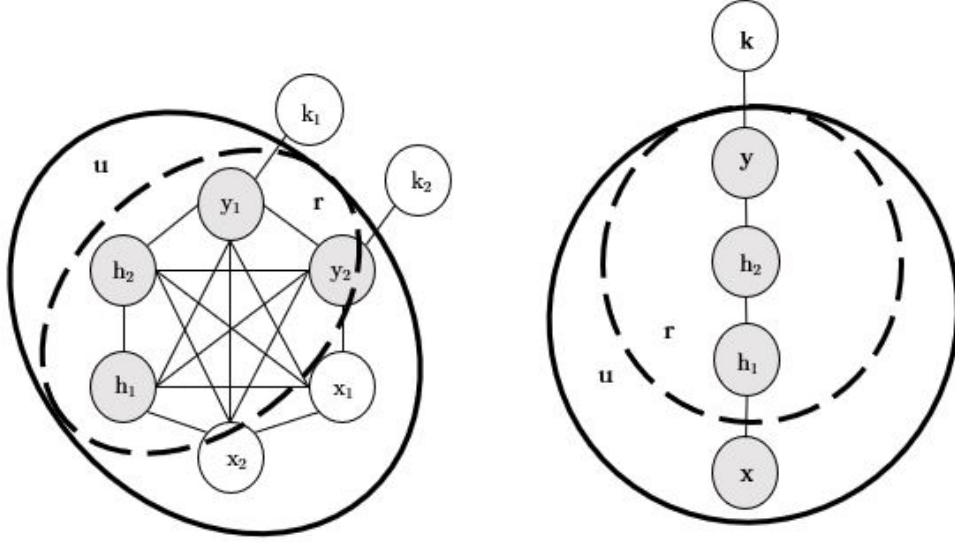


Figure 3.8: Network considered in [212]; picture inspired by [212]

number of hidden neurons but not an exponential one

3.2 An analogue of backpropagation for energy minimization networks

If it is allowable by number of qubits of the annealer architecture at us, it is simply possible to formulate training of a deep neural network as an energy minimization of all the architecture as in the previous chapters. However actual architectures have typically a low number of qubits . So it can be questioned if there are feasible possibilities to extend network dimensions and train the network in an energy minimization fashion via QA. In fact old literature trained neural architectures in a layer-wise manner. This is simple using the last deep neural formulation provided in previous chapter due to capability to obtain layer-wise outputs and easy separability of the problem. As an example, the cascade correlation algorithm, proposed by Fahlman and Lebiere [90], addressed this issue by recruiting new units according to the residual approximation error giving structure to the resulting network [214].

However recent advances in [212] allows for translating back-propagation in energy minimization models and due to success of backpropagation it is relevant to explore a little bit this possibility. Authors of [212] revisit continuous Hopfield model presented in [121]. As underlined by the authors of [212], their result encompasses arbitrary network dynamics whose fixed points/stationary distribution corresponds to a minima of an energy function [212], as depicted in figure 2.14. Using [212] notation, one denotes as $\mathbf{u}$ the collated array of network units containing input units (a single unit is denoted as x_i), output units (a single unit is denoted as y_i), hidden units (a single unit is denoted as h_i), $\mathbf{W}$ the network weights in matrix form (if collated weights are represented as vector $\mathbf{w}$) and $\mathbf{r}$ the state variables (collated vector containing hidden and output units). Authors of [212] considered the network whose energy is represented by

$$E(u) := \frac{1}{2} \sum_i u_i^2 - \frac{1}{2} \sum_{i \neq j} W_{ij} \rho(u_i) \rho(u_j) - \sum_i b_i \rho(u_i). \quad (3.45)$$

where it is present a bias component $\mathbf{b}$ which does not influence further calculation; therefore it is omitted thereafter. Note this is an Hopfield Network, however authors of [212] underline that following results could be extended to every associative memory which recall values via energy minimization.

Resulting network could be trained for pattern recognition in two separate phases. In the first phase, free phase, canonical energy of associative memory, called E and named internal potential, is

minimized

$$\frac{d\mathbf{r}}{dt} = -\frac{\partial E}{\partial \mathbf{r}} \quad (3.46)$$

Energy minimization causes an internal force which in case of symmetric weights is

$$-\frac{\partial E}{\partial r_i} = \rho'(r_i) \left(\sum_{j \neq i} W_{ij} \rho(u_j) \right) - r_i \quad (3.47)$$

The point to which network converges is called the free fixed point u^0 . In a second phase the network is feed-backed based on classification output. In the second phase, called weakly clamped phase, the energy F is minimized

$$F = E + \beta C \quad (3.48)$$

$$C = \frac{1}{2} \|\mathbf{k} - \mathbf{y}\|^2 \quad (3.49)$$

$\beta \geq 0$ is called influence parameter/clamping factor. This parameter allows the network at hand to be "weakly clamped" to the outputs and it is analogue to the present work proposed energy minimization framework for feed-forward neural networks implemented via Dense Associative Memories in quantum annealing. Given energy minimization of F

$$\frac{d\mathbf{r}}{dt} = -\frac{\partial F}{\partial \mathbf{r}} = -\frac{\partial E}{\partial \mathbf{r}} - \beta \frac{\partial C}{\partial \mathbf{r}} \quad (3.50)$$

where energy stops decreasing in a fixed point so $-\beta \frac{\partial C}{\partial h_i} = 0$ and $-\beta \frac{\partial C}{\partial y_i} = \beta(k_i - y_i)$ Note that in this second phase energy decreases because

$$\frac{dF}{dt} = \frac{\partial F}{\partial \mathbf{r}} \cdot \frac{\partial \mathbf{r}}{\partial t} = -\left\| \frac{\partial \mathbf{r}}{\partial t} \right\|^2 \leq 0 \quad (3.51)$$

and that consequently the leaky term drives the network out of the saturation regime, towards weakly clamped fixed point denoted by u^β . In [20] [23] is shown as early changes of neural activities in second phase, caused by the output units moving towards their target, corresponds to the propagation of error derivatives and in [212] is further shown that the weakly clamped phase also implements the (back)-propagation of error derivatives with respect to the synaptic weights. For fixed $\mathbf{w}$ and $\mathbf{m}$, $s_{\mathbf{w},\mathbf{m}}^0$ term denotes a local minimum of E , also called fixed point, which corresponds to the 'prediction' from the model:

$$r_{\mathbf{w},\mathbf{m}}^0 \in \arg \min_r E(\mathbf{w}, \mathbf{m}, r) \quad (3.52)$$

The objective function that is minimized is

$$L(\theta, \mathbf{m}) := C(\mathbf{w}, \mathbf{m}, r_{\mathbf{w},\mathbf{m}}^0) \quad (3.53)$$

and the training objective is $\arg \min_{\mathbf{w}} J(\mathbf{w}, \mathbf{m})$. A fixed point, or energy minimum, denoted by $r_{\mathbf{w},\mathbf{m}}^\beta$ is characterized by

$$\frac{\partial F}{\partial r}(\mathbf{w}, \mathbf{m}, \beta, r_{\mathbf{w},\mathbf{m}}^\beta) = 0 \quad (3.54)$$

and $\frac{\partial^2 F}{\partial r^2}(\mathbf{w}, \mathbf{m}, \beta, r_{\mathbf{w},\mathbf{m}}^\beta)$ is a symmetric positive definite matrix. In [212] it is stated as the gradient of the objective function with respect to $\mathbf{w}$ is given by the formula

$$\frac{\partial L}{\partial \mathbf{w}}(\mathbf{w}, \mathbf{v}) = \lim_{\beta \rightarrow 0} \frac{1}{\beta} \left(\frac{\partial F}{\partial \mathbf{w}}(\mathbf{w}, \mathbf{m}, \beta, r_{\mathbf{w},\mathbf{m}}^\beta) - \frac{\partial F}{\partial \mathbf{w}}(\mathbf{w}, \mathbf{m}, 0, r_{\mathbf{w},\mathbf{m}}^0) \right) \quad (3.55)$$

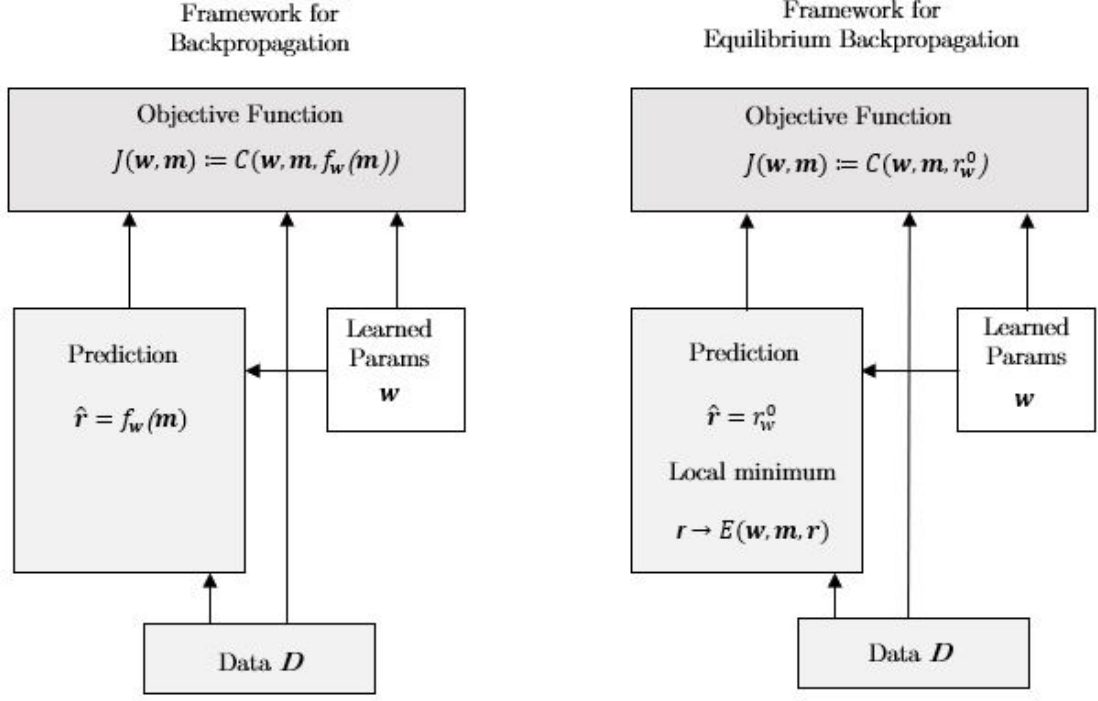


Figure 3.9: equilibrium propagation framework; picture inspired by [212]

or equivalently

$$\frac{\partial L}{\partial \mathbf{w}}(\mathbf{w}, \mathbf{m}) = \frac{\partial \mathcal{C}}{\partial \mathbf{w}}(\mathbf{w}, \mathbf{m}, r_{\mathbf{w}, \mathbf{m}}^0) + \lim_{\beta \rightarrow 0} \frac{1}{\beta} \left(\frac{\partial E}{\partial \mathbf{w}}(\mathbf{w}, \mathbf{m}, r_{\mathbf{w}, \mathbf{m}}^{\beta}) - \frac{\partial E}{\partial \mathbf{w}}(\mathbf{w}, \mathbf{m}, r_{\mathbf{w}, \mathbf{m}}^0) \right) \quad (3.56)$$

Given a data point $\mathbf{m}$ it is possible to train an energy minimization via three steps [212]. First one runs a free phase until the system settles to a free fixed point $r_{\mathbf{w}, \mathbf{m}}^0$ and collect $\frac{\partial F}{\partial \mathbf{w}}(\mathbf{w}, \mathbf{m}, 0, r_{\mathbf{w}, \mathbf{m}}^0)$. Then, one runs a nudged phase for some $\beta \neq 0$ such that $|\beta|$ is "small", until the system settles to a nudged fixed point $r_{\mathbf{w}, \mathbf{m}}^{\beta}$ and collect $\frac{\partial F}{\partial \mathbf{w}}(\mathbf{w}, \mathbf{m}, \beta, r_{\mathbf{w}, \mathbf{m}}^{\beta})$. Finally update the parameter $\mathbf{w}$ according to

$$\Delta \mathbf{w} \propto -\frac{1}{\beta} \left(\frac{\partial F}{\partial \mathbf{w}}(\mathbf{w}, \mathbf{m}, \beta, r_{\mathbf{w}, \mathbf{m}}^{\beta}) - \frac{\partial F}{\partial \mathbf{w}}(\mathbf{w}, \mathbf{m}, 0, r_{\mathbf{w}, \mathbf{m}}^0) \right) \quad (3.57)$$

This approach is appealing also because of neural plausibility and being more advantageous of other analogue approaches [212]. Supposing then, one wants to employ this approach to train dense associative memories, to update weights via equilibrium propagation is only necessary to compute

$$\frac{\partial F}{\partial \mathbf{w}}(\mathbf{w}, \mathbf{m}, \beta', r_{\mathbf{w}, \mathbf{m}}^{\beta'}) \quad (3.58)$$

Where β' is a given learning rate parameter and $\mathbf{d}$ label of $\mathbf{m}$. In general

$$\frac{\partial F}{\partial \mathbf{w}}(\mathbf{w}, \mathbf{m}, \beta', r_{\mathbf{w}, \mathbf{m}}^{\beta'}) = \frac{\partial E}{\partial \mathbf{w}}(\mathbf{w}, \mathbf{m}, r_{\mathbf{w}, \mathbf{m}}^{\beta'}) + \beta' \frac{\partial \mathcal{C}}{\partial \mathbf{w}}(\mathbf{w}, \mathbf{m}, r_{\mathbf{w}, \mathbf{m}}^{\beta'}) \quad (3.59)$$

$$= \frac{\partial E}{\partial \mathbf{w}}(\mathbf{w}, \mathbf{v}, r_{\mathbf{w}, \mathbf{m}}^{\beta'}) + \beta' \frac{\partial \frac{1}{2} \|y^* - \hat{y}\|^2}{\partial \mathbf{w}}(\mathbf{w}, \mathbf{m}, r_{\mathbf{w}, \mathbf{m}}^{\beta'}) \quad (3.60)$$

$$= \frac{\partial E}{\partial \mathbf{w}}(\mathbf{w}, \mathbf{v}, r_{\mathbf{w}, \mathbf{m}}^{\beta'}) + \beta' \frac{\partial \frac{1}{2} \|y^* - \hat{y}\|^2}{\partial \mathbf{w}}(\mathbf{w}, \mathbf{m}, r_{\mathbf{w}, \mathbf{m}}^{\beta'}) \quad (3.61)$$

Taking inside derivatives in respect of weights

$$= \frac{\partial E}{\partial \mathbf{w}} \left(\mathbf{w}, \mathbf{m}, r_{\mathbf{w}, \mathbf{m}}^{\beta'} \right) + \beta' \frac{\partial^{\frac{1}{2}} \left\| k - E \left(\mathbf{w}, \mathbf{m}, r_{\mathbf{w}, \mathbf{m}}^{\beta'} \right) \right\|^2}{\partial \mathbf{w}} \quad (3.62)$$

Applying chain rule

$$= \frac{\partial E}{\partial \mathbf{w}} \left(\mathbf{w}, \mathbf{m}, r_{\mathbf{w}, \mathbf{m}}^{\beta'} \right) + \beta' \left(k - E \left(\mathbf{w}, \mathbf{m}, r_{\mathbf{w}, \mathbf{m}}^{\beta'} \right) \right) \frac{\partial E}{\partial \mathbf{w}} \left(\mathbf{w}, \mathbf{m}, r_{\mathbf{w}, \mathbf{m}}^{\beta'} \right) \quad (3.63)$$

$E \left(\mathbf{w}, \mathbf{m}, r_{\mathbf{w}, \mathbf{m}}^{\beta'} \right)$ is directly obtainable from evolution of network via annealing feed-forward so the only term having to be computed is

$$\frac{\partial E}{\partial \mathbf{w}} \left(\mathbf{w}, \mathbf{m}, r_{\mathbf{w}, \mathbf{m}}^{\beta'} \right) \quad (3.64)$$

For the deep network presented in previous chapter

$$E_{TOT} = E \left(E_{1,1}, \mathbf{w}^{1,1} \right) + E \left(E_{1,2}, \mathbf{w}^{1,2} \right) + E \left(E_2, \mathbf{w}^2 \right) \quad (3.65)$$

Where $E \left(E_2, \mathbf{w}^2 \right)$ is the energy of second hidden layer and output layer and $E \left(E_{1,1}, \mathbf{w}^{1,1} \right)$ and $E \left(E_{1,2}, \mathbf{w}^{1,2} \right)$ energy modelling input, first hidden layer and binarization producing $c_{\alpha,1}$ and $c_{\alpha,2}$. Note here $\mathbf{w}^2, \mathbf{w}^{1,1}, \mathbf{w}^{1,2}$ are subvectors of weight matrices $\mathbf{W}$, or the equivalent collated vector $\mathbf{w}$, containing weights related to respectively second hidden layer to output layer and first hidden to second hidden layer weights. Note that given in previous chapter an equivalent annealer formulation which have the same energy output of the original [147] formulation have been provided, one could conveniently consider original [147] energy formulation to do the derivative computation. In this case

$$E \left(E_{1,1}, \mathbf{w}^{1,1} \right) = c_{\alpha,1} = \text{Sign} \left[\sum_{\mu=1}^m \left(F \left(-\xi_{\alpha}^{\mu} x_{\alpha,2} + \sum_{i=1}^n \xi_i^{\mu} v_i \right) \right) \right] \quad (3.66)$$

$$E \left(E_{1,2}, \mathbf{w}^{1,2} \right) = c_{\alpha,2} = \text{Sign} \left[\sum_{\mu=1}^m \left(F \left(-\xi_{\alpha}^{\mu} x_{\alpha,1} + \sum_{i=1}^n \xi_i^{\mu} v_i \right) \right) \right] \quad (3.67)$$

$$E \left(E_2, \mathbf{w}^2 \right) = c'_{\alpha} = \text{Sign} \left[\sum_{\mu=1}^{m'} \left(F \left(-\xi_{\alpha}^{\mu'} x'_{\alpha} + \sum_{i=1}^{n'} \xi_i^{\mu'} c_{\alpha,i} \right) \right) \right] \quad (3.68)$$

so the total energy

$$E_{TOT} = \text{Sign} \left[\sum_{\mu=1}^m \left(F \left(\xi_{\alpha}^{\mu} x_{\alpha,2} + \sum_{i=1}^n \xi_i^{\mu} v_i \right) \right) \right] + \text{Sign} \left[\sum_{\mu=1}^m \left(F \left(\xi_{\alpha}^{\mu} x_{\alpha,1} + \sum_{i=1}^n \xi_i^{\mu} v_i \right) \right) \right] \\ + \text{Sign} \left[\sum_{\mu=1}^{m'} \left(F \left(\xi_{\alpha}^{\mu'} x'_{\alpha} + \sum_{i=1}^{n'} \xi_i^{\mu'} c_{\alpha,i} \right) \right) \right] \quad (3.69)$$

Sign derivative could be handled with binary neural network back-propagation. This arises naturally from binarization step after nonlinearity calculation which is an exactly analogous to what happen in binarization of activations in binary neural networks methodology [250]. The main approach to compute derivatives in spite of this binarization step is straight through estimation (STE) [49] [228] [22]. In this approach an approximated sign function is employed which allows to compute real derivatives. Note that differently from difficulties of binary neural networks in this setting, binary weights are not required so other aspects of straight trough estimation are not considered (in STE forward and backward pass is done with real weights until the end of training, binarizing network only after training and in fact variant approaches of straight through estimation like [139] do not differ in how

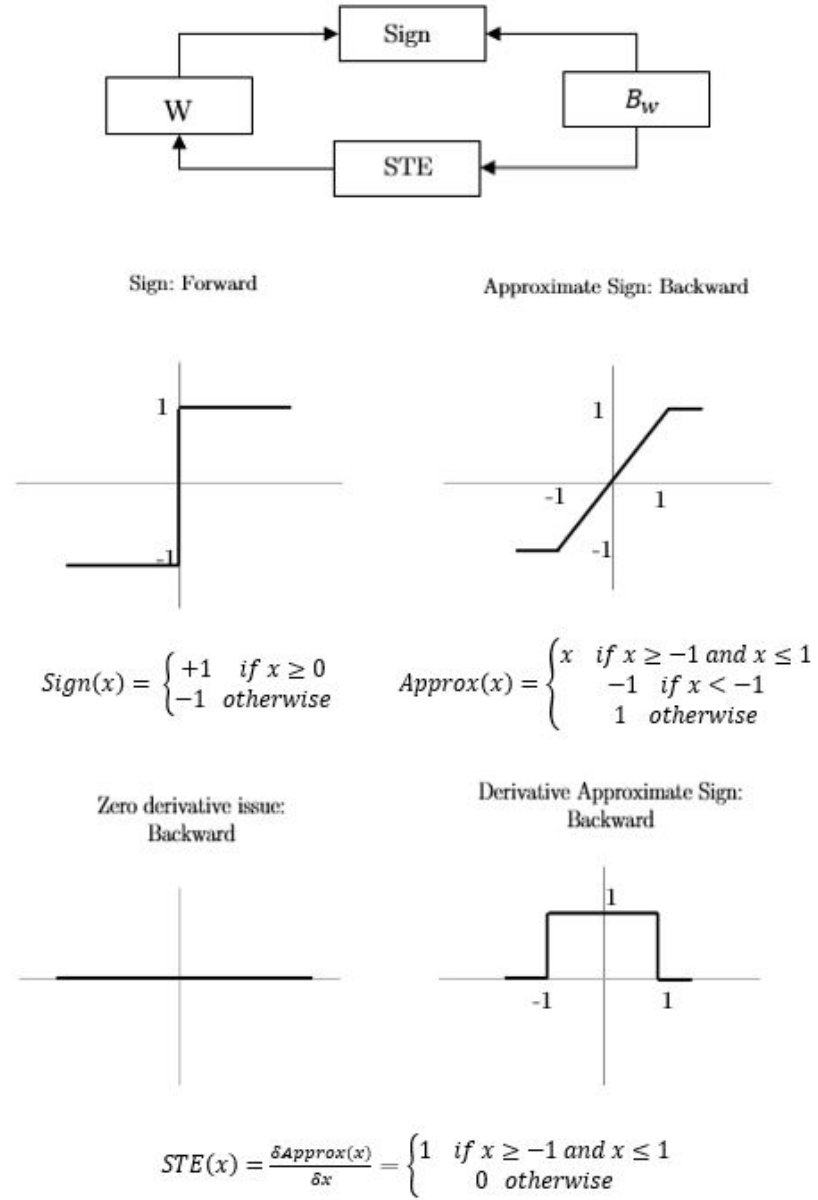


Figure 3.10: STE framework for binary neural networks; picture inspired by [212].

approximate sign function and not considered. Considering ReLU nonlinearity approximation

$$E_{TOT} = \text{Sign} \left[\sum_{\mu=1}^m \left(\text{Rep 2} \left(\xi_{\alpha}^{\mu} x_{\alpha,2} + \sum_{i=1}^n \xi_i^{\mu} v_i \right) \right) \right] + \text{Sign} \left[\sum_{\mu=1}^K (\text{Rep 2} (\xi_{\alpha}^{\mu} x_{\alpha,1} + \sum_{i=1}^n \xi_i^{\mu} v_i)) \right] + \text{Sign} \left[\sum_{\mu=1}^{m'} \left(\text{Rep 2} \left(\xi_{\alpha}^{\mu'} x'_{\alpha} + \sum_{i=1}^{n'} \xi_i^{\mu'} c_{\alpha,i} \right) \right) \right] \quad (3.70)$$

Rep2 derivative, meaning rectified polynomial of order two derivative (which is as said before is used to approximate ReLU nonlinearity in dense associative memory frameworks), is computed as

$$\begin{cases} 2x & \text{if } x \geq 0 \\ 0 & \text{if } x \leq 0 \end{cases} \quad (3.71)$$

However it can be convenient to model it simply as $2x$. This approach is followed by sub-gradient optimization of ReLU. In this case $x = 0$ is not a non-derivability point, so sub-gradient is a singleton 0, however as pointed out in [104] treating it as non-hard nonlinearity can be justified given do not hurt so long as the gradient can propagate along some paths

$$\frac{\partial E_{TOT}}{\partial \mathbf{w}} = \frac{\partial E(E_{1,1}, \mathbf{w}^{1,1})}{\partial \mathbf{w}^{1,1}} + \frac{\partial E(E_{1,2}, \mathbf{w}^{1,2})}{\partial \mathbf{w}^{1,2}} + \frac{\partial E(E_2, \mathbf{w}^2)}{\partial \mathbf{w}^2} \quad (3.72)$$

For the first layer, the vector of derivative is

$$\frac{\partial E(E_{1,1}, \mathbf{w}^{1,1})}{\partial \mathbf{w}^{1,1}} = \left[\frac{\partial E(E_{1,1}, \mathbf{w}^{1,1})}{\partial \xi_1^{\mu}}, \dots, \frac{\partial E(E_{1,1}, \mathbf{w}^{1,1})}{\partial \xi_N^{\mu}}, \frac{\partial E(E_{1,1}, \mathbf{w}^{1,1})}{\partial \xi_{\alpha,1}^{\mu}} \right] \quad (3.73)$$

Where for a generic v_i

$$\frac{\partial E(E_{1,1}, \mathbf{w}^{1,1})}{\partial \xi_i^{\mu}} = \sum_{\mu=1}^m \left(2 \left(\xi_{\alpha,1}^{\mu} x_{\alpha,1} + \sum_{i=1}^n \xi_i^{\mu} v_i \right) \right) v_i \quad (3.74)$$

and similarly

$$\frac{\partial E(E_{1,2}, \mathbf{w}^{1,2})}{\partial \mathbf{w}^{1,2}} = \left[\frac{\partial E(E_{1,1}, \mathbf{w}^{1,1})}{\partial \xi_1^{\mu'}}, \dots, \frac{\partial E(E_{1,1}, \mathbf{w}^{1,1})}{\partial \xi_n^{\mu'}}, \frac{\partial E(E_{1,1}, \mathbf{w}^{1,1})}{\partial \xi_{\alpha,2}^{\mu'}} \right] \quad (3.75)$$

For the second layer, the vector of derivative is

$$\frac{\partial E(E_2, \mathbf{w}^2)}{\partial \mathbf{w}^2} = \left[\frac{\partial E(E_{1,1}, \mathbf{w}^{1,1})}{\partial \xi_1^{\mu''}}, \dots, \frac{\partial E(E_{1,1}, \mathbf{w}^{1,1})}{\partial \xi_n^{\mu''}}, \frac{\partial E(E_{1,1}, \mathbf{w}^{1,1})}{\partial \xi_{\alpha}^{\mu''}} \right] \quad (3.76)$$

Where for a generic $c_{\alpha,i}$

$$\frac{\partial E(E_1, \mathbf{w}^2)}{\partial \xi_i^{\mu''}} = \sum_{\mu=1}^{m''} \left(2 \left(\xi_{\alpha}^{\mu''} x'_{\alpha} + \sum_{i=1}^{n'} \xi_i^{\mu''} c_{\alpha,i} \right) \right) c_{\alpha,i} \quad (3.77)$$

The other nonlinearities are simpler to compute given they do not show hard nonlinearity derivatives. In particular logistic approximation $\log 1 + e^x$ whose derivative is simply logistic function and tanh

approximation $\log \cosh x$ whose derivative is simply $\tanh(x)$, show both no difficulties.

4 Hybrid quantum-classical deep neural networks

It is useful to note that duality does not have restriction on how treating input data before applying dense associative network energy minimization to recover a feed-forward neural network. As an example it is possible to have an associative memory which obtains via duality the following output

$$c_\alpha \approx g \left[\sum_{\mu'=1}^{m'} \xi_\alpha^{\mu'} F'(\xi_k^\mu G(\xi_i^\mu v_i)) \right] \quad (4.1)$$

Where $G(\xi_i^\mu v_i)$ is another nonlinear function applied to the input, that could be possibly precomputed before applying the energy minimization. That holds if

$$\sum_{i=1}^n G(\xi_i^\mu v_i) \gg \sum_{\gamma} \xi_\gamma^\mu x_\gamma \quad (4.2)$$

Which in general is an assumption used in neural networks: representation should be reduced for each layer to abstract useful features for classification [107]. As said before $G(\xi_i^\mu v_i)$ is a constant that could in principle also be retrieved from the energy of another dense associative memory that has as duality property:

$$c_\alpha \approx \text{sgn} \left[\sum_{\mu=1}^m \xi_\alpha^\mu G(\xi_i^\mu v_i) \right] \quad (4.3)$$

where

$$E = - \sum_{\mu=1}^m G(\xi_i^\mu v_i) \quad (4.4)$$

or by a classical network layer. This leads naturally to an hybrid approach that is explored in next sections. This property arises from the fact that dense associative network has a natural interpretation as kernel machine. Recall from first chapter

$$s_i^{(t+1)} = \text{sgn} \left[\sum_{\mu} \alpha_i^\mu \xi_\alpha^\mu F'(\xi_i^\mu v_i) - \theta_i \right] \quad (4.5)$$

The kernel in this case is given by application of $F'(\xi_i^\mu v_i)$ and then α_i^μ and could also be optimized in quantum support vector framework of [245]. Note however that this approach trains only the weights of kernel and does not give help selecting the kernel by itself that could be formed by different neural network layers. This makes necessary to devise a proper quantum method to deal with it. However, this is interesting because it allows to merge research direction regarding both algorithms (qSVM and deep neural network). This chapter leverages kernel formulation of Hopfield network exploring possibility of quantum dense associative memory paired with neural tangent kernel and it will be shown how to employ it for hybrid quantum computation, sampling and ensembling.

4.1 Kernel formulation of deep feed-forward neural network for hybrid classical-quantum training

Before presenting sample via dense associative memories a core concept that will be exploited is presented thereafter: neural tangent kernel. For section 4.1 and 4.2 the notation of [85] is used to make simpler the comprehension¹. The tangent kernel associated with function $f_{\mathbf{w}}(x)$ and parameter vector $\mathbf{v}$ is $K_{f,\mathbf{v}}^g(\mathbf{x}, \mathbf{x}') = \nabla_{\mathbf{w}} f_{\mathbf{w}}(\mathbf{x}) \nabla_{\mathbf{w}} f_{\mathbf{w}}(\mathbf{x}')$, with the gradients taken at $\mathbf{v}$ [131] From this definition, let y_0 be the initial model, before optimization is accomplished, then for the final model y :

$$y = y_0 + \int_{c(T)} \frac{dy}{dt} \quad (4.6)$$

where t is the number of optimization step, while $c(T)$ is the path followed during optimization. If model is differentiable then

$$\frac{dy}{dt} = \sum_{j=1}^d \frac{\partial y}{\partial w_j} \frac{dw_j}{dt} \quad (4.7)$$

[85] underline that, given a model trained with gradient descent, it holds

$$\frac{\mathbf{w}^{s+1} - \mathbf{w}^s}{\epsilon} = -\nabla_{\mathbf{w}} L(\mathbf{w}^s), \quad (4.8)$$

In the $\epsilon \rightarrow 0$ limit previous equation becomes the differential equation

$$\frac{dw(t)}{dt} = -\nabla_{\mathbf{w}} L(w(t)) \quad (4.9)$$

that is known as a gradient flow. As underlined before

$$\frac{dy}{dt} = \sum_{j=1}^d \frac{\partial y}{\partial w_j} \frac{\partial w_j}{\partial t} \quad (4.10)$$

$$= \sum_{j=1}^d \frac{\partial y}{\partial w_j} \left(-\frac{\partial L}{\partial w_j} \right) \quad (4.11)$$

recalling d is the number of parameters and replacing $\frac{\partial w_j}{\partial t}$ by its gradient descent expression. Applying the additivity of the loss and the chain rule of differentiation:

$$\frac{dy}{dt} = \sum_{j=1}^d \frac{\partial y}{\partial w_j} \left(-\sum_{i=1}^m \frac{\partial L}{\partial y_i} \frac{\partial y_i}{\partial w_j} \right) \quad (4.12)$$

$$= -\sum_{i=1}^m \frac{\partial L}{\partial y_i} \sum_{j=1}^d \frac{\partial y}{\partial w_j} \frac{\partial y_i}{\partial w_j} \quad (4.13)$$

defining

$$L'(y_i^*, y_i) = \partial L / \partial y_i \quad (4.14)$$

And applying previous tangent kernel definition (which requires gradient flow condition satisfied)

$$\frac{\partial y}{\partial t} = -\sum_{i=1}^m L'(y_i^*, y_i) K_{f,w(t)}^g(\mathbf{x}, \mathbf{x}_i). \quad (4.15)$$

¹this is relevant in particular for the number of training datapoint which will be denoted as m and not $|D|$ as done before

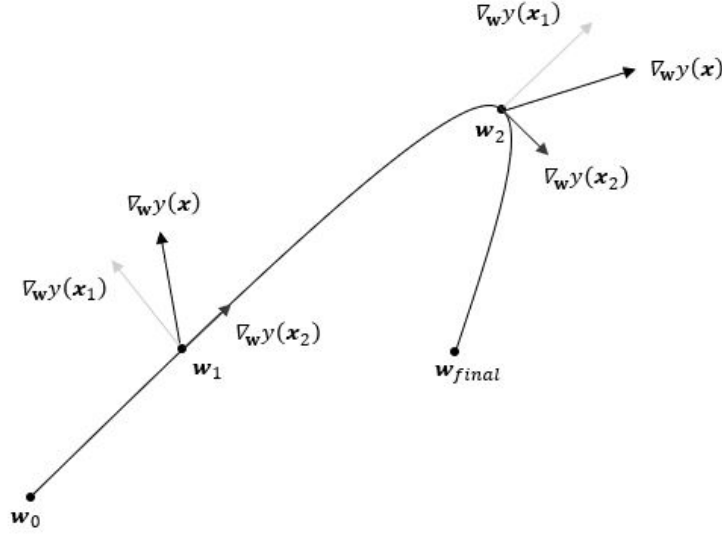


Figure 4.1: Path followed by weights w during training in respect of output y and input $\mathbf{x}$ having data-points $\mathbf{x}_1$ and $\mathbf{x}_2$; picture inspired by [85]

And for

$$y = y_0 + \int_{c(T)} \frac{\partial y}{\partial t} \quad (4.16)$$

authors obtain

$$\lim_{\epsilon \rightarrow 0} y = y_0 - \int_{c(T)} \sum_{i=1}^m L'(y_i^*, y_i) K_{f,w(t)}^g(\mathbf{x}, \mathbf{x}_i) dt, \quad (4.17)$$

which allows to obtain output of the network y given an input $\mathbf{x}$ and training point $\mathbf{x}_i$. Multiplying and dividing by $\int_{c(t)} K_{f,w(t)}^g(\mathbf{x}, \mathbf{x}_i) dt$:

$$\lim_{\epsilon \rightarrow 0} y = y_0 - \sum_{i=1}^m \left(\frac{\int_{c(t)} K_{f,w(t)}^g(\mathbf{x}, \mathbf{x}_i) L'(y_i^*, y_i) dt}{\int_{c(t)} K_{f,w(t)}^g(\mathbf{x}, \mathbf{x}_i) dt} \right) \int_{c(t)} K_{f,w(t)}^g(\mathbf{x}, \mathbf{x}_i) dt \quad (4.18)$$

Let $\bar{L}'(y_i^*, y_i) = \int_{c(t)} K_{f,w(t)}^g(\mathbf{x}, \mathbf{x}_i) L'(y_i^*, y_i) dt / \int_{c(t)} K_{f,w(t)}^g(\mathbf{x}, \mathbf{x}_i) dt$, the average loss derivative weighted by similarity to $\mathbf{x}$. Applying this and Definition 2:

$$\lim_{\epsilon \rightarrow 0} y = y_0 - \sum_{i=1}^m \bar{L}'(y_i^*, y_i) K_{f,c}^p(\mathbf{x}, \mathbf{x}_i) \quad (4.19)$$

This formulation corresponds to a kernel machine with $K(\mathbf{x}, \mathbf{x}_i) = K_{f,c}^p(\mathbf{x}, \mathbf{x}_i)$, $a_i = -\bar{L}'(y_i^*, y_i)$, and $b = y_0$.

$$\lim_{\epsilon \rightarrow 0} y = \sum_{i=1}^m a_i K(\mathbf{x}, \mathbf{x}_i) + b \quad (4.20)$$

The kernel of this formulation could be represented as figure 4.1

Authors of [85] showed how to extend this model to stochastic mini batches and weight regularization. Note also that the result assumes that learning rate is sufficiently small for the trajectory of the weights during gradient descent to be well approximated by a smooth curve. This is standard in the analysis of gradient descent, and is also generally a good approximation in practice, since the learning rate has to be quite small in order to avoid divergence (e.g., 10^{-3})

Quantum Dense Associative memory with Neural Tangent Kernel

As pointed out in [85] this is a kernel machine not an SVM given a_i and y_0 are not parametric values but functions of $\mathbf{x}$. This allow to draw a parallelism between SVM, Dense Associative Memories and neural

network. Note that one layer network/perceptron is equal to an Hopfield network/dense associative memory with quadratic activation or nearest neighbour classifier, a shallow neural network (3 layer) is equal to a Dense Associative Memory or a Support Vector Machine. Finally deep neural network trained with gradient descent are equal to Dense Associative Memories with kernel nonlinearity (see below) or tangent kernel machines. Dense Associative Memories allow to use an arbitrary kernel to model relationship between neurons, so a tangent neural kernel, representing a net with more than 4 layers, could be used to do a forward pass inside the network allowing to obtain the result of network represented by neural tangent kernel.

Recalling duality demonstration:

$$c_\alpha \approx \text{Sign} \left[\sum_{\mu=1}^K \xi_\alpha^\mu F' \left(\sum_{i=1}^N \xi_i^\mu v_i \right) \right] \quad (4.21)$$

Provided the activation function from the visible layer to the hidden layer equals to the derivative of the energy function $f(x) = F'(x)$ and recalling equation 4.16

$$\lim_{\epsilon \rightarrow 0} y \approx \sum_{i=1}^m a_i K(\mathbf{x}, \mathbf{x}_i) + b \quad (4.22)$$

It is clear that setting $\xi_\alpha^\mu = a_i$, having b not influencing calculation and being $F = \int K_{f,c}^p(\mathbf{x}^*, \mathbf{x}_i^*)$ where $\mathbf{x}^* = [x_1, \dots, x_d, x_\alpha]$ and $\mathbf{x}_i^* = [x_1^i, \dots, x_d^i, a_i]$ then

$$c_\alpha \approx \text{Sign} \left[\sum_{i=1}^m a_i K(\mathbf{x}, \mathbf{x}_i) + b \right] \quad (4.23)$$

In this case the energy of the annealer is the output of the net represented by neural tangent kernel, being a continuous value it allows to handle continuous output

Note that $a_i = -L'(y_i^*, y_i)$ is a constant value and $b = y_0 = f_{\text{initial}}(\mathbf{x})$. This last value $b = y_0 = f_{\text{initial}}(\mathbf{x})$ can be obtained via a forward pass in quantum dense associative memories formulation, pre-calculate it classically or simply starting from a situation in which from every input there is the same output². This last option in general is not a good approach because in this case SGD become deterministic. However this situation could be exploited in this way

$$\lim_{\epsilon \rightarrow 0} y = y_0 + \int_{\mathbf{C}(t)} \sum_{j=1}^d \frac{\partial y}{\partial w_j} (w_j^t - w_j^t) \quad (4.24)$$

considering $w_j^t - w_j^t$ as necessary step to put weight to initial condition $w_j^t = 0$ with a constant bias for each layer, given every possible input y_0 has same result and $\frac{\partial y}{\partial w_j}$ is no more an unknown value if one moves with sufficient small step to initial weights. This last formula is explained thereafter. In this work this step-wise approach is added and leveraged: with a sufficiently small learning rate to make valid previous approximation, the line integral $\int_{\mathbf{C}(T)} \frac{\partial y}{\partial t}$ becomes a simpler step-wise line integral: ∂t become an infinitesimal step and $\mathbf{C}(T)$ is a linear path. Here it is exposed briefly how step-wise integral for a single optimization step can be calculated. Considering the case of a neural network with two weights; for simplicity here the weights are denoted with w_1 and w_2

$$\int_C f(w_1, w_2) ds \quad (4.25)$$

the curve C is the one the points come from while here ds is used differently from [85] to underline integral is moving over the path. It is assumed that the curve is smooth and is given by the parametric

²This happen trivially when all weights are zero.

equations,

$$w_1 = h(t) \quad (4.26)$$

$$w_2 = g(t) \quad (4.27)$$

$$a \leq t \leq b \quad (4.28)$$

the parametrization of the curve is often written as a vector function. In this case the curve is given by,

$$\vec{r}(t) = h(t)\vec{i} + g(t)\vec{j} \quad a \leq t \leq b \quad (4.29)$$

The curve is called smooth if $\vec{r}'(t)$ is continuous and $\vec{r}'(t) \neq 0$ for all t . ds here is used to acknowledge the fact that integral is moving along the curve, C and is sometimes called the line integral of f with respect to arc length. The arc length of a curve given by parametric equations is $\int_a^b ds$, where $ds = \sqrt{\left(\frac{dw_1}{dt}\right)^2 + \left(\frac{dw_2}{dt}\right)^2} dt$ and with the parametrization of the function defined before

$$\int_C f(w_1 \dots w_d) ds = \int_a^b f(h(t), g(t)) \sqrt{\left(\frac{dw_1}{dt}\right)^2 + \left(\frac{dw_2}{dt}\right)^2} dt \quad (4.30)$$

For tangent kernel case is useful in present case to exploit another kernel version

$$K(\mathbf{x}^*, \mathbf{x}_i^*) = \int_{c(t)} \sum_{j=1}^d \frac{\partial y}{\partial w_j} \frac{\partial w_j}{\partial t} dt \quad (4.31)$$

which is simply equation 4.10, integrated over the weights path. Note how this kernel does not require applying assumption of gradient descent training expressed in equation 4.8 allowing for training kernel in any possible ways. Differentiability in respect of a loss value is instead guaranteed in continuous weights and inputs setting. Note that w_j is a function of time

$$\begin{aligned} K(\mathbf{x}^*, \mathbf{x}_i^*) &= \int_{c(t)} \sum_{j=1}^d \frac{\partial y}{\partial w_j(t)} \frac{\partial w_j(t)}{\partial t} dt \\ &= \sum_{j=1}^d \int_{c(t)} \frac{\partial y}{\partial w_j(t)} \frac{\partial w_j(t)}{\partial t} dt \end{aligned} \quad (4.32)$$

This approach is analogous to a line integral in respect of the considered weight. In particular noting the line integral of f with respect to x is

$$\int_C f(x, y) dx = \int_a^b f(x(t), y(t)) x'(t) dt \quad (4.33)$$

or in the present case via line integral of $\partial y / (\partial w_j)$ with respect to ∂w_j having

$$\int_{c(t)} f(w_1(t), \dots, w_d(t)) dw_j = \int_{t_1}^{t_2} f(w_1(t), \dots, w_d(t)) w_j'(t) dt \quad (4.34)$$

To solve it it is necessary to parametrize the path. Supposing stepwise approach having $t_1 \leq t \leq t_2$

$$\begin{aligned} \vec{r}(t) &= (t_2 - t) \langle w_1^t, \dots, w_d^t \rangle - (t_1 - t) \langle w_1^{t+1}, \dots, w_d^{t+1} \rangle \\ &= \langle (t_2 - t) w_1^t - (t_1 - t) w_1^{t+1}, \dots, (t_2 - t) w_d^t - (t_1 - t) w_d^{t+1} \rangle \end{aligned} \quad (4.35)$$

then

$$= \sum_{j=1}^d \int_{c(t)} \frac{\partial y}{\partial w_j(t)} (w_j^{t+1} - w_j^t) dt \quad (4.36)$$

It is useful to underline that if $w_j^{t+1} - w_j^t$ is sufficiently small or alternatively learning rate tends to zero in $\frac{\partial y_i}{\partial w_j}$ or $\frac{\partial y}{\partial w_j}$, w_j can be considered constant and equal to its value before the learning step. Supposing to be at third step of gradient descent

$$\sum_{j=1}^d \frac{\partial y}{\partial w_j} \frac{\partial w_j}{\partial t} = \sum_{j=1}^d \frac{\partial \left(y_0 + \int_{c(t-1)} \frac{\partial y}{\partial t} \right)}{\partial w_j} \frac{\partial w_j}{\partial t} \quad (4.37)$$

$$= \sum_{j=1}^d \frac{\partial \left(y_0 + \int_{c(t-1)} \sum_{j=1}^d \frac{\partial y}{\partial w_j} \frac{\partial w_j}{\partial t} \right)}{\partial w_j} \frac{\partial w_j}{\partial t} \quad (4.38)$$

$$= \sum_{j=1}^d \frac{\partial y_0 + \partial \int_{c(t-1)} \sum_{j=1}^d \frac{\partial y}{\partial w_j} \frac{\partial w_j}{\partial t}}{\partial w_j} \frac{\partial w_j}{\partial t} \quad (4.39)$$

$$= \sum_{j=1}^d \frac{\partial y_0}{\partial w_j} \frac{\partial w_j}{\partial t} + \frac{\partial \int_{c(t-1)} \sum_{j=1}^d \frac{\partial y}{\partial w_j} \frac{\partial w_j}{\partial t}}{\partial t} \quad (4.40)$$

$$= \sum_{j=1}^d \frac{\partial y_0}{\partial w_j} \frac{\partial w_j}{\partial t} + \frac{\partial \int_{c(t-1)} \sum_{j=1}^d \frac{\partial y}{\partial w_j} \frac{\partial w_j}{\partial t}}{\partial t} \quad (4.41)$$

4.2 Application of kernel formulation for hybrid quantum-classical deep neural network

A natural hybrid clasical/quantum training algorithm for neural network

As seen in first chapter local search via metropolis algorithms can be a useful way to improve gradient descent.

It's clear that a formulation using local search via annealer and gradient descent is not a form of gradient descent as defined in [85] for which

$$\lim_{\epsilon \rightarrow 0} y = y_0 - \int_{c(T)} \sum_{i=1}^m L'(y_i^*, y_i) K_{f, w(t)}^g(\mathbf{x}, \mathbf{x}_i) dt, \quad (4.42)$$

It's worth noting that this formula being differentiable, if it has a continuous output then satisfies $y = y_0 + \int_{c(T)} \frac{\partial y}{\partial t}$ and that with a sufficiently small learning rate $\int_{c(T)} \frac{\partial y}{\partial t}$ a line integral approximation can be exploited. Suppose doing in sequence an SGD optimization step, followed by a local search step by quantum annealer followed by another SGD step:

$$\begin{aligned} \lim_{\epsilon \rightarrow 0} y = y_0 - \int_{C_1(t')} \sum_{i=1}^m L'(y_i^*, y_i) K_{f, w(t')}^g(\mathbf{x}, \mathbf{x}_i) \partial t' \\ + \int_{C_2(t'')} \frac{dy}{dt''} - \int_{C_3(t''')} \sum_{i=1}^m L'(y_i^*, y_i) K_{f, w(t''')}^g(\mathbf{x}, \mathbf{x}_i) dt''' \end{aligned} \quad (4.43)$$

where $C_1(t')$ represents optimization path followed by first sequence of gradient descent calls, $C_3(t''')$ represents optimization path followed by second sequence of call to gradient descent, $C_2(t'')$ represents optimization path followed by call to annealer local search. T is total training time and

$$T = T' + T'' + T''' \quad (4.44)$$

For simplicity define also $T' = [0, t')$, $T'' = [t', t'')$ and $T''' = [t'', t)$. Local search step represents output difference due to annealer step which, being output continuous and network function differentiable

$$\int_{C_2(t'')} \frac{dy}{dt''} = \int_{C_2(t'')} \sum_{j=1}^d \frac{\partial y}{\partial w_j} \frac{\partial w_j}{\partial t} \quad (4.45)$$

where $\frac{\partial w_j}{\partial t}$ is the step of local search which could be represented as $w_j^{t'''} - w_j^{t''}$ and $\frac{\partial y}{\partial w_j}$ is the derivative of the output in respect of weights. From previous observations it results

$$\begin{aligned} \lim_{\epsilon \rightarrow 0} y = y_0 &- \int_{C_1(t)} \sum_{i=1}^m L'(y_i^*, y_i) K_{f,w(t')}^g(\mathbf{x}, \mathbf{x}_i) dt' + \int_{C_2(t'')} \sum_{j=1}^d \frac{\partial y}{\partial w_j} (w_j^{t'''} - w_j^{t''}) \\ &- \int_{C_3(t''')} \sum_{i=1}^m L'(y_i^*, y_i) K_{f,w(t''')}^g(\mathbf{x}, \mathbf{x}_i) dt''' \end{aligned} \quad (4.46)$$

noting that $\frac{\partial y}{\partial w_j}$ is computed with

$$y = y_0 - \int_{C_1(t')} \sum_{i=1}^m L'(y_i^*, y_i) K_{f,w(t')}^g(\mathbf{x}, \mathbf{x}_i) dt \quad (4.47)$$

which imply one need only to save previous computation step due to gradient descent application to compute the next step integral. It's interesting that a hybrid trained quantum local search and SGD has the convergence property of classical SGD, given that if

$$y'_0 = y_0 - \int_{C_1(t)} \sum_{i=1}^m L'(y_i^*, y_i) K_{f,w(t')}^g(\mathbf{x}, \mathbf{x}_i) dt' + \int_{C_2(t'')} \sum_{j=1}^d \frac{\partial y}{\partial w_j} (w_j^{t'''} - w_j^{t''}) \quad (4.48)$$

Then

$$\lim_{\epsilon \rightarrow 0} y = y'_0 - \int_{C_3(t''')} \sum_{i=1}^m L'(y_i^*, y_i) K_{f,w(t''')}^g(\mathbf{x}, \mathbf{x}_i) dt''' \quad (4.49)$$

is a net trained with SGD that converges to a local optima. Finally to allow quantum search intertwined with SGD the energy is defined as

$$E = \sum_{k=1}^m (y_k^* - \hat{y}(\mathbf{x}_k))^2 + r \quad (4.50)$$

To find best parameters and reduce loss, where

$$\hat{y}(\mathbf{x}_k) = y_{t-1} + \sigma_1 \int_{C_1(t)} \sum_{i=1}^m L'(y_i^*, y_i) K_{f,w(t')}^g(\mathbf{x}_k, \mathbf{x}_i) dt + \sigma_2 \int_{C_2(t)} \sum_{j=1}^d \frac{\partial y}{\partial w_j} (w_j^{t+1} - w_j^t) \partial t \quad (4.51)$$

$$r = (\sigma_1 + \sigma_2 - 1)^2 \quad (4.52)$$

selecting if it is better to go for SGD step or local search³. In synthesis If $w_{j,t+1} - w_{j,t}$ is bigger than the learning rate, $w_{j,t+1}$ could represent a sphere around the weight with radius increased by a SGD step, the motivation is exposed thereafter. Note that a SGD step always reduces the loss if steps are not too much wide and SGD has not already reached a local optima. From this it follows that if annealer opts for a local search instead of gradient descent there are only three possible reasons: a first possibility is that it found a new starting point from which to start with SGD that has a loss value better than gradient step and previous starting point. Another possibility is that some steps of SGD are avoided leading to faster convergence. Finally SGD could have reached local optima from

³it is also possible to parametrize every single weights update, for synthesis, this option is not exposed here

which it could not escape alone. To distinguish between third and first/second possibility it's simply possible to calculate gradient to check if it is not zero. Advantages of this algorithms comprehend the fact that there is not an apriori decision when call local search, same convergence properties of SGD, the fact that annealing usage to compensate gradient descent was experimentally efficient for small datasets machine learning . Further, every time neural network decides for a local search step it generates a new neural network with a new starting point. This is equivalent to train an ensemble (see later). Finally this algorithm is iterative and incremental: it can be parallelized or mixed with a classical machine and in this way globally neural network could be trained only with annealer calls. Note that in the term of the previously exposed energy

$$\begin{aligned} & \left(y_k^* - y_{t-1} + \sigma_1 \int_{C_1(t)} \sum_{i=1}^m L'(y_i^*, y_i) K_{f,w(t')}^g(\mathbf{x}_k, \mathbf{x}_i) dt \right. \\ & \left. + \sigma_2 \int_{C_2(t)} \sum_{j=1}^d \frac{\partial y}{\partial w_j} (w_j^{t+1} - w_j^t) \right)^2 \end{aligned} \quad (4.53)$$

An hyperparameter $\lambda \in [0, 1]$ could be inserted to select how often change research approach giving more weight to an annealer or backpropagation step:

$$\begin{aligned} & \left(y_k^* - y_{t-1} + \lambda \sigma_1 \int_{C_1(t)} \sum_{i=1}^m L'(y_i^*, y_i) K_{f,w(t')}^g(\mathbf{x}_k, \mathbf{x}_i) dt \right. \\ & \left. + (1 - \lambda) \sigma_2 \int_{C_2(t)} \sum_{j=1}^d \frac{\partial y}{\partial w_j} (w_j^{t+1} - w_j^t) \right)^2 \end{aligned} \quad (4.54)$$

Regarding weight wideness it can be noted that a SGD step makes w_j^t change over a surface of an hypersphere of radius equal to ϵ in the weight space of dimension h (or to hypersphere center if gradient step is zero)

$$w_j^{t+1} \in \left\{ S_h(\epsilon) = \frac{2\pi^{\frac{h}{2}}}{\Gamma(\frac{h+2}{2})} \epsilon^{h-1}, w_j^t \right\} \quad (4.55)$$

After a gradient descent step, in presence of a valley weights are still in w_j^t meaning actual learning rate is not capable to exit the valley. If gradient descent step is accepted then search space is now in another hypersphere with same learning rate defined radius 2ϵ , but this would not solve the problem in presence of a local minima. Consequently is useful to explore a radius of $[S_h(2\epsilon), S_h(n+2\epsilon)]$. Being ϵ tends to zero corresponds to a volume of radius $\tau\epsilon$ -surface $S_W(\epsilon)$, meaning

$$\frac{\pi^{h/2}}{\Gamma(\frac{h}{2} + 1)} (\tau + 2) \epsilon^h - \frac{\pi^{h/2}}{\Gamma(\frac{h}{2} + 1)} 2 \epsilon^h \quad (4.56)$$

Where n is an hyperparameter that depends on trade-off number of weights / wideness of space explored. In worst case this heuristic anticipates gradient step make convergence faster. It should finally noted that forward pass during training is not needed, This algorithm is applicable with every feed-forward developed architecture regardless of nonlinearity and regularization methodology employed and if network category belongs to CNN/feedforward/RNN/attention based neural network.

For a negative or positive local search step for a learning rate multiple 2 qubits are necessary, consequently, given volume explored by local search

$$\frac{\pi^{h/2}}{\Gamma(\frac{h}{2} + 1)} (n + 2) \epsilon^h - \frac{\pi^{h/2}}{\Gamma(\frac{h}{2} + 1)} 2 \epsilon^h \quad (4.57)$$

And number of qubits involved is $h*2$

Sampling deep neural networks via quantum associative memory

Sampling via annealer is a commonplace approach that was implemented in different works via different mechanism (e.g. [115][223][249]). Sampling allows to model a distribution ([223]), to train classical architectures ([249]), or could be exploited in ensembling (see next section). Also QA sampling in present context can have different advantages, in particular results is obtainable faster than corresponding classical implemented neural network and can be used to estimate a predictive class score [115].

In this section a different approach in respect of previous works (e.g. [?]) toward sampling a neural network is exposed. Here it is suggested sampling outputs of feedforward neural network, exploiting possible different layerwise output that duality of Dense Associative Memory, and contextually Quantum Dense Associative Memory approach, allow. Note that this approach can only be exploited via Quantum Dense Associative Memory formulation and that it reduces dramatically the number of qubits involved.

To implement sampling, the "naive" formulation of previous chapter is recovered and slightly modified to minimize number of qubits involved. In particular the energy of the sampler will be represented enveloping nonlinearity (or better dense associative memory approximating feedforward network via duality) one into another

Having a single layer neural network implemented via dense associative memories, first layer evolution is

$$\begin{aligned} c_\alpha &= \text{Sign} \left[\beta \sum_{\mu=1}^m \left(F \left(-\xi_\alpha^\mu x_\alpha + \sum_{i=1}^n \xi_i^\mu v_i \right) - F \left(+\xi_\alpha^\mu x_\alpha + \sum_{i=1}^n \xi_i^\mu v_i \right) \right) \right] \\ &\approx \text{sign} \left[\sum_{\mu=1}^m \xi_\alpha^\mu F' \left(\sum_{i=1}^n \xi_i^\mu v_i \right) \right] \end{aligned} \quad (4.58)$$

Note that the following component approximates forward

$$\beta \sum_{\mu=1}^m F \left(-\xi_\alpha^\mu x_\alpha + \sum_{i=1}^n \xi_i^\mu v_i \right) - \left(\xi_\alpha^\mu x_\alpha + \sum_{i=1}^n \xi_i^\mu v_i \right) \quad (4.59)$$

The second hidden layer forward could be modelled as

$$\begin{aligned} c'_\alpha &= \text{Sign} \left[\beta' \sum_{\mu=1}^{m'} \left(F \left(-\xi_\alpha^{\mu'} x'_\alpha + \left(\sum_{\mu=1}^m \xi_\alpha^\mu F' \left(\sum_{i=1}^n \xi_i^\mu v_i \right) \right) \right) \right. \right. \\ &\quad \left. \left. - F \left(-\xi_\alpha^{\mu'} x'_\alpha + \left(\sum_{\mu=1}^m \xi_\alpha^\mu F' \left(\sum_{i=1}^n \xi_i^\mu v_i \right) \right) \right) \right) \right] \end{aligned} \quad (4.60)$$

Note here that differently from before the energy component of previous layer is directly inserted inside the evolution of a qubit representing second layer. This allows obtaining via duality

$$c'_\alpha \approx \text{Sign} \left[\sum_{\mu=1}^{m'} \xi_\alpha^{\mu'} F' \left(\sum_{\mu=1}^m \xi_\alpha^\mu F' \left(\sum_{i=1}^n \xi_i^\mu v_i \right) \right) \right] \quad (4.61)$$

This can be done simply via iterative calls to the annealer, in which each call computes the output of the previous layer.

Also Neural Tangent Kernel version could be splitted layerwise and treated in similar way. Supposing having divided network layers in two subgroup (as in figure 4.2) d and $d' < d$ then it is simply

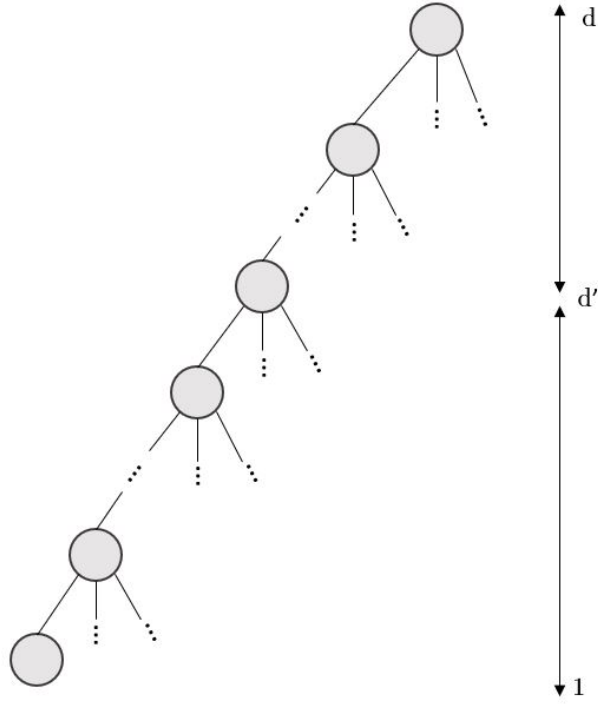


Figure 4.2: Example of two kernel subdivision of network layers

possible obtaining two layers kernel

$$K_2(\mathbf{x}^*, \mathbf{x}_i^*) = \int_{c(t)} \sum_{j=d'}^d \frac{\partial y}{\partial w_j} \frac{\partial w_j}{\partial t} dt \quad (4.62)$$

$$K_1(\mathbf{x}^*, \mathbf{x}_i^*) = \int_{c(t)} \sum_{j=1}^{d'} \frac{\partial y_{d'}}{\partial w_j} \frac{\partial w_j}{\partial t} dt \quad (4.63)$$

Where $y_{d'}$ is the output of the layer. If assumption of equation 4.8 holds then

$$K_2(\mathbf{x}^*, \mathbf{x}_i^*) = \int_{c(t)} \sum_{i=1}^m \frac{\partial L}{\partial y_i} \sum_{j=d'}^d \frac{\partial y}{\partial w_j} \frac{\partial y_i}{\partial w_j} \quad (4.64)$$

$$K_1(\mathbf{x}^*, \mathbf{x}_i^*) = \int_{c(t)} \sum_{i=1}^m \frac{\partial L}{\partial y_i} \sum_{j=1}^{d'} \frac{\partial y}{\partial w_j} \frac{\partial y_i}{\partial w_j} \quad (4.65)$$

Deep feed-forward ensembling with annealer techniques

As said before annealer allows for different simultaneous solutions to energy minimization problem: a property that was exploited in previous works in different manners. First, if annealer implements learning different energy minimization results leading to different solutions to learning problem that can be used to make decisions, as done by [245]. Secondly, if annealer implements sampling, it leads to different solutions amenable to be used in a committee, as done by [210]. In general all these approaches make use of machine learning committee: The combination of the outputs of a group of neural networks [190]. Assume that a training set of m input-output pairs $(\mathbf{x}^1, t_1), \dots, (\mathbf{x}^m, t_m)$ is given and that l networks are trained using this data. For simplicity n -dimensional input vectors and a single output unit are considered. Denote by f_i the network function computed by the i -th network,

for $i = 1, \dots, l$. The network function f produced by the committee of networks is defined as

$$f = \frac{1}{l} \sum_{i=1}^l f_i \quad (4.66)$$

For each network function f_i a m -dimensional vector $\mathbf{e}^i$ can be computed whose components are the approximation error of the function f_i for each input-output pair. The quadratic approximation error Q of the ensemble function f is

$$Q = \sum_{i=1}^m \left(t_i - \frac{1}{l} \sum_{j=1}^l f_j(\mathbf{x}^i) \right)^2 \quad (4.67)$$

This can be written in matrix form by defining a matrix $\mathbf{E}$ whose N rows are the m components of each error vector $\mathbf{e}^i$:

$$\mathbf{E} = \begin{pmatrix} e_1^1 & e_2^1 & \cdots & e_m^1 \\ \vdots & \vdots & \ddots & \vdots \\ e_1^l & e_2^l & \cdots & e_m^l \end{pmatrix} \quad (4.68)$$

The quadratic error of the ensemble is then

$$Q = \left| \frac{1}{l} (1, 1, \dots, 1) \mathbf{E} \right|^2 = \frac{1}{l^2} (1, 1, \dots, 1) \mathbf{E} \mathbf{E}^T (1, 1, \dots, 1)^T \quad (4.69)$$

The matrix $\mathbf{E} \mathbf{E}^T$ is the correlation matrix of the error residuals. If each function approximation produces uncorrelated error vectors, the matrix $\mathbf{E} \mathbf{E}^T$ is diagonal and the i -th diagonal element Q_i is the sum of quadratic deviations for each functional approximation, i.e., $Q_i = \|\mathbf{e}^i\|^2$. In this case

$$Q = \frac{1}{l} \left(\frac{1}{l} (Q_1 + \dots + Q_l) \right) \quad (4.70)$$

and this means that the total quadratic error of the ensemble is smaller by a factor $1/N$ than the average of the quadratic errors of the computed functional approximations. If the quadratic errors are not uncorrelated, that is if $\mathbf{E} \mathbf{E}^T$ is not symmetric, a weighted combination of the l functions f_i can be used. Denote the i -th weight by w_i . The ensemble functional approximation f is now

$$f = \sum_{i=1}^l w_i f_i \quad (4.71)$$

The weights w_i must be computed in such a way as to minimize the expected quadratic deviation of the function f for the given training set. With the same definitions as before and with the constraint $w_1 + \dots + w_l = 1$

$$Q = \frac{1}{l^2} (w_1, w_2, \dots, w_l) \mathbf{E} \mathbf{E}^T (w_1, w_2, \dots, w_l)^T \quad (4.72)$$

The minimum of this expression can be found by differentiating with respect to the weight vector $(w_1, w_2, \dots, w_N)$ and setting the result to zero. This imply calculation of $(\mathbf{E} \mathbf{E}^T)^{-1}$. This method can become prohibitive if the matrix $\mathbf{E} \mathbf{E}^T$ computation requires too many operations and it is ill-conditioned. Note that presented sampling approach allows to compute instantly error per samples given an input simply recovering energy of each spin configurations. For example considering output of a two hidden layer Dense Associative Memory, one can consider quantum dense associative memory

sampling formulation considered in previous section.

$$c_\alpha = \text{sign} \left(F \left(-x_\alpha \xi_\alpha^\mu + \sum_{i=1}^n \xi_i^\mu v_i \right) - F \left(x_\alpha \xi_\alpha^\mu + \sum_{i=1}^n \xi_i^\mu v_i \right) \right) \quad (4.73)$$

$$c'_\alpha = \text{sign} \left(F \left(-x'_\alpha \xi_\alpha^{\mu'} + \left(F \left(-x_\alpha \xi_\alpha^\mu + \sum_{i=1}^n \xi_i^\mu v_i \right) - F \left(x_\alpha \xi_\alpha^\mu + \sum_{i=1}^n \xi_i^\mu v_i \right) \right) \right) \right. \\ \left. - F \left(x'_\alpha \xi_\alpha^{\mu'} + \left(F \left(-x_\alpha \xi_\alpha^\mu + \sum_{i=1}^n \xi_i^\mu v_i \right) - F \left(x_\alpha \xi_\alpha^\mu + \sum_{i=1}^n \xi_i^\mu v_i \right) \right) \right) \right) \quad (4.74)$$

All energies annealer can achieve minimizing energy of previous network configuration are exponential in number of qubits, so in this case

Energy	c'_α	c_α
E_1	1	-1
E_2	-1	1
E_3	-1	-1
E_4	1	1

Being amenable of ensembling representing different possible calculation of forward pass. Noting that

$$\beta \sum_{\mu=1}^{m'} \xi_\alpha^{\mu'} F' \left(\sum_{\mu=1}^m \xi_\alpha^\mu F' \left(\sum_{i=1}^n \xi_i^\mu v_i \right) \right) = F \left(-x'_\alpha \xi_\alpha^{\mu'} + \left(F \left(-x_\alpha \xi_\alpha^\mu + \sum_{i=1}^n \xi_i^\mu v_i \right) - F \left(x_\alpha \xi_\alpha^\mu + \sum_{i=1}^n \xi_i^\mu v_i \right) \right) \right) \\ - F \left(x'_\alpha \xi_\alpha^{\mu'} + \left(F \left(-x_\alpha \xi_\alpha^\mu + \sum_{i=1}^n \xi_i^\mu v_i \right) - F \left(x_\alpha \xi_\alpha^\mu + \sum_{i=1}^n \xi_i^\mu v_i \right) \right) \right) \quad (4.75)$$

and the outputs of feedforward networks implemented via dense associative memories could be retrieved using the layerwise approaches considered in previous section. Note also how this formulation allow obtaining intermediate features via

$$\sum_{\mu=1}^m \xi_\alpha^\mu F' \left(\sum_{i=1}^n \xi_i^\mu v_i \right) = \frac{1}{\beta \sum_{\mu=1}^{m'} \xi_\alpha^{\mu'}} F^{-1'} \left(\beta \sum_{\mu=1}^{m'} \xi_\alpha^{\mu'} F' \left(\sum_{\mu=1}^m \xi_\alpha^\mu F' \left(\sum_{i=1}^n \xi_i^\mu v_i \right) \right) \right) \quad (4.76)$$

Further research in how design optimization approach to improve independence and conditioning of matrix EE^T is necessary, taking note that energy minimization framework allows to include constraints in sampling from network allowing for better results. As said before if networks are not independent a weighted approach is better but also costly. Independence influences also conditioning. In typical dropout, error is not independent given suppressing a single weight maintains the architecture in terms of other shared weights. However here two ways to reduce this correlation are proposed. A first possibility regard implementing skip connections. Using skip connections have suggested to be equivalent to train an ensemble [236] and this could be leveraged in the quantum neural network sampling procedure via reverse annealing, allowing to lead process from an initial state (in the presented case all qubits set to -1). Note that when a qubit does not change sign the next layer calculate this function

$$F \left(-\xi_\alpha^\mu x_\alpha + \sum_{i=1}^n \xi_i^\mu v_i \right) - F \left(\xi_\alpha^\mu + \sum_{i=1}^n \xi_i^\mu v_i \right) \quad (4.77)$$

If $x_\alpha = -1$ then network does not change sign.

$$F \left(-\xi_\alpha^\mu + \sum_{i=1}^n \xi_i^\mu v_i \right) - F \left(-\xi_\alpha^\mu + \sum_{i=1}^n \xi_i^\mu v_i \right) = 0 \quad (4.78)$$

If energy of a two layer quantum neural network sampler (like one defined before) is implemented like

$$E = F \left(-x'_\alpha \xi_\alpha^{\mu'} + \left(F \left(x_\alpha \xi_\alpha^\mu + \sum_{i=1}^n \xi_i^\mu v_i \right) - C \right) + \sum_{i=1}^n \xi_i^\mu v_i \right) - C \quad (4.79)$$

$$C = F \left(-\xi_\alpha^\mu + \sum_{i=1}^n \xi_i^\mu v_i \right) \quad (4.80)$$

this clearly implements skip connections.

Another possibility is leverage quantum training to train multiple different networks. In general as pointed out before different training solutions to full quantum neural network implementation can be independent. However QA can help reducing correlation in network ensemble via hybrid quantum classical neural network training exposed before. In fact every model after applied a step of local search can be used as an ensemble. As pointed out before by [40], if annealing allows to jump local minima every network after a quantum annealer local search step converges to a different local minima if further optimized by SGD, reducing correlation between models. This allows for an ensemble of every network after a quantum local search is applied. This method implemented via simulated annealing [40] improves performances also over other ensemble techniques using only SGD approaches, similarly to snapshot ensembling.

5 Conclusion and further research

The core idea of this work is that classical energy minimization architecture which retrieve pattern based on reaching equilibrium point, should be mainly considered in implementing machine learning architecture on energy minimization platform to exploit fully their potential and make cheaper (in term of complexity) and easier to tackle machine learning problem. In this context equivalence between classical and equilibrium architecture should be considered as a shortcut to solve difficulties arising in translating machine learning problems to quantum annealer formulation. In this spirit the previous chapter following results were exposed: first proposals of space complexity efficient energies for dense associative memories suitable to be minimized in QA frameworks (chapter 2.2), first proposals of space complexity efficient (invariant in the number of datapoints) energy suitable to be minimized in QA frameworks suitable to train shallows networks (chapter 3.1), first proposal for a space complexity efficient and suitable QA energy formulation for quantum deep feed-forward neural network on the annealer (chapter 3.1), first proposal to approach training of deep feed-forward neural network on limited qubits annealing architectures (chapter 3.2), first cost effective methodology to treat sampling and ensembling of deep neural network (chapter 4.2), hybrid classical/quantum training of neural network (chapter 4.1). Further research over this topic can be conducted along this results to test and improve this proposals both with experimental and theoretical approaches. In particular author of this work suggests it would be interesting to compare this work and other relevant QA machine learning algorithms (like [245]) on a suitable task like ttbar, which is already proposed in [27] (in this task, given a set of events generated by simulations of proton-proton collisions at the LHC with final state containing 2 top quarks, the aim is to identify if two tops decay products is an hypothetical new particle) and is relevant for experimental research. A second aspects of interest, regards implementing a convolutional version of previous work as done in [210] . This should be easy because typical activation is element-wise reducing bit consumption allowing to test also on bigger dataset like [154] or implement deep neural network in energy minimization fashion without rely on equilibrium propagation. This work proposed approach to tackle deep neural network training (equilibrium propagation) that could be tested versus a classical dense associative memory implementation (also trained with equilibrium propagation) to discover analogy and differences between quantum and classical base. A possible experiment could be conducted as suggested before and done in equilibrium propagation work [212] on [154]. This could be useful also for more computational neuroscience research purpose transcending

the engineering interest of this work. Note that both this work and [148] implementation do not have more than binary interaction being biologically plausible. As underline before equilibrium propagation is biologically plausible being reminiscent of o Spike-Timing Dependent Plasticity [212]. The whole quantum deep feed-forward network implemented via dense associative network trained via equilibrium is consequently biologically plausible. Given some author proposed quantum phenomena to be of possible impact in brain (e.g. [243], [87]) this model could be of use to test this theories. Note finally the approach of equilibrium propagation could be exploited with other forms of approximated back-propagation in equilibrium models, within the proposed deep feedforward network implmeneted via dense associative memory framework (e.g. [195] [173]). In fact being an inherently energy minimization network implemented in ad-hoc energy minimization platform it would allow easy studies on this type of architecture (which is a difficult aspect underlined by many author working on this topic) and share light on further respective advantages and disadvantages of different algorithms. Also some theoretical aspects should be further analyzed. Firstly, innovation of quantum annealer hardware have to be considered. In fact there could be other use of reverse annealing [186] . Other advanced features that could be of use of annealer like spin-reversal transforms [188] or influence of special annealing schedules should be considered in testing actual implementations. Regarding ensemble approach it would be interesting to find optimization procedure of sampling able to reduce correlation of error vectors of ensemble classifiers (to obtain ensemble with a quadratic error that is linearly reduced in number of classifiers involved in the ensemble) or/and improve conditioning given ensembling to speed up calculation if correlation matrix of the error residuals is not diagonal and it's inverse have to be computed. Finally, It could be interesting to analyse both theoretically and experimentally how quantum dense associative memory relate with quantum phenomena and how much are able to exploit inherently quantum parallelism. Notably some researches underline how quantum Ising model (in the form of x-y quantum Ising model simulating a neural network analyzed by [215], [171]) seems to lack some interesting properties that are expected in quantum systems, given quantum fluctuations play the same role as thermal fluctuations, not allowing exponential memory storage due to quantum parallelism. Consequently it could be interesting to explore if dense associative memory minimization framework (even if it is based on Ising Hamiltonian energy minimization) allow for exponential memory storage also thanks to quantum phenomena.

Bibliography

- [1] S.A. Adachi and M.P. Henderson. Application of quantum annealing to training of deep neural networks. 2015.
- [2] D. Aharonov, W. van Dam, J. Kempe, Z. Landau, S. Lloyd, and O. Regev. Adiabatic quantum computation is equivalent to standard quantum computation. *SIAM Journal of Computing*, 2007.
- [3] M.A. Aizerman, E.M. Braverman, and L.I. Rozner. Theoretical foundations of the potential function method in pattern recognition learning. *Automation and Remote Control*, 1964.
- [4] M. Akazawa. Quantum hopfield network using single-electron circuits. *Analog Integrated Circuits and Signal Processing*, 1997.
- [5] Y. Alexeev, D. Bacon, K.R. Brown, R. Calderbank, L.D. Carr, F.T. Chong, B. DeMarco, D. Englund, E. Farhi, and B. Fefferman. Quantum computer systems for scientific discovery. *PRX Quantum*, 2021.
- [6] M.H.S Amin. Effect of local minima on adiabatic quantum optimization. *Physical Review Letters*, 2008.
- [7] F. Arute, K. Arya, R. Babbush, D. Bacon, J.C. Bardin, R. Barends, R. Biswas, S. Boixo, F.G.S.L. Brandao, A. D Buell, B. Burkett, Y. Chen, Z. Chen, B. Chiaro, R. Collins, W. Courtney, A. Dunsworth, E. Farhi, B. Foxen, A. Fowler, C. Gidney, M. Giustina, R. Graff, K. Guerin, and J.M. Martinis. Quantum supremacy using a programmable superconducting processor. *Nature*, 2019.
- [8] A. Azulay and Y. Weiss. Why do deep convolutional networks generalize so poorly to small image transformations? *ArXiv Computer Vision and Pattern Recognition*, 2018.
- [9] R. Babbush, V. Denchev, N. Ding, S. Isakov, and H. Neven. Construction of non-convex polynomial loss functions for training a binary classifier with quantum annealing. *ArXiv Machine Learning*, 2014.
- [10] D. Bahdanau, K. Cho, and Y. Bengio. Neural machine translation by jointly learning to align and translate. 2015.
- [11] Z. Bain, F. Chudak, W. G. Macready, and G. Rose. The ising model: teaching an old problem new tricks,. *D-Wave Systems Technical Report*, 2010.
- [12] Z. Bain, F. Chudak, W. G. Macready, L. Clark, and F. Gaitan. Experimental determination of ramsey numbers with quantum annealing,. *Physical Review Letters*, 2013.
- [13] C. Bauckhage, R. Sanchez, and R. Sifa. Problem solving with hopfield networks and adiabatic quantum computing. 2020.
- [14] K. Beer, D. Bondarenko, T. Farrelly, T.J. Osborne, R. Salzmann, and R. Wolf. Efficient learning for deep quantum neural networks,. *ArXiv Quantum Physics*, 2019.

- [15] E.C. Behrman, J.E. Steck, and J. Niemel. Quantum associative memory with exponential capacity. 1998.
- [16] E.C. Behrman, J.E. Steck, S.R. Skinner, V.G. Chandrashekar, and L.R. Nash. A quantum hopfield network. *Simulations of quantum neural networks*, 1999.
- [17] M. Benaddy and M. Wakrim. Simulated annealing neural network for software failure prediction. *International Journal of Software Engineering and its Applications*, 2012.
- [18] Y. Bengio. *Learning deep architectures for AI*. Now Foundations and Trends, first edition edition, 2009.
- [19] Y. Bengio, O. Delalleau, and N. Le Roux. The curse of highly variable functions for local kernel machines. 2006.
- [20] Y. Bengio and A. Fischer. Early inference in energy-based models approximates back-propagation. *ArXiv*, 2015.
- [21] Y. Bengio, N. Le Roux, P. Vincent, O. Delalleau, and P. Marcotte. Convex neural networks. 2006.
- [22] Y. Bengio, N. Leonard, and A. Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *ArXiv*, 2013.
- [23] Y. Bengio, T. Mesnard, A. Fischer, S. Zhang, and Y. Wu. Stdp as presynaptic activity times rate of change of postsynaptic activity approximates back-propagation. *Neural Computation*, 2017.
- [24] A. J. Berkeley, M. W. Johnson, P. Bunyk, R. Harris, J. Johansson, T. Lanting, E. Ladizinsky, E. Tolkacheva, M. H. S. Amin, and G. Rose. A scalable readout system for a superconducting adiabatic quantum optimization system. *Superconductivity Sci. Technol*, 2010.
- [25] H.S. Bhatia and F. Philipson. Performance analysis of support vector machine implementations on the d-wave quantum annealer. *International Conference on Computational Science*, 2021.
- [26] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd. Quantum machine learning. *Nature*, 2017.
- [27] A. Blance and M. Spannowsky. Quantum machine learning for particle physics using a variational quantum classifier. 2020.
- [28] A. Blum and R.L. Rivest. Training a 3-node neural network is np-complete. *Neural Networks Volume 5*, 1992.
- [29] S. Boixo, T. Albash, F.M. Spedalieri, N. Chancellor, and D.A. Lidar. Experimental signature of programmable quantum annealing. *Nature Communications*, 2013.
- [30] M. Born and V. Fock. Beweis des adiabatensatzes,. *Zeitschrift für Physik A*, 1928.
- [31] M. Born and V. Fock. Quantum theory, the church-turing principle and the universal quantum computer. 1985.
- [32] E. Boros and A. Gruber. On quadratization of pseudo-boolean functions. *Discrete Applied Mathematics*, 2012.
- [33] B. E. Boser, I. M. Guyon, and V. Vapnik. A training algorithm for optimal margin classifiers. 1992.
- [34] A. Bovier and B. Niederhauser. The spin-glass phase-transition in the hopfield model with p-spin interactions. *International Press, Adv. Theor. Math. Phys.*, 2001.

- [35] L. Breiman. Bagging predictors. *Machine Learning*, 1994.
- [36] H.J. Briegel, D.E. Browne, W. Durr, R. Raussendorf, and M. Van den Nest. Measurement-based quantum computation. *Nature Physics*, 2009.
- [37] J. Bruck. On the convergence properties of the hopfield model. *Proceedings of the IEEE*, 1990.
- [38] P.I. Bunyk. Architectural considerations in the design of a supercomputing quantum annealing processor. *ArXiv Quantum Physics*, 2014.
- [39] C.J.C Burges. A tutorial on support vector machine for pattern recognition. *Data Mining and Knowledge Discovery*, 1998.
- [40] Z. Cai. Sa-gd: Improved gradient descent learning strategy with simulated annealing. *ArXiv Machine Learning*, 2021.
- [41] Y. Cao, G. Guerreschi, and A. Aspuru-Guzik. Quantum neuron: an elementary building block for machine learning on quantum computers. *ArXiv Quantum Physics*, 2017.
- [42] G. Cavallaro, D. Willsch, M. Willsch, K. Michielsen, and M. Riedel. Approaching remote sensing image classification with ensembles of support vector machines on the d-wave quantum annealer. 2022.
- [43] G. Cavallaro, M. Willsch, K. Michielsen, M. Farid, and M. Riedel. Quantum support vector machine algorithms for remote sensing data classification. 2021.
- [44] T.H. Chang, C.H. Lux, and S.S. Tipirneni. Least-squares solutions to polynomial systems of equations with quantum annealing. *Quantum Information Processing volume 18, Article number: 374 (2019)*, 2019.
- [45] A. M. Childs, E. Farhi, and J. Preskill. Robustness of adiabatic quantum computation. *Physical Review A*, 2001.
- [46] M. Childs, E. Farhi, and J. Preskill. Robustness of adiabatic quantum computation. *Physical Review A*, 2001.
- [47] B.A. Cipra. An introduction to the ising model. *The American Mathematical Monthly*, 1987.
- [48] C. Cortes and M. Mohri. AUC optimization vs. error rate minimization. *Proceedings of the 16th International Conference on Neural Information Processing Systems, NeurIPS*, 2003.
- [49] M. Courbariaux, I. Hubara, D. Soudry, R. El-Yaniv, and Y. Bengio. Binarized neural networks: Training deep neural networks with weights and activations constrained to+ 1 or-1. *ArXiv*, 2016.
- [50] M. Cristina Diamantini and C.A. Trugenberger. Quantum pattern retrieval by qubit networks with hebb interactions. *Physical Review Letters*, 2006.
- [51] M. Cristina Diamantini and C.A. Trugenberger. High-capacity quantum associative memories. *Journal of Applied Mathematics and Physics*, 2016.
- [52] G. Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 1989.
- [53] D-Wave. About d-wave: The practical quantum computing company. <https://www.dwavesys.com/company/about-d-wave/>.
- [54] D-Wave. Advantage the most connected and powerful quantum computer built for business. <https://www.dwavesys.com/solutions-and-products/systems/>.
- [55] D-Wave. The d-wave 2000q™ quantum computer technology overview. https://dwavejapan.com/app/uploads/2019/10/D-Wave-2000Q-Tech-Collateral_1029F.pdf.

- [56] D-Wave. D-wave documentation: D-wave qpu architecture: Topologies. https://docs.dwavesys.com/docs/latest/c_gs_4.html#topology-intro-pegasus.
- [57] D-Wave. D-wave ocean software documentation - dimod. https://docs.ocean.dwavesys.com/en/stable/docs_dimod/sdk_index.html.
- [58] D-Wave. D-wave ocean software documentation - dwave-hybrid. https://docs.ocean.dwavesys.com/en/stable/docs_hybrid/sdk_index.html.
- [59] D-Wave. D-wave ocean software documentation - dwave-inspector. https://docs.ocean.dwavesys.com/en/stable/docs_inspector/sdk_index.html.
- [60] D-Wave. D-wave ocean software documentation - minorminer. https://docs.ocean.dwavesys.com/en/stable/docs_minorminer/source/sdk_index.html.
- [61] D-Wave. D-wave ocean software documentation - packages. https://docs.ocean.dwavesys.com/en/stable/_gl=1*10mwv4o*_ga*NDY0MTEwNTIwLjE2NzY3MjAwNTU.*_ga_DXNKH9HE3W*MTY3NjcyNTU0Ni4yLjEuMTY3NjcyNTY5Ny4xMC4wLjA.
- [62] D-Wave. D-wave previews next-generation quantum computing platform. <https://ir.dwavesys.com/news/news-details/2019/D-Wave-Previews-Next-Generation-Quantum-Computing-Platform-02-27-2019/default.aspx>.
- [63] D-Wave. D-wave systems homepage. <https://www.dwavesys.com/>.
- [64] D-Wave. High order model. https://docs.ocean.dwavesys.com/en/stable/docs_dimod/reference/higherorder.html.
- [65] D-Wave. Leap docs - leap ide user guide. https://docs.dwavesys.com/docs/latest/doc_ide_user.html.
- [66] D-Wave. The leap the only real-time quantum cloud service built for business. <https://www.dwavesys.com/solutions-and-products/cloud-platform/>.
- [67] D-Wave. Learning to program the d-wave one. <https://dwave.wordpress.com/2011/05/11/learning-to-program-the-d-wave-one/>.
- [68] D-Wave. Let's get practical*: D-wave details product expansion cross platform roadmap. <https://www.dwavesys.com/company/newsroom/press-release/let-s-get-practical-d-wave-details-product-expansion-cross-platform-roadmap/>.
- [69] D-Wave. Make quadratic. https://docs.ocean.dwavesys.com/en/stable/docs_dimod/reference/generated/dimod.higherorder.utils.make_quadratic.html#dimod.higherorder.utils.make_quadratic.
- [70] D-Wave. Mit technology review, qa: D-wave's geordie rose. <https://www.technologyreview.com/2007/04/06/37553/qa-d-waves-geordie-rose/>.
- [71] D-Wave. product overview: The only end-to-end quantum platform. <https://www.dwavesys.com/solutions-and-products/product-overview/>.
- [72] D-Wave. reformulating problem. https://docs.dwavesys.com/docs/latest/handbook_reformulating.html#cb-techs-reduction-sub-spin.
- [73] D-Wave. solver docs. https://docs.dwavesys.com/docs/latest/guides_solvers.html#guides-solvers.
- [74] P. Date, R. Patton, C.D. Schuman, and T.E. Potok. A classical-quantum hybrid approach for unsupervised probabilistic machine learning. *FICC 2019: Advances in Information and Communication*, 2020.

- [75] Y. Dauphin, R. Pascanu, C. Gulcehre, K. Cho, S. Ganguli, and Y. Bengio. Identifying and attacking the saddle point problem in high-dimensional non-convex optimization. 2014.
- [76] J. Davis and M. Goadrich. The relationship between precision-recall and roc curves. *Proceedings of the 23rd International Conference on Machine Learning*, 2006.
- [77] E.P. De Benedictis. A future with quantum machine learning. *IEEE Computer Volume: 51 Issue: 2*, 2018.
- [78] B. Dema, J. Arai, and K. Horikawa. Support vector machine for multiclass classification using quantum annealers. 2020.
- [79] A. Demian, G Battaglia, E. Santoro, and E. Tosatti. Optimization by quantum annealing: Lessons from hard 3-sat cases. *Physical Review E*, 2005.
- [80] M. Demircigil, J. Heusel, M. Löwe, S. Upgang, and F. Vermet. On a model of associative memory with huge storage capacity. *Journal of Statistical Physics*, 2017.
- [81] V. Denchev. Binary classification with adiabatic quantum optimization., 2013.
- [82] V. Dixit, R. Selvarajan, T. Aldwairi, Y. Koshka, M.A. Novotny, T.S. Humble, and M.A. Alam. Training a quantum annealing based restricted boltzmann machine on cybersecurity data. *IEEE Transactions on Emerging Topics in Computational Intelligence*, 2022.
- [83] Y. Dodge. *The Oxford Dictionary of Statistical Terms*. Oxford University Press, first edition edition, 2006.
- [84] P. Domingos. Every model learned by gradient descent is approximately a kernel machine. *ArXiv Machine Learning*, 2020.
- [85] P. Domingos. Every model learned by gradient descent is approximately a kernel machine., *ArXiv Machine Learning*, 2020.
- [86] J. III Dulny and J. Kim. Developing quantum annealer driven data discovery. *ArXiv Quantum Physics*, 2016.
- [87] F. Dyson. *Infinite in All Directions: Gifford Lectures Given at Aberdeen*. Perennial, 1985.
- [88] J. Engel. Teaching feed-forward neural networks by simulated annealing. *Complex Systems*, 1988.
- [89] G. Faber and A. Giraldi. Models for quantum neural networks. <http://arquivosweb.lncc.br/pdfs/QNN-Review.pdf> (2002).
- [90] S. Fahlman and C. Lebiere. The cascade correlation learning architecture. *Technical Report, Carnegie Mellon University*, 1990.
- [91] E. Farhi, J. Goldstone, S. Gutmann, J. Lapan, A. Lundgren, and D. Preda. A quantum adiabatic evolution algorithm applied to random instances of an np-complete problem. *Science*, 20 April 2001.
- [92] E. Farhi, J. Goldstone, S. Gutmann, and M. Sipser. Quantum computation by adiabatic evolution. *ArXiv Quantum Physics*, 2000.
- [93] A. Farinelli. A quantum annealing approach to biclustering. 2016.
- [94] J. Feng and M. Byungwon. Simulated annealing ann approach for parameter optimization of micro-scaled flow channels formation by electrochemical machining. *International Journal of Electrochemical Science*, 2022.

- [95] R. Fletcher. *Practical Methods of Optimization*. John Wiley and Sons, second edition edition, 1987.
- [96] B. M. Forrest. Content-addressability and learning in neural networks. *Journal of Physics A: Mathematical and General*, 1988.
- [97] W.J. Freeman and G. Vitiello. The dissipative quantum model of brain and laboratory observations. *Journal of Physics A: Mathematical and Theoretical*, 2008.
- [98] Y. Freund and R. E. Schapire. Game theory on-line prediction and boosting. 1996.
- [99] A. Frisch. Ibmq introduction into quantum computing with live demo. 2017.
- [100] A. Frisch. Intel advances quantum and neuromorphic computing research’. *Consumer Electronic Show*, 2018.
- [101] Y. Fu and P. W. Anderston. Applications of statistical mechanics to np-complete problems in combinatorial optimisation,. *Journal of Physics A*, 1986.
- [102] S. Fusi. Memory capacity of neural network model. *ArXiv Neurons and Cognition*, 2021.
- [103] S. Garg and G. Ramakrishnan. Advances in quantum deep learning: An overview. *ArXiv quantum computing*, 2020.
- [104] X. Glorot, A. Bordes, and Y. Bengio. Deep sparse rectifier neural networks. *Proceedings of Machine Learning Research*, 2011.
- [105] X. Glorot, A. Bordes, and Y. Bengio. Deep sparse rectifier neural networks. *AISTATS’2011*, 2011.
- [106] F. Glover, G. Kochenberger, and Y. Du. Quantum bridge analytics i: A tutorial on formulating and using qubo models. *Annals of Operations Research*, 2018.
- [107] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, first edition edition, 2016.
- [108] I. J. Goodfellow, O. Vinyals, and A. M. Saxe. Qualitatively characterizing neuralnetwork optimization problems. 2015.
- [109] W. Haibo, W. Wang, Y. Liu, and B. Alidaee. Integrating machine learning algorithms with quantum annealing solvers for online fraud detection. *IEEE Volume: 10*, 2022.
- [110] S. Hameroff. Quantum computation in brain microtubules? the penrose–hameroff ‘orch or’ model of consciousness. *Philosophical Transactions: Mathematical, Physical and Engineering Sciences*, 1998.
- [111] R. Harris, J. Johansson, A. J. Berkley, M. W. Johnson, T. Lanting, S. Han, P. Bunyk, E. Ladizinsky, T. Oh, I. Perminov, E. Tolkacheva, S. Uchaikin, E. Chapple, C. Enderud, C. Rich, J. Wang, B. Wilson, and G. Rose. Experimental demonstration of a robust andscalable flux qubit. *QPhys. Rev B*, 2010.
- [112] D.O. Hebb. *The Organization of Behavior*. John Wiley Sons. Inc., first edition edition, 1949.
- [113] G. Helmut, F.H. Katzgraber, and S. A. Ruben. Glassy chimeras could beblind to quantum speedup: Designing better benchmarks for quantum annealing machines. *ArXiv*, 2014.
- [114] J. Hertz, A. Krogh, and R. Palmer. *Introduction to the Theory of Neural Computation*. Addison-Wesley, first edition edition, 1991.
- [115] C.F. Higham and A. Bedford. Quantum deep learning: Sampling neural nets with a quantum annealer. *Nature Scientific Reports*, 2021.

- [116] M. Hilbert and P. Lopez. The world’s technological capacity to store, communicate, and compute information. *science*, 2011.
- [117] G. Hinton. Deep belief networks. *Scholarpedia*, 2009.
- [118] G. E. Hinton, L. Deng, D. Yu, G. E. Dahl, A. Mohamed, N. Jaitly, A. Senior, V. Vanhoucke, P. Nguyen, T. N. Sainath, and B. Kingsbury. Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups. *IEEE Signal Processing*, 2012.
- [119] G. E. Hinton and T. Shallice. Lesioning an attractor network: investigations of acquired dyslexia. *Psychological review*, 1991.
- [120] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 1997.
- [121] J. J. Hopfield. Neurons with graded responses have collective computational properties like those of two-state neurons. *Proceedings of the National Academy of Sciences*, 1984.
- [122] J.J. Hopfield. Neural networks and physical systems with emergent collective computational abilities. *PNAS*, 1982.
- [123] H. Horner, D. Bormann, M. Frick, H. Kinzelbach, and A. Schmidt. Transients and basins of attraction in neural network models. *Zeitschrift fur Physik B*, 1989.
- [124] C.W. Hsu, C.C. Chang, and C.J. Lin. A practical guide to support vector classification. *Technical Report (Department of Computer Science, National Taiwan University)*, 2003.
- [125] F. Hu, B. Wang, N. Wang, and C. Wang. Quantum machine learning with d-wave quantum computer. *Advances in Quantum Engineering*, 2019.
- [126] F.N. Iandola, M.W Moskevicz, K. Ashraf, and K. Keutzer. Firecaffe: near-linear acceleration of deep neural netowrk training on computer custers. 2016.
- [127] G. Iatropoulos, J. Brea, and W. Gerstner. Kernel memory networks: A unifying framework for memory modeling. 2022.
- [128] S. Ioffe and C. Szegedy. Batch normalization: Accelerating deep network trainingby reducing internal covariate shift. 2015.
- [129] S. Ioffe and C. Szegedy. Batch normalization: Accelerating deep network trainingby reducing internal covariate shift. 2015.
- [130] H. Ishikawa. Transformation of general binary mrf minimization to the first-order case. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2011.
- [131] A. Jacot, F. Gabriel, and C. Hongler. Neural tangent kernel: Convergence and generalization in neural networks. *Advances in Neural Information Processing Systems*, 2018.
- [132] C. Jin, P. Netrapalli, R. Ge, S.M. Kakade, and M.I. Jordan. On nonconvex optimization for machine learning:. *Gradients, stochasticity, and saddle points. Journal of the ACM*, 2021.
- [133] S.J. Judd. *Neural network design and the complexity of learning*. MIT Press, 1988.
- [134] S.C. Kak. On quantum neural computing. *Advances in Imaging and Electron Physics*, 1995.
- [135] I. Kanter and H. Sompolinsky. Associative recall of memory without errors. *Physical Review*, 1987.
- [136] S. S. Keerthi and C.J. Lin. Asymptotic behaviors of support vector machines with gaussian kernel. *Neural Comput*, 2003.
- [137] K.K. Kenichi, S. Tanaka, and S. Miyashita. Quantum annealing for clustering. 2009.

- [138] A. Khan, N. Akhtar, and M. Qureshi. Real-time credit-card fraud detection using artificial neural network tuned by simulated annealing algorithm. *Proceedings of International Conference on Recent Trends in Information, Telecommunication and Computing*, 2014.
- [139] M. Kim and P. Smaragdis. Bitwise neural networks. *ArXiv*, 2016.
- [140] S. Kirkpatrick and M. P. Vecchi. Optimization by simulated annealing. *Science*, 1983.
- [141] L.B. Kish. End of moore’s law: thermal (noise) death of integration in micro and nano electronics. *Physical Letters A*, 2002.
- [142] C.S. Koh and O.A Mohammed. Detection of magnetic body using artificial neural network with modified simulated annealing. *IEEE Transactions on Magnetics*, 1994.
- [143] Y. Koren. The bellkor solution to the netflix grand prize. *Computer Science*, 2009.
- [144] B. Kosko. Bidirectional associative memories. 1988.
- [145] A. Krizhevsky, I. Sutskever, and G. Hinton. Imagenet classification with deepconvolutional neural networks. 2012.
- [146] A. Krizhevsky, I. Sutskever, and G.E. Hinton. Imagenet classification with deep convolutional neural networks. *Communications of the ACM*, 2012.
- [147] D. Krotov and J. Hopfield. Dense associative memory for pattern recognition. *Advances in Neural Information Processing Systems*, 2016.
- [148] D. Krotov and J. Hopfield. Large associative memory problem in neurobiology. 2021.
- [149] V. Kumar, G. Bass, C. Tomlin, and J. Dulny III. Quantum annealing for combinatorial clustering. *Quantum Information Process, Springer*, 2017.
- [150] E. Kusuma, I. Pantiawati, and S. Handayani. Melanoma classification based on simulated annealing optimization neural network. *Knowledge Engineering and Data Science*, 2021.
- [151] T. Lanting, A. J. Przbyasz, A. Y. Smirnov, F. M. Spedalieri, M. H. Amin, A. J. Berkely, R. Harris, F. Altomare, S. Boixo, P. Bunyk, N. Dickson, C. Enderud, J. P. Hilton, E. Hoskinson, M. W. Johnson, E. Ladizinsky, N. Ladizinsky, R. Neufeld, T. Oh, I. Perminov, C. Rich, M.C. Tom, E. Tolkacheva, S. Uchaikin, A.B. Wilson, and G. Rose. Entanglement in a quantum annealing processor. *Physical Review X* 4, 2014.
- [152] J.B. Lasserre. Global optimization with polynomials and the problem of moments. *SIAM Journal on Optimization*, 2001.
- [153] A. Le Ny. Introduction to (generalized) gibbs measures. *ensaios matematico 2008*, 2008.
- [154] Y. LeCun. Mnist official website. <https://yann.lecun.com/exdb/mnist/>.
- [155] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. 1998.
- [156] R.P. Lippmann, B. Gold, and M.L. Malpass. A comparison of hamming and hopfield neural nets for pattern classification. *Massachusetts Institute of Technology Technical Report 769*, 1987.
- [157] S. Lloyd. Quantum hanky-panky: a conversation with seth lloyd. https://www.edge.org/conversation/seth_lloyd-quantum-hanky-panky14.
- [158] G.L. Long. Duality quantum computing and duality quantum information processing. *International Journal for Theoretical Physics*, 2011.
- [159] M. Marvin and P. Seymour. *Perceptrons: An Introduction to Computational Geometry*. MIT Press, expanded, subsequent edition edition, 1969.

- [160] S. Masha, T. Brunello, and C. Tassi. Quantum hopfield model. *Physics 2020*, 2019.
- [161] W. Mcculloch and W. Pitts. A logical calculus of the ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics*, 1943.
- [162] R. McEliece, E. Posner, E. Rodemich, and S. Venkatesh. The capacity of the hopfield associative memory. *IEEE Transactions on Information Theory*, 1987.
- [163] C. C. McGeoch and C. Wang. Experimental evaluation of an adiabatic quantum system for combinatorial optimization. 2013.
- [164] C.C. McGeoch. *Adiabatic Quantum Computation and Quantum Annealing, Theory and Practice*. MorganClaypool publishes, first edition edition, 2014.
- [165] N. Meinhardt, N.M.P. Neumann, and F. Philipson. Quantum hopfield neural networks a new approach and its storage capacity. *Computational Science – ICCS*, 2020.
- [166] N. Meinhardt, N.M.P. Neumann, and F. Philipson. Quantum hopfield neural networks a new approach and its storage capacity,. *Computational Science – ICCS*, 2020.
- [167] M. Mezard, G. Parisi, and M. A. Virasoro. *Spin Glass Theory and Beyond*. World Scientific Pub Co Inc, 1988.
- [168] E. N. Miller and S. Mukhopadhyay. A quantum hopfield associative memory implemented on an actual quantum processor. *Nature Scientific Reports*, 2021.
- [169] D. Mishkin and J. Matas. All you need is a good init. 2015.
- [170] T.M. Mitchell. *Machine Learning*. McGraw-Hill, "first edition" edition, 1997.
- [171] S. Morita and H. Nishimori. Mathematical foundation of quantum annealing. *Journal of Mathematical Physics*, 2008.
- [172] A. Mott, J. Job, J. Vlimant, D. Lidar, and M. Spiropulu. Solving a higgs optimization problem with quantum annealing for machine learning. *Nature*, 2017.
- [173] J.R. Movellan. Contrastive hebbian learning in the continuous hopfield model. 1990.
- [174] R.K. Nath, H. Thapliya, and T.S. Humble. Quantum annealing for automated feature selection in stress detection. 2021.
- [175] R.K. Nath, H. Thapliyal, and T.S. Humble. A review of machine learning classification using quantum annealing for real-world applications. *SN Computer Science*, 2021.
- [176] R. Nayak and J.D. Sharma. A hybrid neural network and simulated annealing approach to the unit commitment problem. *Computers Electrical Engineering*, 2000.
- [177] R. Neigovzen, J.L. Neves, R. Sollacher, and S.J. Glaser. Quantum pattern recognition with liquid-state nuclear magnetic resonance. *Physical Reviews*, 2009.
- [178] R. Neigozven, J.L. Neves, R. Sollacher, and S.J. Glaser. Quantum pattern recognition with liquid-state nuclear magnetic resonance,. *Physical Review A*, 2009.
- [179] H. Neven, V.S. Denchev, G. Rose, and W.G. Macready. Training a binary classifier with the quantum adiabatic algorithm. *ArXiv Quantum Physics*, 2008.
- [180] H. Neven, V.S. Denchev, G. Rose, and W.G. Macready. Training a large scale classifier with a quantum adiabatic algorithm. *ArXiv Quantum Physics*, 2009.
- [181] H. Nevin, S. V. Denchev, G. Rose, and G.M. Macready. Training abinary classifier with the quantum adiabatic algorithm,. *ArXiv*, 2008.

- [182] H. Nevin, S.V. Denchev, G. Rose, and W. G. Macready. Training a large scale classifier with the quantum adiabatic algorithm,. *ArXiv*, 2009.
- [183] H. Nevin, G. Rose, and G. W Macready. Image recognition with anadiabatic quantum computer i: Mapping to quadratic unconstrained binary optimization,. *ArXiv*, 2008.
- [184] N.T.T. Nguyen and G.T. Kenyon. Image classification using quantum inference on the d-wave 2x. 2018.
- [185] K. Nicholas and T.D. Gedeon. Simulated annealing and weight decay in adaptive learning: The sarprop algorithm. *IEEE Transactions on Neural Networks*, 1998.
- [186] M. Ohkuwa, H. Nishimori, and D. A. Lidar. Reverse annealing for the fully connected p-spin model. 2018.
- [187] M. Ohzeki and H. Nishimori. Quantum annealing: An introduciton and new developments,. *Journal of Computational and Theoretical Nanoscience*, 2011.
- [188] E. Pelofske, G. Hahn, and H. Djidjev. Optimizing the spin reversal transform on the d-wave 2000q. 2019.
- [189] A. Perdomo-Ortiz, N. Dickson, M. Drew-Brook, G. Rose, and A. Aspuru-Guzik. Finding low-energy conformations of lattice protein models by quantum annealing,. *Nature Scientific Reports*, 2012.
- [190] M. P. Perrone and L. N. Cooper. When networks disagree: Ensemble methods for hybrid neural networks. *World Scientific Series in 20th Century Physics*, 1994.
- [191] L. Personnaz, I. Guyon, and G. Dreyfus. Collective computational properties of neural networks: New learning mechanisms,. *Physical Review A*, 1986.
- [192] M. Perus. Neural networks as a basis for quantum associative memory. *Neural Network World*, 2000.
- [193] F. Philpson, R.S. Wezeman, and I Chiscop. Three quantum machine learning approaches for mobile user indoor-outdoor detection. *Machine Learning for Networking*, 2020.
- [194] M. Piattini, G. Peterssen, R. Pérez-Castillo, J.L. Hevia, M.A. Serrano, G. Hernández, I.G.R. de Guzman, C.A. Paradela, M. Polo, and E. Murina. The talavera manifesto for quantum software engineering and promgramming, 2020.
- [195] F.J. Pineda. Generalization of back-propagation to recurrent neural networks., 1987.
- [196] T.E. Potok, C.D. Schuman, S.R. Young, R.M. Patton, F. Spedalieri, J. Liu, K.T. Yao, G. Rose, and G CHakma. A study of complex deep learning networks on high performance, neuromorphic and quantum computers. *Machine Lecarning in HPC environments (MLHPC), Workshop on, IEEE*, 2016.
- [197] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery. *Numerical Recipes: The Art of Scientific Computing*. Cambridge University Press, third edition edition, 2007.
- [198] K. Rahul, R.K. Banyal, P. Goswami, and V. Kumar. Machine learning algorithms for big data analytics. *Computational Methods and Data Engineering*, 2021.
- [199] H. Ramsauer, B. Schafl, J. Lehner, P. Seidl, M. Widrich, T. Adler, L. Gruber, M. Holzleitner, M. Pavlovic, K.G. Sandve, V. Greiff, D. Kreill, M. Kopp, G. Klaumbauer, J. Brandstetter, and S. Hochreiter. Hopfield network is all you need. *ArXiv Neural and Evolutionary Computing*, 2021.
- [200] B. Rebentrost and L. Weedbrook. Quantum hopfield neural networks. *Physical Review A*, 2018.

- [201] L.M.R. Rere, M.I. Fanany, and A.M. Arymurthy. Simulated annealing algorithm for deep learning. *Procedia Computer Science*, 2015.
- [202] G.G. Rigatos and S.G. Tzafestas. Neurodynamics and attractors in quantum associative memories. *Integrated Computer-Aided Engineering*, 2007.
- [203] R. Roja. *Neural Networks, A Systematic Introduction*. Springer, first edition edition, 1996.
- [204] J. Roland and N.J. Cerf. Quantum search by local adiabatic evolution. *Physical Review A*, 2001.
- [205] G. Rose and W. G. Macready. An introduction to quantum annealing,. dwave.files.wordpress.com/.../20070810_d-wave_quantum_annealing.pdf.
- [206] F. Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 1958.
- [207] P. Rotondo, M. Marcuzzi, J. P. Garrahan, I. Lesanovsky, and M. Müller. Open quantum generalisation of hopfield neural networks. *Journal of Physics A: Mathematical and Theoretical*, 2020.
- [208] A. Ruiz. Quantum annealing, brief description on the state of the art of some local optimization methods. 2014.
- [209] D. Rumelhart, G. Hinton, and R. William. Learning representations by back-propagating errors. *Nature*, 1986.
- [210] M. Sasdelli and T. Chin. Quantum annealing formulation for binary neural networks. *Proceedings of the Digital Image Computing: Techniques and Applications*, 2021.
- [211] I. Sato, S. Tanaka, K. Kurihara, S. Miyashita, and H. Nakagawa. Quantum annealing for dirichlet process mixture models with applications to network clustering. *Neurocomputing*, 2013.
- [212] B. Scellier and B. Yoshua. Equilibrium propagation: Bridging the gap between energy-based models and backpropagation. *Frontiers of Computational Neuroscience*, 2017.
- [213] M. Schenk, E.F. Combarro, M. Grossi, V. Kain, K.S.B. Li, M. Popa, and S. Vallecorsa. Hybrid actor-critic algorithm for quantum reinforcement learning at cern beam lines. *ArXiv Quantum Physics*, 2022.
- [214] W. Schiffmann, M. Joost, and R. Werner. Comparison of optimized backpropagation algorithms. 1993.
- [215] M. Schuld, I. Sinayskiy, and F. Petruccione. The quest for a quantum neural network. *Quantum Information Processing*, 2014.
- [216] M. Schuld, I. Sinayskiy, and F. Petruccione. The quest for a quantum neural network. *Quantum Information Processing*, 2014.
- [217] B. Schölkopf, C. J. C. Burges, and A. J. Smola. *Advances in Kernel Methods —Support Vector Learning*. MIT Press, Cambridge, MA., first edition edition, 1999.
- [218] H. Seddiqi and T.S. Humble. Adiabatic quantum optimization for associative memory recall. *Frontiers of Physics*, 2014.
- [219] R. Sexton, J.D. Johnson, and R.E. Dorsey. Beyond backpropagation: Using simulated annealing for training neural networks. *Journal of End User Computing*, 1999.
- [220] P.W. Shor. Algorithms for quantum computation: discrete logarithms and factoring. 1994.
- [221] M. Steffen, W. van Dam, T. Hogg, G. Breyta, and I. Chuang. Experimental implementation of an adiabatic quantum optimization algorithm,. *Physical Review Letters*, 2003.

- [222] A. J. Storkey and R. Valabregue. The basins of attraction of a new hopfield learning rule. *Neural Networks*, 1999.
- [223] V. Sundararaghavan and S. Srivastava. Generative and discriminative training of boltzmann machine through quantum annealing. *Nature Scientific Reports*, 2023.
- [224] J.G Sutherland. The holographic cell: A quantum perspective. *Frontier decision support concepts: help desk, learning, fuzzy diagnoses, quality evaluation, prediction, evolution*, 1994.
- [225] I. Sutskever, O. Vinyals, and Q. V. Le. Sequence to sequence learning with neural networks. 2014.
- [226] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich. Going deeper with convolutions. technical report. 2014.
- [227] B.M. Terhal. Quantum supremacy, here we come. *Nature Physics*, 2018.
- [228] T. Tieleman and G. Hinton. Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude., 2012.
- [229] N.K. Treadgold and T.D. Gedeon. Combining simulated annealing and gradient descent for improved convergence in neural networks. 1998.
- [230] C.A. Trugenberger. Probabilistic quantum memories. *Physical Review Letters*, 2001.
- [231] C.A. Trugenberger. Probabilistic quantum memories. *Physical Review Letters*, 2001.
- [232] C.A. Trugenberger. Quantum pattern recognition. *Quantum Information Processing*, 2002.
- [233] V. Vapnik. *Estimation of Dependences Based on Empirical Data*. Springer Verlag, first edition edition, 1982.
- [234] V. Vapnik. *The Nature of Statistical Learning Theory*. Springer-Verlag, first edition edition, 1995.
- [235] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. 2017.
- [236] A. Veit, M. Wilber, and S. Belongie. Residual networks behave like ensembles of relatively shallow networks. 2016.
- [237] D. Ventura and T. Martinez. Quantum associative memory with exponential capacity. *International Joint Conference on Neural Networks*, 1998.
- [238] F. Verstraete, M.M. Wolf, and J.I. Cirac. Quantum computation and quantum-state engineering driven by dissipation. *Nature Physics*, 2009.
- [239] L. Viana and A. Coolen. Attraction domain in neural networks. *Journal de Physique*, 1993.
- [240] A. Ward and N. Bambos. Quantum annealing assisted deep learning for lung cancer detection. *SN Applied Sciences*, 2020.
- [241] S. Wendy. Associative learning and the hippocampus, 2005. <https://www.apa.org/science/about/psa/2005/02/suzuki>.
- [242] P. J. Werbos. Applications of advances in nonlinear sensitivity analysis. 1981.
- [243] E.P. Wigner. Remarks on the mind-body question. 1967.
- [244] C. K. I. Williams and C. E. Rasmussen. Gaussian processes for regression. 1996.

- [245] D. Willsch, M. Willsch, H.D. Raedt, and K. Michielsen. Support vector machines on the d-wave quantum annealer. *ArXiv Quantum Physics*, 2016.
- [246] M. Wilson, T. Vandal, T. Hogg, and E.G. Rieffel. Quantum-assisted associative adversarial network: applying quantum annealing in deep learning. *Quantum Machine Intelligence*, 2020.
- [247] Wired. Wired, d-wave defies world of critics with 'first quantum cloud'. <https://www.wired.com/wiredenterprise/2012/02/dwave-quantum-cloud/all/1>.
- [248] N. Xu, J. Zhu, D. Lu, X. Zhou, X. Peng, and J. Du. Quantum factorization of 143 on a dipolar-coupling nuclear magnetic resonance system. *Physical Review Letters*, 2012.
- [249] F.F. Yang, M. Sasdelli, and T. Chin. Training multilayer perceptrons by sampling with quantum annealers. *ArXiv*, 2023.
- [250] C. Yuan and S.S. Aguin. A comprehensive review of binary neural network. *Artificial Intelligence Review*, 2023.
- [251] X. Yuan, Z. Chen, Y. Liu, Z. Xie, X. Jin, Y. Liu, X. Wen, and H. Tang. Quantum support vector machines for aerodynamic classification. *ArXiv Quantum Physics*, 2022.
- [252] M. Zak and C.P. Williams. Quantum neural nets. *International Journal of Theoretical Physics*, 1997.

Appendix A Notation

This section provides a reference regarding notation used throughout this work.

A.1 Generic notation employed

Here is reported generic notation employed. Next section will report instead specific symbols used in this work.

Numbers and Arrays

a denotes scalar (integer or real). Note in this work Greek letters are used for constants or complex numbers (the context make it apparent)

$\mathbf{a}$ denotes a vector

$\mathbf{A}$ denotes a matrix

$|\alpha|$ denotes the magnitude of the complex number: $|\alpha| = |x + iy| = \sqrt{x^2 + y^2}$.

i denotes imaginary unit of a complex number

$\|\cdot\|$ denotes the matrix norm

Set and Graphs

A denotes a set

R denotes the set of real numbers

$\{0, 1\}$ denotes the set containing 0 and 1

$\{0, 1, \dots, n\}$ denotes the set of all integers between 0 and n

$[a, b]$ denotes the real interval including a and b

$(a, b]$ denotes the real interval excluding a but including b

Indexing

a_i denotes an element i of vector a , with indexing starting at 1

$A_{i,j}$ denotes an element i, j of matrix $\mathbf{A}$

$\mathbf{a}_i$ denotes a row vector of a matrix $\mathbf{A}$

Linear Algebra Operations

$\mathbf{A}^\top$ denotes transpose of matrix $\mathbf{A}$

$\mathbf{A} \odot \mathbf{B}$ denotes element-wise (Hadamard) product of $\mathbf{A}$ and $\mathbf{B}$

Calculus

$\frac{dy}{dx}$ denotes derivative of y with respect to x

$\frac{\partial y}{\partial x}$ denotes partial derivative of y with respect to x

$\nabla_x y$ denotes gradient of y with respect to x

$\int f(x)dx$ denotes definite integral over the entire domain of x

Probability and Information Theory

$P(a)$ denotes a cumulative probability distribution

$p(a)$ denotes a probability distribution

$a \sim P$ denotes random variable a has distribution P

$a \sim P$ denotes expectation of $f(x)$ with respect to $P(x)$

$\mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \mathbf{s})$ denotes Gaussian distribution over $\mathbf{x}$ with mean $\boldsymbol{\mu}$ and covariance $\mathbf{s}$

Functions

$f : A \rightarrow B$ denotes the function f with domain A and range B . Note that also capital letters are employed to denote functions and that in this work f/F and g/G are typically used to denote generic (or user specified) functions

$f \circ g$ Composition of the functions f and g

$f(\mathbf{x}; \boldsymbol{\theta})$ denotes a function of $\mathbf{x}$ parametrized by $\boldsymbol{\theta}$ (Sometimes $f(\mathbf{x})$ is simply written omitting the argument $\boldsymbol{\theta}$ to lighten notation)

$\log x$ denotes natural logarithm of x

$\exp(x)$ denotes exponential with base e of x

$\cosh(x)$ denotes hyperbolic cosine of x

$\text{Sign}(x)$ denotes sign function

$s(x)$ denotes logistic sigmoid, $\frac{1}{1+\exp(-x)}$

Datasets and Distribution

p_{data} denotes the data generating distribution

$\hat{p}_{\text{data}}$ denotes the empirical distribution defined by the training set

X denotes a set of training examples

$\mathbf{x}_i$ denotes the i -th example (input) from a dataset

y_i denotes the target associated with $\mathbf{x}_i$ for supervised learning

A.2 Specific symbol employed

general symbols

K denotes set of labels employed in classification, k number of labels and $\mathbf{k}$ array of labels for each data-point in dataset $\mathcal{D}$. To denotes a single label y_i^* is employed

$\mathcal{D}$ denotes dataset, with m number of datapoints (typically i is employed as index for iteration). Note however that for dense associative memories m is used to denote memories, if are used to implement feedforward network m is used to denote memories which store weights while $|\mathcal{D}|$ is used to explicit training dataset

$\mathcal{X}$ denotes input space, n is its dimensionality

$\mathbf{w}$ denotes a vector of parameters of a model. $\hat{\mathbf{w}}_n$ (or simply $\boldsymbol{\theta}$) represents its estimation over n data points .

η denotes some quantity of user choice

ε denotes infinitesimal quantity

h denotes Vapnik Chervonenkis (VC) dimension

$\mathbf{x}$ denotes generic input use, if input are bipolar $\mathbf{s}$ is used to avoid ambiguities with sigmoid symbol.

SVM

$k(\mathbf{x}, \mathbf{x}')$ denotes a generic inner product kernel $= \Phi(\mathbf{x})^T \Phi(\mathbf{x}')$ where Φ denotes kernel mapping
 ρ denotes margin of a SVM

C denotes regularization parameter of a SVM

b denotes bias component of a classifier

α_i denotes Lagrangian multiplier

ζ_i denotes slack variable

neural networks

$L(\mathbf{w})$ denotes cost function (loss) of a weight configuration

$E(t)$ denotes energy at time t (if time is not relevant in the formula dependency on t could be omitted)

$\mathbf{s}^{(t)}$ denotes global state of Hopfield at time t . Single neuron is represented as $s_i^{(t)}$. In general for

a dense associative memory is employed, as bit index, μ memory index (a superscript if memories are stored as different vectors), m for the number of memory stored (note also that n is the length of memories), ξ^μ memory vector.

μ denotes mask vector of dropout

γ denotes Hebbian learning rate

$\mathbf{w}^t$ denotes neural network weights at a specific time step. $w(t)$ function retrieves weights $\mathbf{w}^t$. d is the total number of parameters

Quantum computation

$|\phi\rangle$ denotes a superposition state containing two complex numbers α and β . south poles correspond to the standard basis, meaning states $|0\rangle$ and $|1\rangle$

$\mathcal{H}_t$ denotes Hamiltonian at time t , with eigenvalues λ

$\hbar$ denotes Planck's constant

δ_s denotes the spectral gap, the positive difference between the eigenvalues of the ground state and of the first excited state, at any time s .

δ_m denotes the minimum spectral gap

$\tau(s)$ determines the rate at which the Hamiltonian changes as a function of s .

D^n denotes solution space of a local search problem

$\mathcal{H}_D$ denotes a transverse field Hamiltonian (also called the disordering Hamiltonian) $\mathcal{H}(t)$ denotes time-dependent Hamiltonian

h^* denotes external field in Ising model

Appendix B Chapters integrations

Here are attached some integrations of present work chapters which were removed to make lighter fruition of content. However these sections provide focus regarding content of present work.

B.1 Machine learning

This section contains integration of chapter 1.1 of present work

Asynchronous bidirectional associative network dynamic

The typical asynchronous bidirectional associative network dynamic is exposed thereafter using as reference figure B.1 [203]. The network maps an n -dimensional row vector $\mathbf{x}_0$ to a k -dimensional row vector $\mathbf{y}_0$. The $n \times k$ weight matrix of the network is denoted by $\mathbf{W}$ so that the mapping computed in the first step can be written as

$$\mathbf{y}_0 = \text{sgn}(\mathbf{x}_0 \mathbf{W}) \quad (\text{B.1})$$

In the feedback step $\mathbf{y}_0$ is treated as the input and the new computation is

$$\mathbf{x}_1^T = \text{sgn}(\mathbf{W} \mathbf{y}_0^T) \quad (\text{B.2})$$

After m iterations the system has computed a set of $m+1$ vector pairs $(\mathbf{x}_0, \mathbf{y}_0), \dots, (\mathbf{x}_m, \mathbf{y}_m)$ which fulfill the conditions

$$\mathbf{y}_i = \text{sgn}(\mathbf{x}_i \mathbf{W}) \quad (\text{B.3})$$

and

$$\mathbf{x}_{i+1}^T = \text{sgn}(\mathbf{W} \mathbf{y}_i^T) \quad (\text{B.4})$$

After some iterations a fix-point $(\mathbf{x}, \mathbf{y})$ is found if $\mathbf{y} = \text{sgn}(\mathbf{x} \mathbf{W})$ and $\mathbf{x}^T = \text{sgn}(\mathbf{W} \mathbf{y}^T)$ hold. Hebbian learning can be used to compute an adequate matrix $\mathbf{W}$. If $\mathbf{W}$ is defined as $\mathbf{W} = \mathbf{x}^T \mathbf{y}$, as prescribed by Hebbian learning, then

$$\mathbf{y} = \text{sgn}(\mathbf{x} \mathbf{W}) = \text{sgn}(\mathbf{x} \mathbf{x}^T \mathbf{y}) = \text{sgn}(\|\mathbf{x}\|^2 \mathbf{y}) = \mathbf{y} \quad (\text{B.5})$$

and also

$$\mathbf{x}^T = \text{sgn}(\mathbf{W} \mathbf{y}^T) = \text{sgn}(\mathbf{x}^T \mathbf{y}^T) = \text{sgn}(\mathbf{x}^T \|\mathbf{y}\|^2) = \mathbf{x}^T \quad (\text{B.6})$$

For a set of m vector pairs the matrix $\mathbf{W}$ is set to

$$\mathbf{W} = \mathbf{x}_1^T \mathbf{y}_1 + \mathbf{x}_2^T \mathbf{y}_2 + \dots + \mathbf{x}_m^T \mathbf{y}_m \quad (\text{B.7})$$

If $\mathbf{y}_0$ is now used for a new iteration from the right, excitation of the units in the left layer can be summarized in an excitation vector $\mathbf{e}$ computed according to

$$\mathbf{e}^T = \mathbf{W} \mathbf{y}_0. \quad (\text{B.8})$$

The vector pair $(\mathbf{x}_0, \mathbf{y}_0)$ is a stable state of the network if $\text{sgn}(\mathbf{e}) = \mathbf{x}_0$. All vectors $\mathbf{e}$ close enough to $\mathbf{x}_0$ fulfill this condition. These vectors differ from $\mathbf{x}_0$ by a small angle and therefore the product $\mathbf{x}_0 \mathbf{e}^T$

is larger than for other vectors of the same length but further away from $\mathbf{x}_0$. The product

$$E = -\mathbf{x}_0 \mathbf{e}^T = -\mathbf{x}_0 \mathbf{W} \mathbf{y}_0^T \quad (\text{B.9})$$

is therefore smaller (because of the minus sign) if the vector $\mathbf{W} \mathbf{y}_0^T$ lies closer to $\mathbf{x}_0$. The scalar value E can be used as a kind of index of convergence to the stable states of an associative memory. E denotes the energy function of the network.

$$E(\mathbf{x}_i, \mathbf{y}_i) = -\frac{1}{2} \mathbf{x}_i \mathbf{W} \mathbf{y}_i^T \quad (\text{B.10})$$

Adding thresholds is equivalent to the introduction of an additional unit with constant output 1 into each layer. The weight of each edge from a constant unit to each one of the others is the negative threshold of the connected unit. It is straightforward to deduce that the energy function of the extended network can be written as

$$E(\mathbf{x}_i, \mathbf{y}_i) = -\frac{1}{2} \mathbf{x}_i \mathbf{W} \mathbf{y}_i^T + \frac{1}{2} \theta_r \mathbf{y}_i^T + \frac{1}{2} \mathbf{x}_i \theta_\ell^T \quad (\text{B.11})$$

The row vector of thresholds of the k units in the left layer is denoted in the above expression by θ . The row vector of thresholds of the n units in the right layer is denoted by θ_r . Using the energy function it can be shown that a BAM arrives at a stable state after a finite number of iterations. Expanding in different order the products, the energy function can be written in the two equivalent forms

$$E(\mathbf{x}, \mathbf{y}) = -\frac{1}{2} \sum_{i=1}^k e_i y_i \quad (\text{B.12})$$

$$E(\mathbf{x}, \mathbf{y}) = -\frac{1}{2} \sum_{i=1}^n g_i x_i \quad (\text{B.13})$$

The state of unit i on the left layer changes only when the excitation g_i has a different sign than x_i , the present state. The state is updated from x_i to x'_i , where x'_i now has the same sign as g_i . Since the other units do not change their state, the difference between the previous energy $E(\mathbf{x}, \mathbf{y})$ and the new energy $E(\mathbf{x}', \mathbf{y})$ is

$$E(\mathbf{x}, \mathbf{y}) - E(\mathbf{x}', \mathbf{y}) = -\frac{1}{2} g_i (x_i - x'_i) \quad (\text{B.14})$$

Since both x_i and $-x_i$ have a different sign than g_i it follows that

$$E(\mathbf{x}, \mathbf{y}) - E(\mathbf{x}', \mathbf{y}) > 0 \quad (\text{B.15})$$

Markovian dynamic of hopfield network

Here a clarifying example from [203]: given the network and its corresponding dynamics in figure. Every transition has the same probability, one third in this case. It could be noted that there are two states, $(-1, 1, -1)$ and $(1, -1, 1)$, not reachable from any other state. These states are called "urstates" or "garden of Eden configurations". In this case there is only a single stable state, $(-1, -1, -1)$. In the second example instead two minima arise, a local one $(-1, 1, -1)$, and a global one $(1, -1, 1)$.

Learning rules and basin of attraction focus on

A learning rule has typically some desired properties e.g. locality, being incremental, being immediate, shape and dimension of basin of attraction. A rule is said to be local if update of a particular connection depends only on information available to the neurons on either side of the connection. As said before, locality is important for neural plausibility purposes. If the learning process can modify an old network configuration to memorise a new pattern without referring to any of the previously learnt patterns, the learning rule is called incremental. An immediate learning rule updates are performed immediately. As anticipated, final property that is influenced by learning rule is the basin of

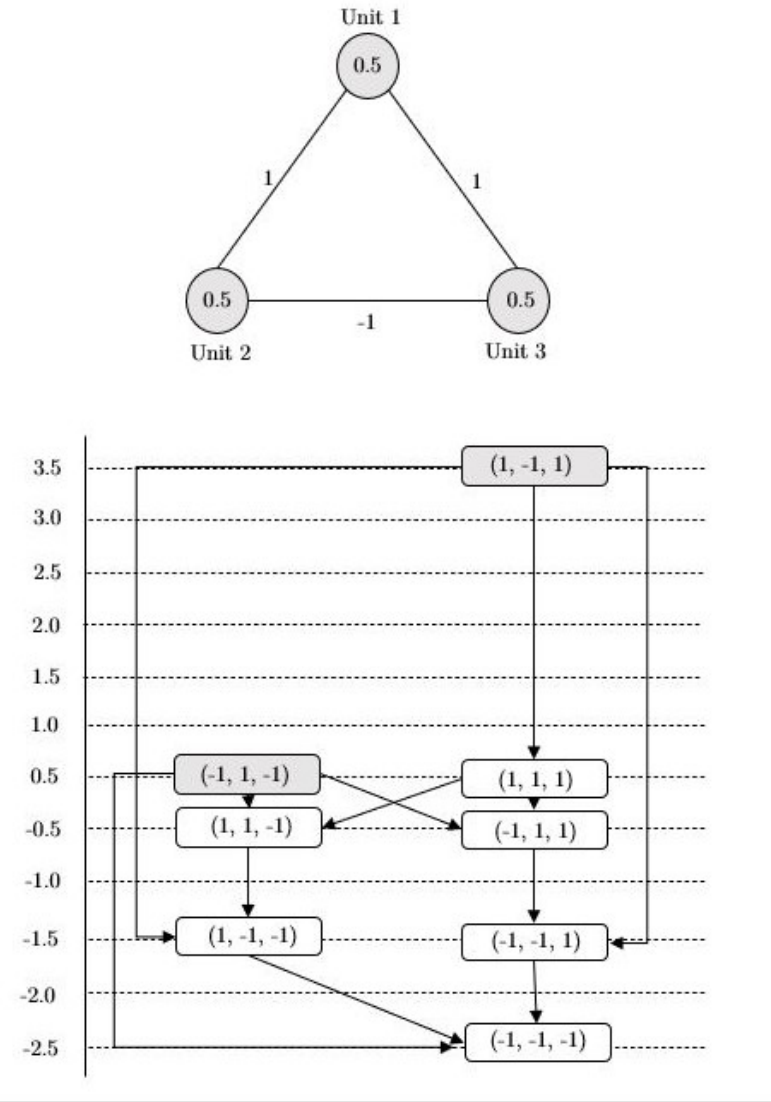


Figure B.1: Example of Hopfield Neural Network and it's Markov dynamic. In this example each state is represented by an oval located in the corresponding energy level. Gray oval represent "urstates"; picture inspired by [203]

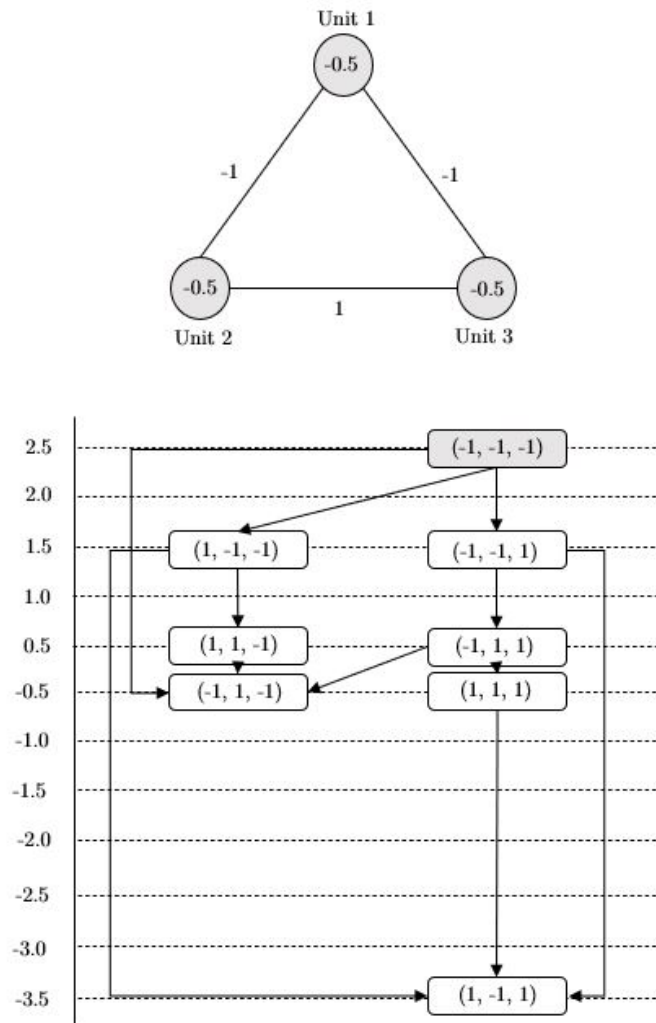


Figure B.2: Example taken from [203]. As before but with local minima depicted; picture inspired by [203]

attraction. This concept is defined thereafter, however it should be noted basins of attraction of a network influence absolute capacity of the network itself. There are two other relevant learning rules: Storkey learning rule and Projection learning rule. Note that all of them converge to Hebbian learning rule for orthogonal patterns. The Storkey learning rule defines the synaptic weights in an iterative fashion as

$$w_{ij}^t = w_{ij}^{t-1} + \frac{1}{n} \xi_i^t \xi_j^t - \frac{1}{n} \xi_i^t h_{ji}^t - \frac{1}{n} h_{ij}^t \xi_j^t \quad (\text{B.16})$$

where ξ^t is the memory to be learned in the t -th iteration for $t = 1$ to m . Here subscript t is used to underline iterative aspect of memory, typically μ or ν is used as memory index. Then

$$h_{ij}^t = \sum_{k=1, k \neq i, j} w_{ik}^{t-1} \xi_k^t \quad (\text{B.17})$$

is the local field at the i -th neuron [222]. The final synaptic weights storing p memories is given by $w_{ij} = w_{ij}^p$. The asymptotic Storkey capacity under perfect recall is $n/\sqrt{2 \ln n}$. The Storkey rule is also incremental.

Another approach, the projection rule defines the synaptic weights for m memories as

$$w_{ij} = \frac{1}{n} \sum_{\mu, \mu'=1}^m \xi_i^\mu C_{\mu\mu'}^{-1} \xi_j^{\mu'} \quad (\text{B.18})$$

where $C_{\mu\mu'} = \frac{1}{n} \sum_{k=1}^n \xi_k^\mu \xi_k^{\mu'}$ is the covariance matrix and C^{-1} is the inverse of C . This rule has a theoretical capacity of n for linearly independent patterns and approximately $n/2$ for interfering memories [191] [135]. However, the projection rule is neither local nor incremental.

As said before, choice of the learning rule defines Basins of Attraction. A basin of attraction of a fixed point or stable state of a network is defined to be the set of states that are attracted to that fixed point. Basins of attraction in general should be large, allowing more states to be attracted to stored patterns instead spurious patterns, should be rounded, allowing states with similar Hamming Distance to behave in same fashion meaning the network is more predictable, it should be even distributed, increasing predictability and avoid biases towards a specific stored or spurious pattern. Basins of attraction are canonically measured by their hamming radius [222][123][239][96][162] and to compare two learning rule average size of attraction basin over all stored patterns, this formula is used

$$r = \frac{1}{|Q|} \sum_{p \in Q} r(p) \quad (\text{B.19})$$

Basin of attraction are further divided in direct and undirect. A state is within the direct basin of attraction of a fixed point if the dynamics of the network always take the system state to the fixed point and the network state will never move away from the fixed point throughout this process while a state is within indirect basin of attraction if it is eventually attracted to the fixed point with high probability. Complete attraction is the term used to denote union of this two types of basins of attraction. Even if direct basins of attraction estimation is relevant, because convergence is faster within direct basin of attraction, the complete attraction basin is that which most influence the ability of the network to perform an associative memory task and so is the typically considered parameter of interest. This also because estimating direct basins of attractions requires finding shortest distance state from inputs that return incorrect updates, which is not of easy calculation.

Duality demonstration

The update rule for the network defined before is

$$c_\alpha = g \left[\beta \sum_{\mu=1}^m \left(F \left(-\xi_\alpha^\mu x_\alpha + \sum_{\gamma \neq \alpha} \xi_\gamma^\mu x_\gamma + \sum_{i=1}^n \xi_i^\mu v_i \right) - F \left(\xi_\alpha^\mu x_\alpha + \sum_{\gamma \neq \alpha} \xi_\gamma^\mu x_\gamma + \sum_{i=1}^n \xi_i^\mu v_i \right) \right) \right] \quad (\text{B.20})$$

in this case author used x_α to indicate only a single classification neuron involved in update, while x_γ indicate classification neurons not involved in update, so related to other classes for a multi-class problem. To perform classification c_α is typically limited between $[-1, 1]$. Here it is considered a substitution - ε in classification neuron x_α . If $-\varepsilon \rightarrow 0$, which happens when a neuron goes from -1 to 1 during update step, then

$$\lim_{-\varepsilon \rightarrow 0} c_\alpha = \text{sign} \left[\beta \sum_{\mu=1}^m \left(F \left(\lim_{-\varepsilon \rightarrow 0} -\xi_\alpha^\mu x_\varepsilon + \sum_{\gamma \neq \alpha} \xi_\gamma^\mu x_\gamma + \sum_{i=1}^n \xi_i^\mu v_i \right) - F \left(\lim_{-\varepsilon \rightarrow 0} -\xi_\alpha^\mu x_\varepsilon + \sum_{\gamma \neq \alpha} \xi_\gamma^\mu x_\gamma + \sum_{i=1}^n \xi_i^\mu v_i \right) \right) \right] \quad (\text{B.21})$$

McLaurin could then be applied. McLaurin of order n of a function $f(x)$ is given by:

$$f^{M,n}(y) = f(0) + f'(0)y + \frac{1}{2!}f''(0)y^2 + \frac{1}{3!}f'''(0)y^3 + \dots + \frac{1}{n!}f^{(n)}(0)y^n + o(y^n) \quad (\text{B.22})$$

Applying it over formula B.21 allows to obtain

$$f_1(-\xi_\alpha^\mu x_\alpha) = F \left(-\xi_\alpha^\mu x_\alpha + \sum_{\gamma \neq \alpha} \xi_\gamma^\mu x_\gamma + \sum_{i=1}^n \xi_i^\mu v_i \right) \quad (\text{B.23})$$

$$f_2(\xi_\alpha^\mu x_\alpha) = F \left(\xi_\alpha^\mu x_\alpha + \sum_{\gamma \neq \alpha} \xi_\gamma^\mu x_\gamma + \sum_{i=1}^n \xi_i^\mu v_i \right) \quad (\text{B.24})$$

$$c'_\alpha = c_\alpha = \text{sign} \left[\beta \sum_{\mu=1}^m (f_1(-\xi_\alpha^\mu x_\alpha) - f_2(\xi_\alpha^\mu x_\alpha)) \right] \quad (\text{B.25})$$

obtaining

$$c'_\alpha \approx \text{sign} \left[\beta \sum_{\mu=1}^m (f_1^{M,2}(-\xi_\alpha^\mu x_\alpha) - f_1^{M,2}(\xi_\alpha^\mu x_\alpha)) \right] \quad (\text{B.26})$$

$$\approx \text{sign} \left[\beta \sum_{\mu=1}^m \left(F' \left(-\xi_\alpha^\mu x_\alpha + \sum_{i=1}^n \xi_i^\mu v_i \right) * (-\xi_\alpha^\mu x_\alpha) - F' \left(\xi_\alpha^\mu x_\alpha + \sum_{i=1}^n \xi_i^\mu v_i \right) * (\xi_\alpha^\mu x_\alpha) \right) \right] \quad (\text{B.27})$$

given terms $f_1(0)$ and $f_2(0)$ of McLaurin series can be simplified:

$$F \left(+ \sum_{\gamma \neq \alpha} \xi_\gamma^\mu x_\gamma + \sum_{i=1}^n \xi_i^\mu v_i \right) - F \left(+ \sum_{\gamma \neq \alpha} \xi_\gamma^\mu x_\gamma + \sum_{i=1}^n \xi_i^\mu v_i \right) \quad (\text{B.28})$$

And given for both terms $+f'(0)x$ it should be considered

$$f'_1(0) = F' \left(-\xi_\alpha^\mu x_\alpha + \sum_{\gamma \neq \alpha} \xi_\gamma^\mu x_\gamma + \sum_{i=1}^n \xi_i^\mu v_i \right) * (-\xi_\alpha^\mu x_\alpha) \quad (\text{B.29})$$

$$f'_2(0) = F' \left(\xi_\alpha^\mu x_\alpha + \sum_{\gamma \neq \alpha} \xi_\gamma^\mu x_\gamma + \sum_{i=1}^n \xi_i^\mu v_i \right) * (\xi_\alpha^\mu x_\alpha) \quad (\text{B.30})$$

If $\sum_{i=1}^n \xi_i^\mu v_i \gg \sum_\gamma \xi_\gamma^\mu x_\gamma$ (which is a safe assumption in representation learning where one wants to obtain a reduced representation to proceed to classification) then

$$f'_1(0) - f'_2(0) = f'_1(0) - f'_1(0) \quad (\text{B.31})$$

Applying subtraction

$$c_\alpha \approx \text{sign} \left[\frac{1}{2\varepsilon} \sum_{\mu=1}^m F' \left(\sum_{i=1}^n \xi_i^\mu v_i \right) * (-2\xi_\alpha^\mu x_\alpha) \right] \quad (\text{B.32})$$

Returning to original limit

$$\begin{aligned} \lim_{-\varepsilon \rightarrow 0} c_\alpha &= \lim_{-\varepsilon \rightarrow >0} c'_\alpha \approx \lim_{-\varepsilon \rightarrow >0} \text{sign} \left[\frac{1}{2\varepsilon} \sum_{\mu=1}^m F' \left(\sum_{i=1}^n \xi_i^\mu v_i \right) * (-2\xi_\alpha^\mu x_\alpha) \right] \\ &\approx \text{sign} \left[\sum_{\mu=1}^m \xi_\alpha^\mu F' \left(\sum_{i=1}^n \xi_i^\mu v_i \right) \right] \end{aligned} \quad (\text{B.33})$$

This learning schema supposes some fixed elements: v_i is a clamped input to the network, ξ_i^μ are memories representing training data and ξ_α^μ represents weights of output layer

Neural network architectural archetypes

Architectural archetypes have emerged during neural network developments, in particular regarding activation functions in relation to their architectural roles. Output layers activations include linear output layer. Given features $\mathbf{h}$, a layer of linear output units produces a vector $\hat{\mathbf{y}} = \mathbf{W}^\top \mathbf{h} + \mathbf{b}$. Linear output layers are often used to produce the mean of a conditional Gaussian distribution.[107]. Another common output layer is Sigmoid output layer. Sigmoid output layer are used to define a Bernoulli distribution over y conditioned on x . In this case the neural net needs to predict only $P(y = 1 \mid \mathbf{x})$ that, to be a valid probability, must lie in the interval $[0, 1]$. Sigmoid output unit is defined by

$$\hat{y} = s(\mathbf{w}^\top \mathbf{h} + b) \quad (\text{B.34})$$

where s is the logistic sigmoid function. Finally softmax functions, called softmax output units, are used to represent the probability distribution over n different classes and is given by

$$\text{softmax}(\mathbf{z})_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)} \quad (\text{B.35})$$

Typical hidden units activations include Rectified linear units, logistic sigmoid activation function and hyperbolic tangent activation function. The first one is formally defined as

$$\text{ReLU}(z) = \max\{0, z\} \quad (\text{B.36})$$

ReLU that was already used in pre '80 network go outside usage during '80-'90 because of its non-differentiable point. However is now a major component of most successful network. Authors of [105] underlined also neural plausibility of this non-linearity: neurons tend to have linear proportional activation. Logistic sigmoid activation function is formally defined as

$$s(z) = \frac{1}{1 + \exp(-z)} \quad (\text{B.37})$$

and is the first historical used non-linearity. Hyperbolic tangent activation function is defined as

$$\tanh(z) = \frac{\exp(z) - \exp(-z)}{\exp(z) + \exp(-z)} \quad (\text{B.38})$$

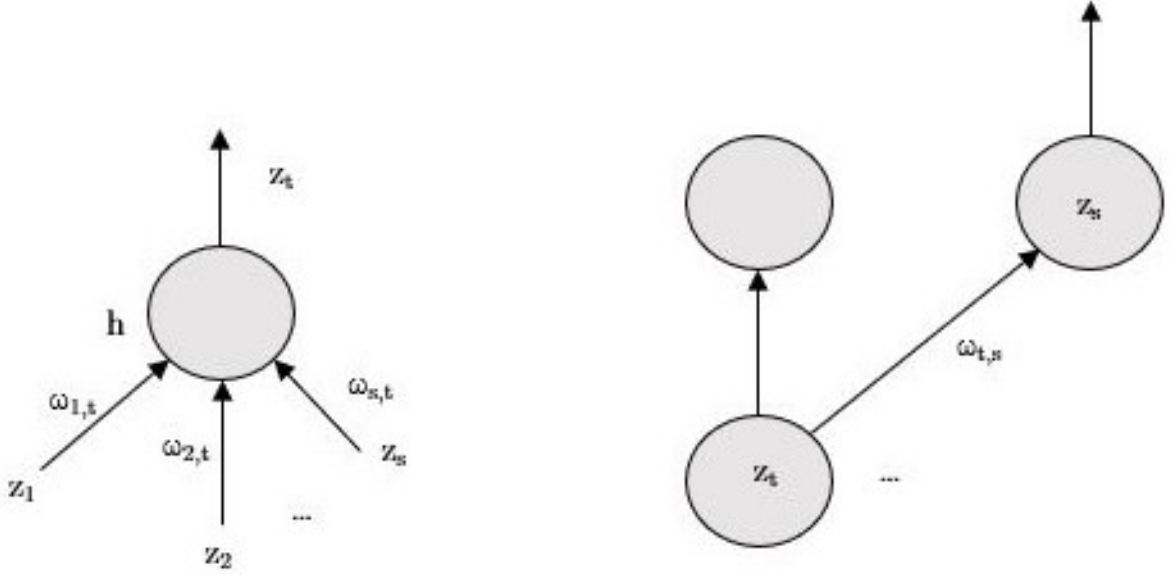


Figure B.3: Network employed in back-propagation description; picture inspired by [107]

is typically used in recurrent networks (like LSTM [120]) for numerical stability reasons

Backpropagation, brief explanation

To apply gradient descent is necessary to compute neural network derivatives in automatic fashion. Automatic differentiation is a discipline concerned in computing derivative algorithmically that encompass the algorithmic family called reverse mode accumulation, of which back-propagation belongs to. As said before back-propagation is at the core of modern implementation of neural networks and since its first proposal 1981 [242] it was rediscovered by different authors like [209]. Back-propagation requires three specific phase. Feed-forward Propagation phase accepts input $\mathbf{x}$, passes through intermediate stages and obtains output. Following that, error evaluation phase uses the computed output to compute a scalar cost depending on the loss function. The error of the network on a training set can then be estimated with

$$L(X; \mathbf{w}) = \sum_{i=1}^m \frac{1}{2} (y_i - \hat{y}(\mathbf{x}_i; \mathbf{w}))^2 \quad (\text{B.39})$$

if as example a simple linear model is considered for output $\hat{y} = \sum_j w_j x_{ij}$

$$\frac{\partial L(\mathbf{x}_i)}{\partial w_j} = \underbrace{(\hat{y}_i - y_i^*)}_{\text{error}} x_{ij} \quad (\text{B.40})$$

Last phase is called backward pass: Back-propagation allows information to flow backwards from cost to compute the gradient. Given a general unit activation in a multi layer network

$$z_t = h \left(\sum_j w_{jt} z_j \right) \quad (\text{B.41})$$

where

$$a_t = \sum_j w_{jt} z_j \quad (\text{B.42})$$

the derivative of the loss with respect of weights is

$$\frac{\partial L}{\partial w_{jt}} = \frac{\partial L}{\partial a_t} \frac{\partial a_t}{\partial w_{jt}} = \frac{\partial L}{\partial a_t} z_j \quad (\text{B.43})$$

where

$$\frac{\partial L}{\partial a_t} = \delta_t \quad (\text{B.44})$$

If unit t sends output to units S

$$\begin{aligned} \delta_t &= \sum_{s \in S} \frac{\partial L}{\partial a_s} \frac{\partial a_s}{\partial a_t} \\ &= h'(a_t) \sum_{s \in S} w_{ts} \delta_s \end{aligned} \quad (\text{B.45})$$

where

$$a_s = \sum_{j: j \rightarrow s} w_{js} h(a_j) \quad (\text{B.46})$$

Note that computing gradient in a graph has a square complexity over number of node n .

B.2 Quantum computation integration

This section contains integration of chapter 1.2 of present work

Quantum gate paradigm

Operators in quantum computation are defined on a Hilbert space and describe how one wave function is changed into another. A quantum logic gate operating on a qubit can be visualized via a transformation that rotates the state vector to another location on the previously mentioned Bloch sphere. The fundamental rotation operators are expressed by the Pauli matrices $\sigma^x, \sigma^y, \sigma^z$, here the conventional notation is employed instead of bold capital letter to denote matrices. Together with the identity transformation I these are represented in matrix notation by [164]

$$I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \quad (\text{B.47})$$

$$\sigma^x = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad (\text{B.48})$$

$$\sigma^y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} \quad (\text{B.49})$$

$$\sigma^z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \quad (\text{B.50})$$

All measurable properties of a quantum system are in fact represented as operators whose eigenvalues are the quantum allowed values for that property. As said before all this operators must be linear. An eigenvector of a square matrix $\mathbf{A}$ is defined as a non-zero vector $\mathbf{v}$ such that multiplying $\mathbf{A}$ by $\mathbf{v}$ yields a constant multiple of $\mathbf{v}$: that is, $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$ for some scalar λ . The coefficient λ is the eigenvalue of $\mathbf{A}$ corresponding to $\mathbf{v}$. The Pauli matrices each have two eigenvalues $+1$; -1 . to be complete eigenstates of y and z operators are

$$\begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad (\text{B.51})$$

$$\begin{pmatrix} 0 \\ 1 \end{pmatrix} \quad (\text{B.52})$$

while Pauli-x matrix

$$|x+\rangle = \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix} \quad (\text{B.53})$$

$$|x-\rangle = \begin{pmatrix} \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} \end{pmatrix}, \quad (\text{B.54})$$

The quantum gate model of computation is developed around the concept of a register Q of n qubits that is operated on by quantum circuits $C_1, C_2 \dots$ working in parallel and in series. Quantum gates must be reversible meaning for any quantum logic operator it must be possible to determine the input bits from reading the output bits. This allow to compute any classical function f defined on n bits [31]. A given computation involves [164]

- initializing Q to some known state. Suppose one has a quantum register Q containing n qubits. The superposition state of Q is represented by a vector $|\Phi\rangle$ of length $l = 2^n$.
- applying the circuits to induce state transformations. Let C_f denotes such a circuit applied to register Q : the result is a new superposition state, $C_f|\Phi\rangle \rightarrow |\Phi'\rangle$. A given circuit C_f can be represented as an $l \times l$ matrix.
- measuring Q at the end. Measurement causes a probabilistic "collapse" to a classical state, which is taken as the answer returned by the computation.

Given NISQ era problems, circuit based quantum computing hardware implementation has a limited number of qubits. Most of actual solution show around 50 qubits[227] [99][100]

Proposed implementation of quantum circuit computational models try take into account noise in various fashion. Briefly quote some of them, Dissipative Quantum Computing [238], Measurement-based Quantum Computing [36], Duality Quantum Computing [158]. Dissipative Quantum Computing [238] is based on open quantum system, represented by the total system which obeys to quantum unitary evolution while principal system is composed of circuits, which due to noise evolve non-unitary. Initial state in superposition is passed through logic quantum gates and final desired state can be read after T time with probability inversely proportional to T . Measurement-based Quantum Computing [36] also called One-way Quantum Computing because the measurement which destroy quantum superposition is done only at the end, consists in : after the starting entangle state is passed through quantum logic gates. Finally a duality quantum computer exploits the duality property that quantum systems can behave like both waves and particles. It admits two new operations, a divider operator and a combiner operator. The divider operator decomposes the initial wave function into sub-waves that are attenuated copies of the initial wave evolving along different paths. Each of the paths can contain quantum gates represented by unitary operators. After the sub-waves pass through the quantum gates they are collected together by the combiner operator to form a final state.

D-wave platform in detail

Founded in 1999 by Haig Farris, Geordie Rose, Bob Wiens and Alexandre Zagoskin as a branch of University of British Columbia, with headquarters near Vancouver, Canada [68]. In 13 February 2007 it proposed Orion, first prototype of hybrid quantum computer, meaning a classical machine with a quantum 16 bit accelerator based on Ising model [70]. On May 11, 2011, D-Wave Systems announced the D-Wave One, an integrated quantum computer system running on a 128-qubit processor. The D-Wave One was claimed to be the world's first commercially available quantum computer system [67]. In early 2012, D-Wave Systems revealed a 512-qubit quantum computer D-Wave Two [247]. On August 20, 2015, D-Wave released general availability of their D-Wave 2X computer, with 1000 qubits in a Chimera graph architecture [164]. In February 2019 D-Wave announced the next-generation system that would become the Advantage [62]. D-Wave is actually the platform with highest number of qubits. D-Wave's advancing the science of quantum computing has resulted in over 200 U.S. patents granted and over 100 peer-reviewed papers published in leading scientific journals[63][53] and

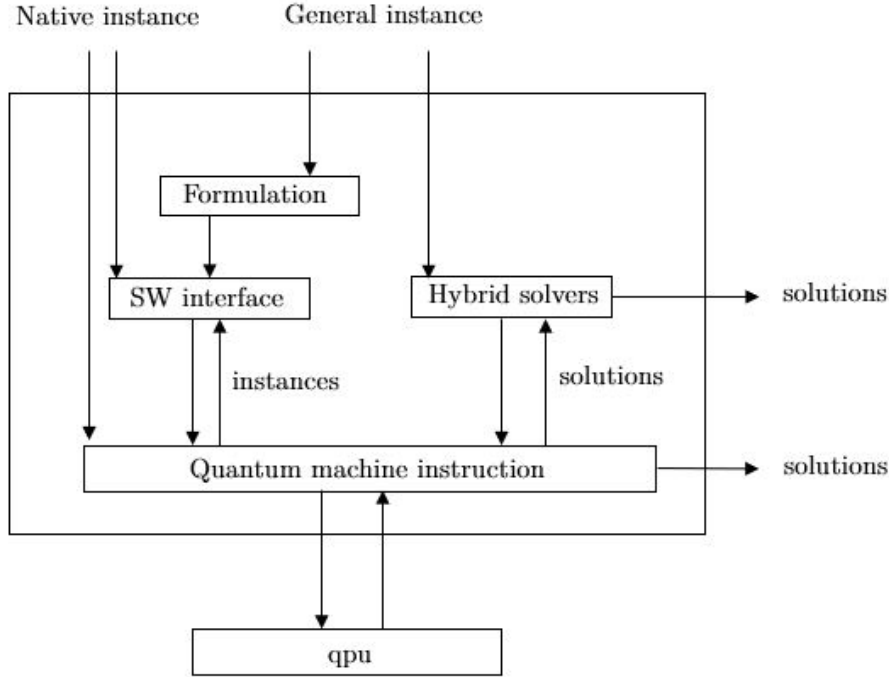


Figure B.4: Depiction of D-Wave system; picture inspired by [164]

D-Wave's systems are being used by some of the world's most advanced organizations, including NEC, Volkswagen, DENSO, Lockheed Martin, USC, and Los Alamos National Laboratory [68]. It is also worth noting that the new Clarity road-map contains new initiative to engineer its first scalable and practical error-corrected gate-model quantum computing system[68].

D-Wave is a full-stack technology provider in which annealer is provided along with other services [71]. A D-Wave platform comprises two main components: A processor chip that solves Ising Model by physical realization of a quantum annealing algorithm and a front end server. The annealer chip is super-cooled below 20mK temperature, which is necessary to achieve quantum effects (this is due to superconducting flux qubit realization [111]), within many layers of environmental shielding. A D-Wave chip holds an arrangement of superconducting flux qubits, specifically CCJ rfSQUID s. The rf SQUIDS allow a high level of tunability in situ, which means it is possible to correct for some types of fabrication errors. Tunability also means that qubits are runtime programmable. Each qubit is made of niobium (Nb) in the shape of a double ring interrupted and controlled by two Josephson junctions. The flux, flow of current, in the ring depends on biases represented by Φ values. Current can flow counterclockwise, or clockwise, or in both directions at once, when the qubit is in quantum superposition. Flux creates spins $|\uparrow\rangle$ and $|\downarrow\rangle$ that represent the state of the qubit. Couplers form the interconnections between pairs of qubits. Couplers are also fabricated from loops of niobium. Qubits and couplers are etched onto silicon wafer circuits consisting of four Nb layers separated by insulation. Each qubit in a D-Wave Two chip is connected to six "control knobs" called Φ -DACs, which carry signals from the processor and perform digital-to-analog conversion and signal latching. One Φ -DAC carries the problem bias corresponding to h_i , and the other five are used to tune and correct for variations from specification that arise in fabrication. Each SQUID qubit is connected to a dc (direct current) SQUID, which has a voltage state that depends on the flux imparted by the qubit and which can be read by applying a small voltage pulse. The remedy involves insertion of a third SQUID called a quantum flux parametron (QFP) that serves as a buffer and allows the dc SQUID to safely latch the qubit state. [24] there are actually two chips available, the D-Wave 2000Q [55] processor architecture

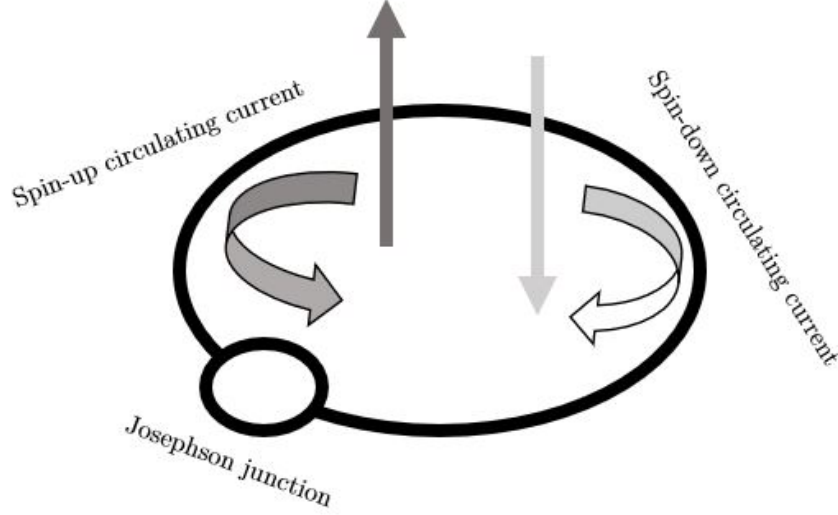


Figure B.5: Physical realization of qubits; picture inspired by [164]

with over 2,048 qubits and 6000 couplers in a Chimera graph architecture and Advantage architecture [54] [221] a new processor architecture with over 5,000 qubits and 15-way qubit connectivity. It has 2.5x more connections and more than double the number of qubits than the D-Wave 2000Q system, is 30x faster and delivers equal or better solutions 94% of the time . The conventional Intel front end server instead is connected to the chip via control lines and an I/O subsystem. The front end receives instructions from the user and is accessed using a cloud computing model called Leap. The front end sends a problem instance to the hardware and receives a set of solutions in reply. Leap is used to submit problems providing access, via a problem API token, to a portfolio of quantum processing units (QPUs), hybrid solvers, classical solvers [66][73] in multiple regions. Leap also allows for administer projects, containing the Leap IDE, a cloud-based integrated development environment(IDE) accessible via The Leap dashboard. It is a customization of the cloud-based developer environment Gitpod. Leap IDE workspace is configured for a single GitHub repository, that is, the project. In addition, a Leap IDE workspace comes preinstalled with Ocean software development kit [65]. Ocean software is a suite of tools D-Wave Systems provides on the D-Wave GitHub repository for solving hard problems with quantum computers[61] . Some of most important packages include dimod (Shared API for binary quadratic samplers which Provides a binary quadratic model (BQM) class that contains Ising and quadratic unconstrained binary optimization (QUBO) models used by samplers such as the D-Wave system.[57], dwave-hybrid, a general minimal Python framework for building hybrid asynchronous decomposition samplers for quadratic unconstrained binary optimization (QUBO) problems. [58]), dwave-inspector, a tool for visualizing problems submitted to, and answers received from, a D-Wave structured solver such as a D-Wave 2000Q quantum computer. [59], minor-miner, a tool for finding graph minor-embeddings, developed to embed Ising problems onto quantum annealers (QA) [60].

Suppose the user has an instance I in hand for a given problem P . I could be in native or non-native form. native instances are formulated via Ising or QUBO formulations. can be submitted to the front end to be loaded onto the quantum annealing chip for direct solution in hardware. The quantum annealing algorithm used by D-Wave incorporates the initial Hamiltonian

$$\mathcal{H}_I = \sum_i \sigma_i^x \quad (\text{B.55})$$

and the problem Hamiltonian

$$\mathcal{H}_{IM} = \sum_i h_i \sigma_i^z + \sum_{i < j} J_{ij} \sigma_i^z \sigma_j^z \quad (\text{B.56})$$

where h_i and J_{ij} are constrained to match the hardware working graph. The annealing path is defined by a pair of envelope functions $A(s), B(s)$ where $s(t) : 0 \rightarrow 1$ as $t : 0 \rightarrow t_f$ for a total transition time t_f . The rate-of-change function $s(t)$ is not exactly linear in t but instead "slows down" somewhat in the middle of the transition [111]. Thus D-Wave quantum annealers implement the following Hamiltonian:

$$\mathcal{H}(s) = A(s)\mathcal{H}_I + B(s)\mathcal{H}_{IM} \quad (\text{B.57})$$

the weights h_i and J_{ij} correspond to electronic signals called biases that are applied to the qubits and to the couplings that connect pairs of qubits. Note that not all logical Hamiltonians can be embedded in a current D-Wave machine. D-Wave requires that $h_i \in [-2, 2]$, $J_{i,j} \in [-1, 1]$, uses a quantization step size of approximately 0.01 and only allows for nonzero couplings between qubit pairs q_i and q_j that are connected according to some graph topology G . [124][175]

Due to physical hardware limitations the problem solved by the D-Wave annealer is generally not identical to the physical Hamiltonian. Rather, the D-Wave solves a perturbed model. The problem will have the same ground state as the original if $\|\tilde{\mathcal{H}} - \hat{\mathcal{H}}\|_\infty \leq \Delta'$, where Δ' is the size of the energy gap between the ground and the first excited state of $\tilde{\mathcal{H}}$ and $\|\cdot\|_\infty$ denotes the H_∞ operator norm. Note that the ratio $\|\tilde{\mathcal{H}} - \hat{\mathcal{H}}\|_\infty / \Delta'$ can be considered analogous to the conditioning of the problem in this context.[175]

In general, to implement a classical circuit on a QA machine, the circuit must be transformed into a Hamiltonian whose ground states correspond to valid input/output combinations. Using basic gates generated via the above process, it is possible to implement general classical circuits using the additive property of Hamiltonians [152]. While this methodology is convenient, it is wasteful in terms of ancillary bit requirements, and therefore, a handcrafted solution is preferable for most problems. To measure the optimality of an implementation, it is typical to consider both the number of physical qubits N' required to embed the Hamiltonian and the size of the physical energy gap Δ' , which as pointed out before is not easy to be done [175].

A non-native instance involve two possibilities: translate it into a native instance or submit it as is to a hybrid classical-quantum solver. Actually there are two possibilities for hybrid solvers which improves over the previous legacy qSage [58]: Leap's Hybrid Solvers, D-Wave's Leap quantum cloud service provides cloud-based hybrid solvers to which you can submit arbitrary quadratic models, and dwave-hybrid Hybrid Solvers, dwave-hybrid provides you with a Python framework for building a variety of flexible hybrid workflows.

Some restrictions to the general problem are imposed by the hardware design. First, the h_i and J_{ij} weights must match the physical connectivity structure on the chip. Qubit connectivity could be based on [56] a Chimera structure denoted C_k . Each unit cell in a Chimera structure is a complete bipartite graph $K_{4,4}$ on 8 nodes. Cells are connected in a $k \times k$ grid, with four qubits connected to horizontal neighbors and four qubits connected to vertical neighbors with internal couplers. To realize the Chimera topology in hardware, groups of qubits, number depends on cell dimension, are arranged in criss-cross fashion to form the cells. External couplers connect colinear pairs of qubits, pairs of parallel qubits, in the same row or column. Chimera qubits are characterized as having nominal length 4, meaning each qubit is connected to 4 orthogonal qubits through internal couplers, degree 6, meaning each qubit is coupled to 6 different qubits. The D-Wave 2000Q QPU supports a C16 Chimera graph: its more than 2000 qubits are logically mapped into a 16x16 matrix of unit cells of 8 qubits. Alternatively, In Advantage Pegasus structure, qubits are "oriented" vertically or horizontally, as in the Chimera topology, but similarly aligned qubits are also shifted. For QPUs with the Pegasus topology it is conceptually useful to categorize couplers as internal, external, and odd. Internal couplers connect pairs of orthogonal, with opposite orientation, qubits. External couplers connect vertical qubits to adjacent vertical qubits and horizontal qubits to adjacent horizontal qubits. Odd couplers connect similarly aligned pairs of qubits. Pegasus features qubits of degree 15 and native K_4 and $K_{6,6}$ sub-graphs. Pegasus qubits are considered to have a nominal length of 12 and degree of 15. Note that the

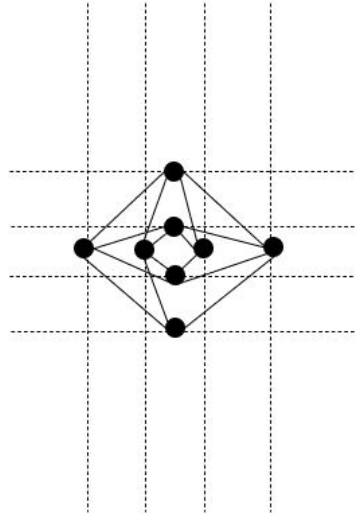


Figure B.6: Chimera cell; picture inspired by [164]

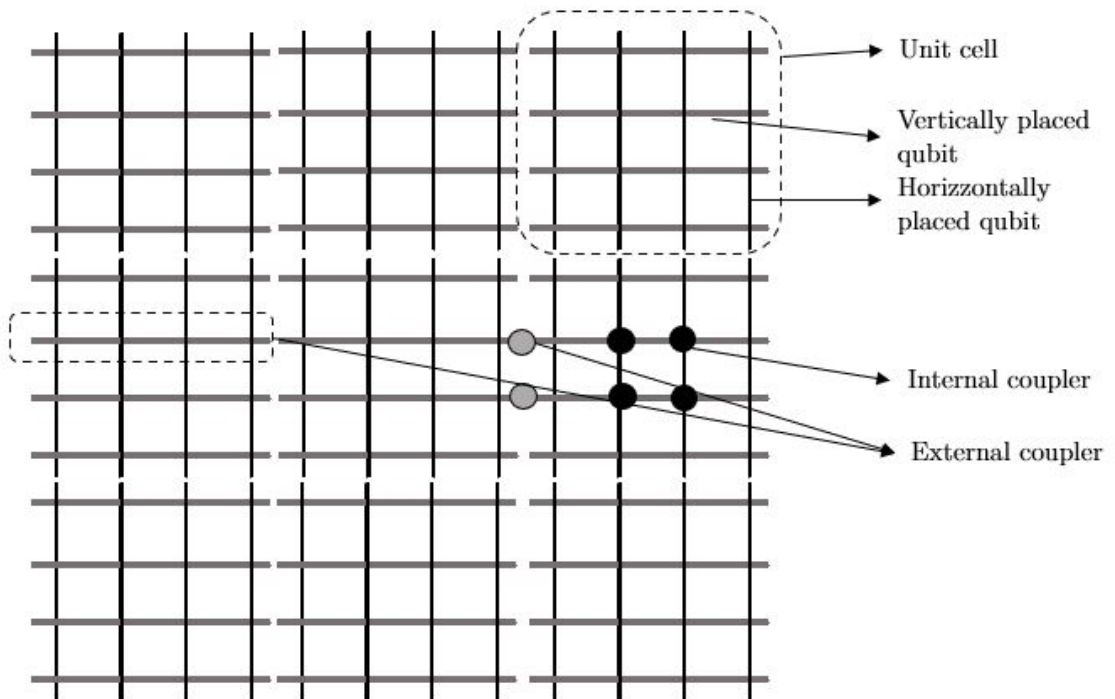


Figure B.7: Chimera grid; picture inspired by [164]

	2000Q	Advantage
performance		
Better solution sat problems		23x often than 2000Q
Time to solution 3d lattice problem		2x faster than 2000Q
Annealing quantum processor design		
Qubits	2000+	5000+
Couplers	6000+	35000+
Couplers per qubit	6	15
Topology		
graph	chimera	Pegasus
Graph size	C16	P16
connectivity	Degree 6	Degree 15
Lattice	8x8x8	15x15x12
Chain length (for problem size n=64)	17	6

Figure B.8: Comparison 2000Q and advantage graph

current chip fabrication process leaves some number of qubits and couplers inoperable. Therefore each individual chip has a specific hardware working graph. The gridlike topology is attractive because [38] it is robust against certain types of cross-talk, allow building larger C_k graphs without increasing individual qubit connectivity, the layout can be realized by a fabrication containing at most two qubit layers. The native problem solved by the chip (Ising Model) remains NP-hard when restricted to this non-planar structure and a complete graph on k vertices can be minor-embedded into the upper diagonal of C_{k^2} , which is optimal [164]. A last restriction concerns the precision and range of weights h_i and J_{ij} . Although the front end accepts arbitrary floating point values as weights, the analog control circuitry that applies the biases to qubits and couplers has fixed range and limited precision. ICE / intrinsic control error are specific techniques could be applied to reduce this risk (e.g result selection technique [29], error suppression technique [46], gauge transformation [163])

After obtained a native instance, it is necessary to map the general logical connectivity graph to the hardware working graph, meaning to find a minor embedding of G into H . A minor of H is any graph H' that can be constructed from H by collapsing two adjacent vertices into one, combining their incident edges, or by removing an edge. Source vertex $v_i \in V_g$ can be split into multiple vertices to form a connected sub-graph t_i such that $t_i \subseteq H$. The edges of t_i form a chain that must be mapped onto H . Weights on chain edges are set to large-magnitude negative values, called ferromagnetic couplings, so that all vertices in t_i have the same spin in the optimal solution. Source edge $(v_i, v_j) \in E_g$ must map to an edge $(w_i, w_j) \in H$ such that $w_i \in t_i$ and $w_j \in t_j$. Weight h_i may be assigned to one vertex in t_i or be splitted among multiple vertices.

The computation takes place in stages as follows.

- Programming/Initialization. The weights h_i, J_{ij} are loaded onto qubits and couplers biases. The qubits are placed in superposition according to $\mathcal{H}_I$ which is scaled by $A(0)$. Programming raises the chip temperature, so this step also includes a wait time for the chip to cool back down to operating temperature. The time involved in this operations is chip programming time t_p
- Anneal. A transition takes place whereby the forces on the qubits change according to the path functions, which is represented by anneal time t_a
- Readout. The transition ends. The qubits now have classical spin states according to $\mathcal{H}_{IM}$, which is scaled by $B(1)$. Qubit values are read to yield the solution S . It is represented by readout time t_r
- Re-sampling. Since the chip operates in an open system, there is always a positive, sometimes significant, probability that the computation does not finish in ground state. Given the relatively high initialization times it is cost-effective to repeat the anneal-readout cycle many times, typically $k = 1,000$ or $k = 10,000$.

Thus a given chip requires time $t_k = t_p + k(t_a + t_r)$ to return a sample of k solutions to one instance. In current chip models, classical overhead costs dominate computation time. Quantum annealing takes around 20 microseconds, but the total time to solve any given instance is bounded below by $t_p + t_a + t_r$ when $k = 1$, which corresponds to hundreds of milliseconds. Is not easy comparing classical and quantum annealer implementations to see if it achieves a quantum speedup given quality of solution depends on several factors and as probabilistic approach underlying performances describe trade-off between time (anneal time e number sample) and solution quality. Further, technology is still developing so results are difficult to be compared. However some comparative studies [29] [113] show time to solution seems to be a good metric

Some observations have to be done regarding quantum annealer. In particular, it has to be underlined that difficulties in finding optimal anneal time are the same of AQC. Also D-Wave has not showed quantum supremacy [74] given measuring quantum speedup and quantum advantage is not clear [151][29] [163]. Finally regarding problem size, it is relevant to underline that every NP-hard problem can be translated to an Ising Model instance with no more than polynomial expansion in size and any Ising Model instance on n variables can be minor-embedded onto a Chimera graph using a quadratic number of qubits in worst case. Embeddability correspond then roughly to edge count

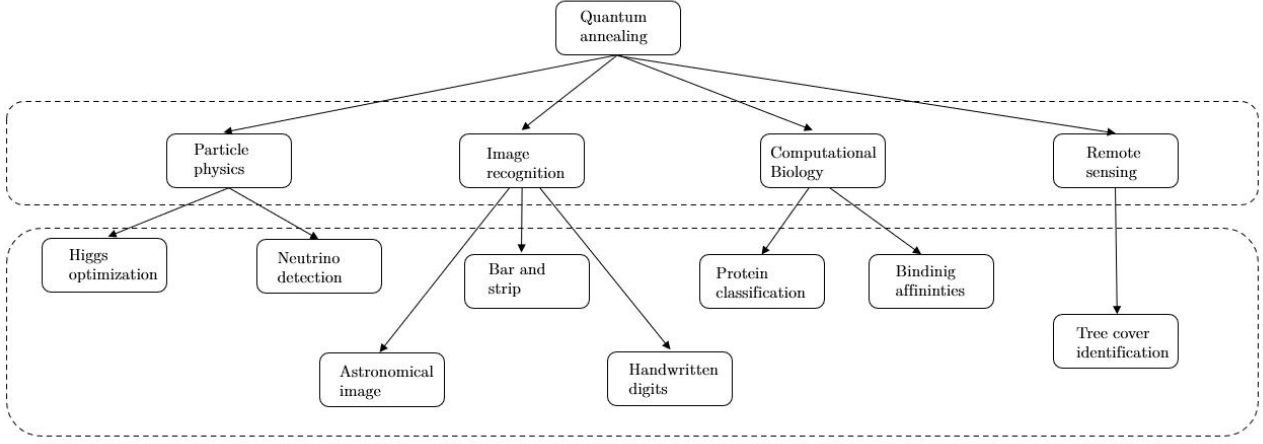


Figure B.9: Annealer applications in machine learning

$$\begin{aligned}
 |quron\rangle &= \text{rest } |0\rangle + \text{active } |1\rangle \\
 |\text{rest}|^2 + |\text{active}|^2 &= 1
 \end{aligned}$$

.

Figure B.10: Depiction of quron approach [215]

Different problems can be accommodated on D-wave, e.g. decision problem, approximation problems (which ask for a solution that lie within a given distance from optimal), sharp-P problems (ask for count of the number of optimal solutions to a given instance), partition problems (ask for counting the number of solutions for each cost in the solution space), Boltzmann/Gibbs sampling (ask for sampling from the space of solution according to probability distribution that depends on their cost), specific purpose problems (like protein folding [189] or machine learning which require solving large number of hard optimization problems via repeated queries), maximization problem (e.g. weighted maximum cut), minimization problems (e.g. finding Ramsey number [12]) and Ising problem [11],

B.3 Quantum machine learning

Quantum Associative Memories

A natural way to implement QNNs is to replace the McCulloch-Pitts neuron $x = \{-1, 1\}$ by a 'quron' $|x\rangle$ of the two-dimensional Hilbert space $\mathcal{H}^2$ with basis $\{|0\rangle, |1\rangle\}$. The state $|\psi\rangle$ of a network with n qurons thus becomes a multi-particle quantum state of the 2^n -dimensional Hilbert space $\mathcal{H}^{2^n} = \mathcal{H}^2 \otimes \dots \otimes \mathcal{H}^2$ with basis $\{|0, 0, \dots, 0\rangle, \dots, |1, 1, \dots, 1\rangle\}$ reading

$$|\psi\rangle = \sum_{i=1}^{2^n} a_i |x_1 x_2 \dots x_n\rangle_i \quad (\text{B.58})$$

where the $a_i, i \in \{1, \dots, 2^n\}$ refer to the complex amplitudes assigned to the respective network basis states [215]. A Quantum Circuit approach based on quron is the most diffused approach to implements neural networks however there are other possibilities considered in literature (like amplitude encoding, see thereafter). Associative memories, Hopfield network in particular, were considered the prototype of neural networks. First neural network implementation consequently deals with memory recall problem as exposed by classical Hopfield models. In this section some starting approach not easy categorizable

are exposed to ground initial steps in this field. After that quantum circuit and adiabatic quantum computing implementations are exposed.

Kak in [134] introduced the idea of "quantum neural computation" by interpreting the necessary condition for a stable state $x^0 = (x_1^0, \dots, x_N^0)$ in a 'Hopfieldlike' network defined by the weight matrix w with entries w_{ij} ,

$$\sigma(wx_0) = x_0 \quad (\text{B.59})$$

as an eigenvalue equation of a quantum system, $\hat{w}|x_0\rangle = \lambda|x_0\rangle$. The weight matrix w thereby becomes an operator with eigenvector $|x_0\rangle$ and eigenvalue $\lambda = 1$. Updating a network then corresponds to a quantum measurement that selects the eigenstates of a system. Kak notes that the sigmoid function is a non-linearity that has no equivalent in the quantum mechanical formalism but does not discuss this crucial point any further.

[192] emphasises the analogy between what he calls a 'Hopfield-like' network and quantum theory, deriving from an updating function without threshold, the time evolution of a quantum state,

$$\psi(r_2, t_2) = \iint G(r_1, t_1, r_2, t_2) \psi(r_1, t_1) dr_1 dt_1 \quad (\text{B.60})$$

Here, G is a projection operator in position representation defined as $G = \sum_{l=1}^k \psi^{l*}(r_1, t_1) \psi^l(r_2, t_2)$, r_1, r_2 are position and t_1, t_2 time variables, and $\{\psi^l(r, t)\}$ is a complete basis of the quantum system. G is derived from Hebb's learning rule and constitute the "hologram" or memory by itself as in analogy to holographic memory [224]. Peruš models a pattern recall through a collapse of the wave function. A valid input state ψ is supposed to be almost orthogonal to all memorised vectors, or basis states, ψ^l except from one vector ψ^j ($l \neq j$), so that G applied to the input, as in equation before, starting from superposition of single-pattern auto-correlation, retrieves ψ^j with a high probability $|\psi\psi^j|^2$. If pattern is not orthogonal the system does not collapse to a precise memorized state but it instead to an interference of eigenwave functions. Peruš' approach has been further developed by ideas of interacting quantum dots presented below.

[50] presents a model very similar to ones used for AQC approach to the problem of [178] which is discussed in the related sections. Substituting each neuron with a qubit (quron approach), it introduces a transverse Hamiltonian to make system evolution unitary and reversible

$$\mathcal{H} = J \sum_{ij} w_{ij} \sigma_i^y \sigma_j^z \quad (\text{B.61})$$

note that author used two Pauli matrices that do not commute in the memory model of neural network, in alternative classical model would be obtained. This is analogous to AQC model of [178]. Inputs ξ^{ext} are introduced adding an external transverse magnetic field along the y axis, i.e. modifying the Hamiltonian to

$$\mathcal{H} = J \sum_{ij} w_{ij} \sigma_i^y \sigma_j^z + g \sum_i h_i^{\text{ext}} \sigma_i^y \quad (\text{B.62})$$

where $h_i^{\text{ext}} = \sum_j w_{ij} \xi_j^{\text{ext}}$. This Hamiltonian generates a unitary evolution of the network:

$$|\psi(t)\rangle = \exp(i\mathcal{H}t)|\psi_0\rangle \quad (\text{B.63})$$

where $|\psi_0\rangle = |\psi(t=0)\rangle$. Initial configuration of the network the uniform superposition of all computational basis states, is

$$|\psi_0\rangle = \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle \quad (\text{B.64})$$

notably this network expose phase transitions in function of memory density, meaning stored memory over number of neuron, and coupling constant J having a paramagnetic, no solution, ferromagnetic, associative memory behaviour, and quantum spin glass phase, metastable phase. The author underlined the maximum capacity of this network is no different from a classical model of Hopfield network.

A full QNN proposal inspired by [192] appears in [15]. The time evolution of network is refor-

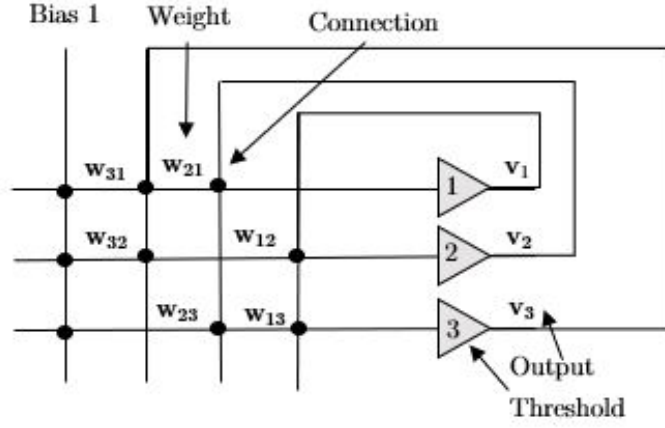


Figure B.12: [4] proposed circuit; picture inspired by [4]

unfortunately do not give a derivation of U from a given Hopfield network that shows its characteristic dynamics. A major problem in such approaches is also the global coherence between the qurons should have to be maintained for a high dimension neural network

To overcome limit capacity of previous work same author of [50] suggested in [51] to use probabilistic quantum memories[230] that exploit wave function collapse upon measurement. Quantum associative memories involve, besides the memory register itself a certain number b of control qubits all initialized in the state $|0\rangle$. Using entanglement to "store" an arbitrary number p of binary patterns p^i of length n in a quantum superposition of just n qubits,

$$|m\rangle = \frac{1}{\sqrt{p}} \sum_{i=1}^p |p^i\rangle \quad (\text{B.70})$$

The idea of the memory architecture consists of two steps. First it generates the state $|m\rangle$ by a unitary evolution M from a simple prepared state, say $|0_1, \dots, 0_n\rangle$, $|m\rangle = M |0_1, \dots, 0_n\rangle$. The quantum memory itself is the unitary operator M that codes the p patterns. It defines implicitly a Hamiltonian through the formal relation $M = \exp(i\mathcal{H})$, a Hamiltonian that represents pattern-dependent interactions among the qubits. It is constructed using algorithm of [230]. Secondly, given an input state $|i\rangle = |i_1, \dots, i_n\rangle$, generate from $|m\rangle$ a superposition of the pattern states, this unitary evolution mixes the memory register and the control qubits, that is no more uniform but whose amplitudes define a probability distribution peaked on the pattern states with minimal Hamming distance front the input. This implies unitary evolution to the initial input state, and then the control qubits are measured called recognition step. Only if one obtains a certain specific result, one proceeds to measure the memory register, identification step. Both recognition are probabilistic and both the recognition efficiency and the identification accuracy depend on the distribution of the stored patterns and their performance can be tuned. This two step procedure is called probabilistic post-selection of the measurement result and guarantees that the memory register is in a superposition of the stored patterns such that the measurement probabilities are peaked on those patterns that minimize the Hamming distance to the input. A measurement of the memory register will thus associate input and stored patterns according to this probability distribution. This architecture allow store any desired polynomial number, in terms of the number of qubits.

Most of the work previously presented are focused on parallelism however also local minima are a problem for neural network.

Author of [4], first in 1997, underlined the possibility to solve problem of global minimum finding in quantum Hopfield neural network [4]. The author proposed a single-electron circuit, never implemented, that changes its state to decrease its free energy. A tunnel junction with an excess electron is used as a threshold element, representing neurons. If the barrier is thin enough and empty sites are available in the right range of energies, carriers from one side can tunnel to these empty sites:

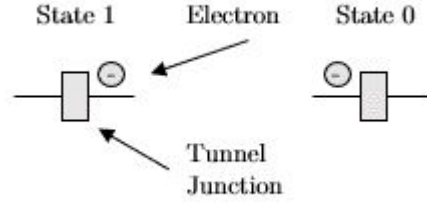


Figure B.13: [4] tunnel junction; picture inspired by [4]

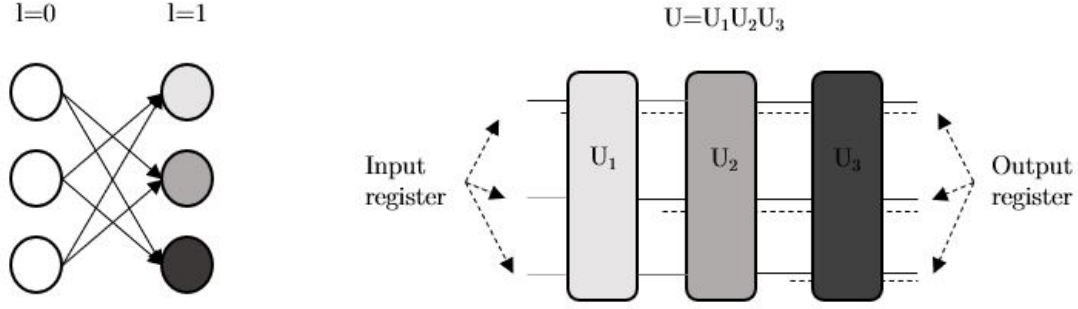


Figure B.14: [166] unitary evolution to retrieve Hopfield network; picture inspired by [166]

tunneling events are the only way to achieve phase transition. The author defined the state of the tunnel junction as "1" if the electron is on the right of the junction and as "0" if it is on the left. The connection between two tunnel junctions can be established by a pair of coupling capacitors. There is a direct mathematical isomorphism of this formulation of the free energy for this circuit with that of Hopfield model. For Akazawa model the implementation constrains the memory stored so it was never implemented and there is not a clear cut parallelism to neural architecture, which makes debatable satisfaction of second point of [215].

Authors of [166] took directly into account Schuld criteria to implement quantum Hopfield neural network. They followed a proposal of deep quantum neural networks [14] and applied it to feed forward analogy of HNN. Differently from quron approach, a quantum neuron is here considered a general unitary operator U_i acting on n input qubits and m output qubits. The Hopfield network is then implemented in a feedforward "analogy", in which it is imagined Hopfield network states evolve through layer computations in which weights of each layers are the same throughout all network. However they consider only a single recurrent step not all steps toward minima. Input qubits are initialized in a register in a possibly unknown mixed state ρ^{in} and then each unitary act on a fresh ancilla qubit. Ancilla qubits are measured to determine classical state, while input qubits remains unvaried. The objective in this case is training unitaries, and this is done measuring fidelity between the output and the state that is wanted to be obtained and evolving unitaries accordingly.

Most circuit approaches are in the category called Quantum Associative Model, a term to identify Hopfield associative memories emulated via quantum mechanism without explicitly trying to model architectural components of Hopfield net by them-self

Historically author of [237] proposed first quantum associative memory (QuAM). Let

$$X^P = \left\{ \left| x_1^{(1)}, \dots, x_N^{(1)} \right\rangle, \dots, \left| x_1^{(P)}, \dots, x_N^{(P)} \right\rangle \right\} \quad (\text{B.71})$$

again be the P patterns to be stored in the QAM'. Memory $|M\rangle$ is therefore a superposition of all memorised states

$$|M\rangle = \frac{1}{\sqrt{P}} \sum_{p=1}^P \left| x_1^{(p)}, \dots, x_N^{(p)} \right\rangle \quad (\text{B.72})$$

requiring a modification of Grover search to look for specified patterns, as some patterns should not be stored in this superposition. In [237], authors proposed a QUAM with a theoretical storage capacity of $O(2^n)$, while using only n neuron. Practically [237] limits functionality of the net to pattern completion. This is achieved via superposition of $|M\rangle$ with a qutrits system able to represent also incomplete patterns. The information learned from the training set will be encoded in the coefficients of the system c_{zp} via a set of unitary operators over the system. Operators increase the probability amplitudes of those eigenstates associated with features that match the input vectors in the training set, and also cause interference between the amplitudes of competing eigenstates increasing the probability amplitudes of those eigenstates that correspond to features that match their associated input training vector regardless of output class. This pattern completion approach was rendered a full associative memory in [231] [232]. It should be noted that all these proposals were not actually implemented.

[200] exploits original possibility to codify the network inside the amplitude of the quantum system, allowing exponential capacity and an exponential speedup in logarithm of d , an improvement respect [237], intertwined with the naturally visible structure of Hopfield networks which allows a simple encoding of the information into the amplitudes of a quantum state. In particular authors devised following operators

- qHop operate Hopfield via efficient quantum inversion of matrix that require eigenvalues estimation of Hopfield weight matrix achieved by following point
- qHeb
 - encodes weight matrix into a density matrix stored in a qubit register, via a reading of a qRAM
 - allows for estimating eigenvalues of density matrix via sparse Hamiltonian simulation followed by quantum phase estimation

as said qheb subsumes to be able to read-in activation patterns. For general usage this translates into having an implemented qRAM which is not actually practical so this proposal is not implemented in hardware. The usage of qHeb has a complexity dependent on memory stored requiring higher number of neurons to be used in comparison of the data: the exponential storage is therefore limited in some way. This implementation does not consider the dissipative dynamics of neural networks resorting only on quantum operations carried by unitary operators.

First actual implementation of Hopfield model in quantum hardware is in the work of Miller and Mukhopadhyay and based on concept of quron. They based their system on the quantum neuron developed by [41]. It utilizes controlled rotations dependent on a superposition of input states, which is based on weight matrix and bias term set $\beta_i = \frac{\pi}{4} - \gamma \left(\sum_{j=1}^n w_{ij} \right)$, to create a controlled rotation $CR_y(2\phi)$. A measurement of $|0\rangle$ on the input qubit indicates that the output qubit has been rotated $R_y(2q(\phi))$ with $q(\phi) = \arctan^2(\tan^2 \phi)$. If $|1\rangle$ is measured, the circuit has applied $R_y(\frac{\pi}{2})$ to the output qubit, in which case this rotation is reversed and the circuit is repeated until success is achieved. The output probability of being in $|1\rangle$ state is used as an analog to the classical neuron, in which $\phi > \frac{\pi}{4}$ produces $P(|1\rangle)$ close to 1 and $\phi < \frac{\pi}{4}$ produces a $P(|1\rangle)$ close to 0, corresponding to the classical $x_i = \pm 1$ states: in fact the parameter ϕ relates to the classical θ parameter by $\phi = \gamma\theta + \frac{\pi}{4}$ where γ is a normalization factor such that $0 < \phi < \frac{\pi}{2}$ and threshold behaviour of neural network is obtainable setting θ to 0.

Given Hopfield dynamic obeys to

$$x_i = \begin{cases} +1 & \sum_{j=1}^n w_{ij} x_j > h_i \\ -1 & \text{else} \end{cases} \quad (\text{B.73})$$

However authors modified their structure for implementation in IBMQ in a single, uninterrupted quantum circuit removing intermediate measurements, and following reset operations. To update an initial qubit state $|s_i\rangle$ to a new state $|s_i'\rangle$, the rotation by 2ϕ is performed on an ancilla (target) qubit initialized to state $|0\rangle$, then the output of the neuron on the ancilla qubit is swapped to the

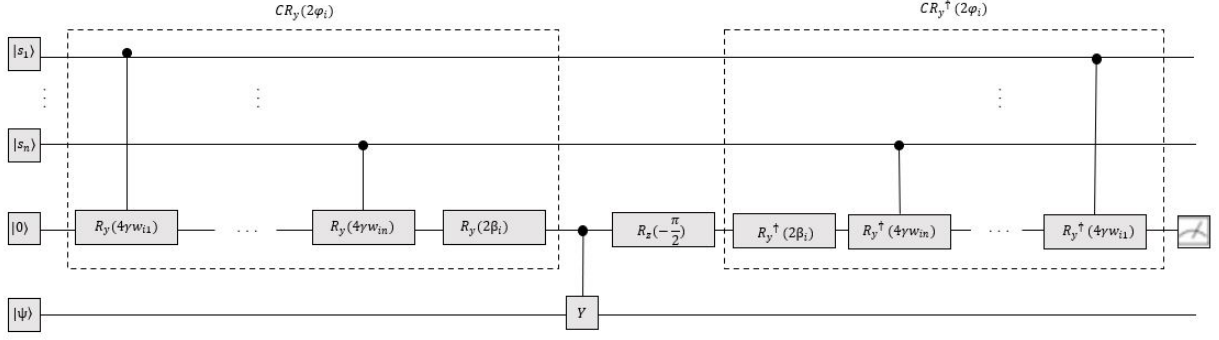


Figure B.15: Architecture of [41]; picture inspired by [41]

qubit which held the original state using a quantum SWAP gate. Multiple updates are performed in succession to reach a desired endpoint of the circuit, after which the majority vote method can be used to extract the result to a classical output. The results show however that performances degrade rapidly increasing number of patterns in real hardware implementation compared to simulated noise. This implementation however does not consider at all the dissipative nature of neural network, it is irrelevant per se the usage of a non-linearity.

There were also AQC implementation of Quantum Hopfield network

Authors of [92] presented a quantum algorithm for the satisfiability problem, and other combinatorial search problems, that works on the principle of quantum adiabatic evolution. Their Hamiltonian is defined by

$$H(t) = H_{C_1}(t) + H_{C_2}(t) + \dots + H_{C_M}(t) \quad (\text{B.74})$$

where each H_{C_a} depends only on clause C_a and acts only on the bits in C_a . $H(t)$ is defined for t between 0 and T and is slowly varying. The initial state, which is always the same and easy to construct, is the ground state of $H(0)$. For each a , the ground state of $H_{C_a}(T)$ encodes the satisfying assignments of clause C_a . The ground state of $H(T)$ encodes the satisfying assignments of the intersection of all clauses. Clause C is associated with the 3 bits labeled i_C , j_C , and k_C . For each clause C they define an energy function

$$h_C(z_{i_C}, z_{j_C}, z_{k_C}) = 0, \quad (\text{B.75})$$

if

$$(z_{i_C}, z_{j_C}, z_{k_C}) \quad (\text{B.76})$$

satisfies clause C

$$h_C(z_{i_C}, z_{j_C}, z_{k_C}) = 1 \quad (\text{B.77})$$

if

$$(z_{i_C}, z_{j_C}, z_{k_C}) \quad (\text{B.78})$$

violates clause C. They then define the total energy h as the sum of the individual h_C 's,

$$h = \sum_C h_C \quad (\text{B.79})$$

They replace the bit z_i by a spin- $\frac{1}{2}$ qubit labeled by $|z_i\rangle$ where $z_i = 0, 1$ and construct base Hamiltonian H_P accordingly. For a given problem, specifying H_P is straightforward but finding its ground state may be difficult so they consider an n -bit Hamiltonian H_B that is straightforward to construct but whose ground state is simple to find. Let $H_B^{(i)}$ be the 1-bit Hamiltonian acting on the i -th bit

$$H_B^{(i)} = \frac{1}{2} \left(1 - s_x^{(i)} \right) \quad (\text{B.80})$$

with

$$s_x^{(i)} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad (\text{B.81})$$

so

$$H_B^{(i)} |x_i = x\rangle = x |x_i = x\rangle |x_i = 0\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} \quad (\text{B.82})$$

and

$$|x_i = 1\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix} \quad (\text{B.83})$$

Continuing to take 3-SAT as working example, clause C is associated with the bits i_C, j_C , and k_C . Now define

$$H_{B,C} = H_B^{(i_C)} + H_B^{(j_C)} + H_B^{(k_C)} \quad (\text{B.84})$$

and

$$H_B = \sum_C H_{B,C}. \quad (\text{B.85})$$

The ground state of H_B is $|x_1 = 0\rangle |x_2 = 0\rangle \cdots |x_n = 0\rangle$. This state, written in the z basis, is a superposition of all 2^n basis vectors $|z_1\rangle |z_2\rangle \cdots |z_n\rangle$

$$|x_1 = 0\rangle |x_2 = 0\rangle \cdots |x_n = 0\rangle = \frac{1}{2^{n/2}} \sum_{z_1} \sum_{z_2} \cdots \sum_{z_n} |z_1\rangle |z_2\rangle \cdots |z_n\rangle \quad (\text{B.86})$$

and

$$H_B = \sum_{i=1}^n d_i H_B^{(i)} \quad (\text{B.87})$$

where d_i is the number of clauses in which bit i appears in the instance of 3-SAT being considered. After that authors considered the Grover problem casted in the fashion of SAT problem. It has a single, generalized, clause, h_G , which depends on all n bits with a unique but unknown satisfying assignment $w = w_1, w_2, \dots, w_n$. The problem Hamiltonian corresponding to h_G is

$$H_P |z\rangle = \begin{cases} |z\rangle, & z \neq w \\ 0, & z = w \end{cases} \quad (\text{B.88})$$

$$= 1 - |z = w\rangle \langle z = w|$$

where $|z\rangle = |z_1\rangle |z_2\rangle \cdots |z_n\rangle$. So, they construct $H(t) = \tilde{H}(t/T)$ with H_B given with $d_i = 1$ for all i from 1 to n . evolving using $H(t)$. The problem is "oracular," that is, it uses no knowledge about the structure of H_P which could aid in finding w . Writing $\tilde{H}(s)$ explicitly

$$H(s) = (1-s) \sum_{j=1}^n \frac{1}{2} \left(1 - s_x^{(j)} \right) + s(1 - |z = w\rangle \langle z = w|) \quad (\text{B.89})$$

There were no direct implementation and testing of classic Hopfield networks, however the work of [218], that was only realized on a simulated adiabatic quantum computation platform, due to its robustness to noise in respect of circuit based, is direct implementable in the annealer. This paper answer to a precedent publication that asked for this learning rule comparison [178] that in order to apply AQC to the computational task of associative pattern recognition, encode the complete problem in the Hamiltonian

$$\mathcal{H}_p = \mathcal{H}_{\text{mem}} + \Gamma \mathcal{H}_{\text{inp}} \quad (\text{B.90})$$

with $\mathcal{H}_{\text{mem}}$ representing the knowledge about stored patterns, $\mathcal{H}_{\text{inp}}$ representing the computational input and an appropriate weight factor $\Gamma > 0$. Since the problem is directly encoded into the final Hamiltonian $\mathcal{H}_p$, no ancilla qubits are required. They encode the final adiabatic Hamiltonian as a sum of two independent term; independency offers the author the possibility of compute evolution of the system approximating it by separate periods of unitary evolution, each according to the individual

terms building the total instantaneous Hamiltonian. The total Hamiltonian is formed by

- Input Hamiltonian $\mathcal{H}_{\text{inp}}$

$$\mathcal{H}_{\text{inp}} = \sum_i \xi_i^{\text{inp}} s_i^z \quad (\text{B.91})$$

as said before being independent, summation with problem Hamiltonian, can be obtained and imposed independently of memory pattern dynamic computed only once. For a given state $|\xi\rangle$ representing a pattern ξ , the energy corresponding to $\mathcal{H}_{\text{inp}}$ depends on the Hamming distance $\hat{h} = 0, \dots, n$ between ξ^{inp} and $\hat{\xi} = [\xi_1, \dots, \xi_n]$, i.e., the number of positions for which the corresponding entries are different [218]

- A Memory Hamiltonian

$$\mathcal{H}_{\text{mem}} = -\frac{1}{2} \sum_{i \neq j} w_{ij} s_i^z s_j^z \quad (\text{B.92})$$

where s_i^z is the Pauli z matrix on qubit i and w_{ij} depends on learning algorithm chosed setting the activation thresholds in place of initializing the network to a known initial state meaning the dynamics of $\mathcal{H}_p$ depends on both memory and input information (leading to both information exploitation) . Authors formally defined the energy minimization condition as

$$z = \arg \min_{z'} E(z'; \theta) \quad (\text{B.93})$$

in which the vector θ represents the activation thresholds $\theta_i = \lceil z_{0,i}$ and Γ is an energy scale for the applied bias, as done in [178]. The activation threshold θ therefore serves as an energetic bias towards network states that best match the input z_0 . The bias usage was considered for breaking energy symmetry, so not storing spurious patterns.

The hyper-parameter gamma is regulated to point towards stored memory, and some bounds could be analytically derived. Similar to before-mentioned cases:

$$H(t) = \left(1 - \frac{t}{\tau}\right) H_B + \frac{t}{\tau} H_P \quad (\text{B.94})$$

$$H_P = \sum_{i=1}^n \sum_{j=1}^n Q_{ij} s_z^i s_z^j + \sum_{i=1}^n q_i s_z^i \quad (\text{B.95})$$

With $h_i = \theta_i$ and $J_{ij} = w_{ij}$. They choose the computational basis in terms of tensor product states of the $+1$ and -1 eigenstates of operators s_z^i . The authors recognized that the exact scaling for the minimal T with respect to Ising model size and parametrization is not clear. Simulated problem instances input parameters include in general number of neurons n , number of stored patterns p , the applied learning rule (Hebb, Storkey, or projection), the annealing time T , the applied bias Γ , the input key z_0 Note that the large number of simulations limit the authors to a small number of neurons; this and the simulation make debatable the fulfillment of the first two quantum neural network criteria exposed before. The results of these simulations (see Appendix D for a description in detail) clearly show the dependence of recall capabilities by learning rule also for AQO, adding another parameter to be evaluated in quantum implementation of HNN.

Hopfield networks answer to tasks like finding an unknown value in an unsorted database, the network memory. An Hopfield neural network that uses AQO for memory recall is therefore equivalent to these Grover search implementations when the oracle is expressed via a one-memory network: the Hebb rule is in fact based on projection operation. A Hopfield network extends search in a general context where the oracle must discriminate between tagged and untagged keys, stored patterns and a new pattern. This is expected to influence also the spectral gap and minimum adiabatic time.

This model was further analyzed in its thermodynamic limit in [160] and usage of transverse field to induce quantum phenomena in Hopfield network influence on solutions [207]. Notably attractors of the classic memory remain attractors also for quantum model, meaning this network as defined maintain same spurious state, however it also includes a novel time-dependent stationary phase (limit

cycles solutions) dependent on transverse field intensity. Interestingly this results seem in line with [50] so one could reasonably think adiabatic quantum model with classical Hopfield does not offer increase in capacity per se.

Neural networks on annealer implementations

Authors of [115] proposed to simulate a neural network to obtain classification from input samples, supposing network is already trained, to provide faster classification. This allows to obtain hints for a time consuming classification procedures and to exploit ensembling of solutions.

They tried to model a neural network composed of visible layer, hidden pooling layer, that behaves like a max function in respect of a subgroup of visible units, and softmax output layer interconnected to hidden layer via linear units.

Let $\mathbf{V} = \{v_{ij}\}_{i=1\dots n_p, j=1\dots n'_p}$, $\mathbf{h}^p = \{h_j^p\}_{j=1\dots n'_p}$, and $\mathbf{h}^c = \{h_k^c\}_{k=1\dots n_c}$ be binary random variables where $\mathbf{V}$ denotes visible units, $\mathbf{h}^p$ and $\mathbf{h}^c$ denotes hidden pooling and classification units, respectively. n, n_p, n'_p, n_c are the number of visible units, the number of visible units subsumed by a pooling unit, the number of hidden pooling units and the number of output units (which in this case is equivalent to class number), respectively.

For pooling, n'_p subsets of $\mathbf{V}$, each containing four neighbouring elements are connected to a $\mathbf{h}^p$. All $\mathbf{h}^p$ are connected to all $\mathbf{h}^c$ forming a fully connected classification layer. Note that $\mathbf{h}^c$ are interconnected to mimic a softmax activation (see later).

The energy of this system is then

$$E(\mathbf{V}, \mathbf{h}^p, \mathbf{h}^c) = -\mathbf{q}^v \mathbf{V} - \sum_j \sum_i^{n'_p, n_p} \left(h_j^p q_{ij}^{vp} V_{ij} + q_j^p h_j^p \right) - \mathbf{h}^{cT} \mathbf{q}^{pc} \mathbf{h}^p - \mathbf{q}^{cT} \mathbf{h}^c - \sum_{j>k}^{n'_p, n'_p-1} h_j^c q_{jk}^{cc} h_k^c \quad (\text{B.96})$$

where $\mathbf{q}^v, \mathbf{q}^p, \mathbf{q}^c$ are the biases of $\mathbf{V}, \mathbf{h}^p, \mathbf{h}^c$ respectively, and $\mathbf{q}^{vp}, \mathbf{q}^{pc}, \mathbf{q}^{cc}$ are the coupling strengths between $\mathbf{V}$ and $\mathbf{h}^p$, $\mathbf{h}^p$ and $\mathbf{h}^c$ and between $\mathbf{h}^c$ and $\mathbf{h}^c$ respectively.

In order to make quadratic binary model behaving like the neural network classification model, authors found parameter values, $\mathbf{Q} = \{\mathbf{q}^v, \mathbf{q}^p, \mathbf{q}^c, \mathbf{q}^{pv}, \mathbf{q}^{cp}, \mathbf{q}^{cc}\}$ as described thereafter.

Real valued information about the input features entered the system through the biases $\mathbf{q}^v$ placed on the visible units $\mathbf{V}$: a single unit bias q_i^v is normalised allowing for a real value between $-\alpha$ and α . Given pooling is interpreted as max function, the objective of pooling for the hidden units $\mathbf{h}^p$ is to establish whether features are present. If features are present, then $\mathbf{h}^p$ is on and contributions are made to the classification units $\mathbf{h}^c$. Consider the contribution that a pooling subset, h_j^p makes to the energy equation,

$$- \sum_i \left(q_i^v V_{ij} + h_j^p q_{ij}^{vp} V_{ij} + q_j^p h_j^p \right) \quad (\text{B.97})$$

to increase the likelihood that $\mathbf{h}^p$ is on if a feature is present authors neutralized $\mathbf{h}^p$ by setting $q_j^p = 0$ and weighted the interaction between $\mathbf{V}$ and $\mathbf{h}^p$ by setting $q_i^{vp} j = \alpha$, meaning absolute value of normalized feature values. Concerning the interaction between h_p and h_c , authors set $\mathbf{q}^{pc} = \mathbf{W}$. Note that this relies on assumption that the weights of a CNN classification layer, $\mathbf{W}$, have been optimised so that $y = \mathbf{x}^T \mathbf{W}$ agrees with the class of $\mathbf{x}$ where $\mathbf{x}$ is a input feature vector. To mimic a softmax activation is necessary constraining the model so that $\sum_k h_k^c = 1$. Introducing interconnection between classification units, this is achieved by finding the values of $\mathbf{q}^c$ and $\mathbf{q}^{cc}$ that minimizes $(\sum_k^{n_c} h_k^c - 1)^2$ using the following expansion

$$\left(\sum_k^{n_c} h_k^c - 1 \right)^2 = 2 \sum_{j>k}^{n_c, n_c-1} h_j^c q^c c_{jk} h_k^c - \sum_j^{n_c} h_j^c + 1 \quad (\text{B.98})$$

with solutions $\mathbf{q}^{cc} = 2$ and $\mathbf{q}^c = -1$. Considering classification on MNIST, they tried to sample from biases and weights obtained from a convolutional neural network, given convolutional neural network are known to have low number of units. Biases and weights were then transferred to the

QPU via minor-miner and samples with the lowest energy were averaged to estimate the expected value of each classification nodes. This allows to obtain a reasonable classification 10 time faster than a classical only. Using their approach they were able to find an embedding and obtain samples from a four layer model comprising 358 nodes: 196 visible nodes, 128 convolutional feature nodes, 24 fully connected hidden nodes and 10 classification nodes. Note that this approach supposes having already optimize neural network before and give no space to train this model to achieve classification tasks. It also have limits in embeddability. Pooling in this case has the role of, and could be considered the only, nonlinearity. However if a feature is inactive there is no bound on value of $\mathbf{h}^p$. This could be a problem if all pooled group is zero. Further, feature maps from input to pooling layer are not preserved, only the fact that they are present or not: this means implementing max only for binary feature values. In this implementation Softmax is constructed without providing vector of probability, but instead putting all the confidence on only one of the output unit. Finally note that the network here implemented is still shallow: three layers, 1 visible, 1 hidden, 1 output.

Authors of [210] implemented a binary shallow neural network inside D-wave quantum annealer. The problem of modelling a binary neural network is formulated as quadratically constrained binary optimization (QCBO):

$$\min_{\{\mathbf{w}^{l'}, \mathbf{z}^{l'}\}_{l'=0}^{l-1}, \{\mathbf{y}^{l'}\}_{l'=1}^{l-1}, y^l} \left(\frac{y^l - \hat{y}}{2} \right)^2 \quad (\text{B.99})$$

s.t.

$$\begin{aligned} y^l &\in \{-1, 1\} \\ W_{ij}^{l'}, y_i^{l'}, Z_{ij}^{l'} &\in \{-1, 1\} \\ Z_{ij}^{l'} &= W_{ji}^{l'} y_i^{l'} \\ \left(\sum_i Z_{ij}^{l'} \right) y_j^{l'} &\geq 0. \end{aligned}$$

where layers set is denoted as $\ell \in \{0, 1, \dots, L-1\}$, binary weights as $\mathbf{W}^\ell$ and inputs at each layer as $\mathbf{y}^\ell$. Last condition accounts for a sign function nonlinearity between each layer, meaning the output of the j -th neuron at layer $l' + 1$ is

$$y_j^{l'+1} = \text{sgn} \left(\sum_i Z_{ij}^{l'+1} \right). \quad (\text{B.100})$$

Collating all the binary variables into the N -element spin vector one obtains

$$\mathbf{s} = (y_0^1, y_1^1, \dots, Z_{0,0}^0, Z_{0,1}^0 \dots Z_{0,0}^1, Z_{0,1}^1, \dots, W_{0,0}^0, W_{0,1}^0, \dots, y^l)$$

one can then rewrite the QCBO formulation as

$$\min_{\mathbf{s}} \left(\frac{s_{\text{last}} - \hat{y}}{2} \right)^2 \quad (\text{B.101})$$

s.t. it satisfies same constraints from QCBO energy above presented. s_{last} is the final element of $\mathbf{s}$. Finally, authors encode the problem in QUBO, which implies converting spin variables $\mathbf{s} \in \{-1, 1\}^n$ into the equivalent binary variables via

$$\mathbf{q} = \frac{(\mathbf{s} + 1)}{2} \in \{0, 1\}^N \quad (\text{B.102})$$

The equivalent QUBO formulation authors obtained is then

$$\min_{\mathbf{q} \in \{0,1\}^n} \left(q_{y^L} - \hat{b} \right)^2 + \sum_{l', i, j} g \left(q_{Z_{i,j}^{l'+1}}, q_{W_{i,j}^{l'}}, q_{y_i^{l'}}, b_{i,j}^{l'} \right) + \sum_{l', i, j} k \left(a_{i,j}^{l'}, \dots, q_{y_j^{l'+1}}, q_{Z_{k,j}^{l'}} \right) \quad (\text{B.103})$$

The first term of the QUBO energy function is the squared 0 – 1 indicator loss with the label $\hat{b} = (\hat{y} + 1)/2$. Given a training datum $(\mathbf{y}^0, \hat{y})$, the 0 – 1 loss is defined as

$$L_{0-1}(\hat{y}, y^l) = \begin{cases} 0 & \hat{y} = y^l; \\ 1 & \text{otherwise} . \end{cases} \quad (\text{B.104})$$

The second term enforces the element-wise multiplications, represented as third condition of QCBO energy formulation. The bipolar QCBO product formulation can be rewritten using binary variables as $q_{Z_{ij}^{\ell+1}} = \text{XNOR}(q_{W_{ji}^\ell}, q_{y_i^\ell})$ where hereafter the subscript s in q_s indexes the binary variable in q corresponding to the spin variable s . Introducing a penalty parameter $P \in \mathbb{R}^+$, violations to the logical statement $q_3 = \text{XNOR}(q_1, q_2)$ can be penalized via the cubic polynomial

$$1 - q_1 - q_2 - q_3 + 2q_1q_2 + 2q_2q_3 + 2q_1q_3 - 4q_1q_2q_3 \quad (\text{B.105})$$

and the cubic term $q_1q_2q_3$ can be reduced to a quadratic term with an additional ancillary binary variable b , i.e.,

$$q_1q_2q_3 \approx bq_3 + (3b + q_1q_2 - 2q_1b - 2q_2b). \quad (\text{B.106})$$

The ancillary bits required by the element-wise multiplications due to the cubic terms above are equal to the number of the weights. The last term enforces the summations and non-linearities. The required output of non-linearity corresponds to the highest bit of binary representation of summation of the provided inputs to non-linearity. This could be done "counting" how many Z bits are ones, representing each Z in base two, and using the most significant bit $q^{(\text{out})}$ as output. To enforce this constraints, they built a the quadratic term h :

$$h(a_0, \dots, a_{n-2}, q^{(\text{out})}, q_0^{(\text{in})}, \dots, q_{2^n-1}^{(\text{in})}) = \left(-\sum_{i=0}^{n-2} 2^i a_i - 2^{n-1} q^{(\text{out})} + \sum_{k=0}^{2^n-1} q_k^{(\text{in})} \right)^2 \quad (\text{B.107})$$

where the output bit $q^{(\text{out})}$ is the bit representing non-linearity output, the number of inputs of a layer is restricted to dimension $2^n - 1$, and a_k are auxiliary bits needed for enforcing constraints. The number of ancilla bits required by the non-linearities scales only logarithmically with the input size of the layer ($\log_2(n)$). Using standard minorminer embedding for a two-layer BNN with 3 input neurons and 3 hidden neurons, and 1 output neuron, authors analyzed this implementation in a binary classification problem with the UCI "adult" dataset. They validate correctness of formulation and obtain faster learning in respect of using classical algorithms. Note that the only non-linearity modelled is the sign function and the proposed network allow only for binary weights and input. It's interesting to note that in the Markov model of time evolution of Ising Hamiltonian/Hopfield model, a state is update only if the sign of the difference of the energy of the two state is positive, meaning sign function is somehow implicit in the Markovian path of the states of Quantum Annealer. This fact is not exploited in this formulation. Also the author approximate the product between input and weight with XNOR function. This is done to preserve bipolar behaviour in this product. Even if using bipolar implementation is mathematical convenient in pattern association tasks this is not necessary in principle as we will see in present work implementation. Finally, note that the total number of variables employed for m train data, l layers with n units for each layer is $l * k * m * n \log(n)$.

B.4 Present work integration

Chapter 2 integration

One could be tempted to not deal with substitutions anymore, leading to this possible approach to face the problem. Noting that if $\xi_i^\mu s_i = -1$ than $e^{\xi_i^\mu s_i} = e^{-1}$ and conversely if $\xi_i^\mu s_i = 1$ than $e^{\xi_i^\mu s_i} = e^1$;

recall energy function

$$E = - \sum_{\mu=1}^m \exp \left(\sum_i \xi_i^{\mu} s_i \right) \quad (\text{B.108})$$

As before supposing two stored memories with length three

$$\begin{aligned} \mu_1 &= [\mu_{1,1}, \mu_{1,2}, \mu_{1,3}] \\ \mu_2 &= [\mu_{2,1}, \mu_{2,2}, \mu_{2,3}] \end{aligned}$$

Considering the first term of the energy as before

$$-e^{(s_i \xi_i^{\mu_1} s_i)} = - (e^1)^t (e^{-1})^h \quad (\text{B.109})$$

where t is the number of products $\xi_i^{\mu_1} s_i$ with values equal 1 and h is the number of products $\xi_i^{\mu_1} s_i$ with values equal to -1. Note also that $t + h = n$. In particular being k the number of products $\xi_i^{\mu_1} s_i = 1$

$$t = \frac{(s_i \xi_i^{\mu} s_i) - n}{2} \quad (\text{B.110})$$

Wanting to minimize a term

$$\min_t - (e^1)^t (e^{-1})^{n-t} \quad (\text{B.111})$$

Suppose having a single stored memory $\exp(\sum_i \xi_i^{\mu} s_i)$. This value can always be set to n, meaning selecting values of s_i that set the product $\xi_i^{\mu} s_i$ equal to 1 while in general values of s_i interacts also with other memories in the outer summation $-\sum_{\mu=1}^m \exp(\sum_i \xi_i^{\mu} s_i)$. So it could be anticipated that only a subset of neurons will superimpose to a given memory, meaning the result of the network has a number of spins in common to a given pattern, that is represented with a term of the total energy function of dense associative memories. So the number of spins in common t_1 between solutions provided and a given memory stored could be modelled with

$$- \sum_{t_1=1}^n f(\xi^{\mu} \mathbf{s}, t_1) (e^{-1})^{n-t_1} (e^1)^{t_1} \quad (\text{B.112})$$

This formula substitutes a term of the total energy function of dense associative memory given a contribution to the minimization proportional to how much a given memory is "satisfied", meaning it quantifies of much of memory spins are given as a result of the network. $f(s_i, t_1)$ should be a function capable to identify when $\xi^{\mu} \mathbf{s}$ has value t_1 . This function has value 1, when product $\xi^{\mu} \mathbf{s}$ has value t_1 . In this way the value for which is multiplied when this happens it gives contribution to energy function, and this value as seen before corresponds to original contribution of a term in dense associative memory energy function. Also this function computes a value belonging to range 0 to -inf proportionally to distance t_1 . This aspect is important for the retrieval of the solution, see end of the

chapter. A function satisfying this constraint could be simply

$$f(\xi^\mu s, t_1) = - \left(\frac{(\sum_i \xi_i^\mu s_i) - n}{2} - t_1 \right)^2 + 1 \quad (\text{B.113})$$

$$= - \left(\frac{((\sum_i \xi_i^\mu s_i) - n)^2}{4} + t_1^2 - t_1 \left(\left(\sum_i \xi_i^\mu s_i \right) - n \right) \right)^2 + 1 \quad (\text{B.114})$$

$$= - \frac{((\sum_i \xi_i^\mu s_i) - n)^2}{4} - t_1^2 + t_1 \left(\left(\sum_i \xi_i^\mu s_i \right) - n \right) + 1 \quad (\text{B.115})$$

$$= - \frac{((\sum_i \xi_i^\mu s_i))^2 + n^2 - 2n(\sum_i \xi_i^\mu s_i)}{4} - t_1^2 + t_1 \sum_i \xi_i^\mu s_i - nt_1 + 1 \quad (\text{B.116})$$

$$= - \frac{((\sum_i \xi_i^\mu s_i))^2}{4} - t_1^2 + t_1 \sum_i \xi_i^\mu s_i - \frac{n(\sum_i \xi_i^\mu s_i)}{2} - nt_1 + \frac{n^2}{4} + 1 \quad (\text{B.117})$$

$$= - \frac{\left((\sum_i \xi_i^\mu s_i)^2 + 2 \left(\sum_{j<i} \xi_i^\mu s_i \xi_j^\mu s_j \right) \right)^1}{4} - t_1^2 + t_1 \sum_i \xi_i^\mu s_i \quad (\text{B.118})$$

$$\begin{aligned} & - \frac{n(\sum_i \xi_i^\mu s_i)}{2} - nt_1 + \frac{n^2}{4} + 1 \\ & = + \frac{- \sum_i (\xi_i^\mu s_i)^2 - 2 \left(\sum_{j<i} \xi_i^\mu s_i \xi_j^\mu s_j \right)}{4} - t_1^2 + t_1 \sum_i \xi_i^\mu s_i \\ & \quad - \frac{n(\sum_i \xi_i^\mu s_i)}{2} - nt_1 + \frac{n^2}{4} + 1 \end{aligned} \quad (\text{B.119})$$

Given that $\xi_i^\mu s_i$ squared is equal to 1

$$f(\xi^\mu s, t_1) = + \frac{- \left(\sum_{j<i} \xi_i^\mu s_i \xi_j^\mu s_j \right)}{2} + t_1 \sum_i \xi_i^\mu s_i - \frac{n(\sum_i \xi_i^\mu s_i)}{2} \quad (\text{B.120})$$

$$\begin{aligned} & - nt_1 - t_1^2 + \frac{n^2}{4} - \frac{n}{4} + 1 \\ & = + \frac{- \left(\sum_{j<i} \xi_i^\mu s_i \xi_j^\mu s_j \right)}{2} + \left(t_1 - \frac{n}{2} \right) \sum_i \xi_i^\mu \sigma_i \\ & \quad - nt_1 - t_1^2 + \frac{n^2}{4} - \frac{n}{4} + 1 \end{aligned} \quad (\text{B.121})$$

However it should be noted that this function f is an hyper-parameter. Using different functions f , different penalties for non-zero distance from the memorized memory appears. However care should be taken for functions that exhibit higher than two interactions that would lead to same problems of decomposability that had to be faced before. Returning to the case of two memories with length three

$$\begin{aligned} \mu_1 &= [\mu_{1,1}, \mu_{1,2}, \mu_{1,3}] \\ \mu_2 &= [\mu_{2,1}, \mu_{2,2}, \mu_{2,3}] \end{aligned}$$

With total energy

$$E = -e^{(s_i \xi_i^{\mu_1} s_i)} - e^{(s_i \xi_i^{\mu_2} s_i)} \quad (\text{B.122})$$

This method suffers however of approximation issues. Recall every term corresponding to a memory satisfaction contribution in the original dense associative memory is substituted with

$$-\sum_{t=1}^n f(\xi^\mu \mathbf{s}, t_1) (e^{-1})^{n-t_1} (e^1)^{t_1} \quad (\text{B.123})$$

where ideally f function should not involve more than binary interaction to not have to introduce auxiliary variables. In fact not having more than binary interactions leads to a simple variant learning rule for classical Hopfield neural network suffering of original limited capacity. This occurs because partial contributions to a stored memory, meaning the fact that an associative memory output is partially coincident with other memories stored, are weighted linearly in respect of the number of spins in common between a given memory and an output provided. This phenomenon implies a tendency of the net to evolve toward states that minimize energy satisfying partially the greatest number of other memories instead of trying to satisfy totally a single memory without caring about partially superposition to other memories. As said before f , should have an output value belonging to range 0 to $-\infty$ proportionally to distance t_1 . Note that this positive contribution is the same independently of number of spins not satisfied. This in fact imply the linear contribution of each memory to the total energy minimization. If care is used in selecting a suitable f , such that this function gives appropriate penalty for not satisfied bit, this could be however a simple and extremely cost effective approach amenable of local approximation of a dense associative memory energy function.

As an example, a similar approach could be used also for other dense associative memory functions, in particular rectified polynomial of order two could be treated separating first the energy in two cases having a value of zero if the input y , in this case dot product of vector of spins and memory, is negative Case $y \geq 0$

$$\sum_{\mu=0}^m \sum_{j=0}^{z_\mu} \gamma_{\mu,j} j^2 \left(a(y-j)^2 - \frac{1}{\gamma_{\mu,j}} \right) \quad (\text{B.124})$$

Where $\gamma_{\mu,j}$ is a set of constants, z_μ is maximum degree of superposition between memory and network spins, given y is positive, m is the total number of memories stored and

$$a(1)^2 - \frac{1}{\gamma_{\mu,j}} = 1 \quad (\text{B.125})$$

$$a = 1 + \frac{1}{\gamma_{\mu,j}} \quad (\text{B.126})$$

in a way that $\left(a(y-j)^2 - \frac{1}{\gamma_{\mu,j}} \right)$ has the role of discriminating degree of superposition between network output and memory, given the fact that y is positive, having

$$\gamma_{\mu,j} j^2 \left(a(0)^2 - \frac{1}{\gamma_{\mu,j}} \right) = -j^2 \quad (\text{B.127})$$

Where $\gamma_{\mu,j'}$ is another set of constants. And for other cases with $\gamma_{\mu,j} \gg j^2 \left(a(n)^2 - \frac{1}{\gamma_{\mu,j}} \right)$ and $n \neq 0$

$$\gamma_{\mu,j} j^2 \left(a(n)^2 - \frac{1}{\gamma_{\mu,j}} \right) = 0 \quad (\text{B.128})$$

Case $y < 0$

$$\sum_{\mu=0}^m \sum_{j'=z'_\mu}^{-1} \gamma_{\mu,j'} (y-j')^2 \quad (\text{B.129})$$

The total E results in

$$\sum_{\mu=0}^m \left(\sum_{j=0}^{z_\mu} \gamma_{\mu,j} j^2 \left(a(y-j)^2 - \frac{1}{\gamma_{\mu,j}} \right) + \sum_{j=z'_\mu}^{-1} \gamma_{\mu,j'} (y-j')^2 \right) \quad (\text{B.130})$$